

Fogo em floresta

Vinicius Trindade Dias Abel - 2020007112

Parte 1

Implementação

```
In [125... import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import numba
from numba import jit
import random
```

```
In [126... @jit(nopython=True)
def vizinhos(N):
    """
    Calcula a tabela de vizinhos para uma grade 2D com condições de contorno per
    """
    L = int(np.sqrt(N)) # Calcula o tamanho de cada lado da rede, assumindo que
    viz = np.zeros((N, 4), dtype=np.int32) # Inicializa a tabela de vizinhos (N

    for k in range(N): # Itera sobre cada nó da rede.
        # Vizinho à direita
        viz[k, 0] = k + 1
        if (k + 1) % L == 0: # Se ultrapassou o limite direito, volta ao início
            viz[k, 0] = k + 1 - L

        # Vizinho abaixo
        viz[k, 1] = k + L
        if k >= (N - L): # Se ultrapassou o limite inferior, conecta à linha su
            viz[k, 1] = k + L - N

        # Vizinho à esquerda
        viz[k, 2] = k - 1
        if k % L == 0: # Se ultrapassou o limite esquerdo, conecta ao lado dire
            viz[k, 2] = k + L - 1

        # Vizinho acima
        viz[k, 3] = k - L
        if k < L: # Se ultrapassou o limite superior, conecta à linha inferior.
            viz[k, 3] = k + N - L

    return viz # Retorna a tabela de vizinhos.
```

```
In [127... def simula_floresta(L, p, f, iteracoes, vizualizar):
    """
    Simula uma floresta com crescimento de árvores e incêndios periódicos.
    """
    N = L * L # Número total de nós na rede.
    viz = vizinhos(N) # Calcula os vizinhos de cada nó da floresta.
    tam_incendios = [] # Armazena tamanho dos incêndios.
```

```

# Inicializa a floresta
floresta = np.zeros(shape=N, dtype=np.int8)

for t in range(iteracoes):
    # Estado inicial da iteração
    estado_inicial = floresta.copy()

    # Crescimento de árvores
    crescimento = np.random.rand(N) < p
    floresta[(floresta == 0) & crescimento] = 1

    # Queda de raio
    raio_caiu = False
    local_raio = -1
    if np.random.rand() < f:
        local_raio = np.random.randint(0, N)
        raio_caiu = True
        if floresta[local_raio] == 1:
            floresta[local_raio] = 2 # Vermelho para árvore atingida
        else:
            floresta[local_raio] = 3 # Amarelo para local vazio

    estado_raio = floresta.copy()

    # Propagação do fogo
    if raio_caiu and floresta[local_raio] == 2:
        queimando = [local_raio]
        cluster = []
        while queimando:
            idx = queimando.pop()
            if idx not in cluster:
                cluster.append(idx)
                for v in viz[idx]:
                    if floresta[v] == 1:
                        floresta[v] = 4
                        queimando.append(v)
        tam_incendios.append(len(cluster))

    elif raio_caiu and floresta[local_raio] == 3:
        floresta[local_raio] = 0

    # Estado do cluster queimado
    estado_cluster = floresta.copy()

    # Limpa árvores queimadas
    for i in range(N):
        if floresta[i] == 4:
            floresta[i] = 0
        elif floresta[i] == 2:
            floresta[i] = 0

    # Visualizações
    if vizualizar:
        visualizar_floresta(estado_inicial, estado_raio, estado_cluster, L,
            print("\n" + "=" * 80 + "\n"))

return tam_incendios

```

In [128...

```

def visualizar_floresta(estado_inicial, estado_raio, estado_cluster, L, t, titulo)
    """
    Visualiza as três etapas de uma iteração:
    - Estado inicial.
    - Estado após a queda do raio.
    - Estado do cluster queimado.
    """
    # Definição das cores e seus significados
    cmap = ListedColormap(["white", "green", "red", "yellow", "orange"]) # Core
    labels = [
        ("Vazio", "white"),
        ("Árvore", "green"),
        ("Árvore atingida por raio", "red"),
        ("Vazio atingido por raio", "yellow"),
        ("Árvore queimando - Cluster", "orange"),
    ]

    # Configuração dos subplots
    fig, axes = plt.subplots(1, 3, figsize=(20, 7))

    # Título geral
    fig.suptitle(titulo, fontsize=16, fontweight="bold", y=0.95)

    # Estado inicial
    axes[0].imshow(estado_inicial.reshape((L, L)), cmap=cmap, interpolation="none")
    axes[0].set_title(f"Estado Inicial (t={t + 1})")
    axes[0].set_xticks(np.arange(-0.5, L, 1))
    axes[0].set_yticks(np.arange(-0.5, L, 1))
    #axes[0].grid(color="black", linestyle="-", linewidth=0.5) # Malha quadricu
    axes[0].tick_params(left=False, bottom=False, labelleft=False, labelbottom=False)

    # Estado após o raio
    axes[1].imshow(estado_raio.reshape((L, L)), cmap=cmap, interpolation="none",
    axes[1].set_title(f"Após Raio e Crescimento de árvore (t={t + 1})")
    axes[1].set_xticks(np.arange(-0.5, L, 1))
    axes[1].set_yticks(np.arange(-0.5, L, 1))
    #axes[1].grid(color="black", linestyle="-", linewidth=0.5) # Malha quadricu
    axes[1].tick_params(left=False, bottom=False, labelleft=False, labelbottom=False)

    # Estado do cluster queimado
    axes[2].imshow(estado_cluster.reshape((L, L)), cmap=cmap, interpolation="none")
    axes[2].set_title(f"Cluster Queimado (t={t + 1})")
    axes[2].set_xticks(np.arange(-0.5, L, 1))
    axes[2].set_yticks(np.arange(-0.5, L, 1))
    #axes[2].grid(color="black", linestyle="-", linewidth=0.5) # Malha quadricu
    axes[2].tick_params(left=False, bottom=False, labelleft=False, labelbottom=False)

    # Adicionando a Legenda como quadrados coloridos
    legend_elements = [
        plt.Line2D([0], [0], marker='s', color=color, label=label, markersize=10)
        for label, color in labels
    ]
    fig.legend(handles=legend_elements, loc="lower center", ncol=3, fontsize=10,

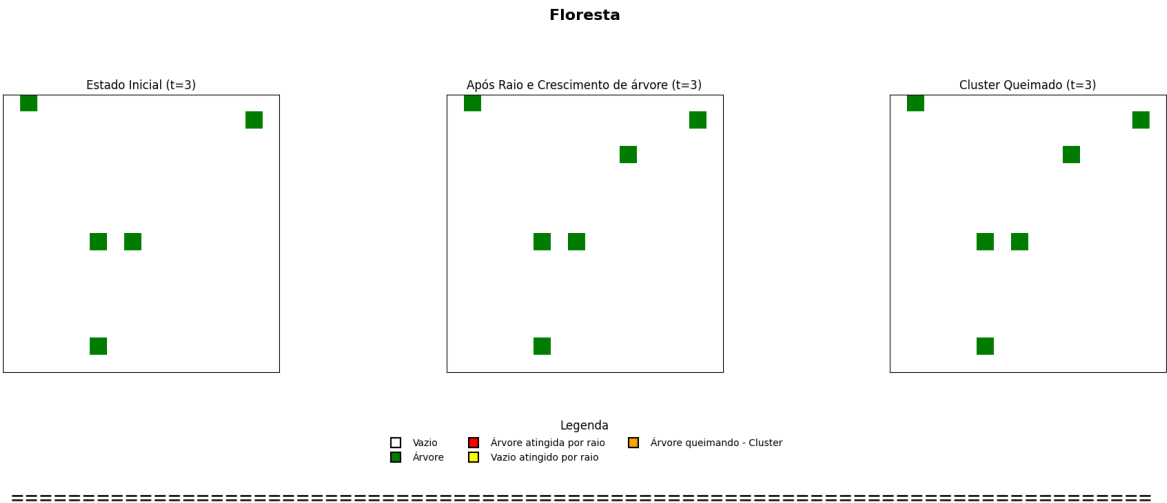
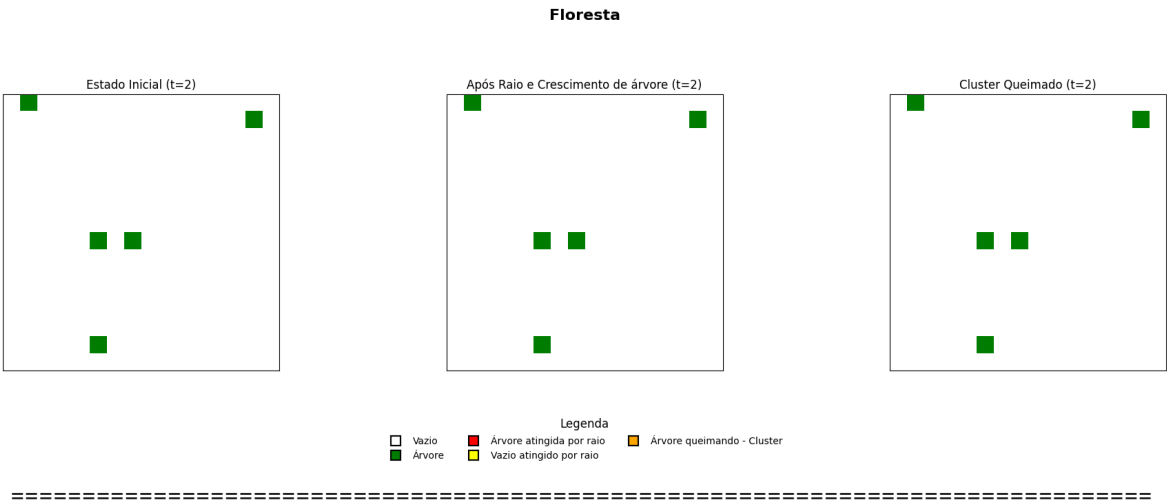
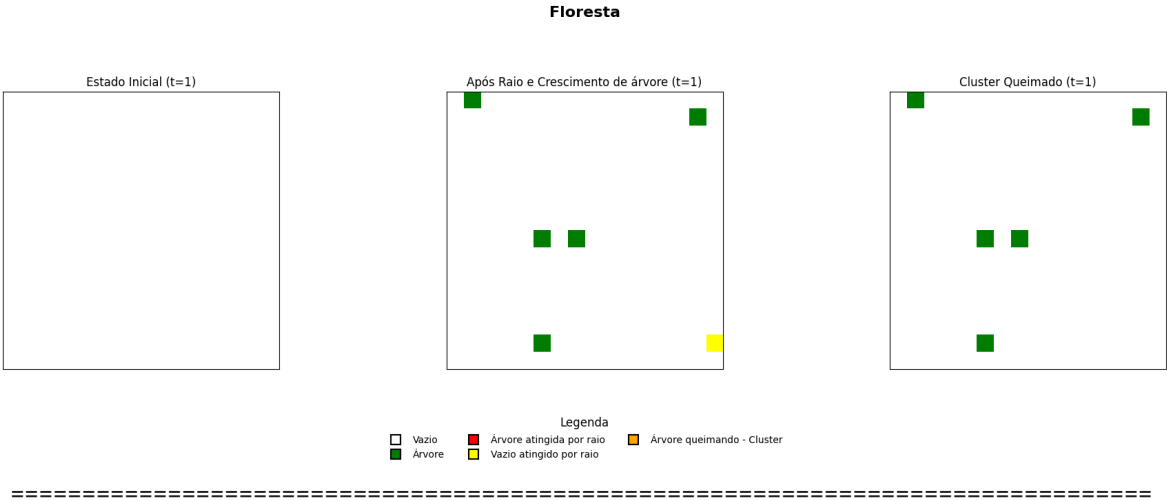
    # Ajuste do Layout
    plt.tight_layout(pad=5.0)
    plt.subplots_adjust(bottom=0.2) # Espaço para a Legenda
    plt.show()

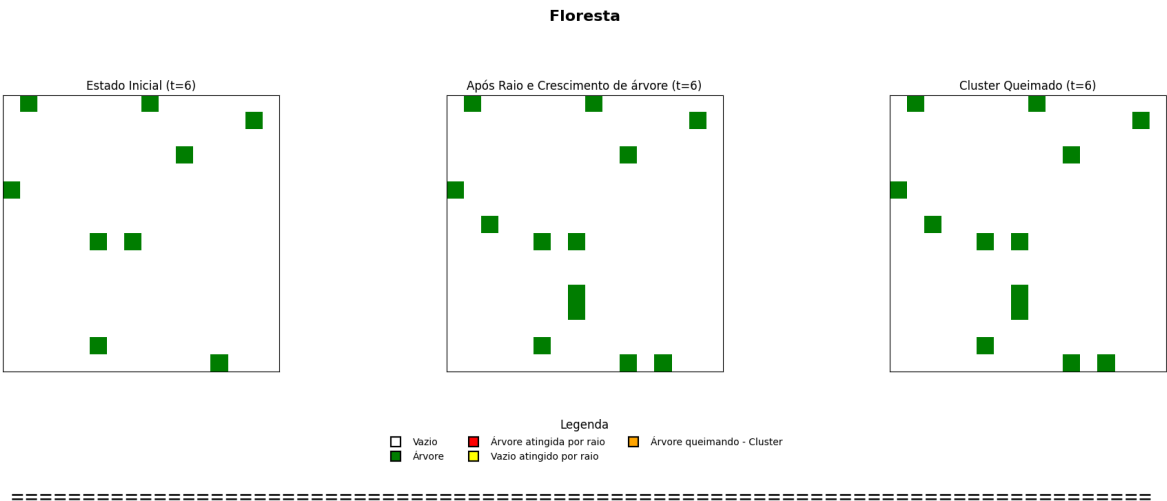
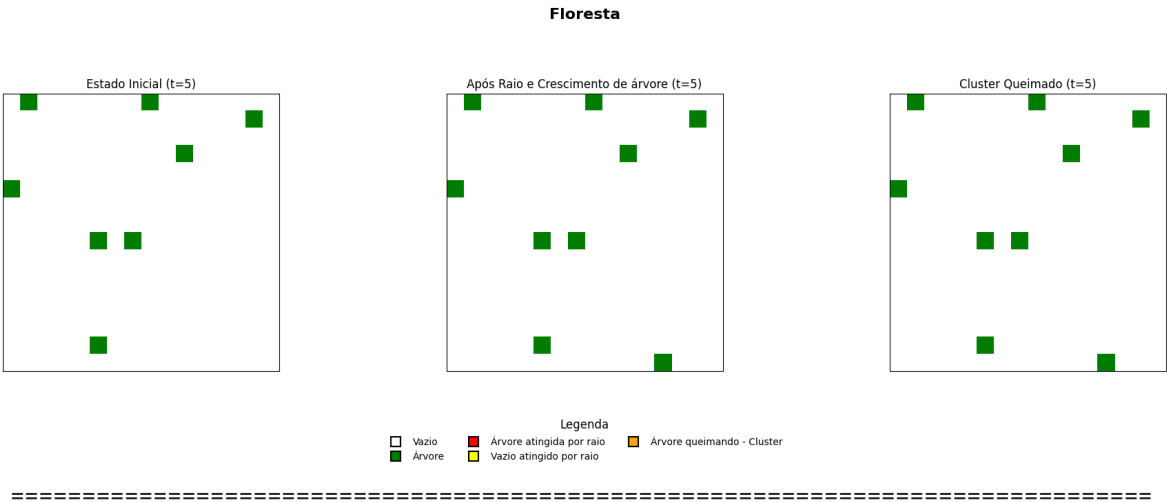
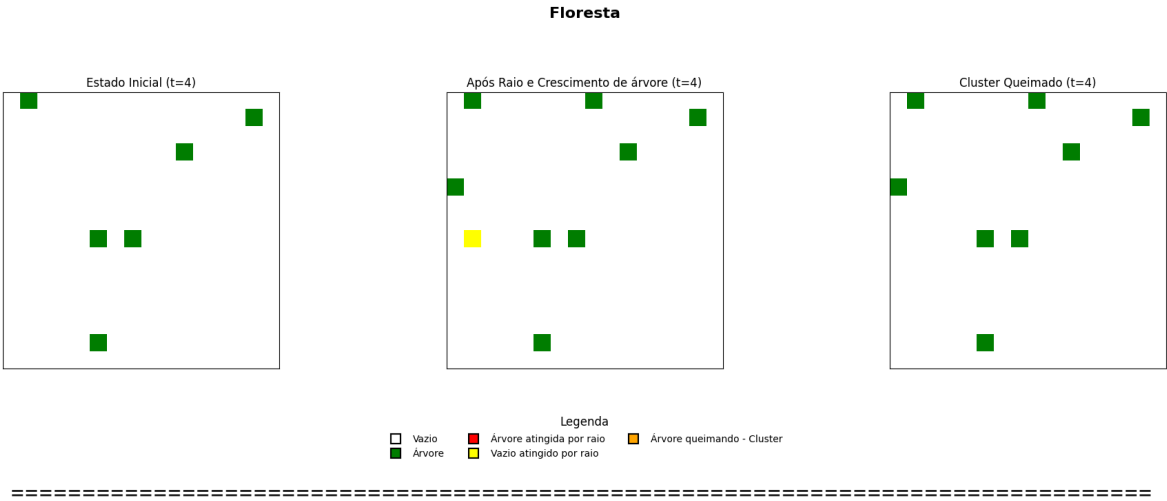
```

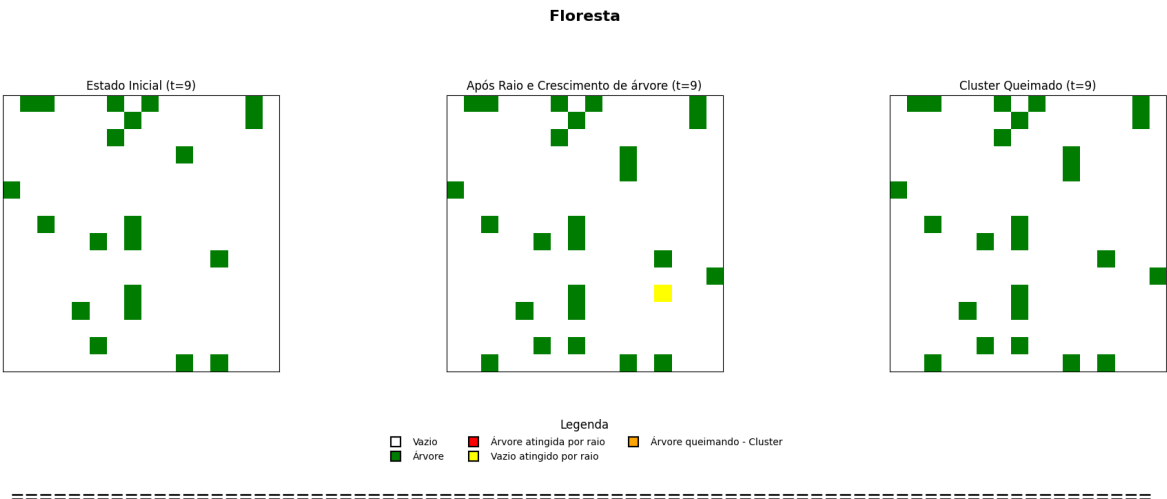
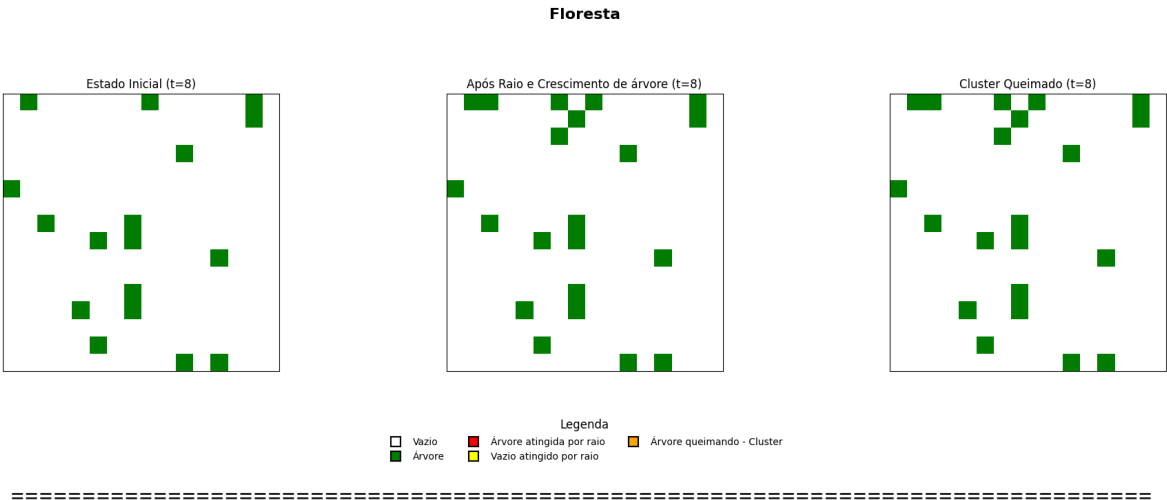
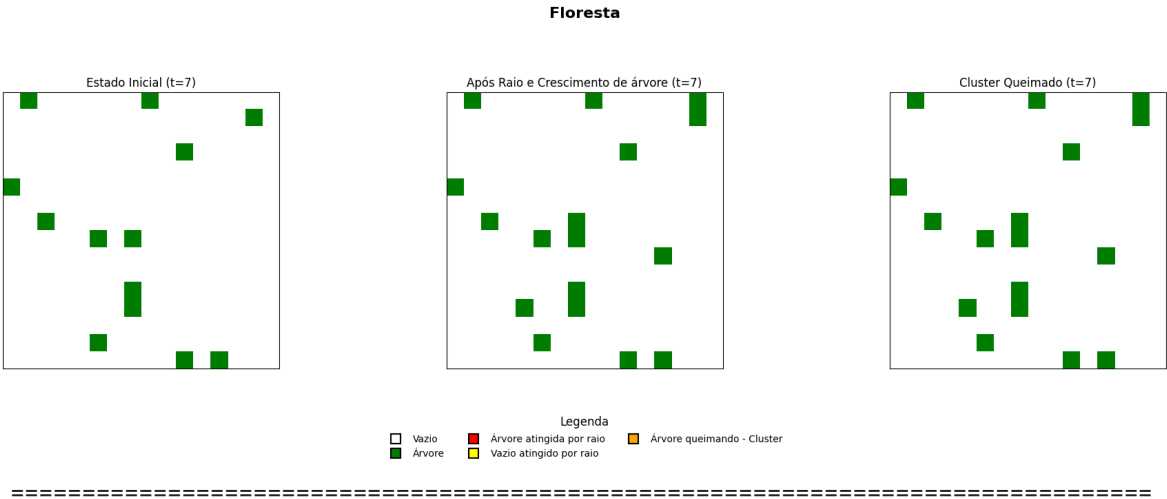
a.

```
In [129... L = 16
p = 0.01
f = 0.2
iteracoes = 100
vizualizar = True

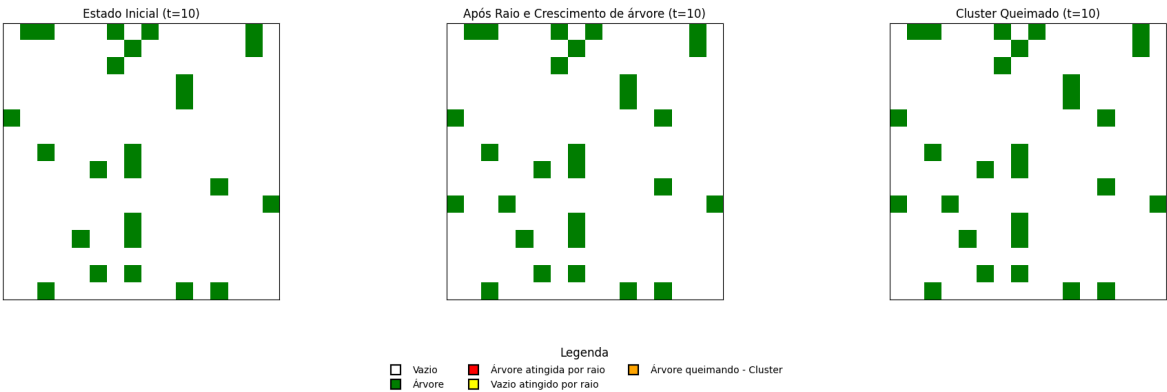
In [130... _ = simula_floresta(L, p, f, iteracoes, vizualizar)
```



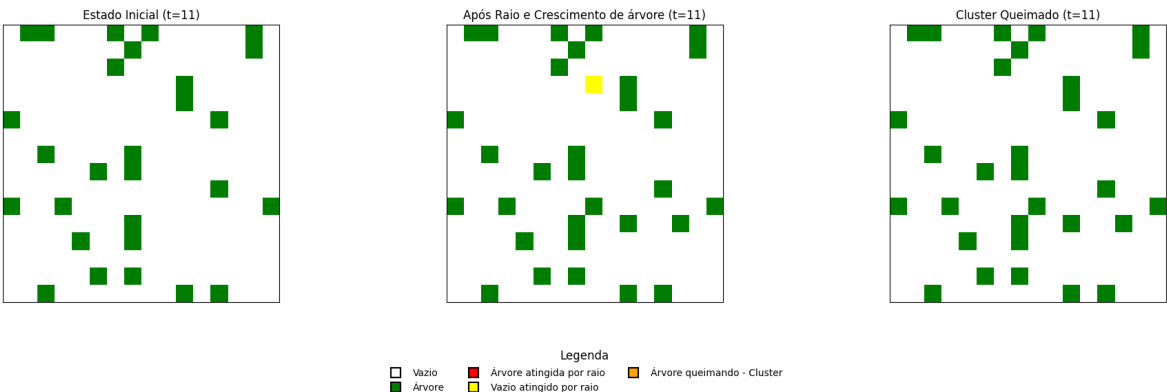




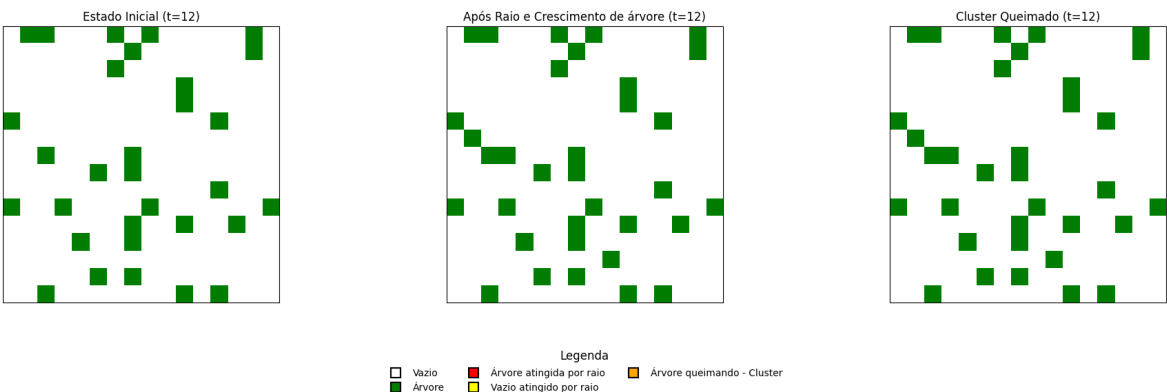
Floresta

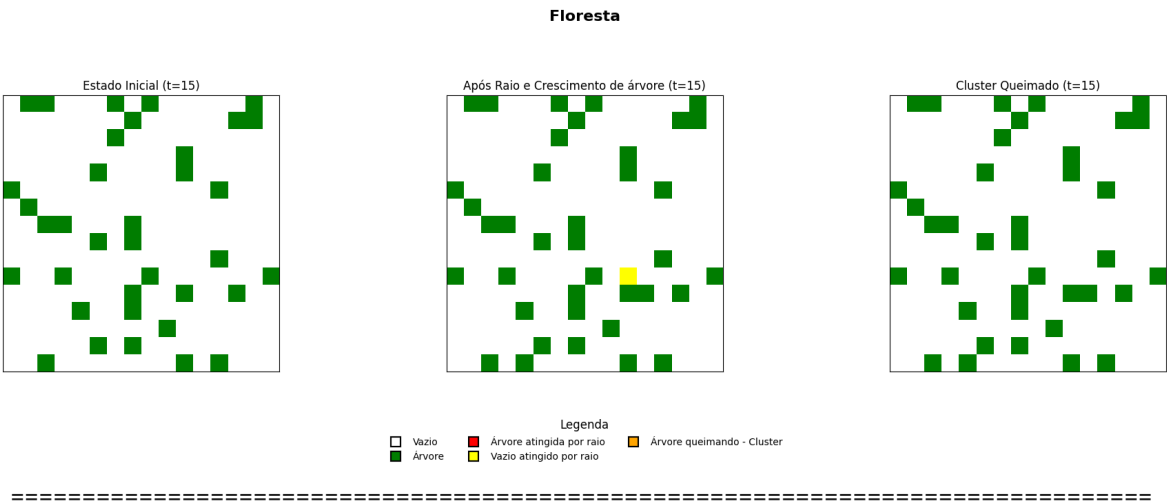
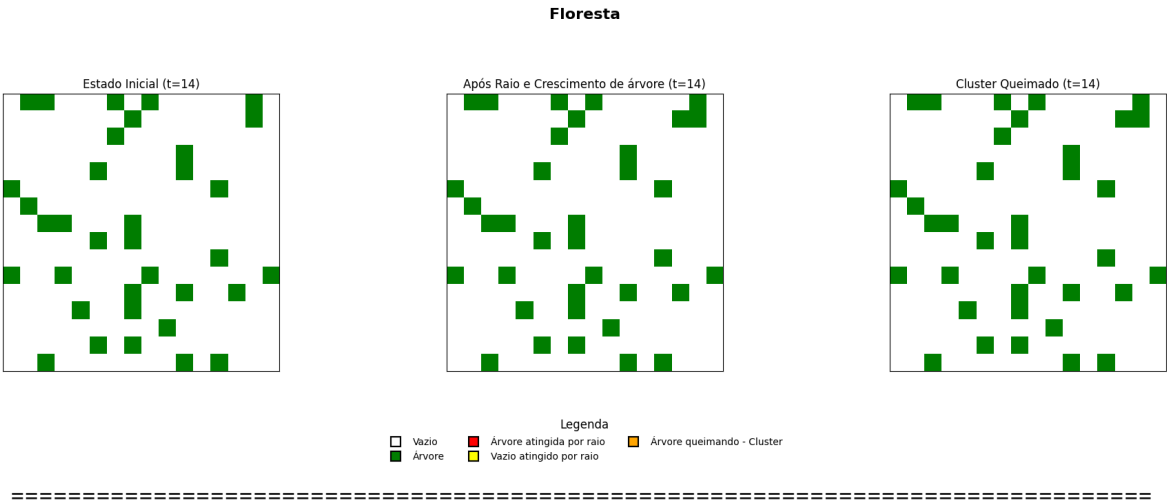
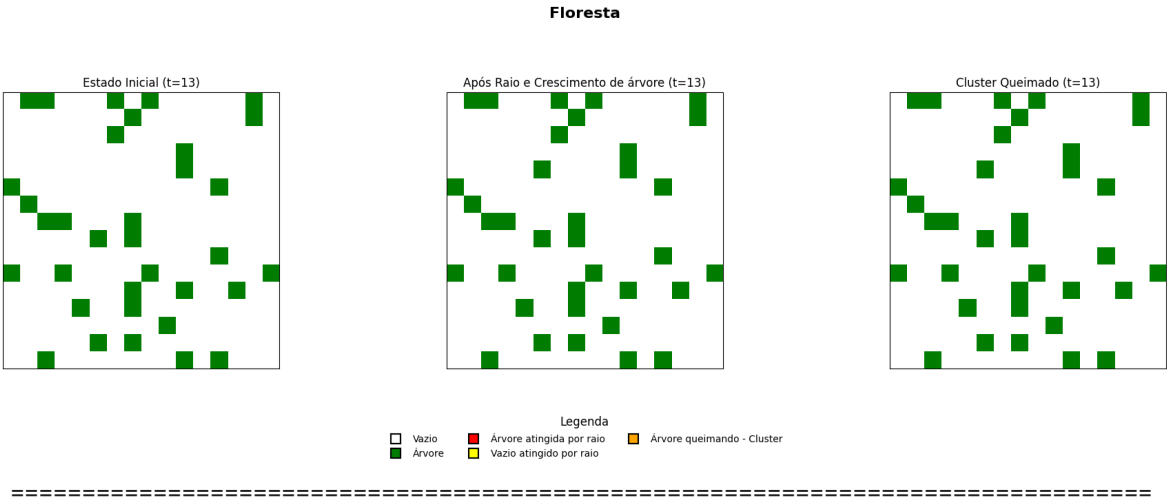


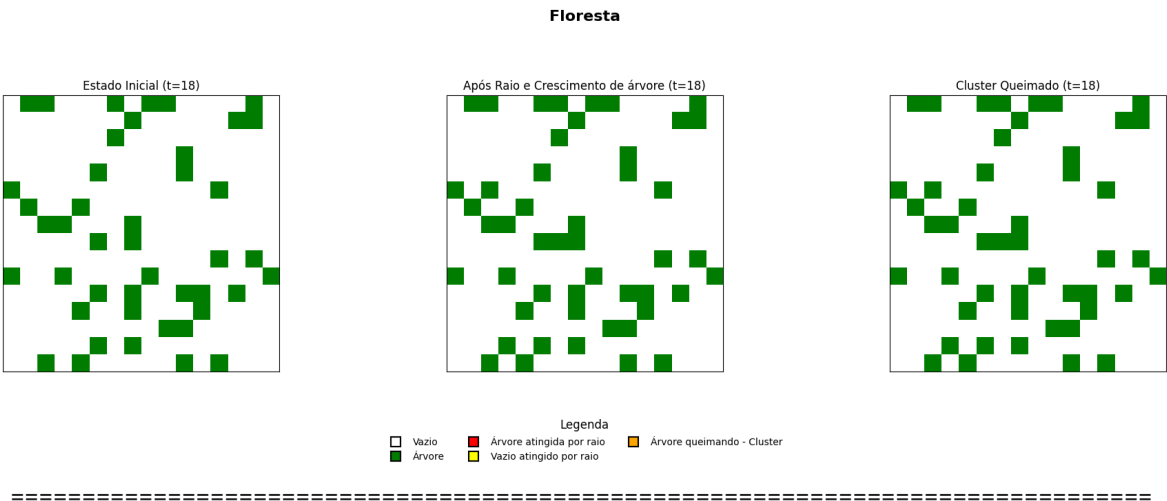
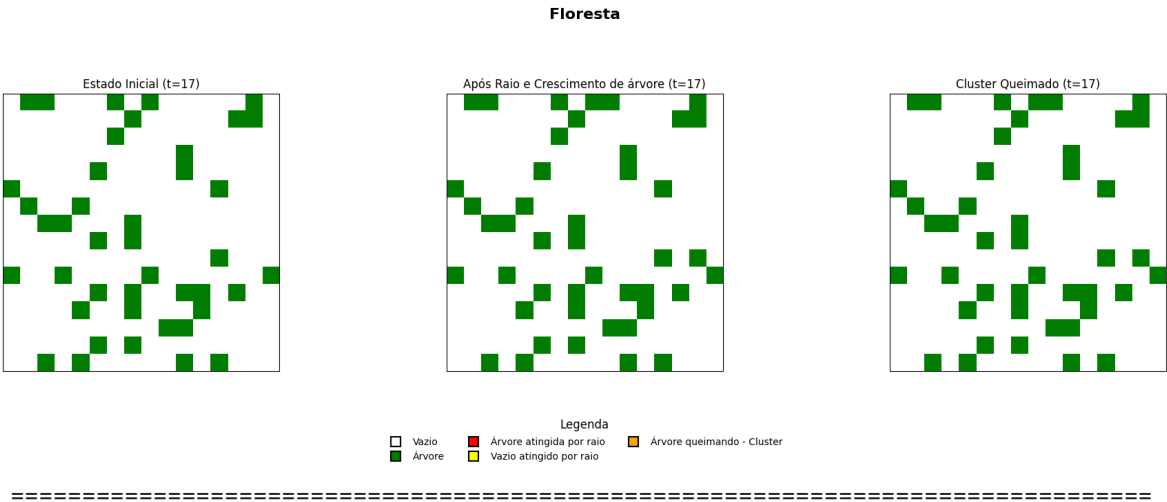
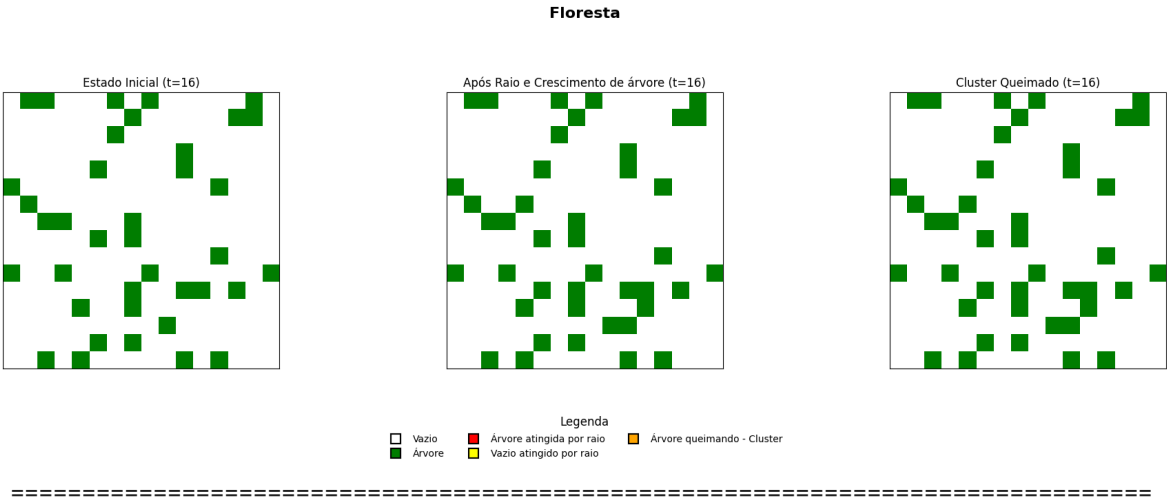
Floresta



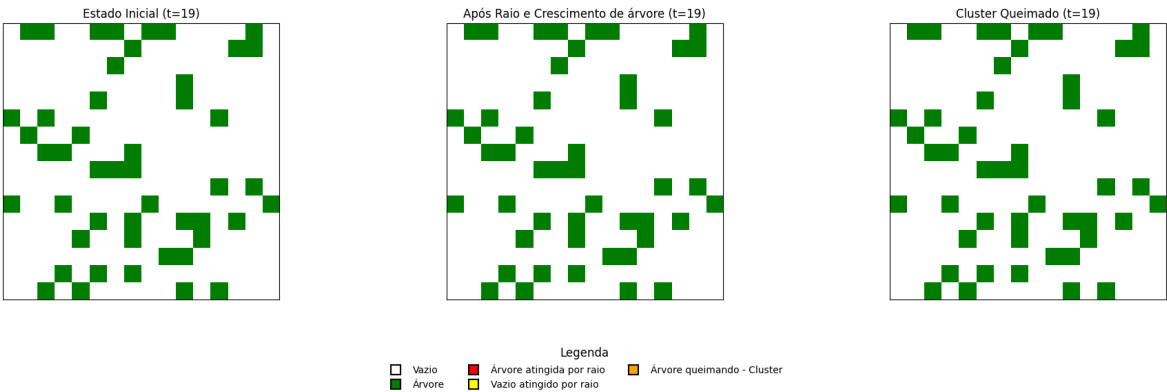
Floresta



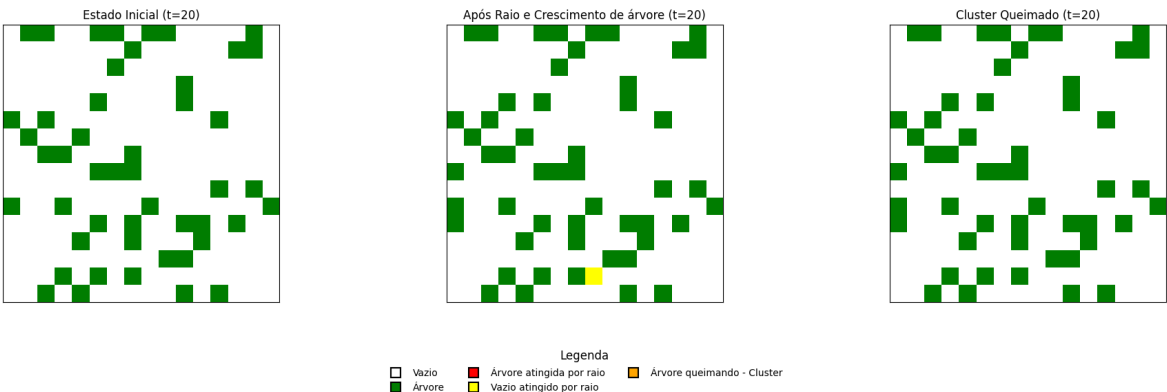




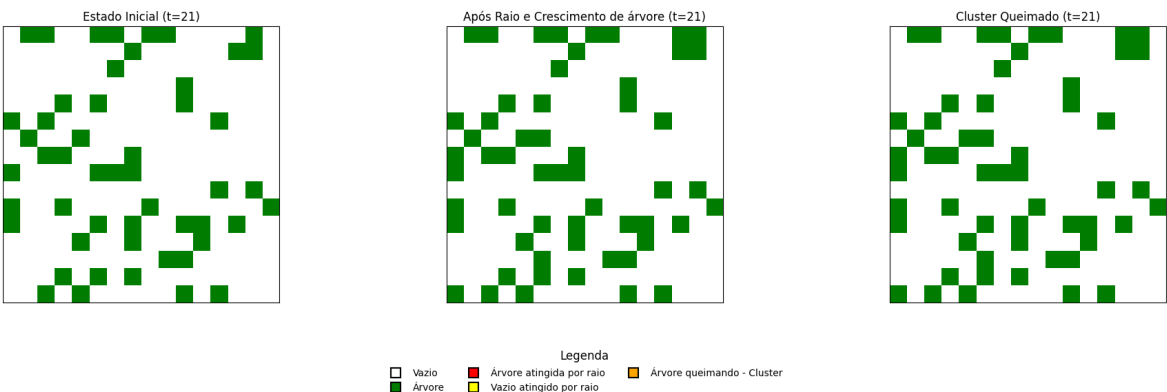
Floresta

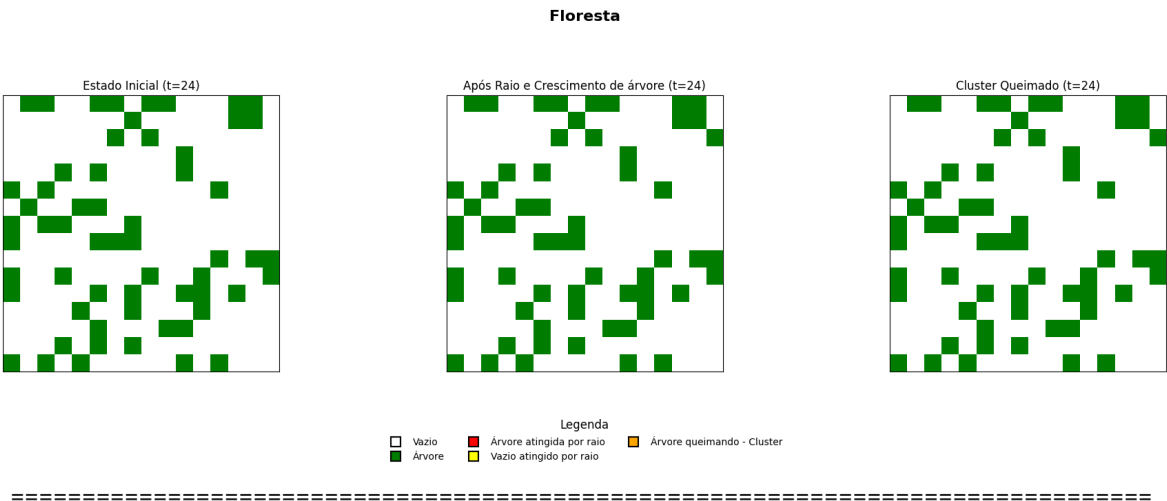
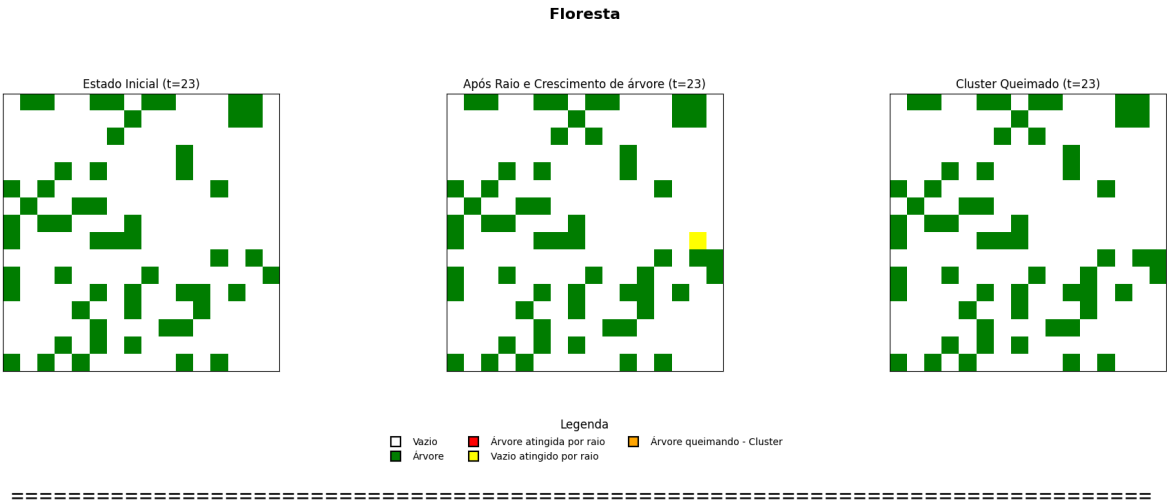
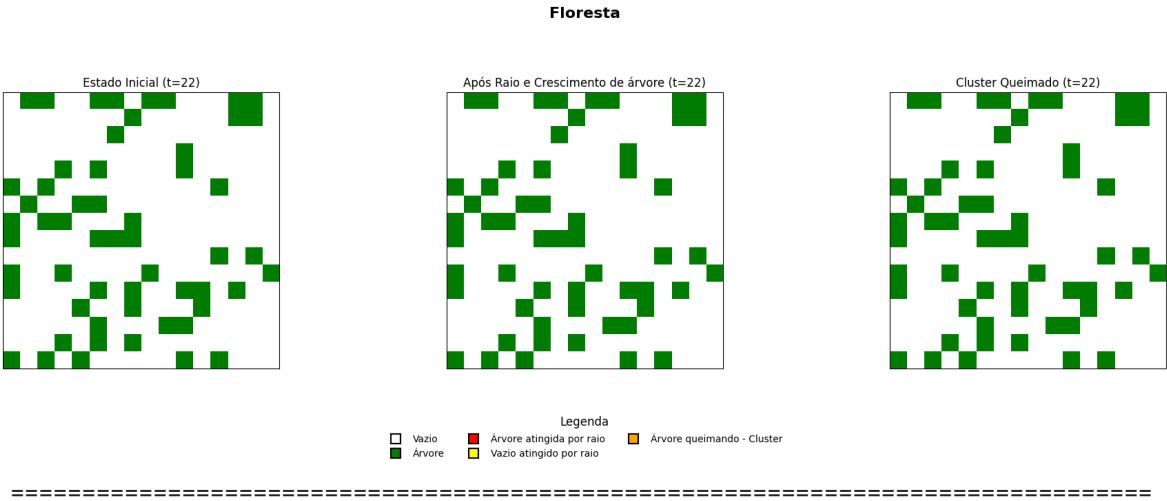


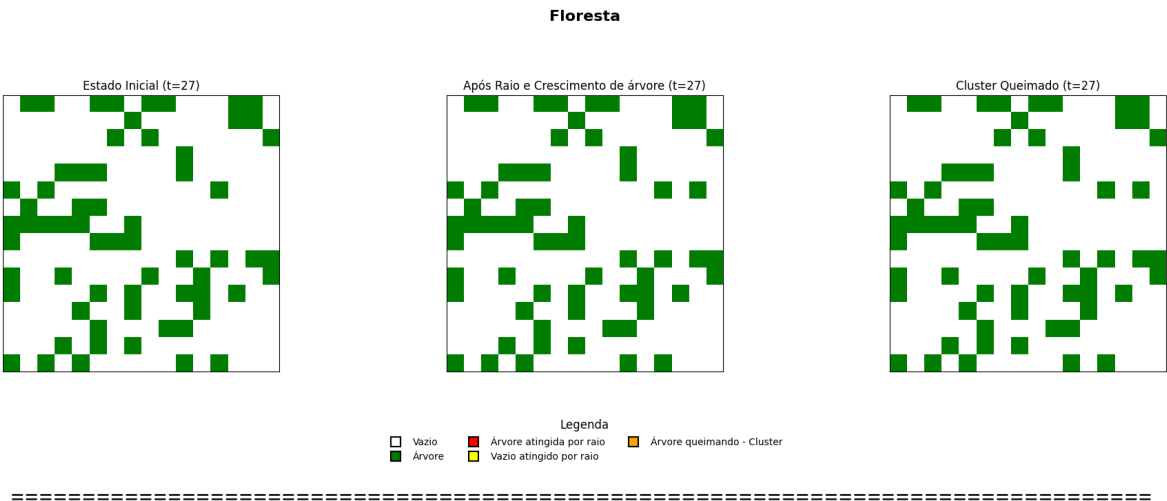
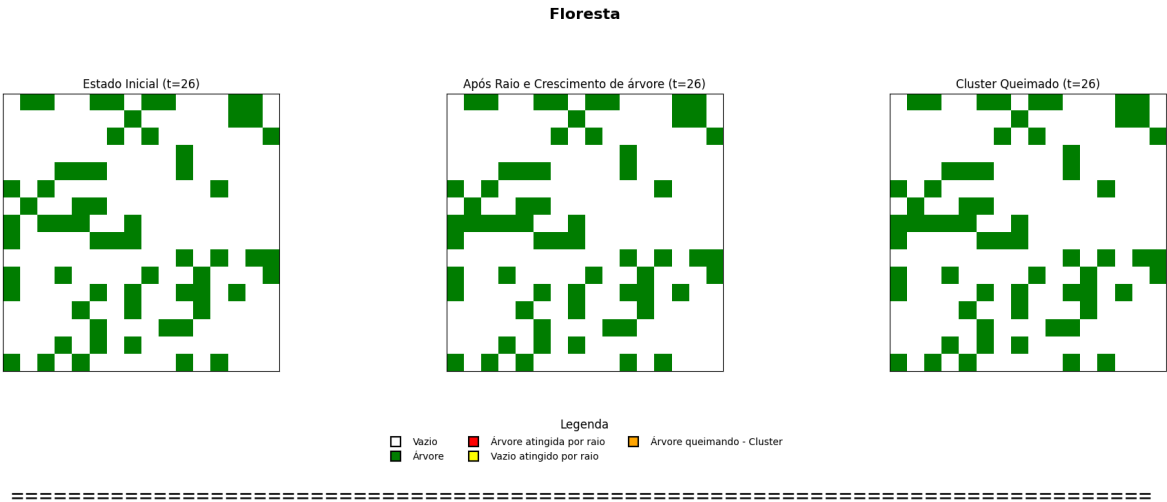
Floresta

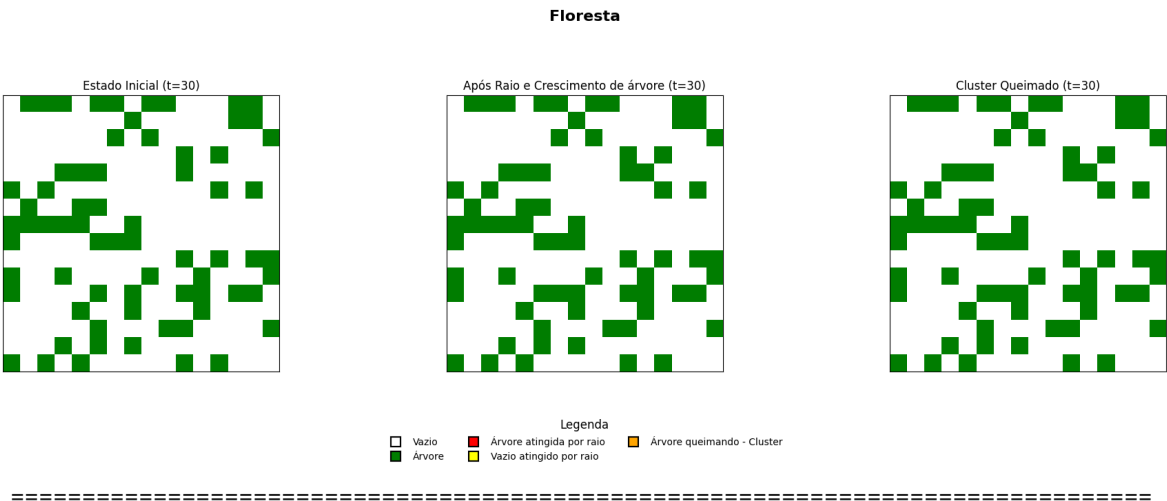
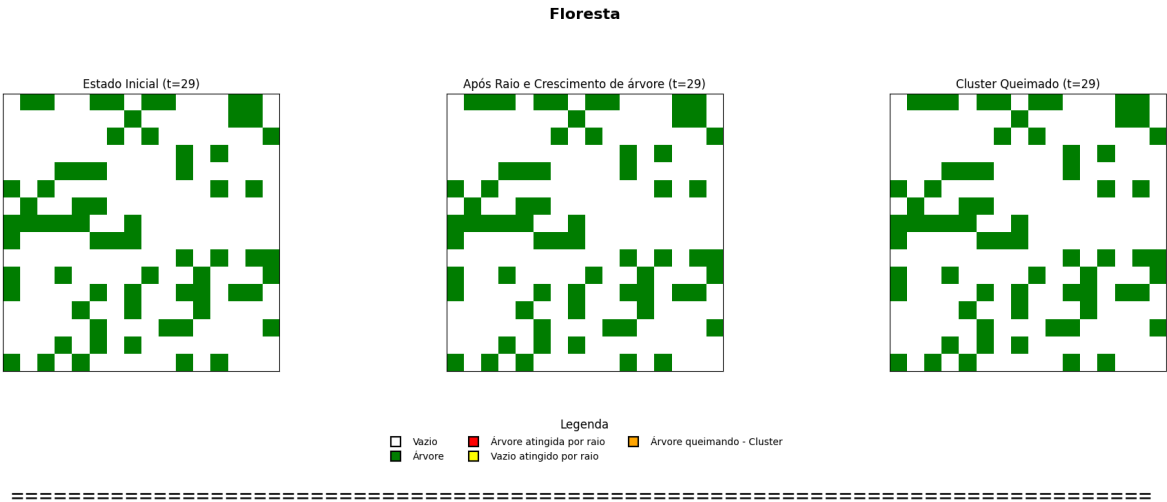


Floresta

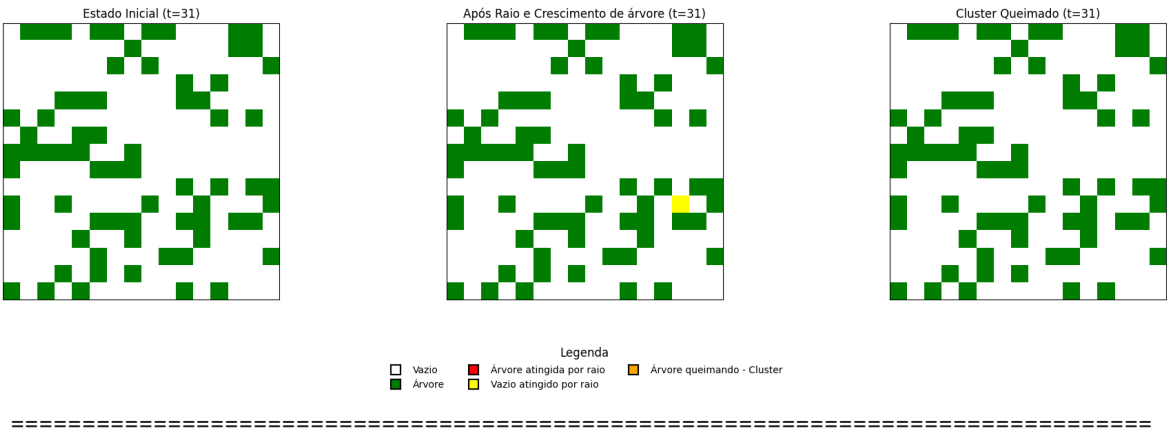




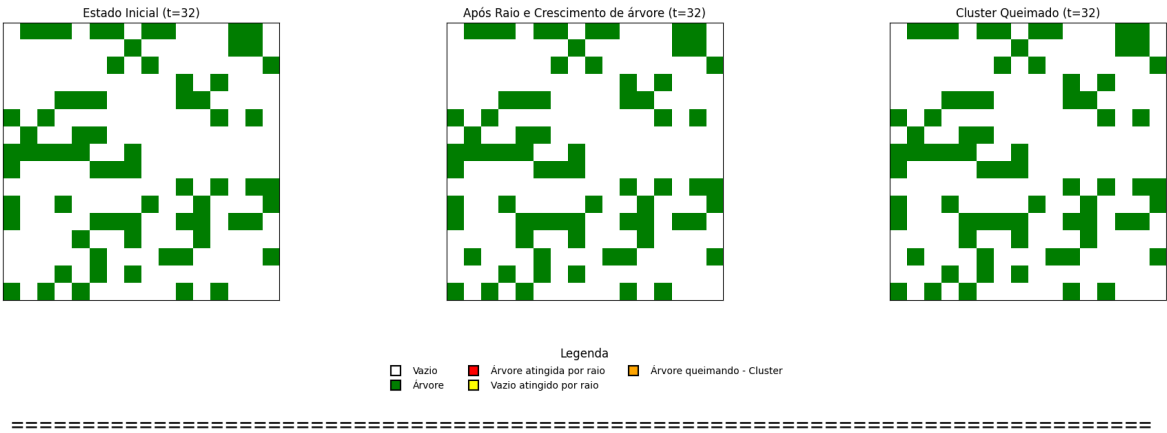




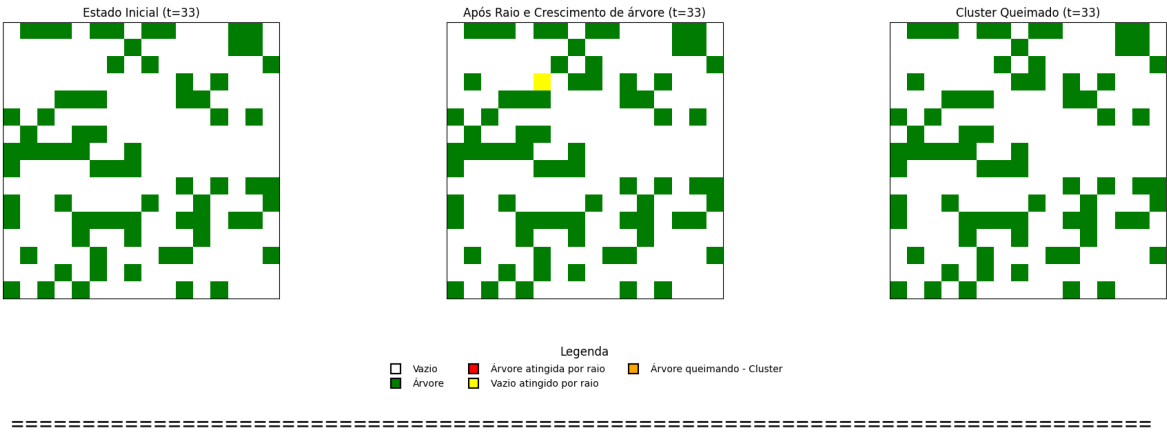
Floresta

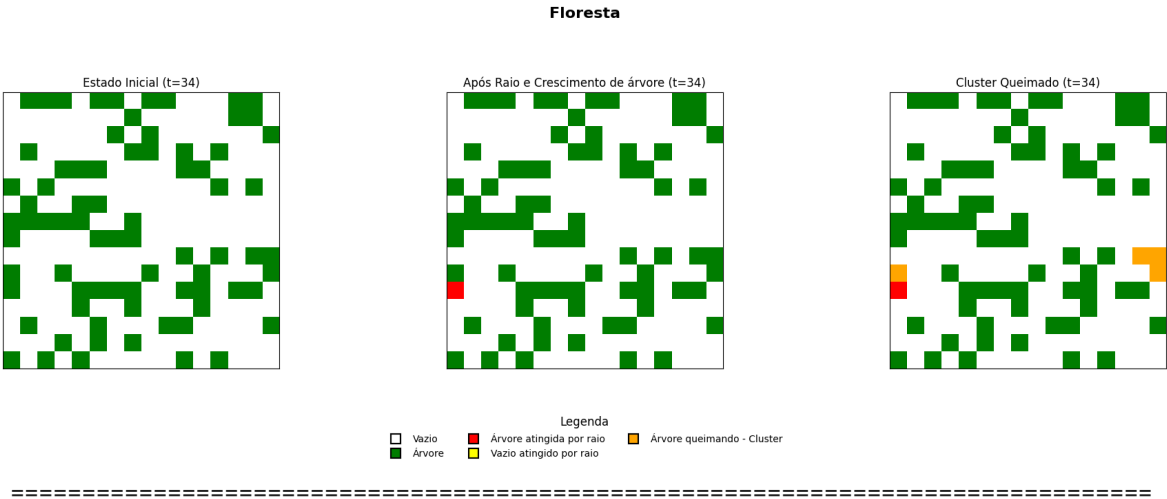


Floresta

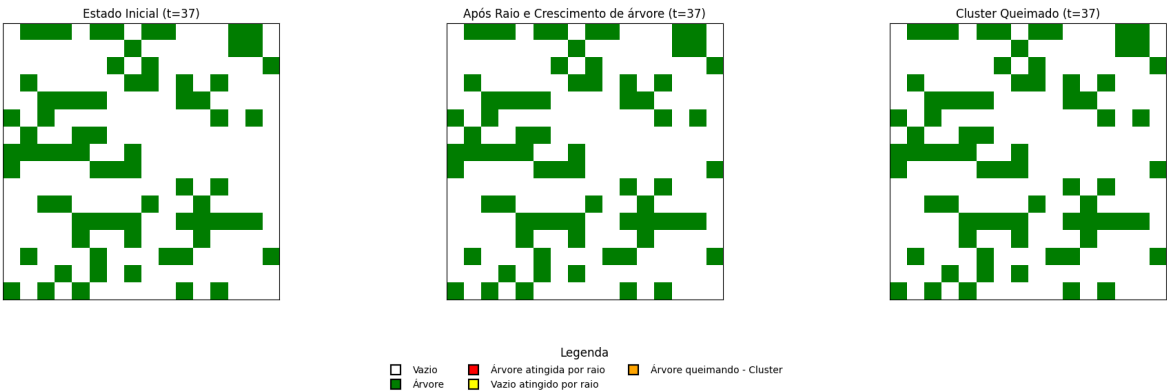


Floresta

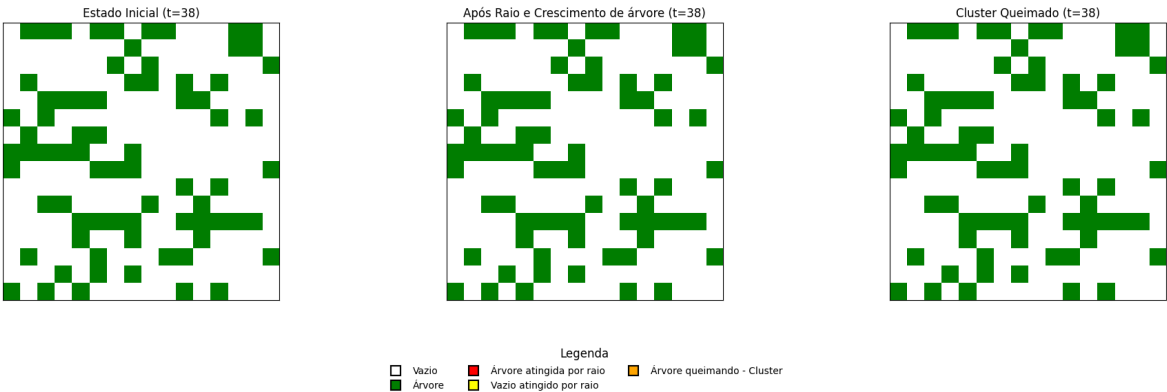




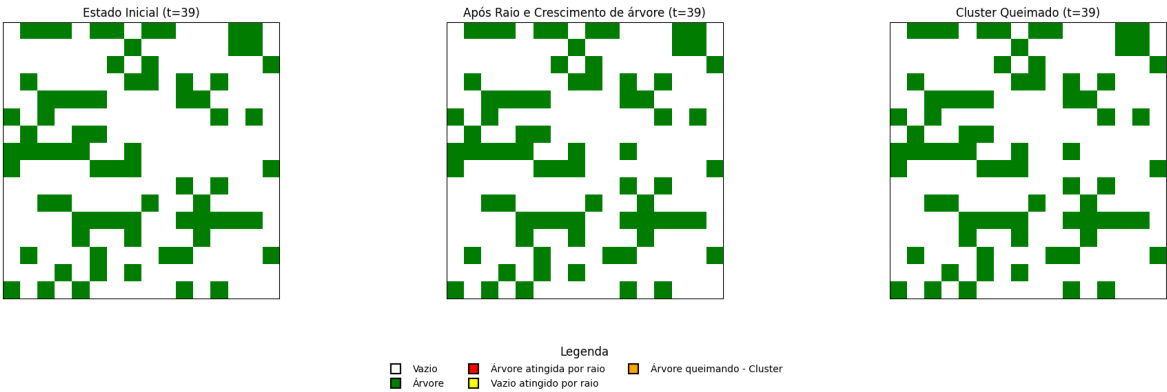
Floresta



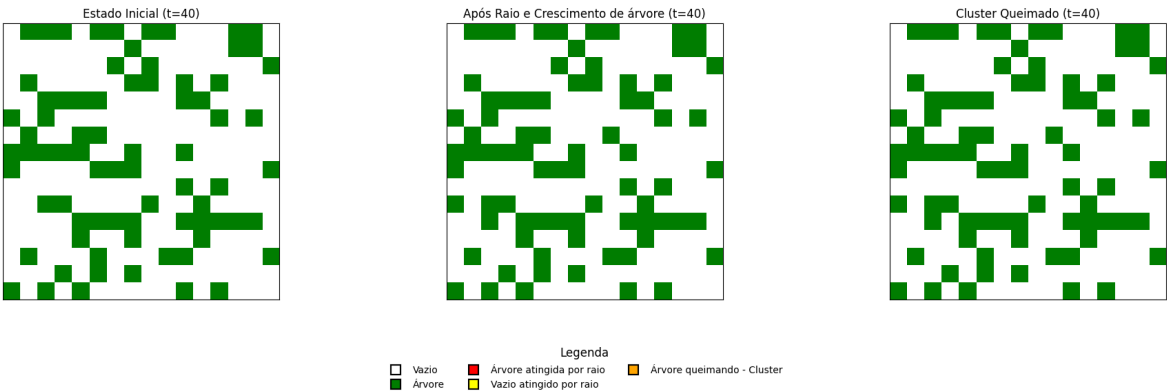
Floresta



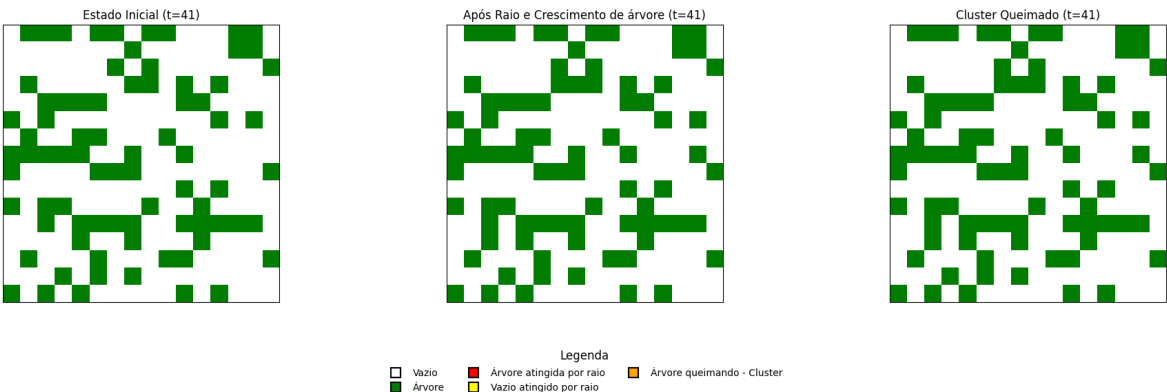
Floresta



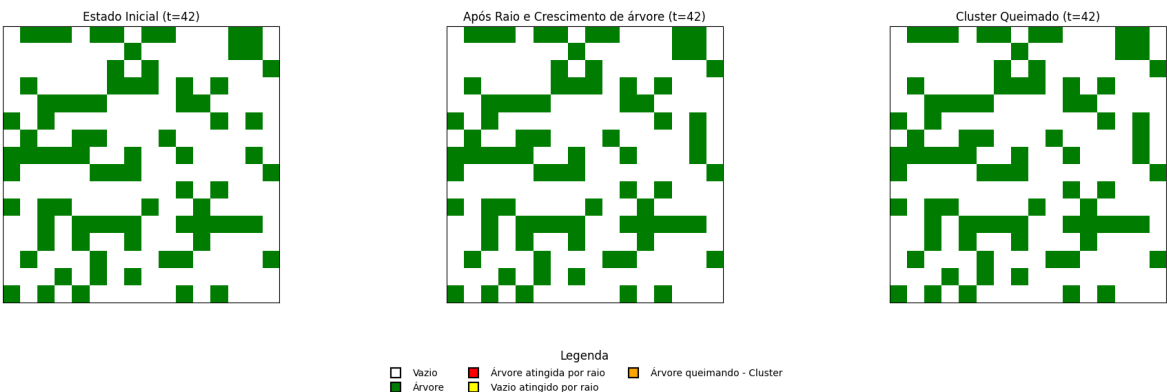
Floresta



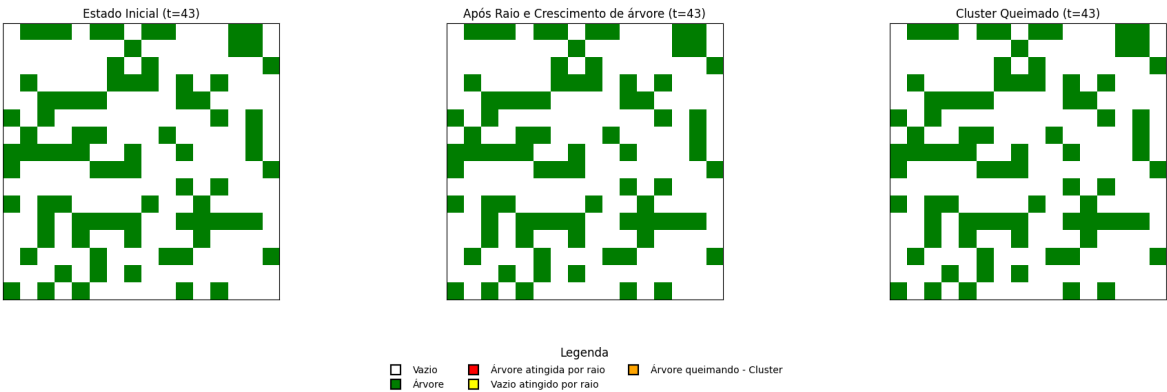
Floresta



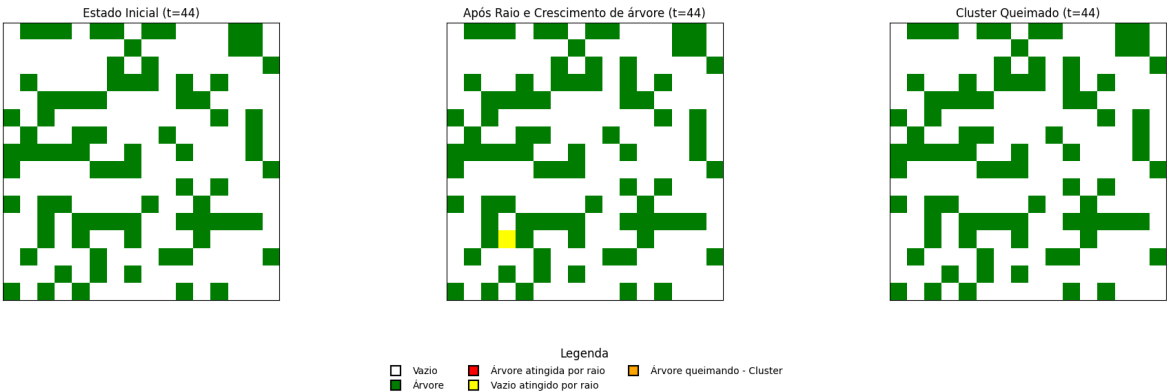
Floresta



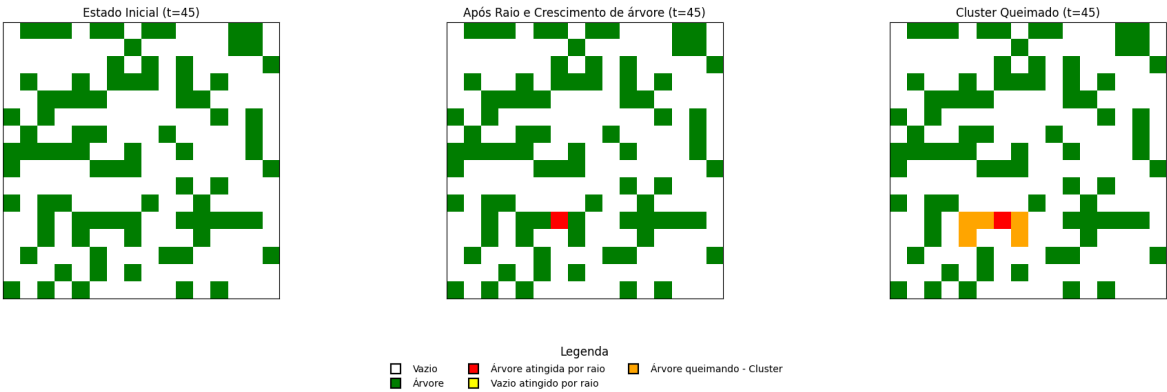
Floresta

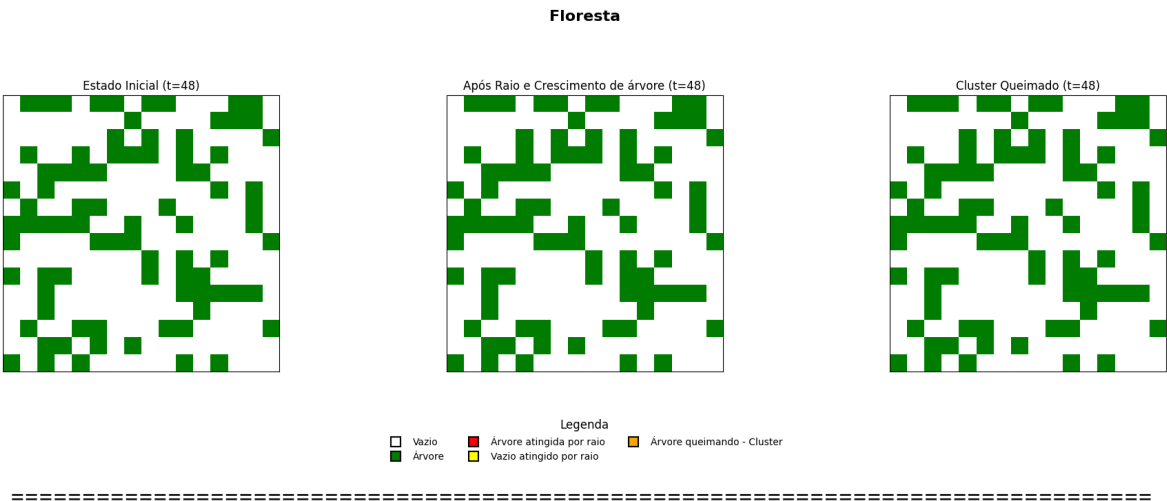
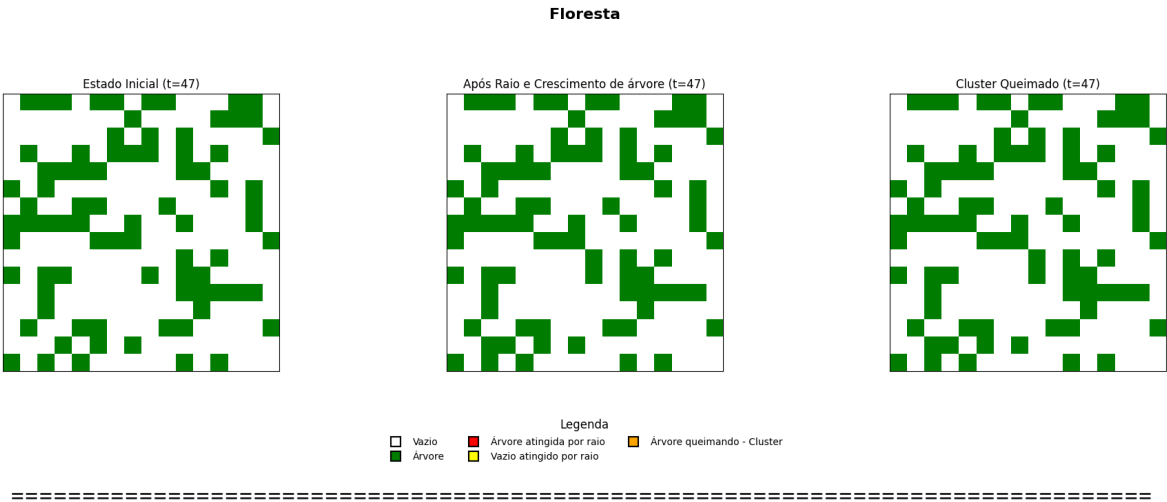
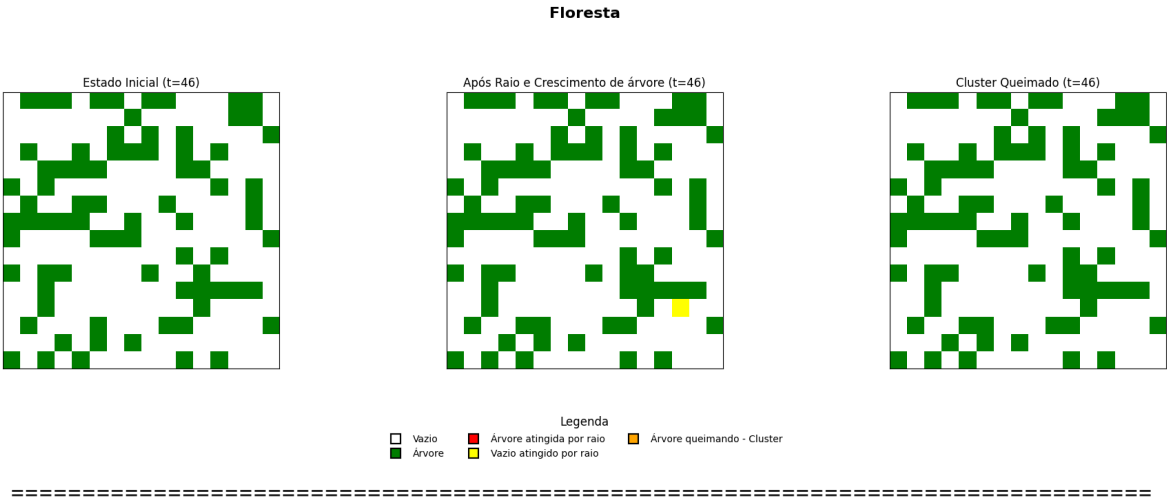


Floresta

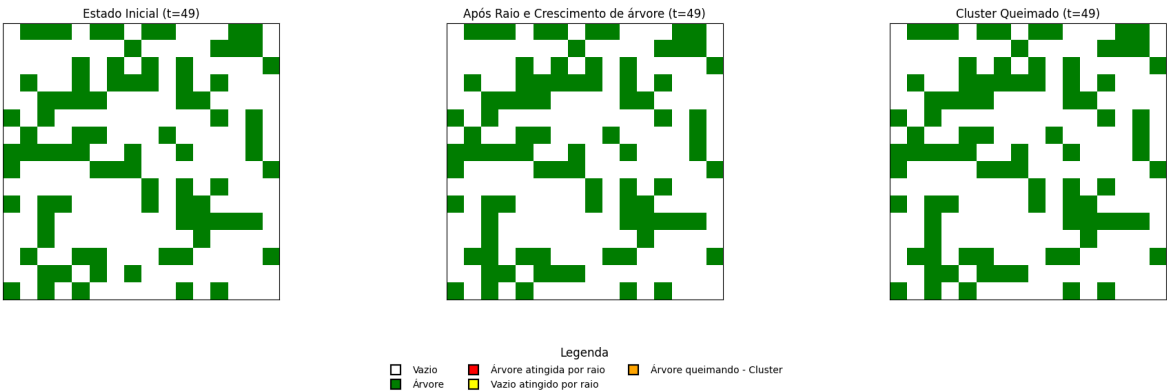


Floresta

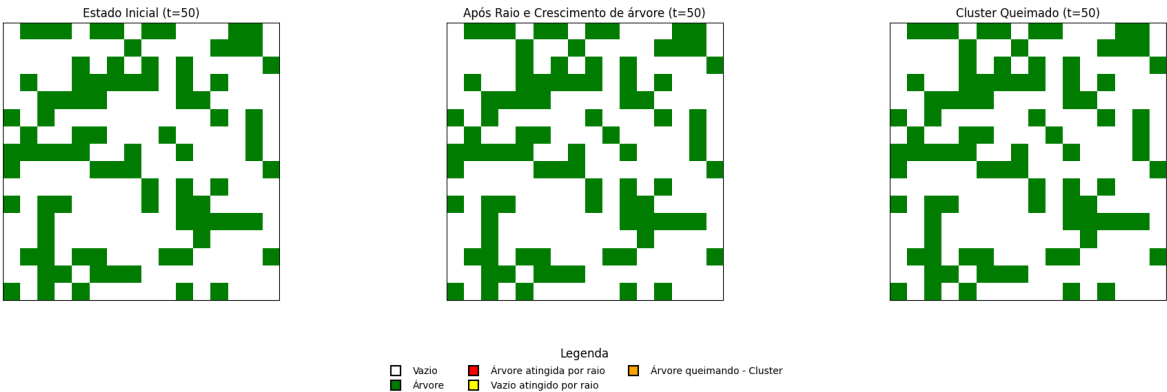




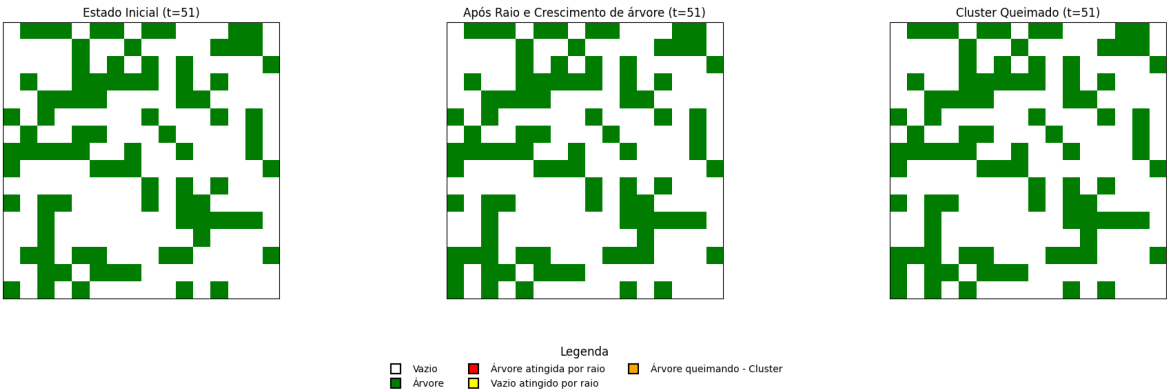
Floresta



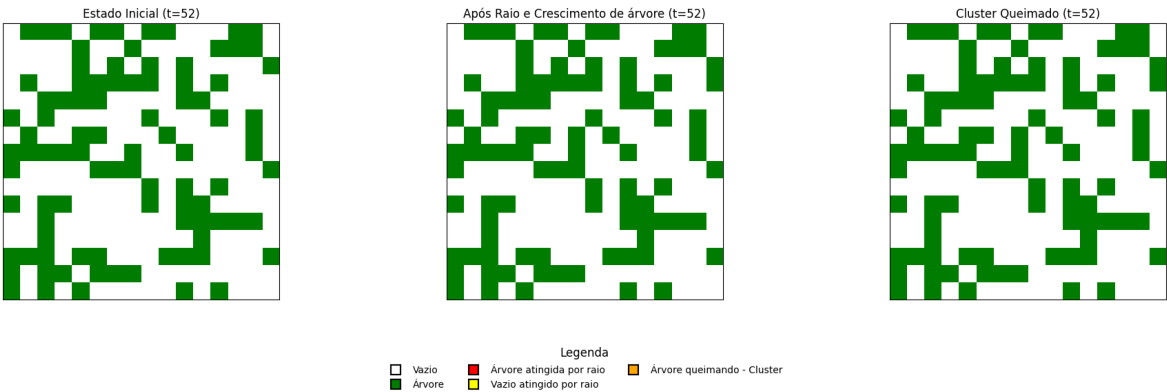
Floresta



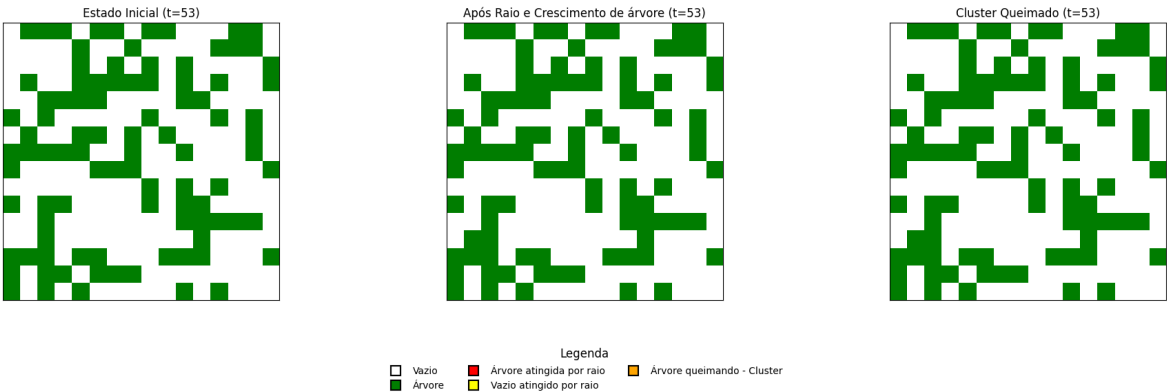
Floresta



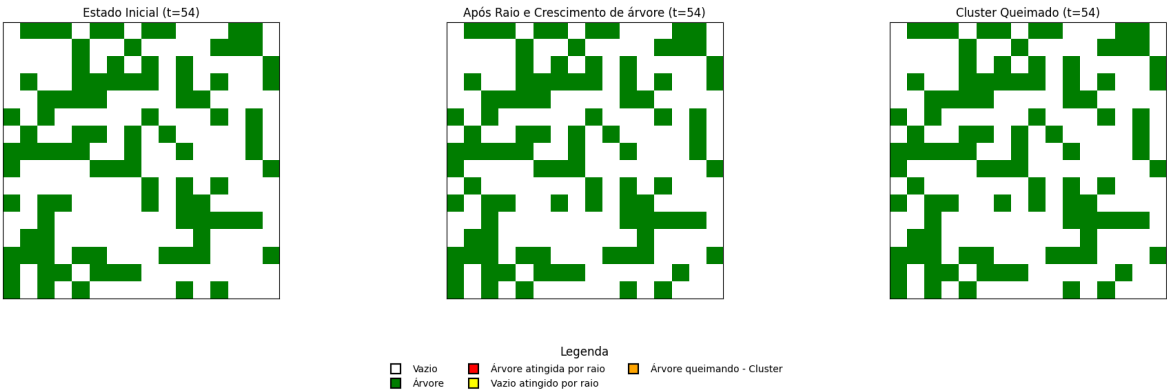
Floresta



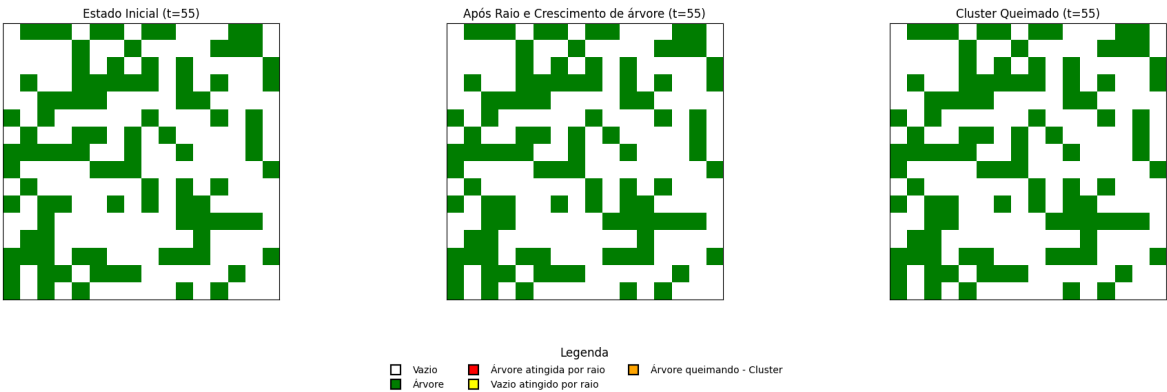
Floresta



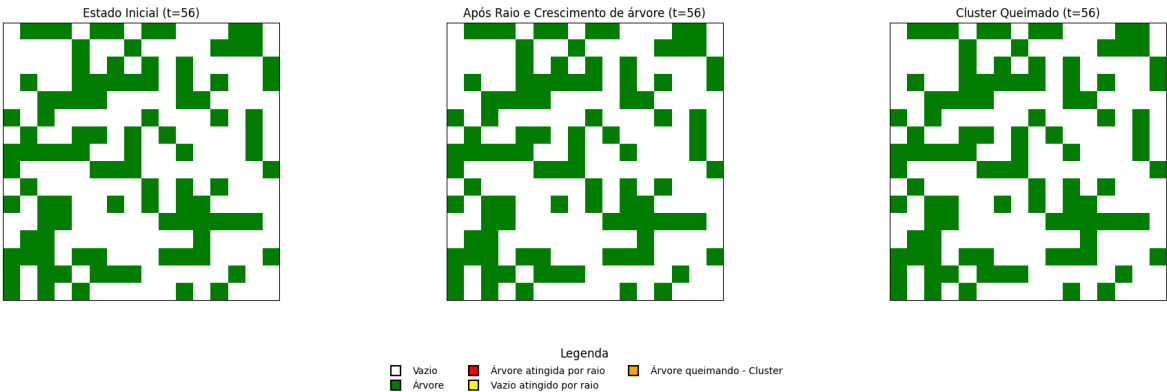
Floresta



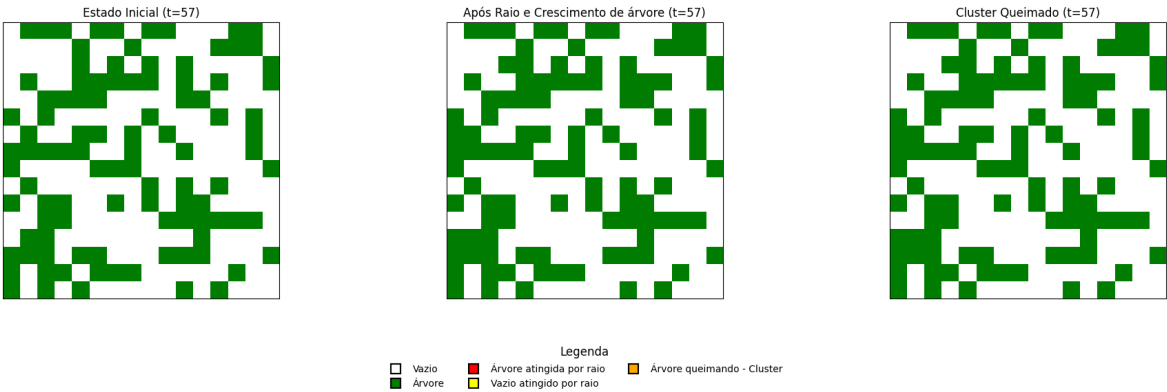
Floresta



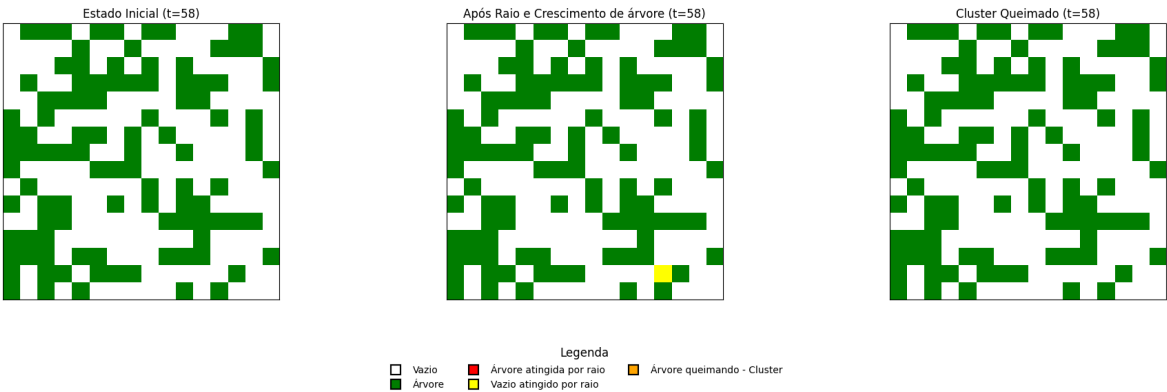
Floresta



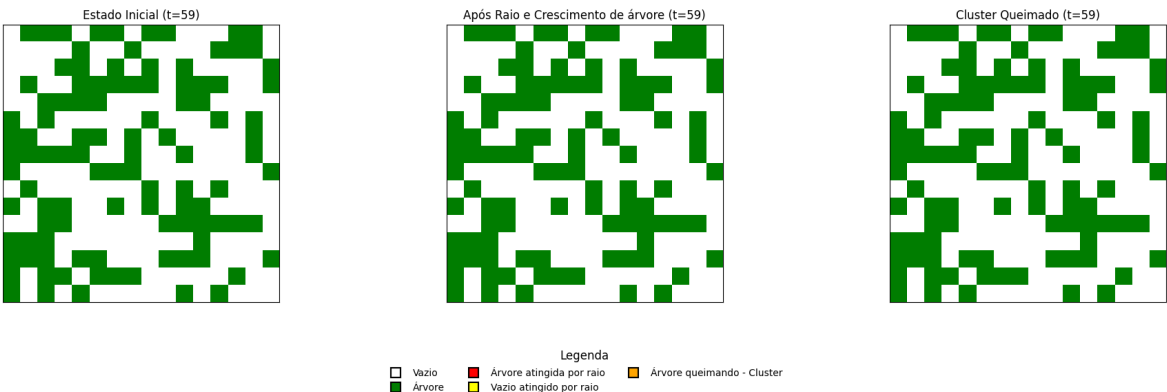
Floresta



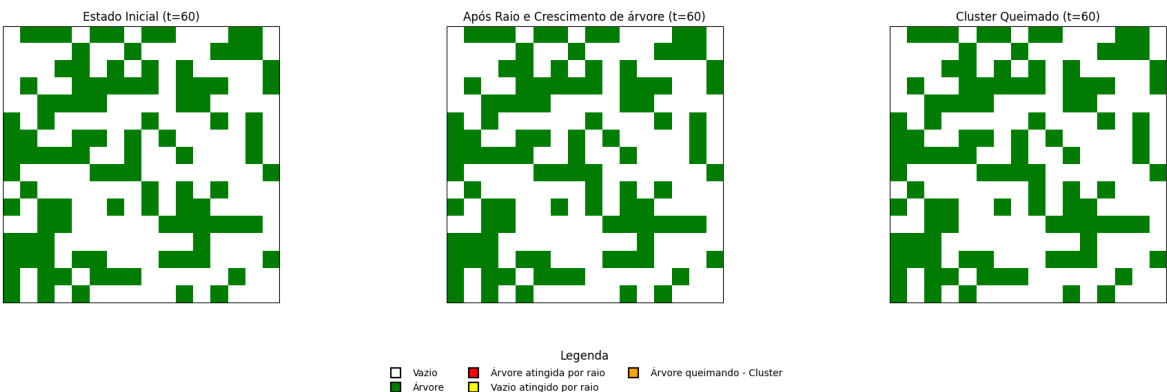
Floresta



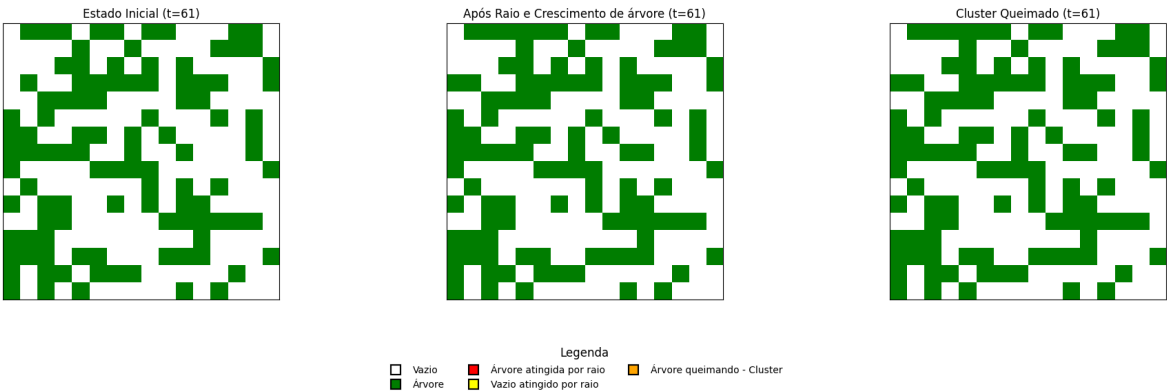
Floresta



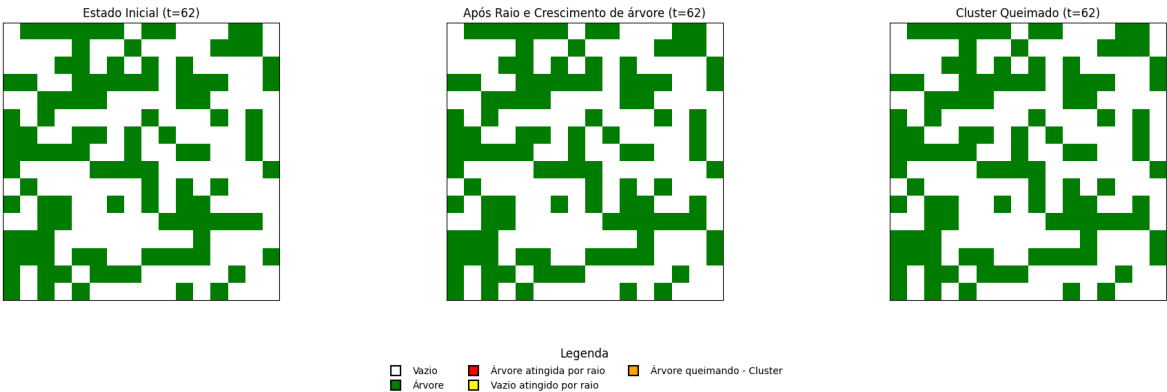
Floresta



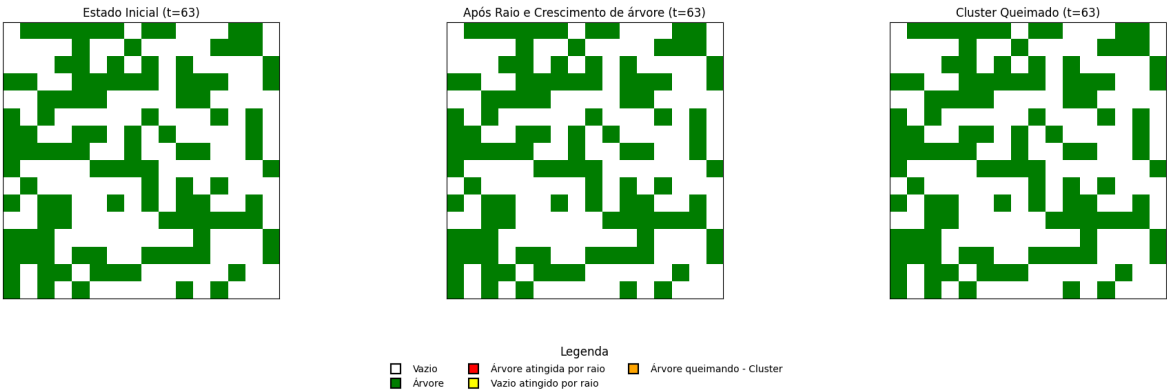
Floresta



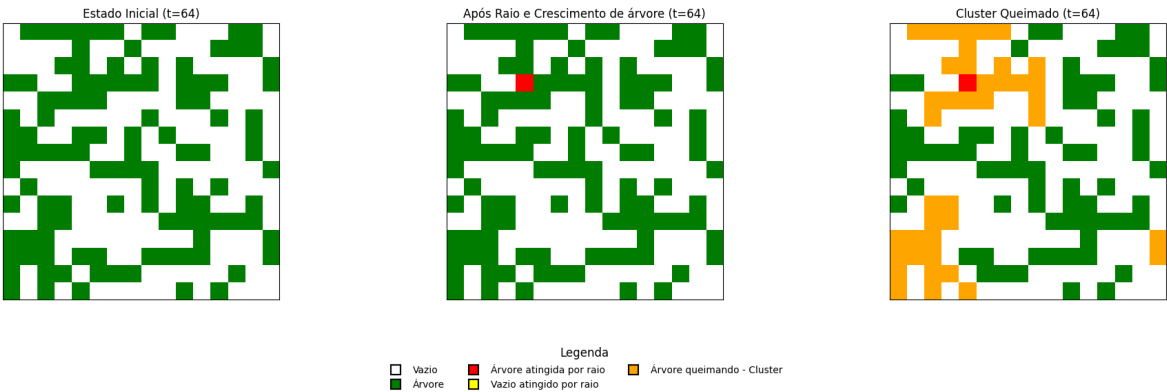
Floresta



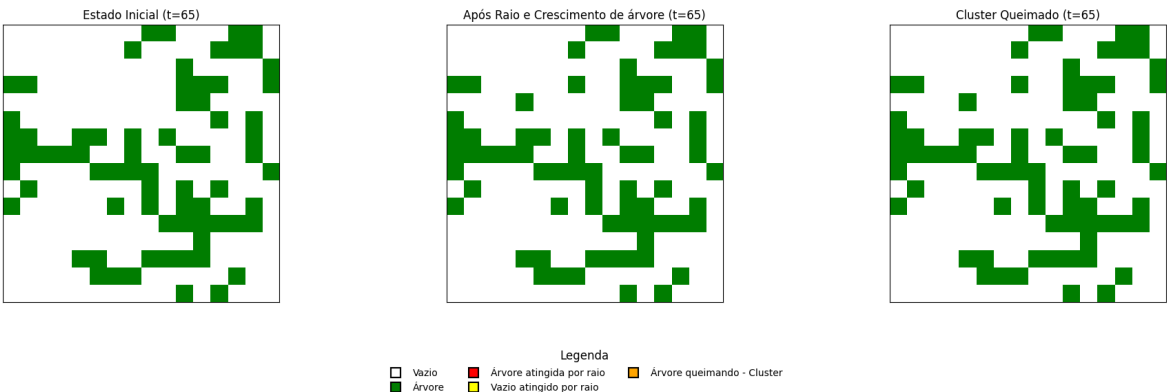
Floresta



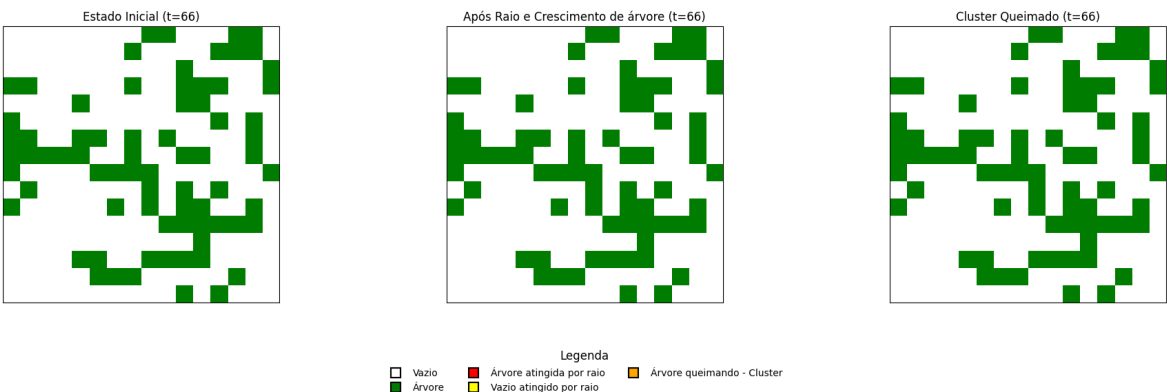
Floresta



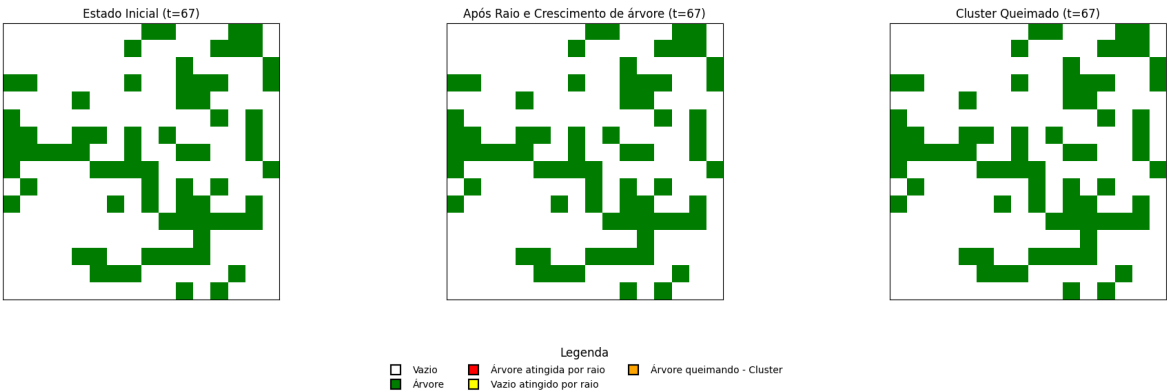
Floresta



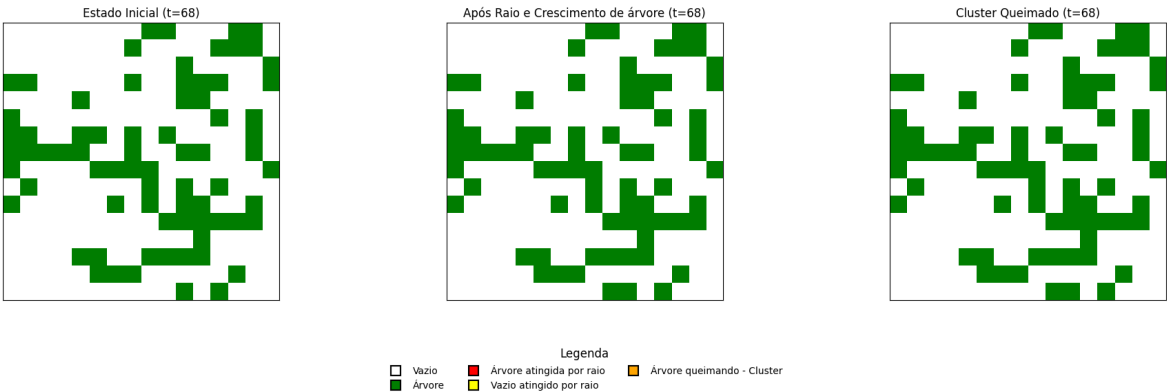
Floresta



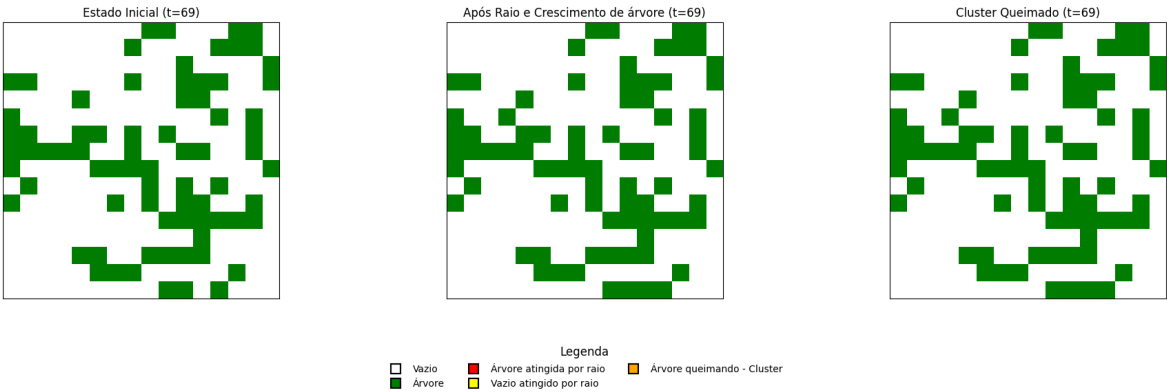
Floresta



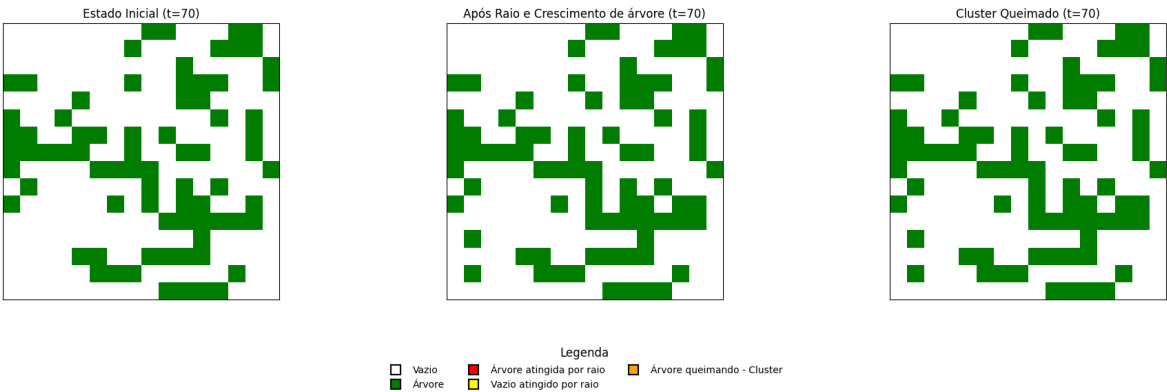
Floresta



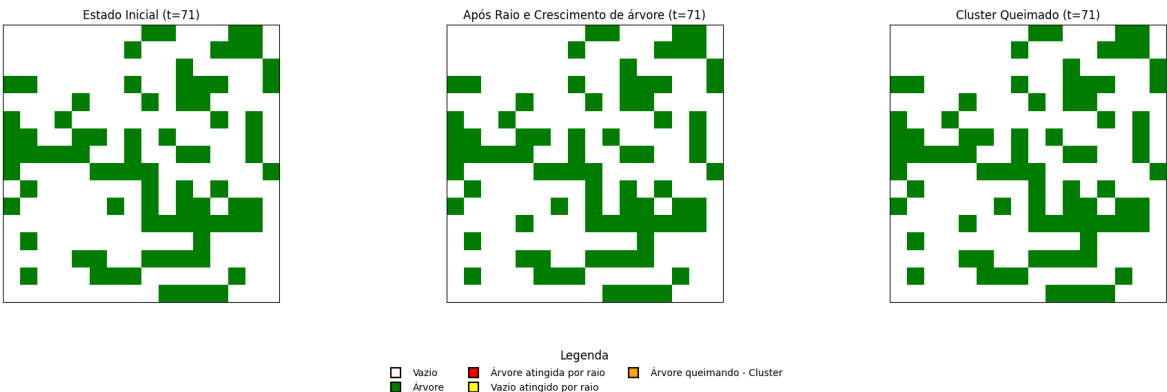
Floresta



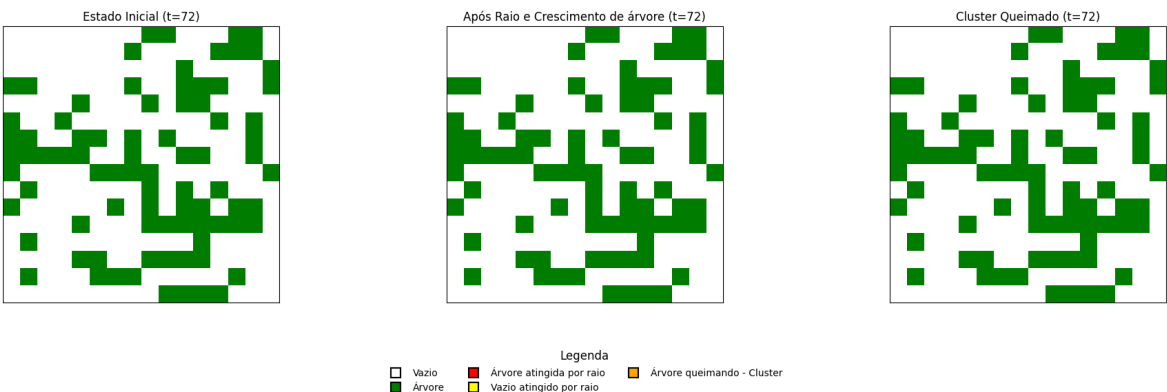
Floresta



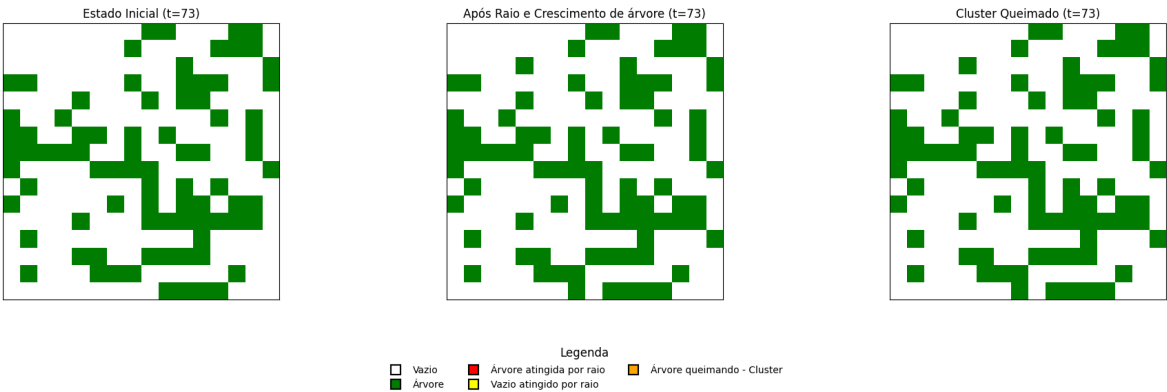
Floresta



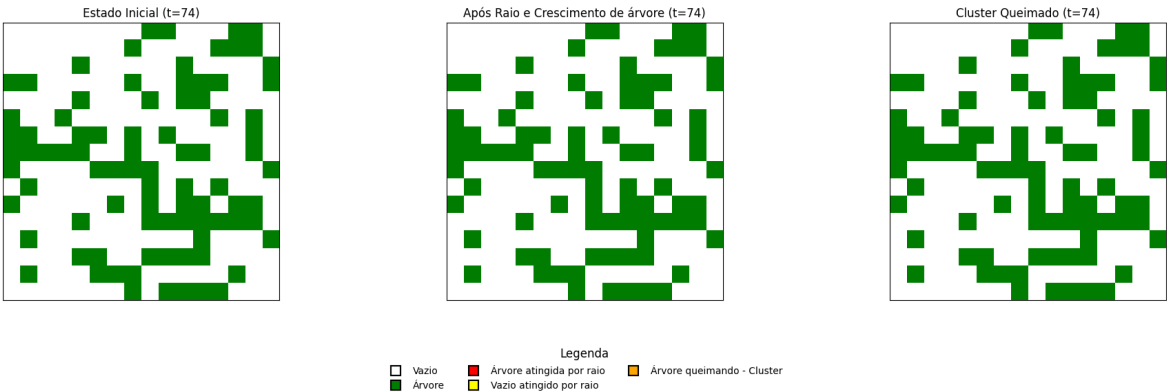
Floresta



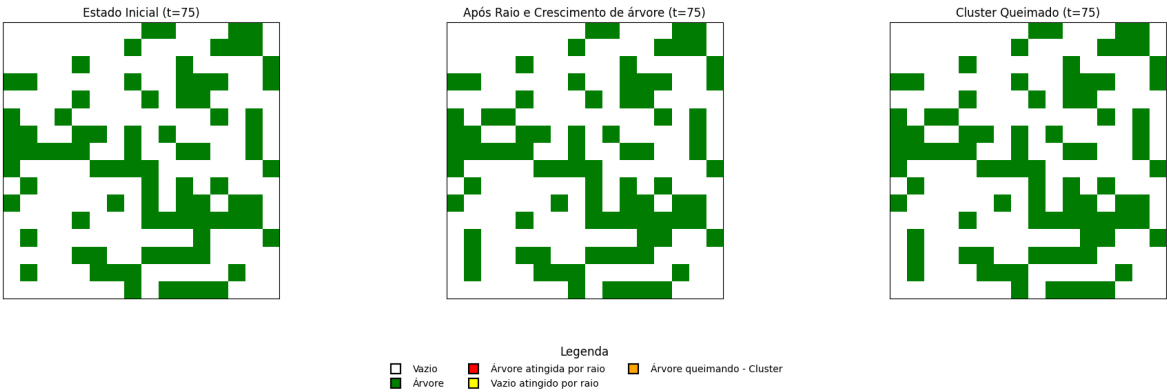
Floresta



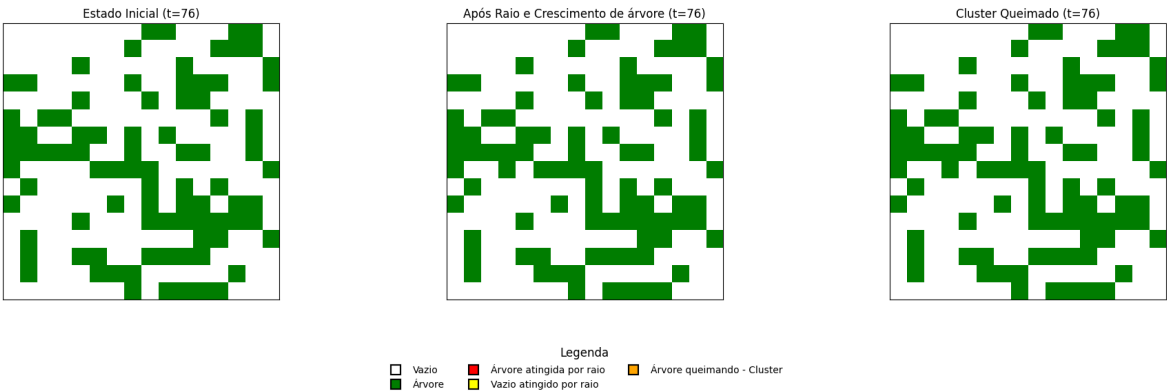
Floresta



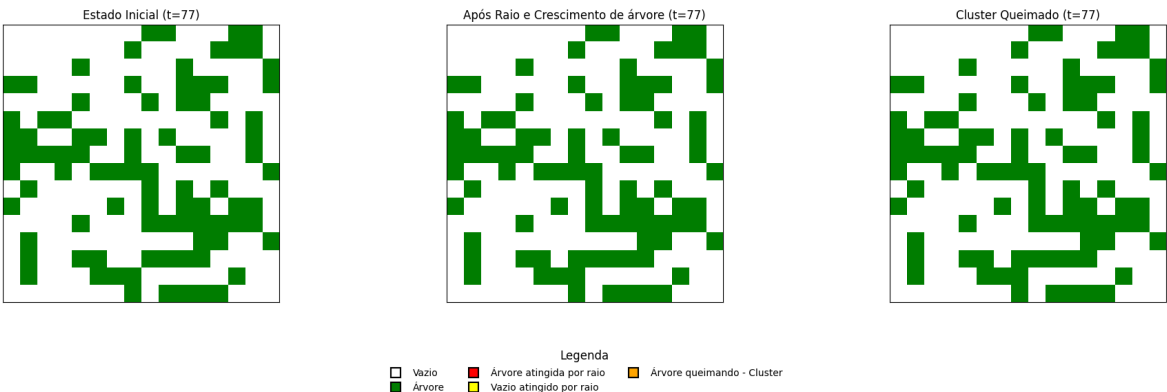
Floresta



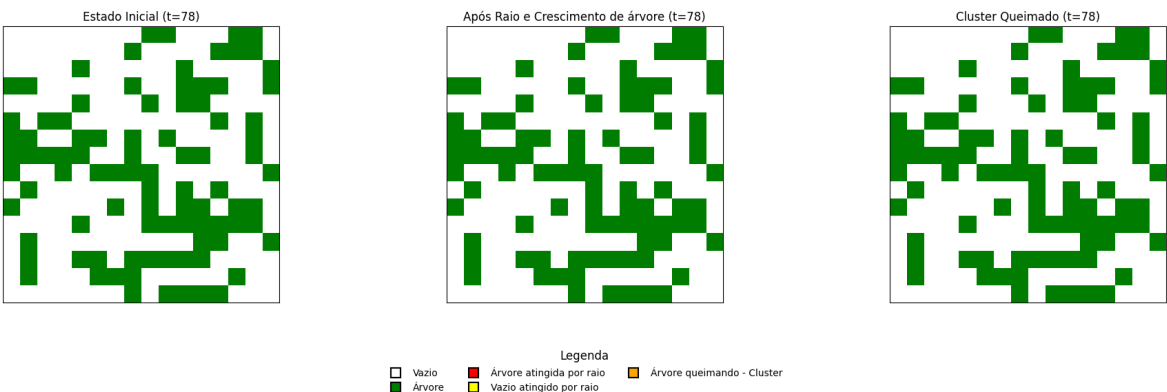
Floresta



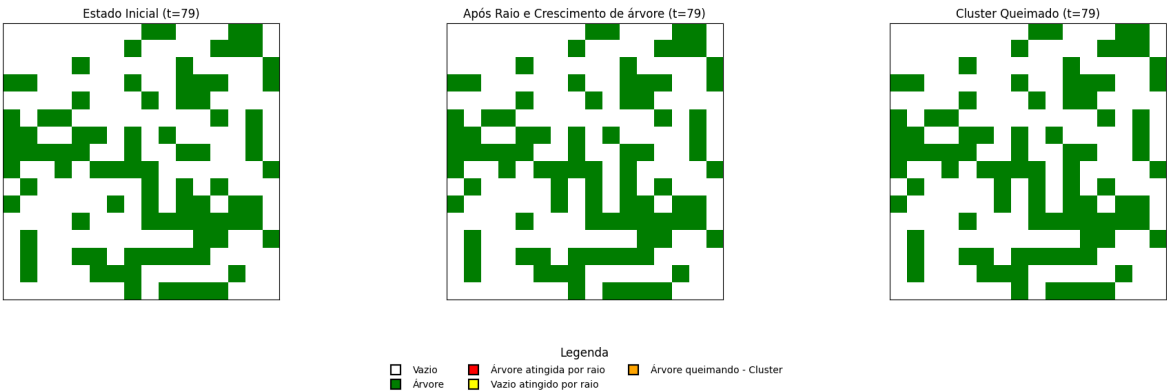
Floresta



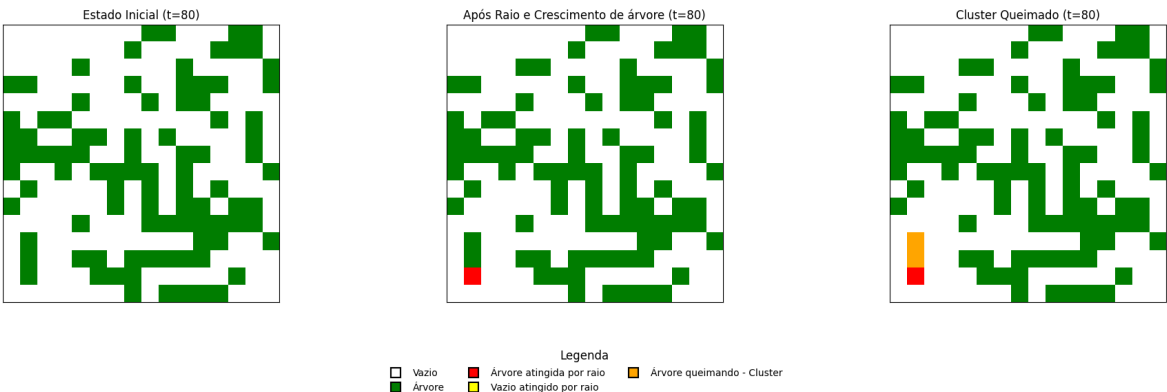
Floresta



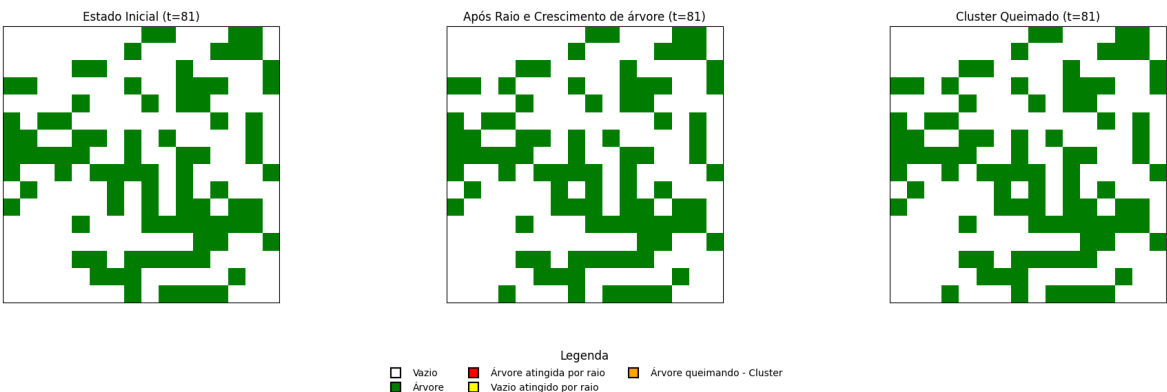
Floresta

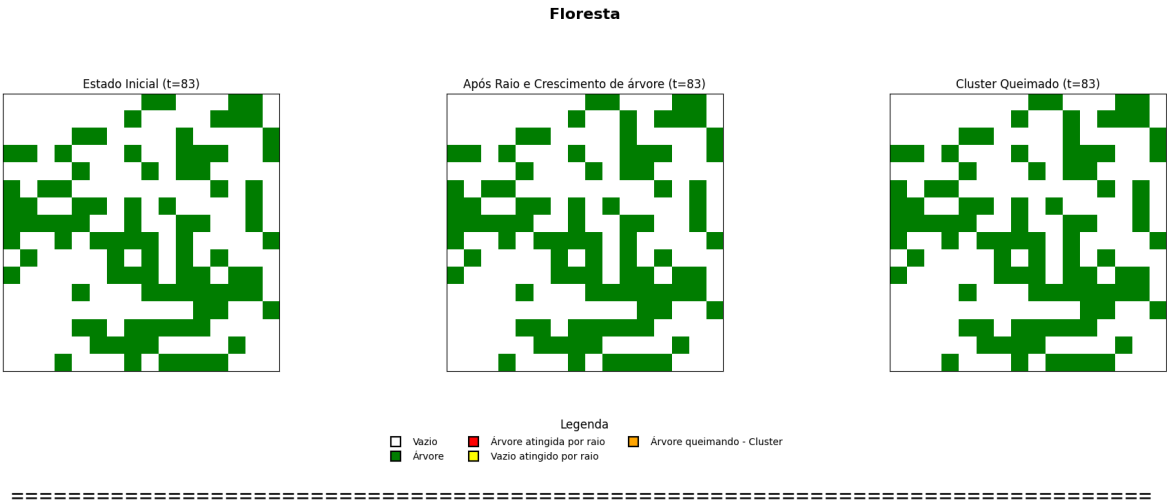
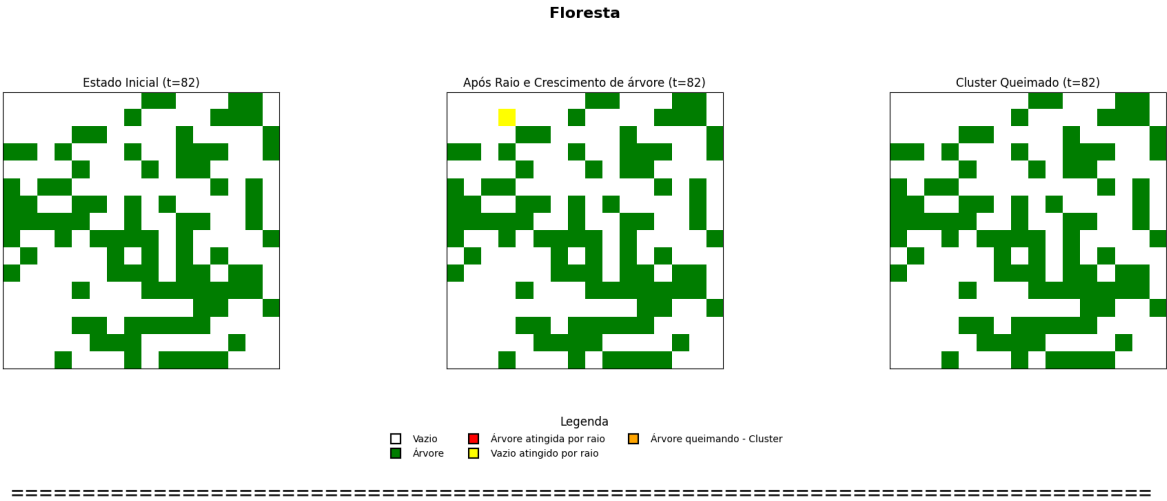


Floresta

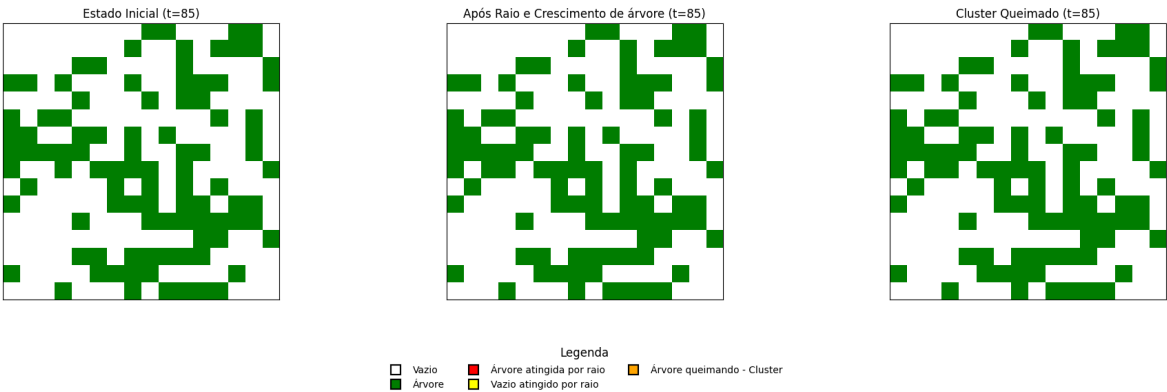


Floresta

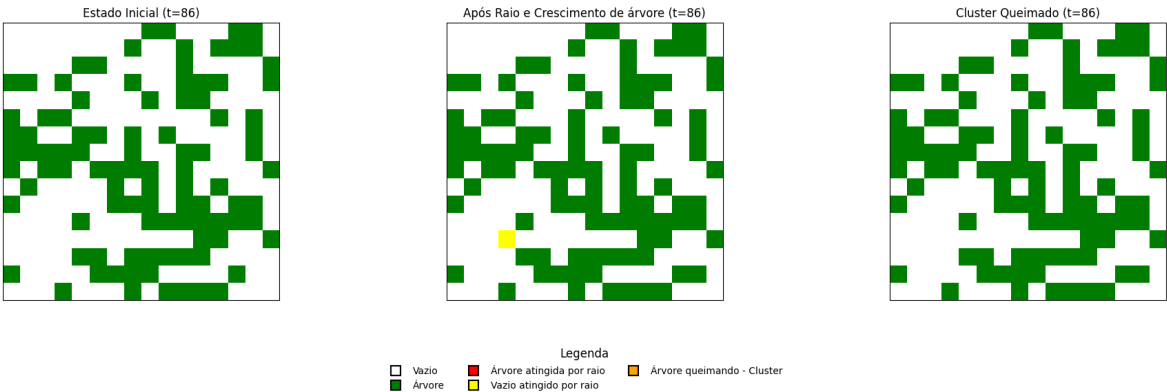




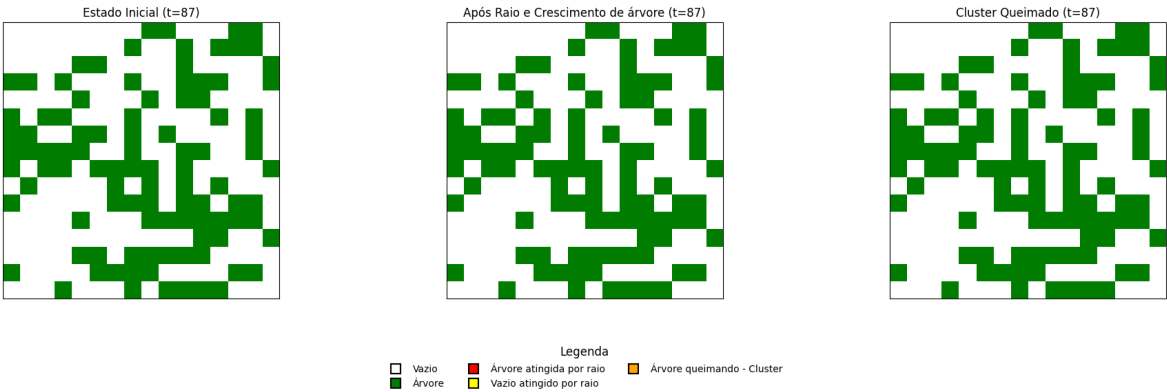
Floresta



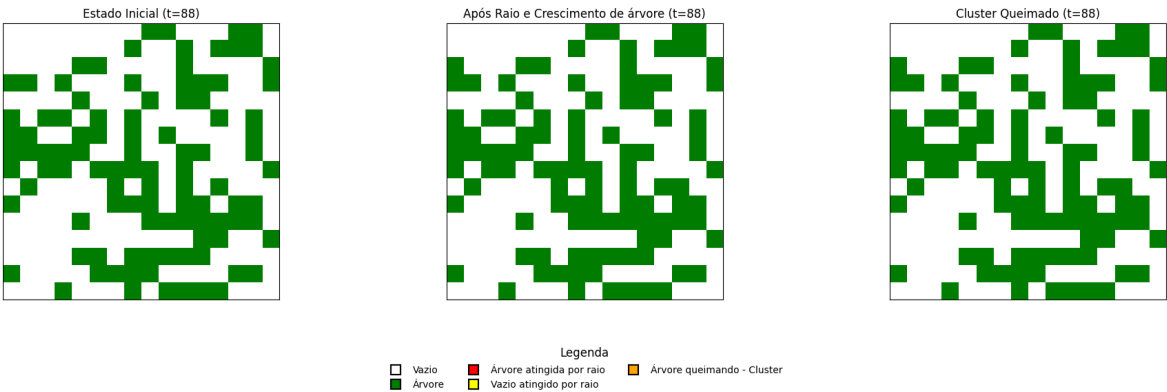
Floresta



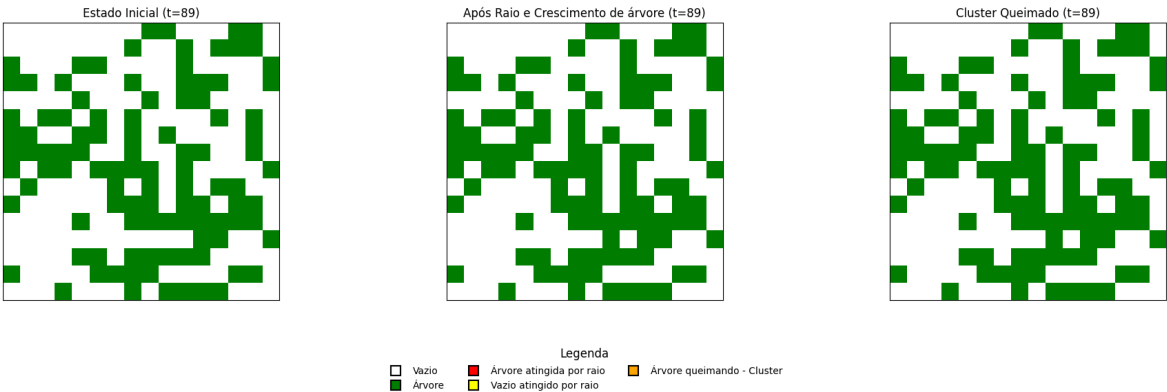
Floresta



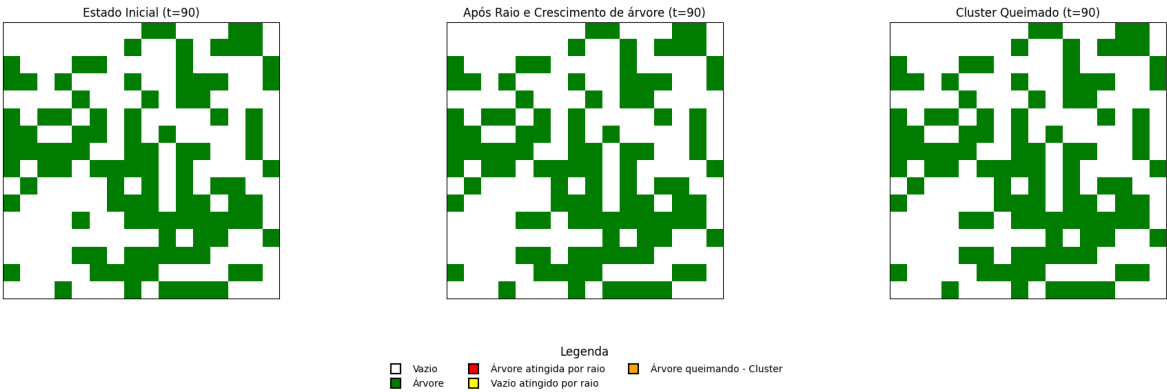
Floresta



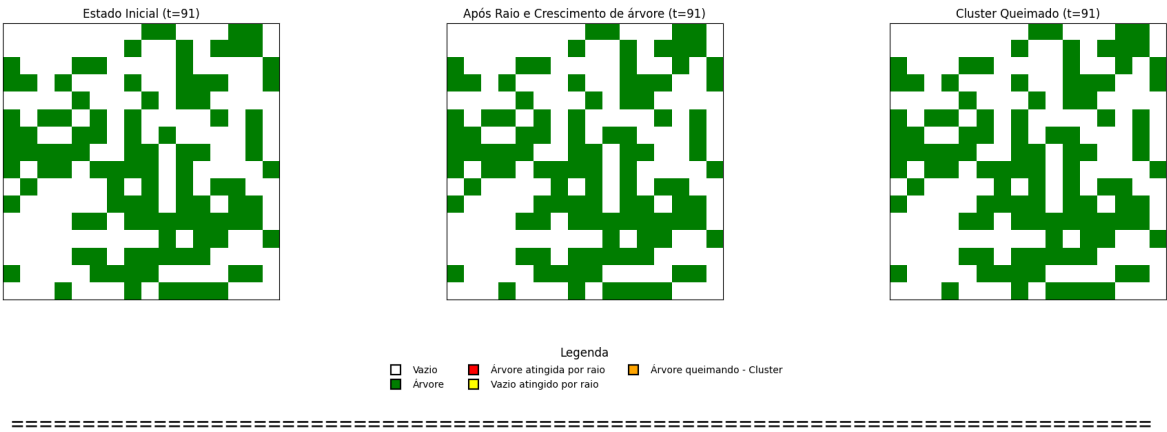
Floresta



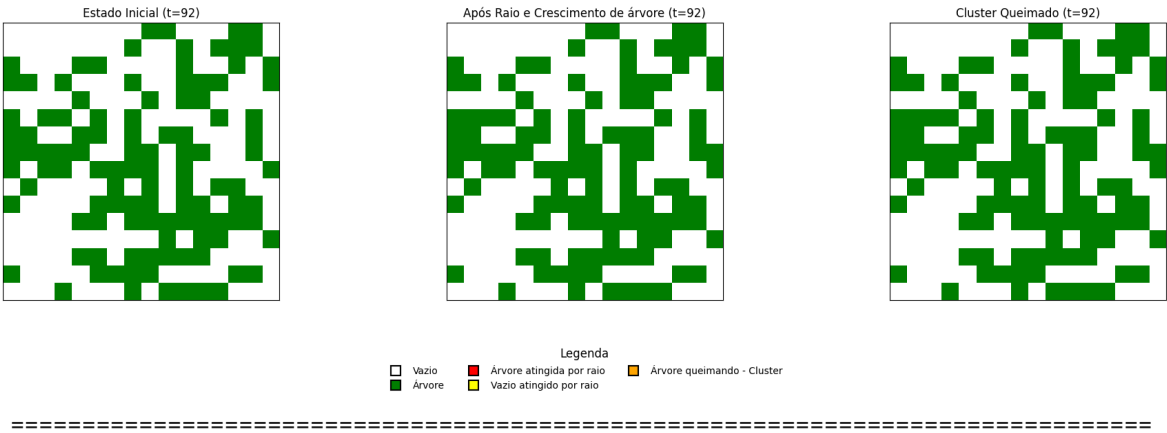
Floresta



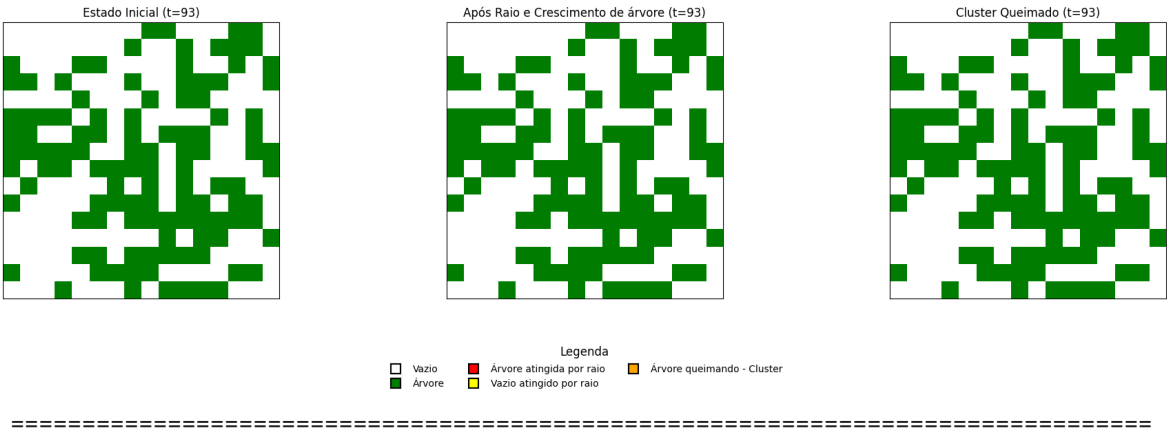
Floresta



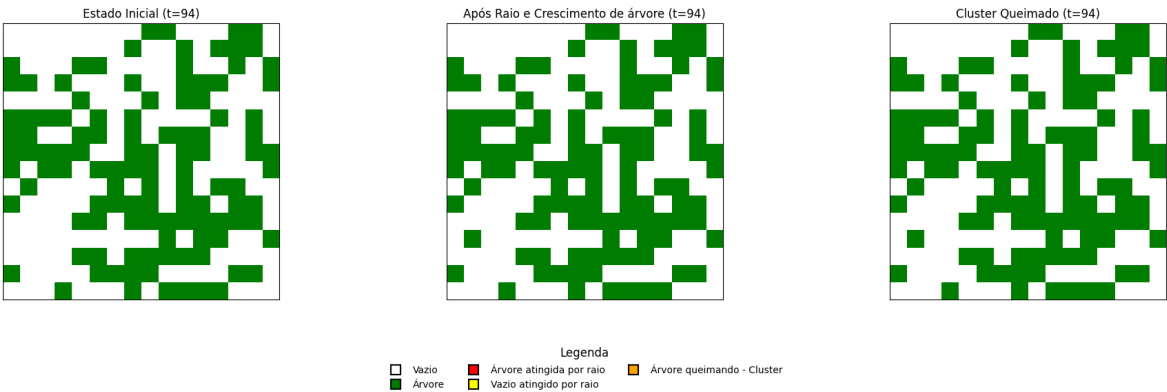
Floresta



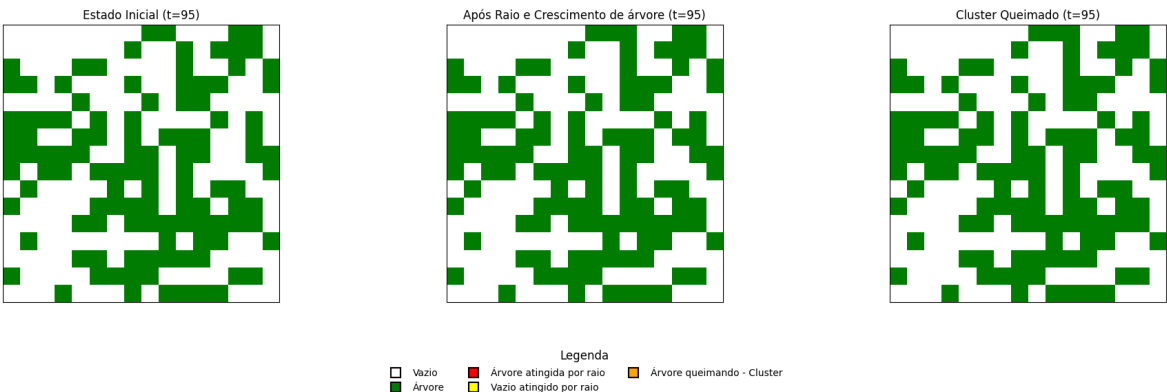
Floresta



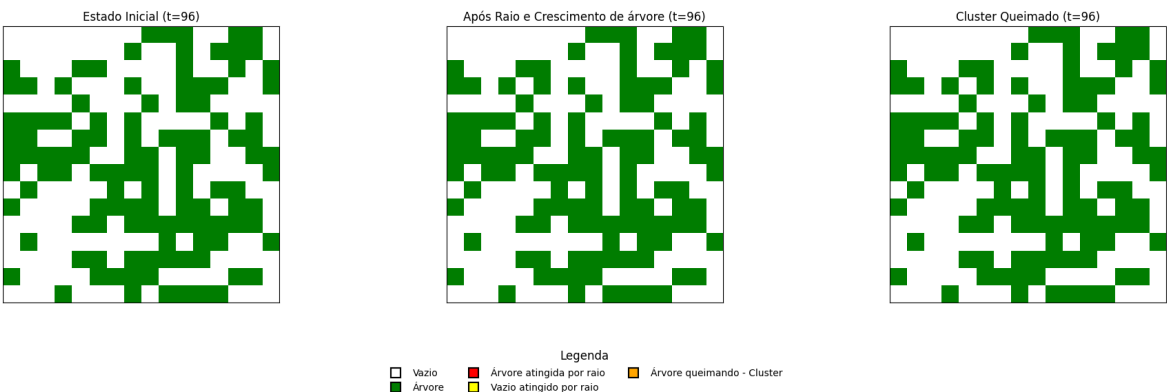
Floresta



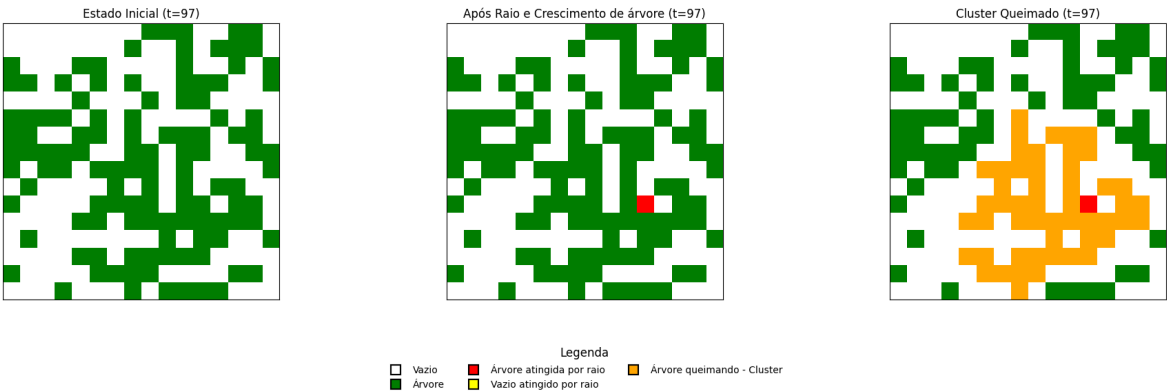
Floresta



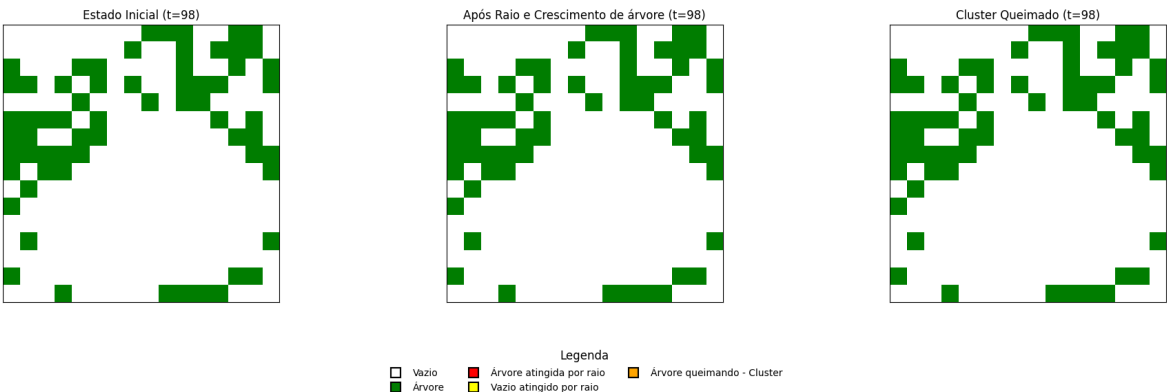
Floresta



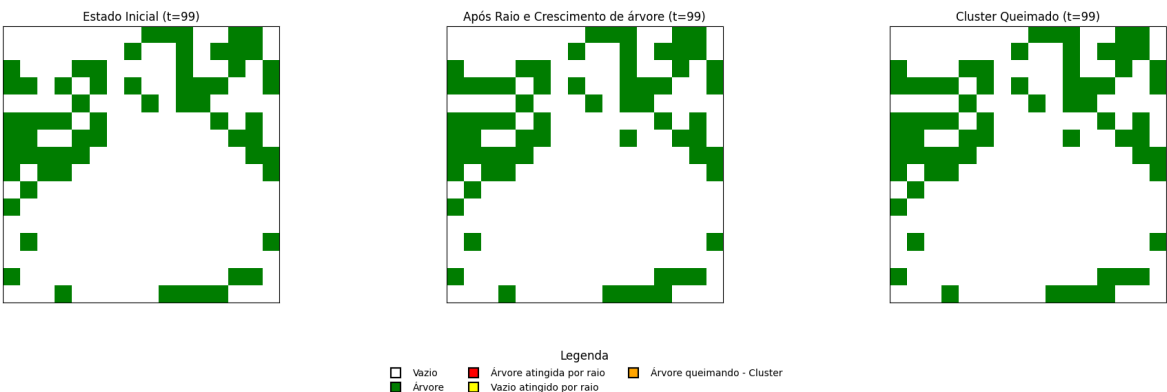
Floresta

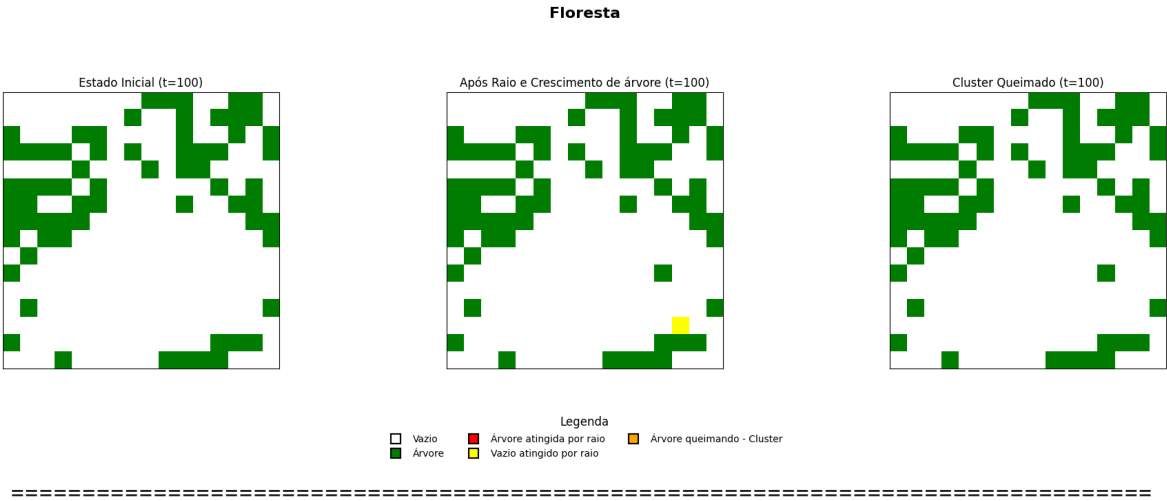


Floresta



Floresta





b.

Incêndios muito pequenos - Baixo p e alto f

$p = 0.005$

$f = 0.6$

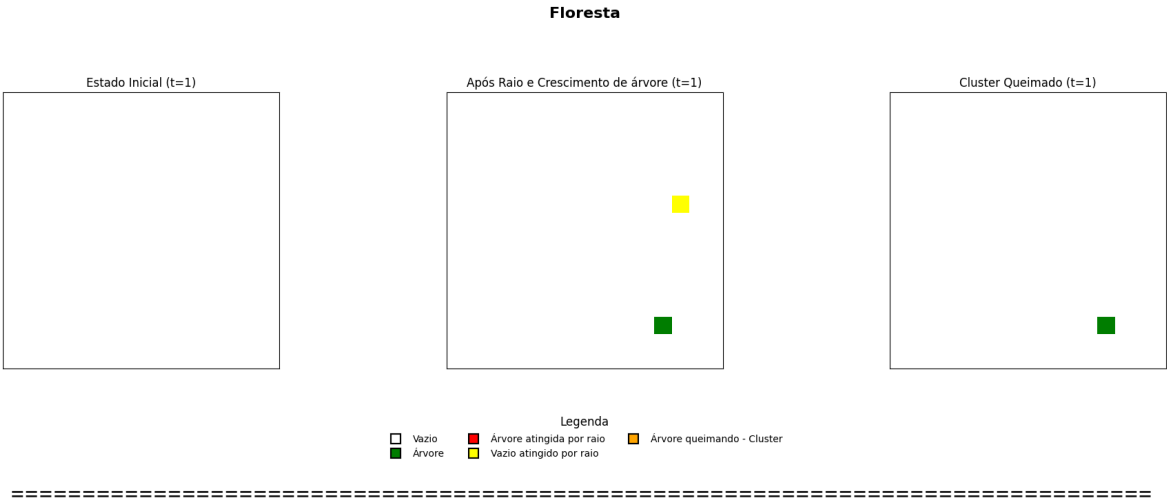
In [131...

$p = 0.005$

$f = 0.6$

In [132...

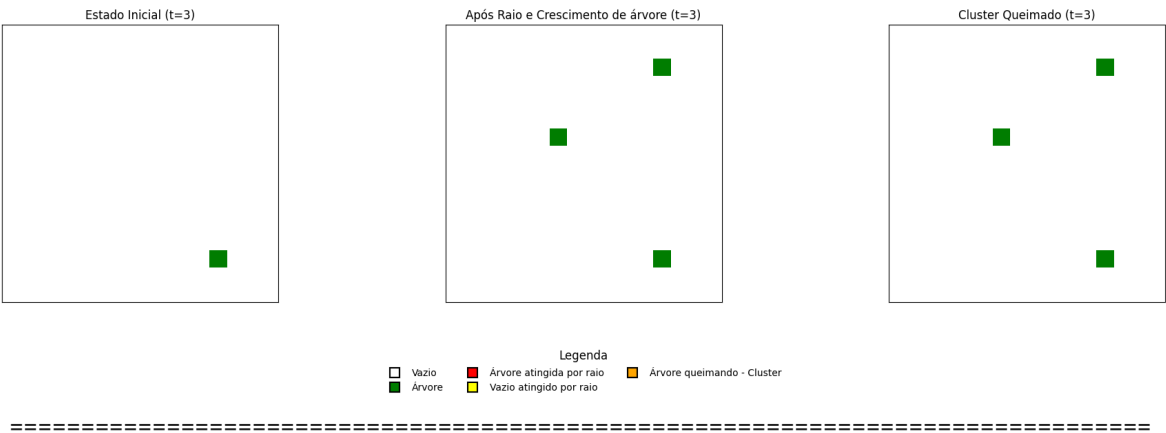
```
_ = simula_floresta(L, p, f, iteracoes, visualizar)
```



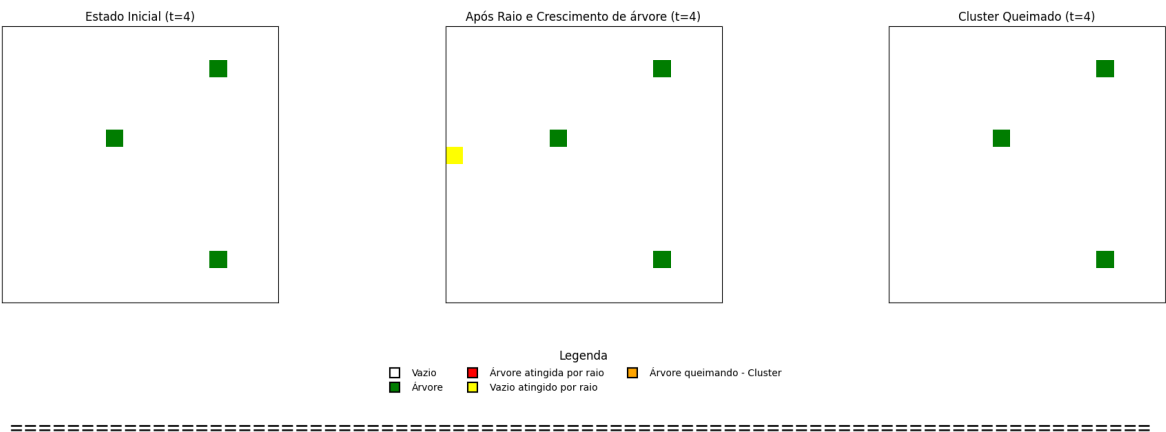
Floresta

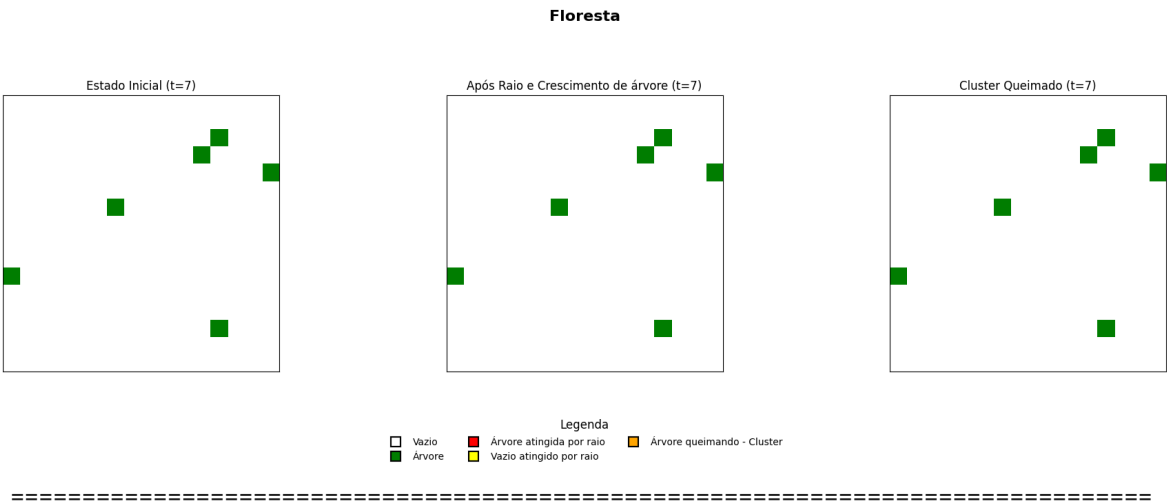
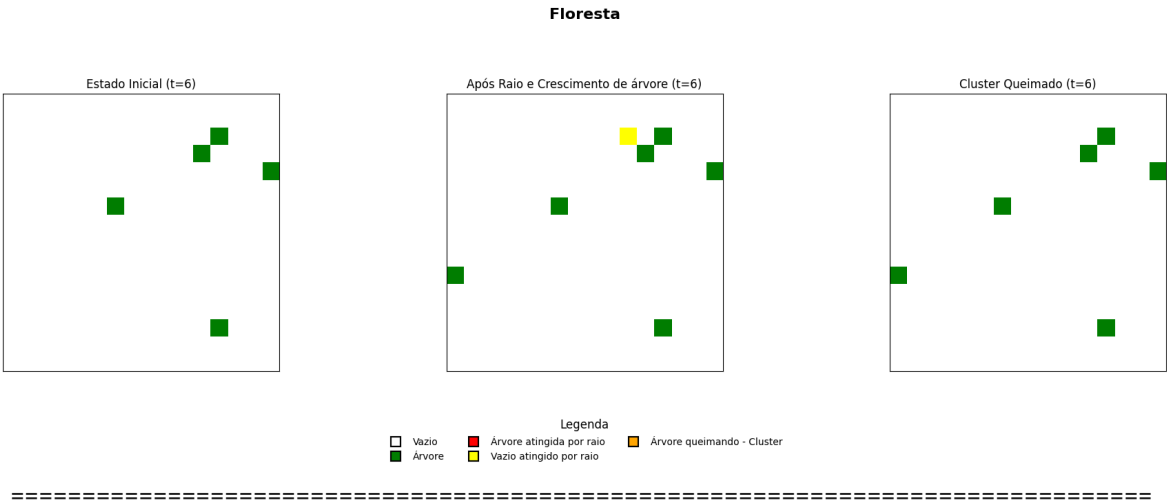
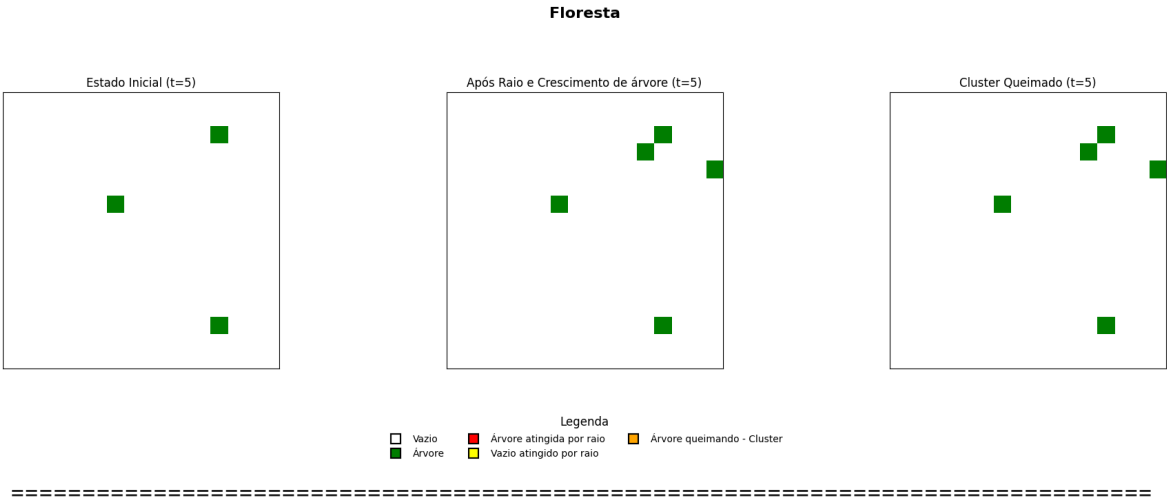


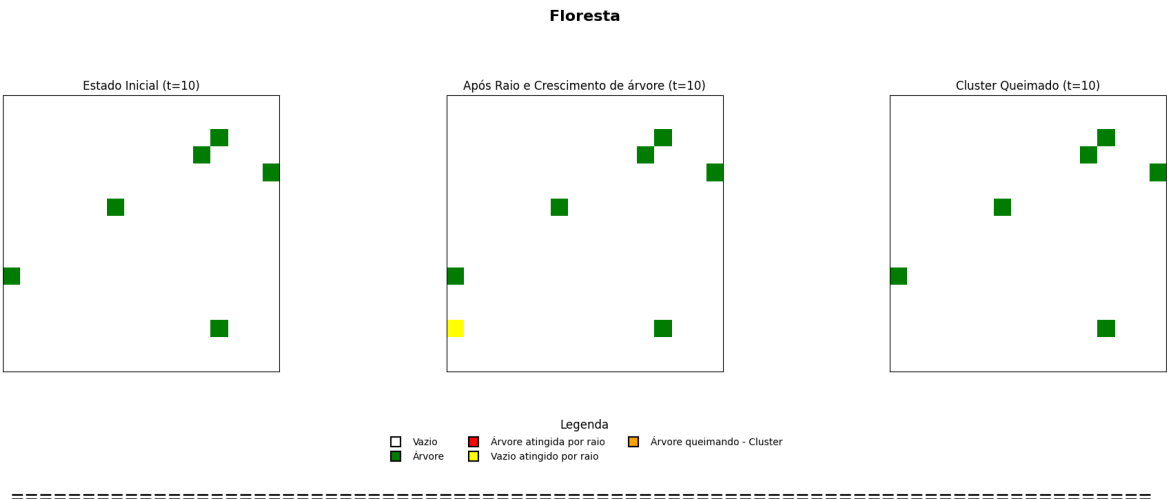
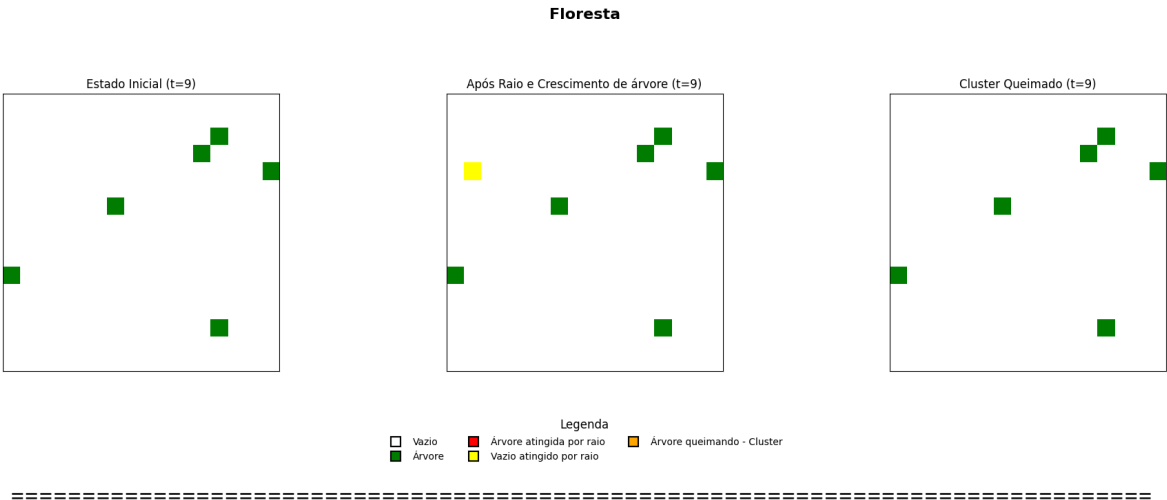
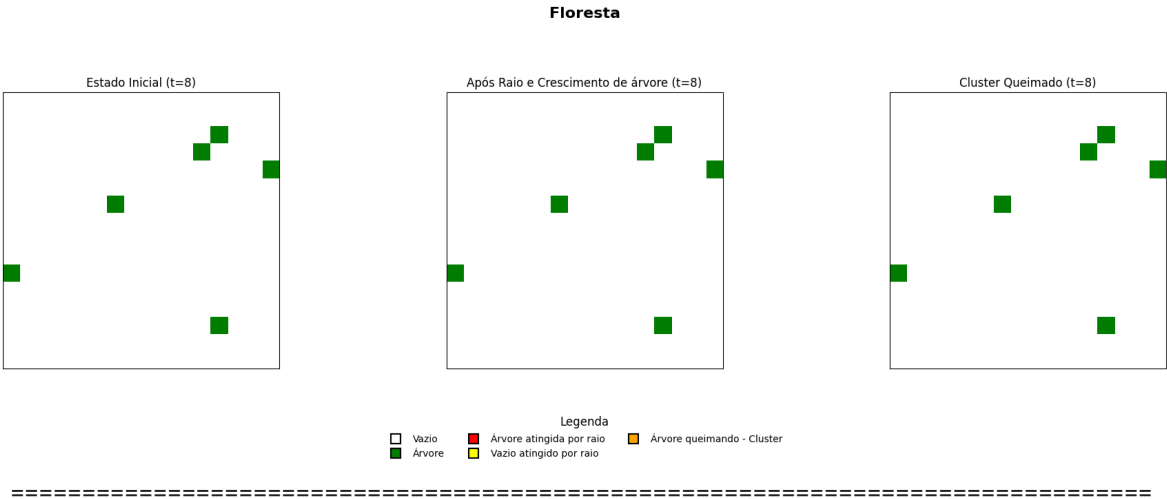
Floresta

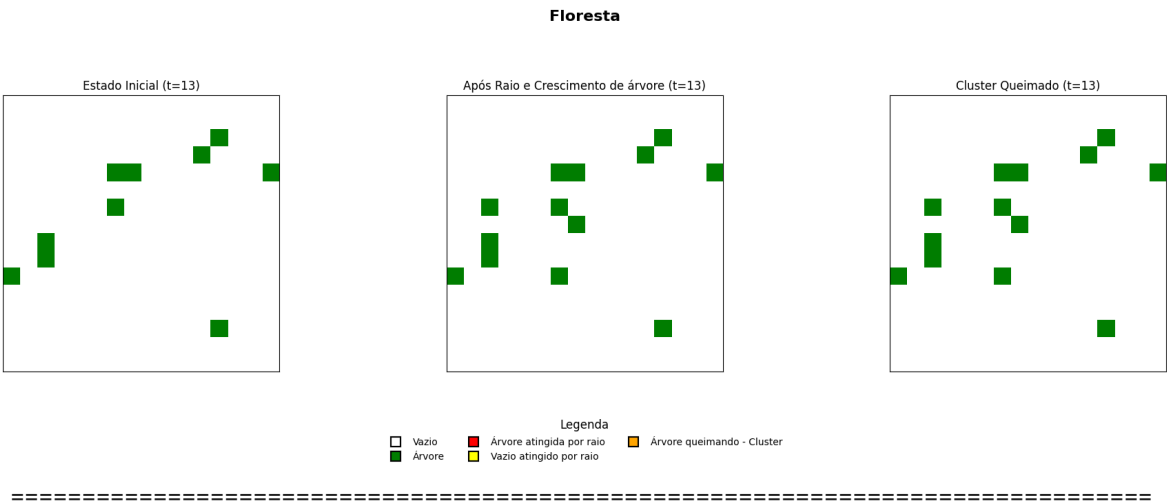
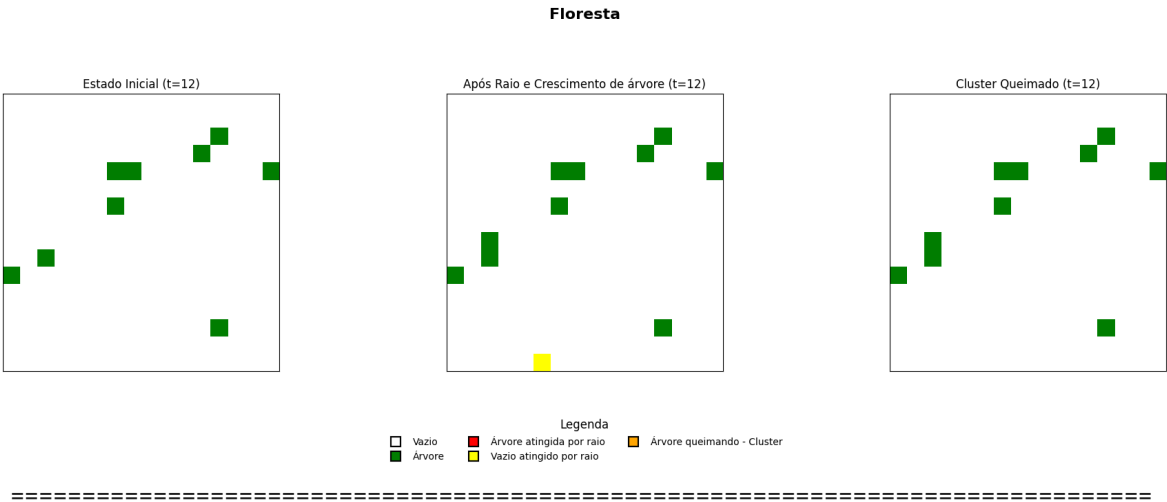
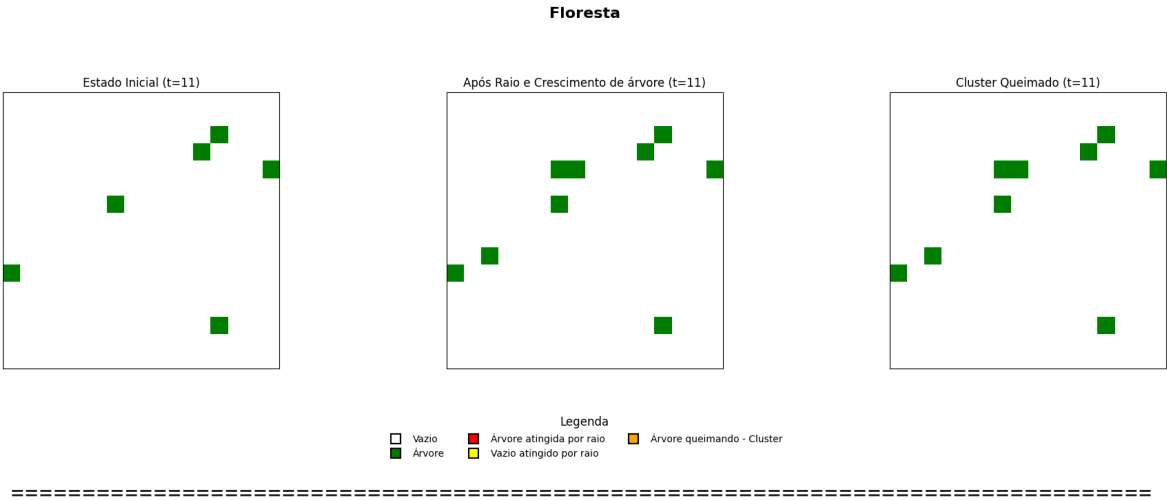


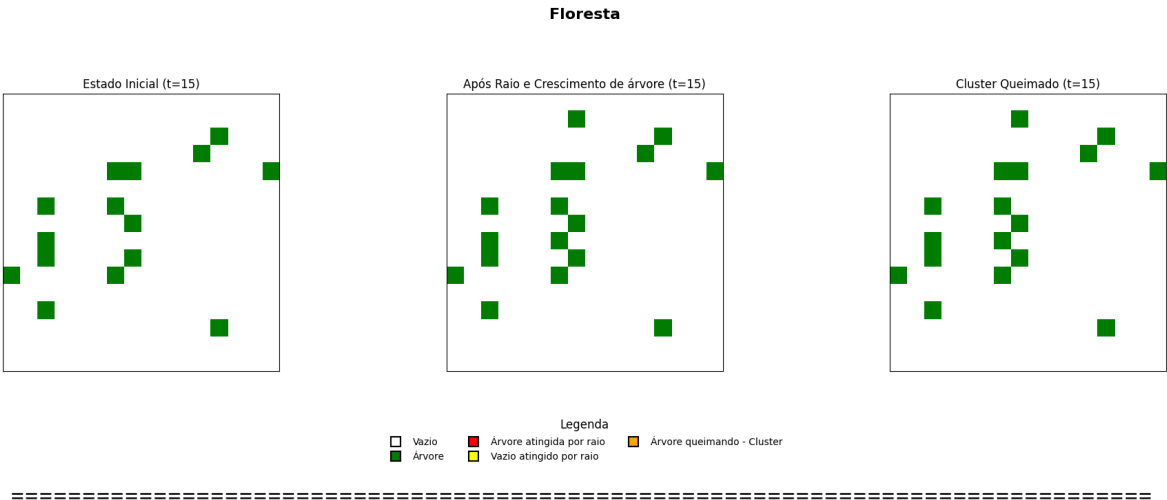
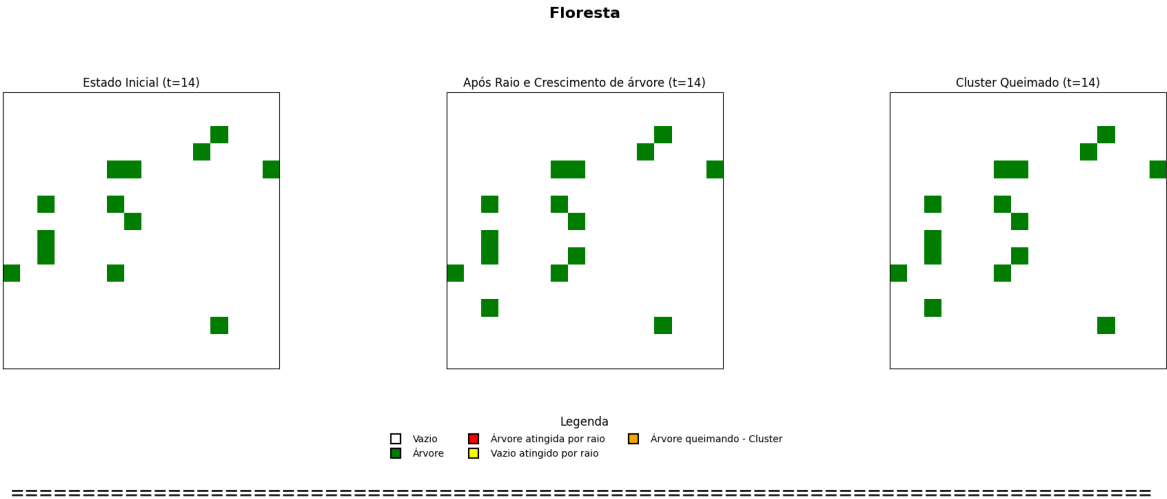
Floresta

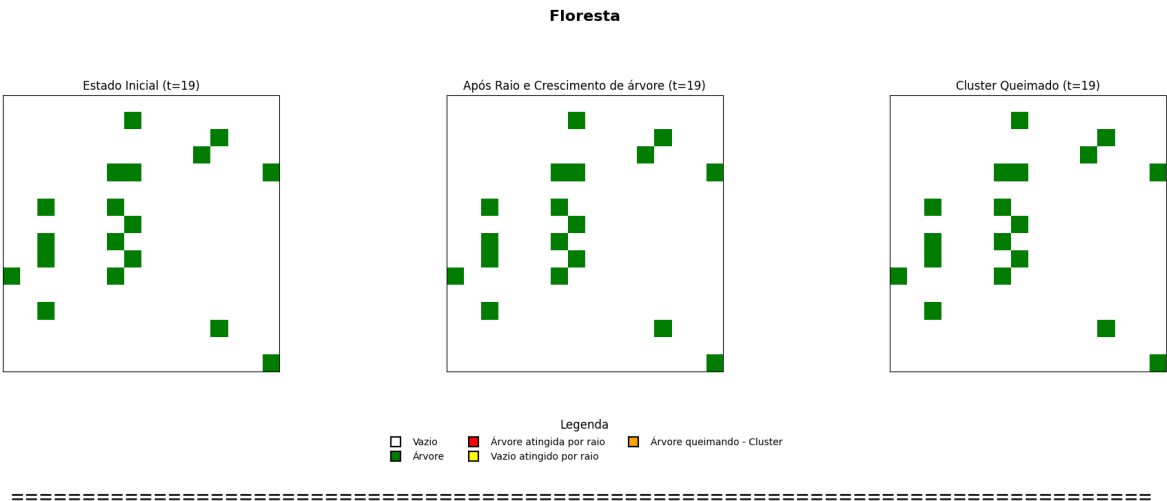


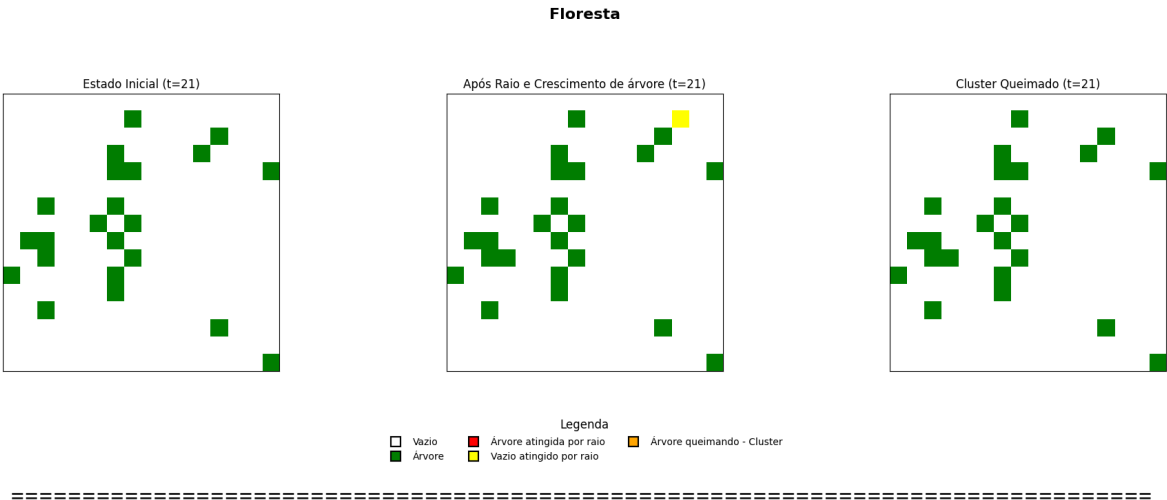
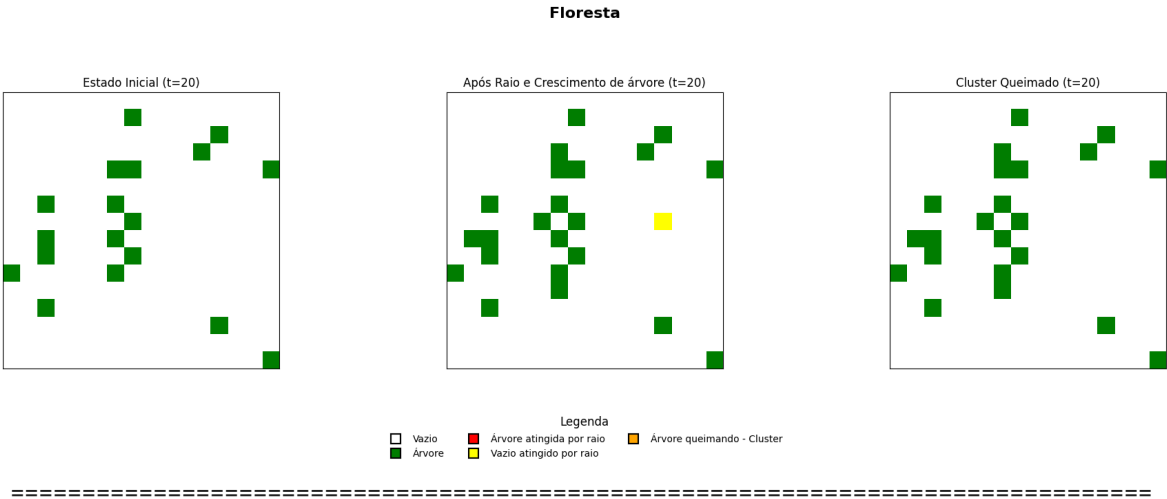


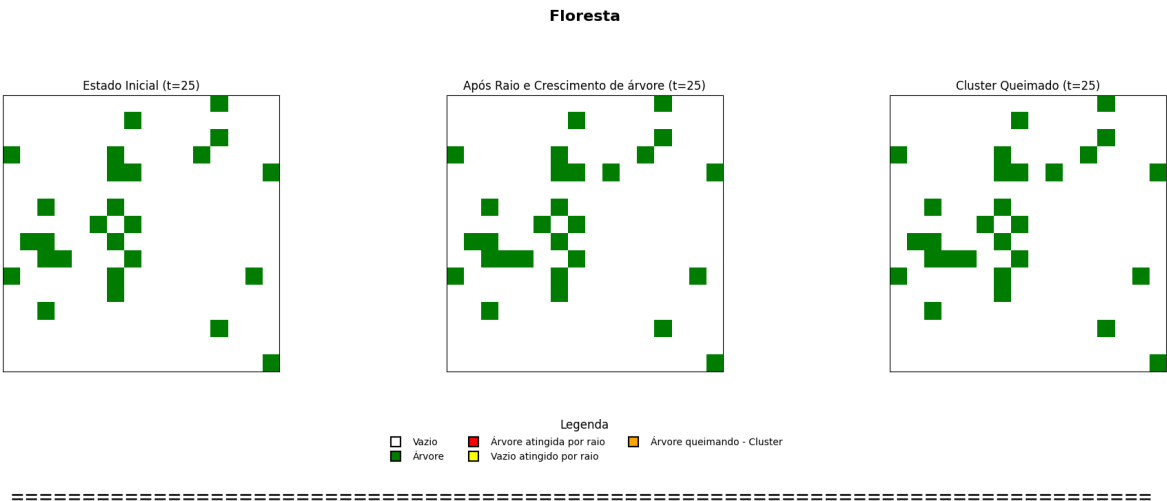
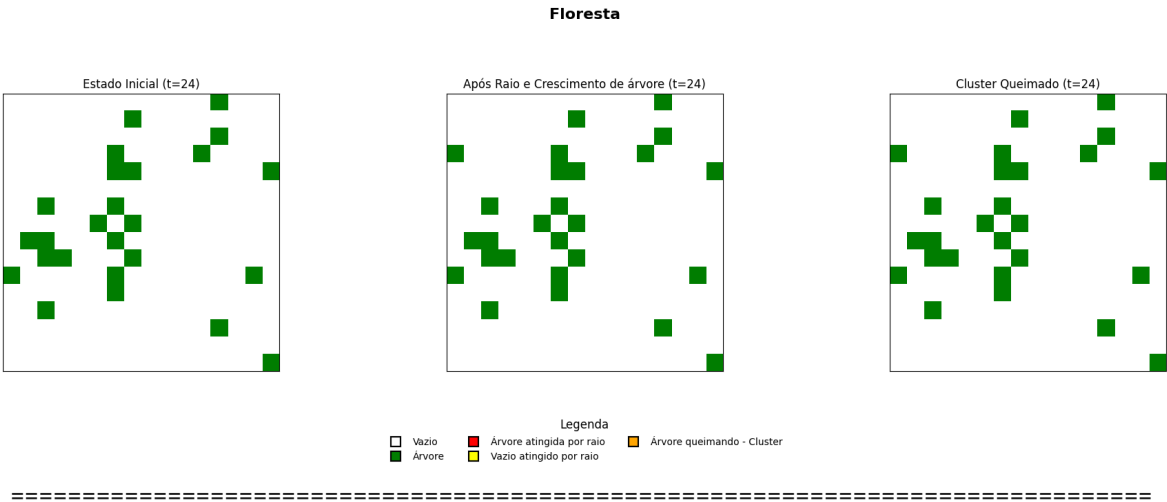
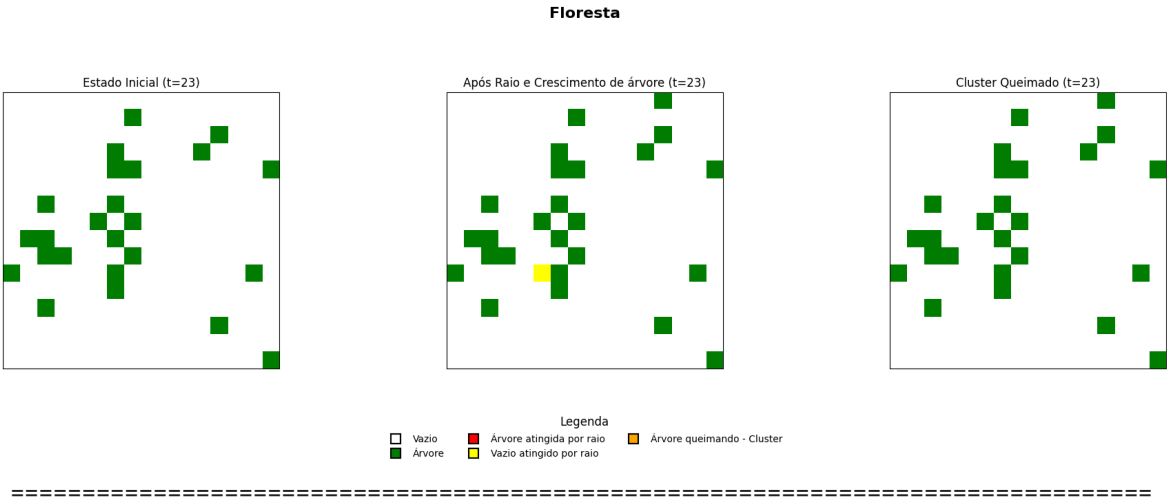


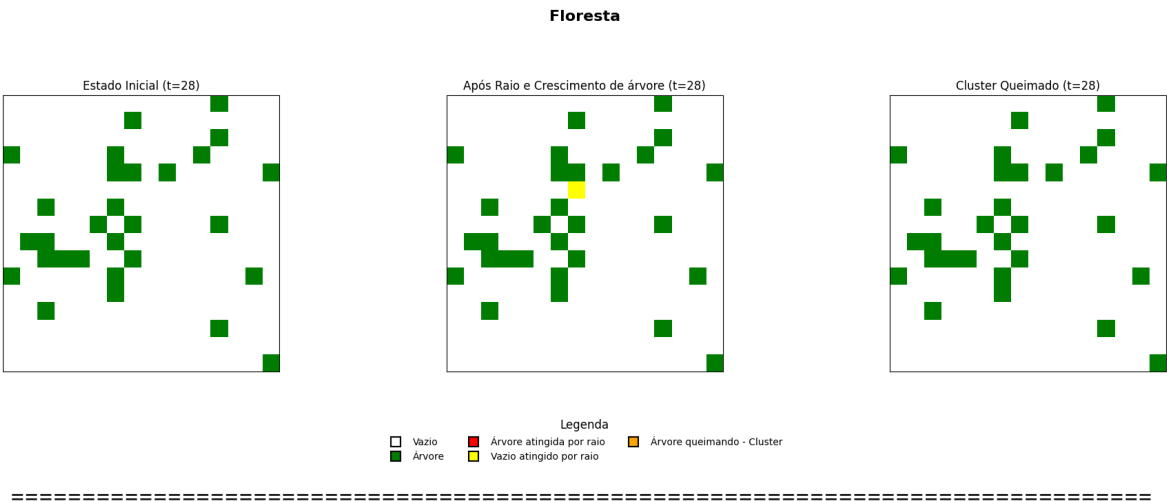
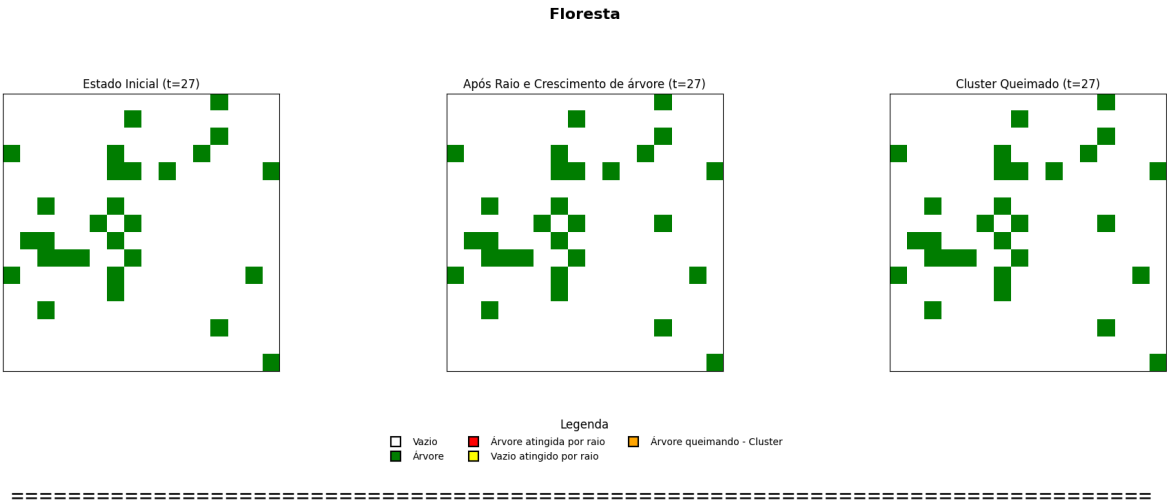
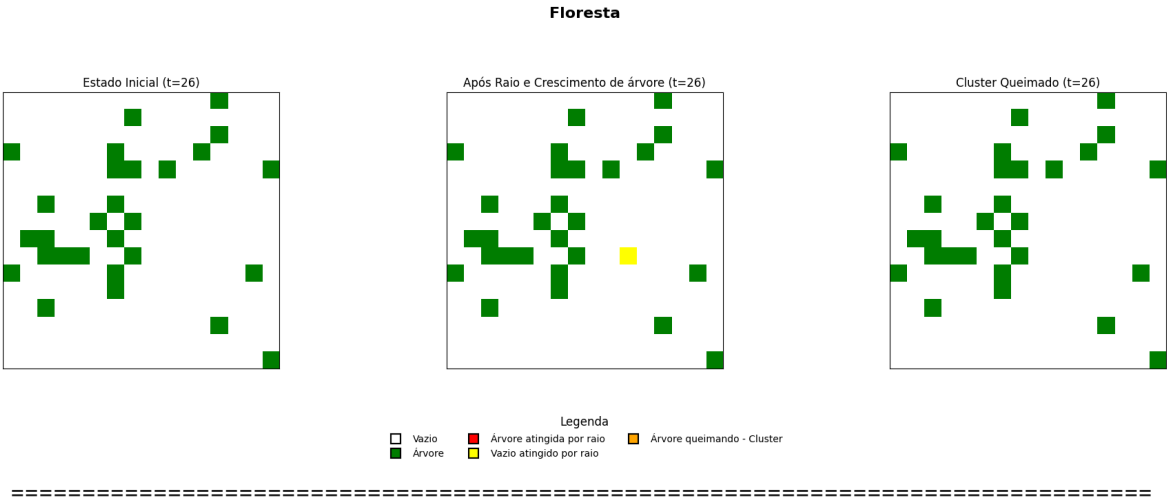


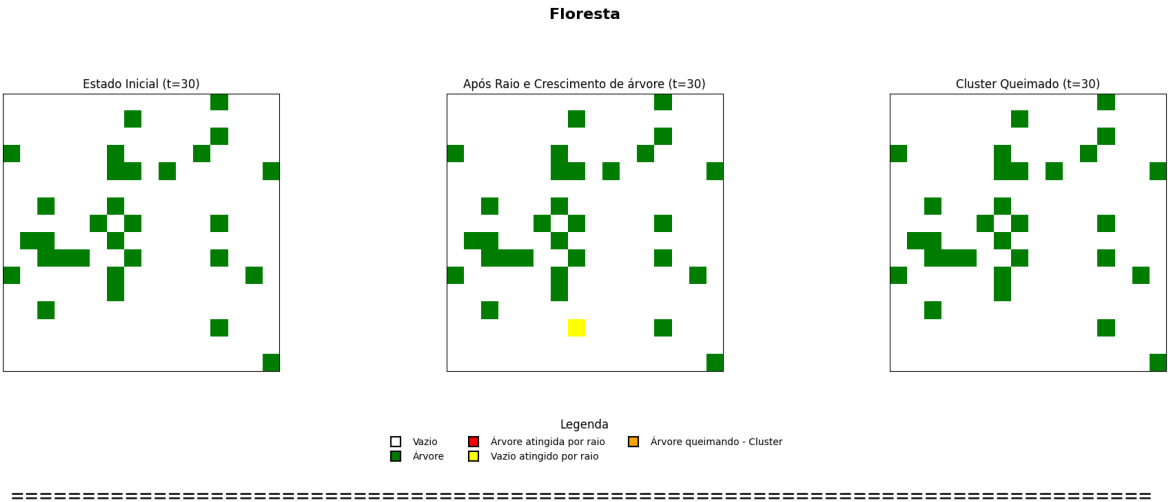
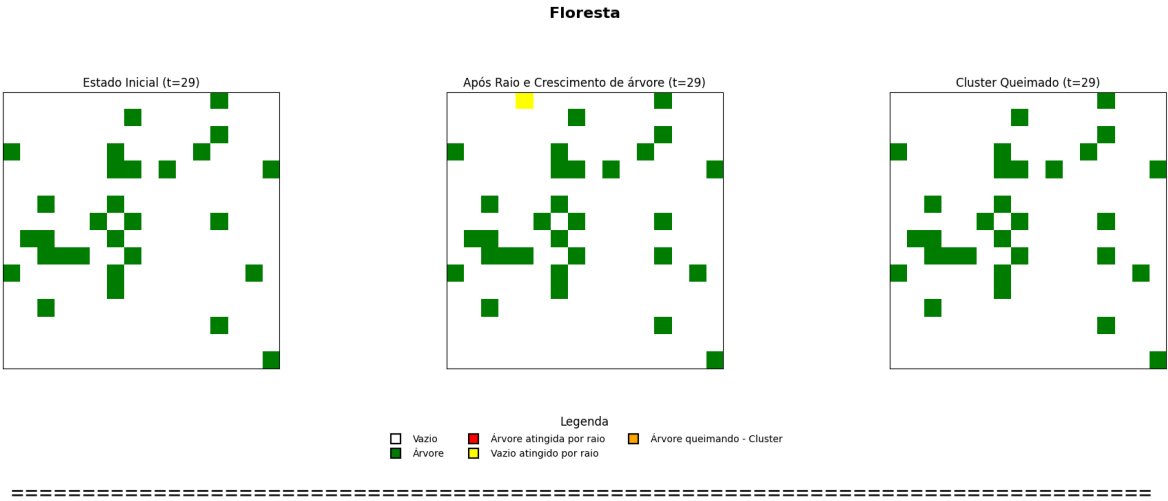


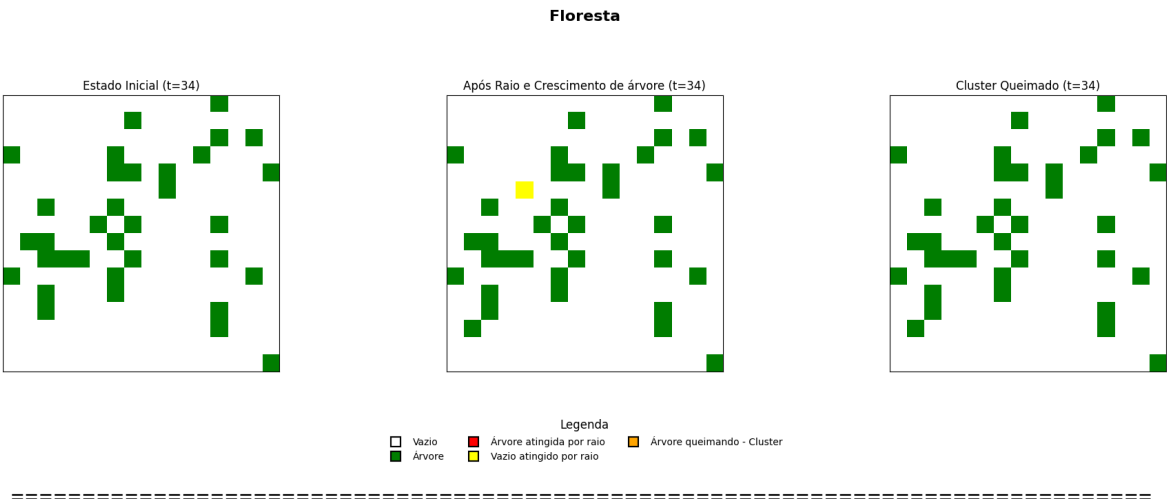
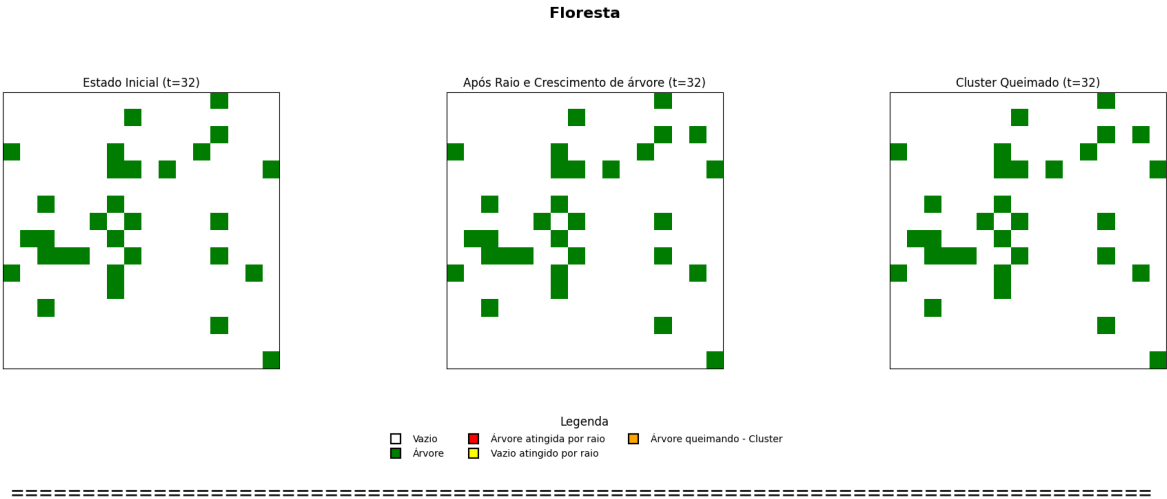


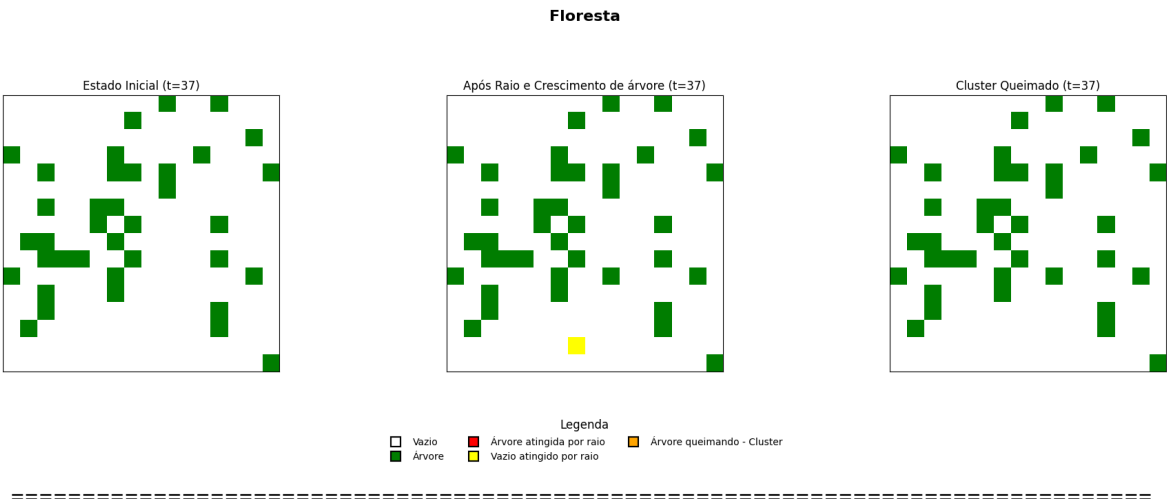
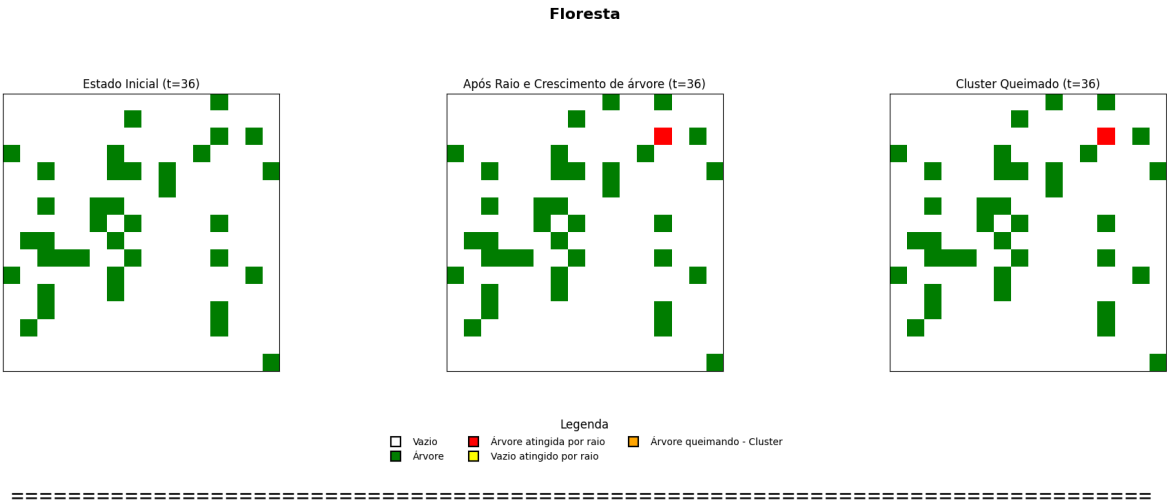
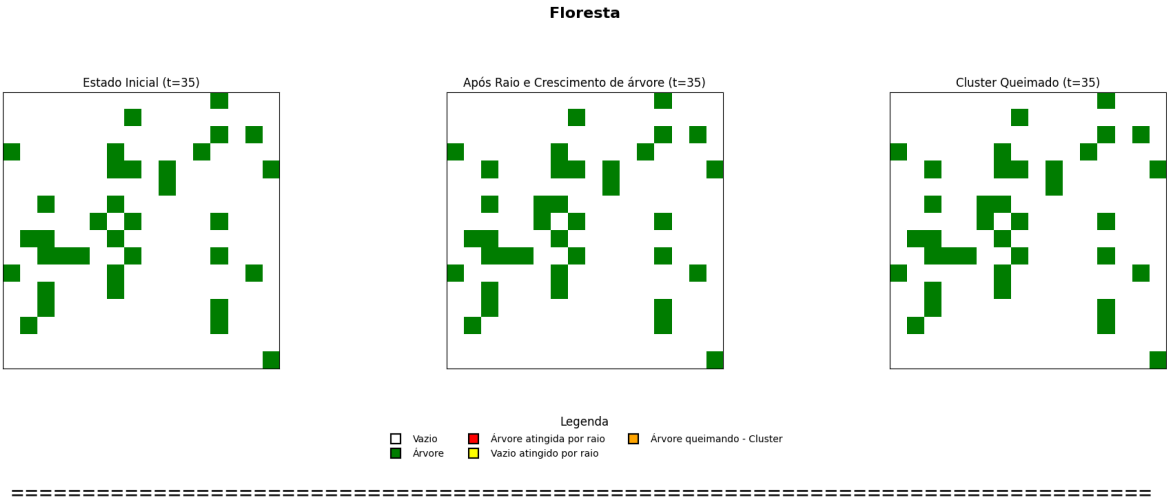


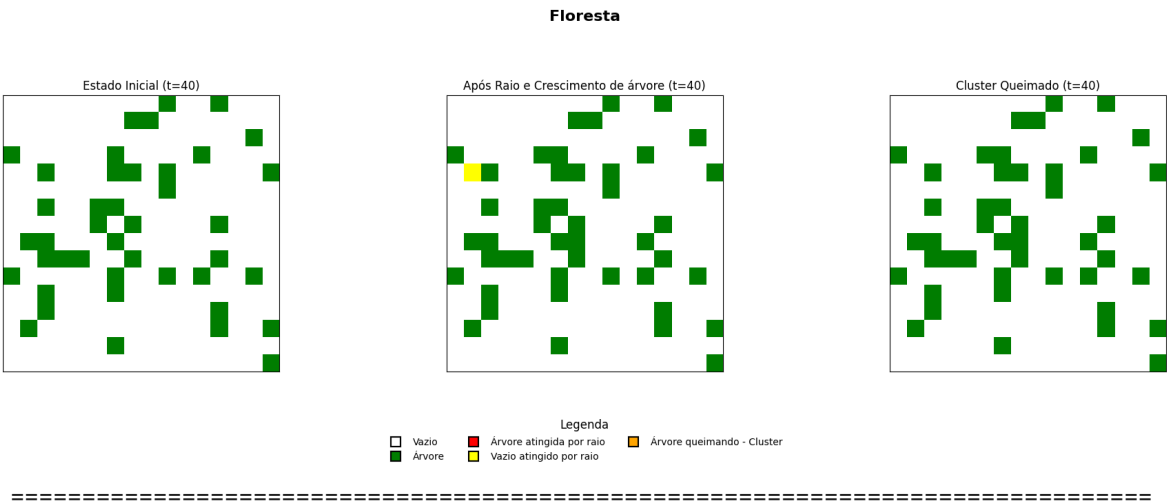
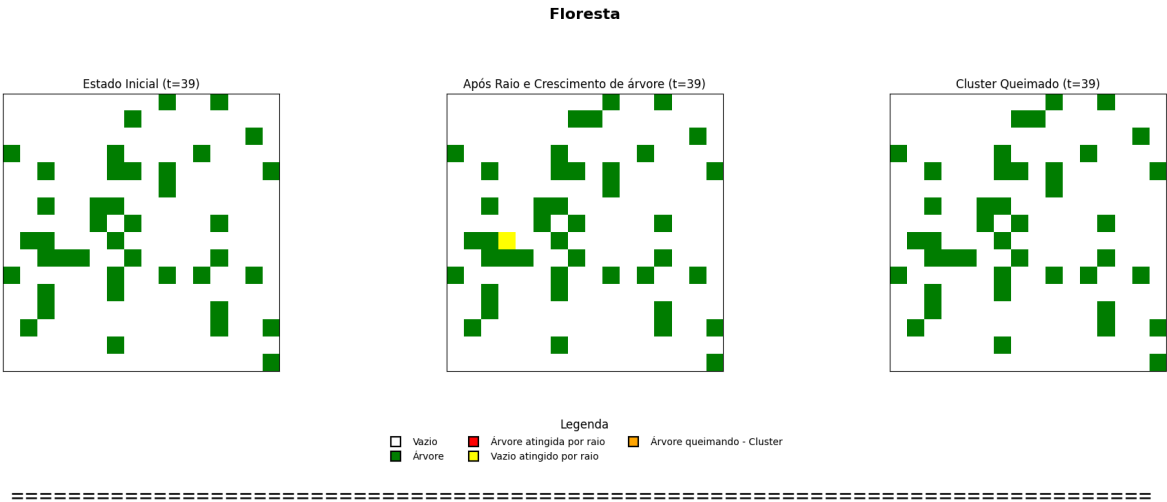
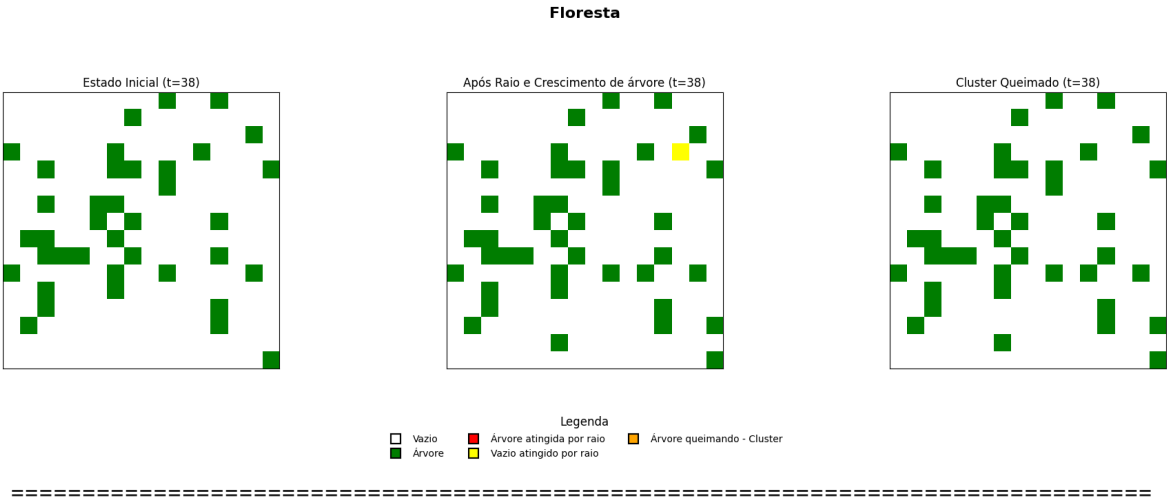


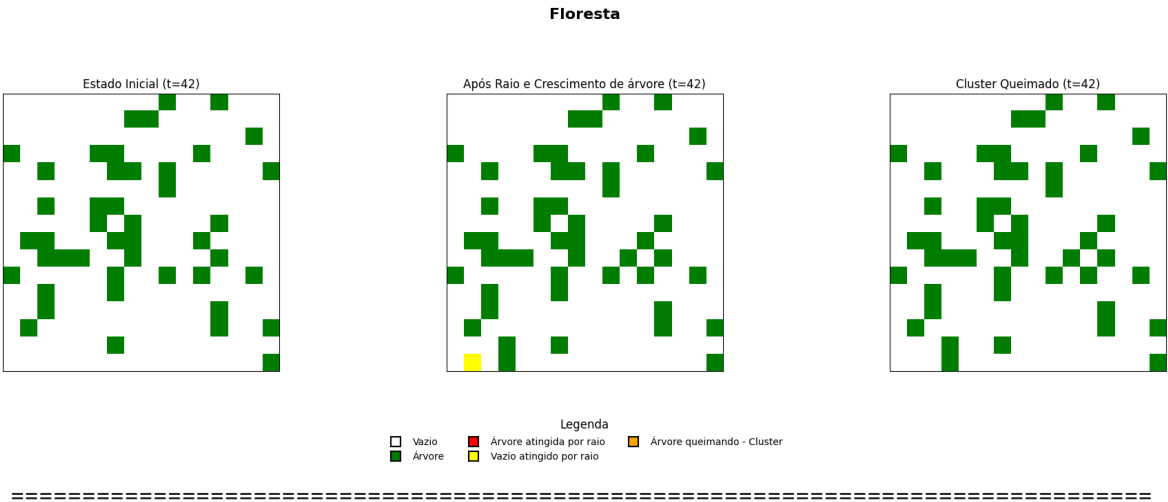




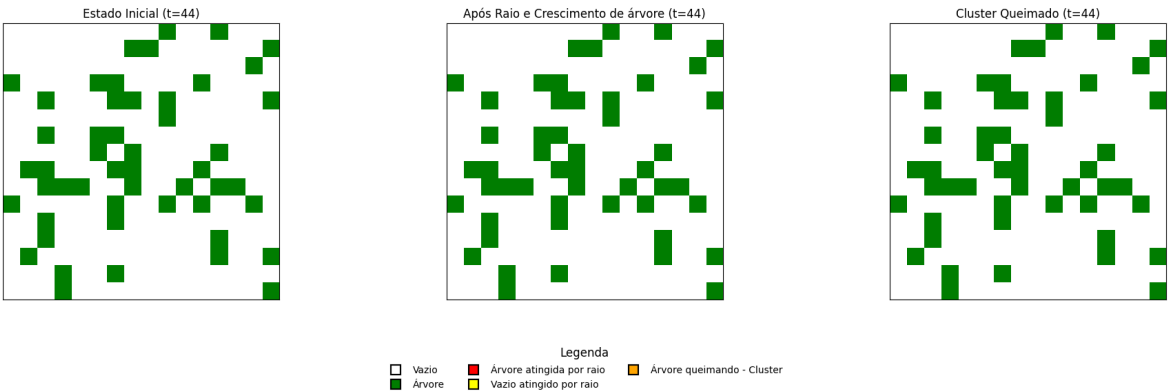




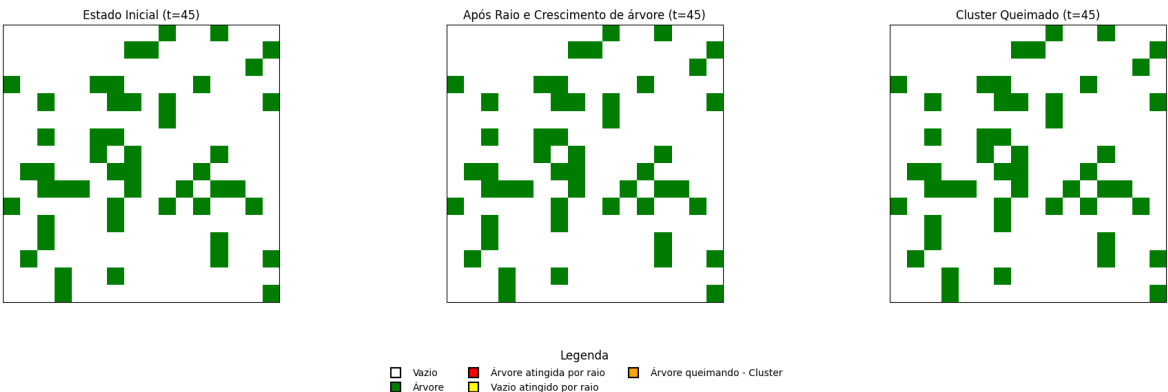




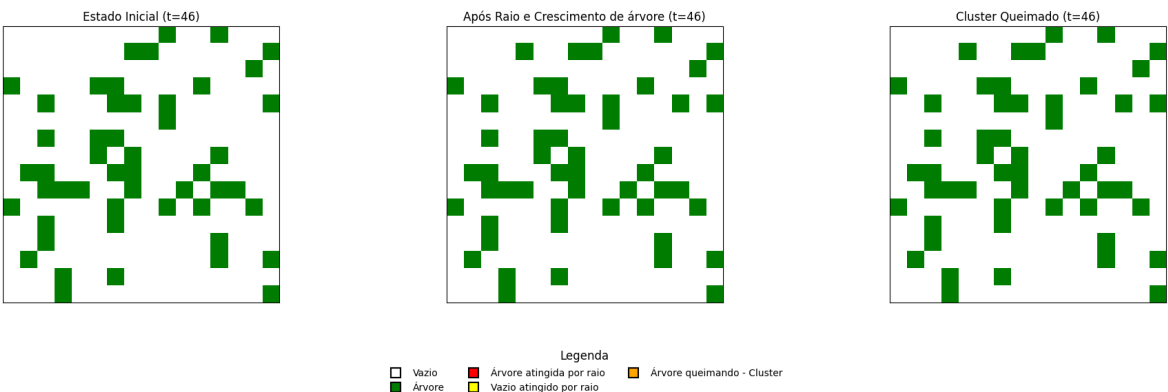
Floresta



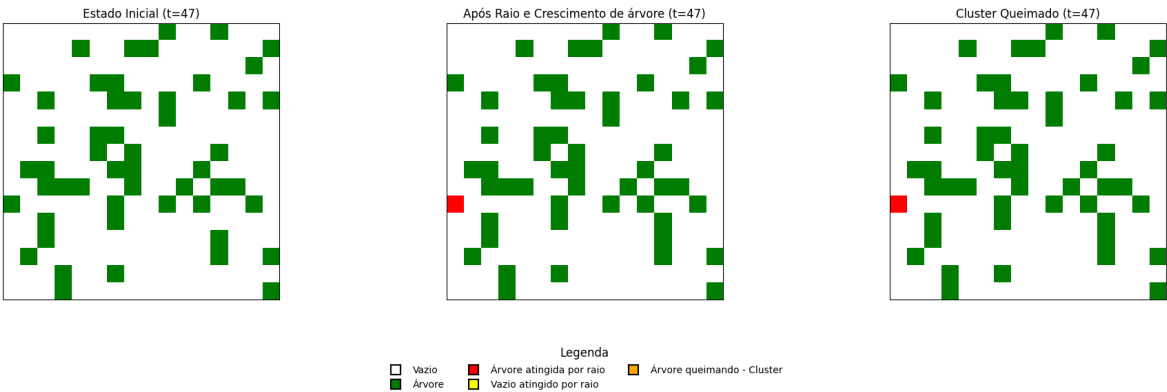
Floresta



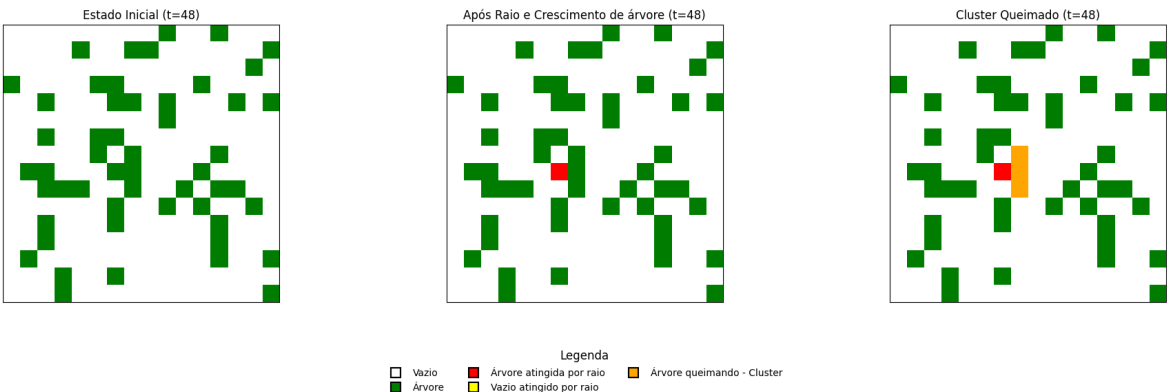
Floresta



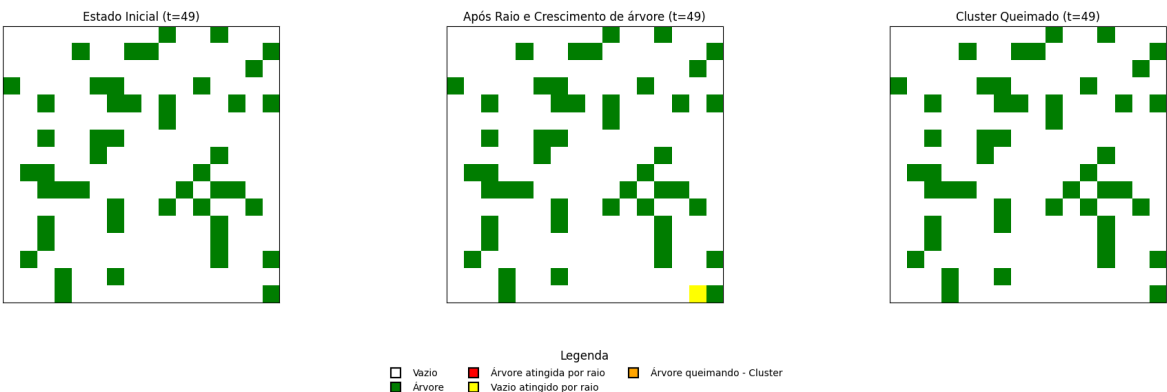
Floresta

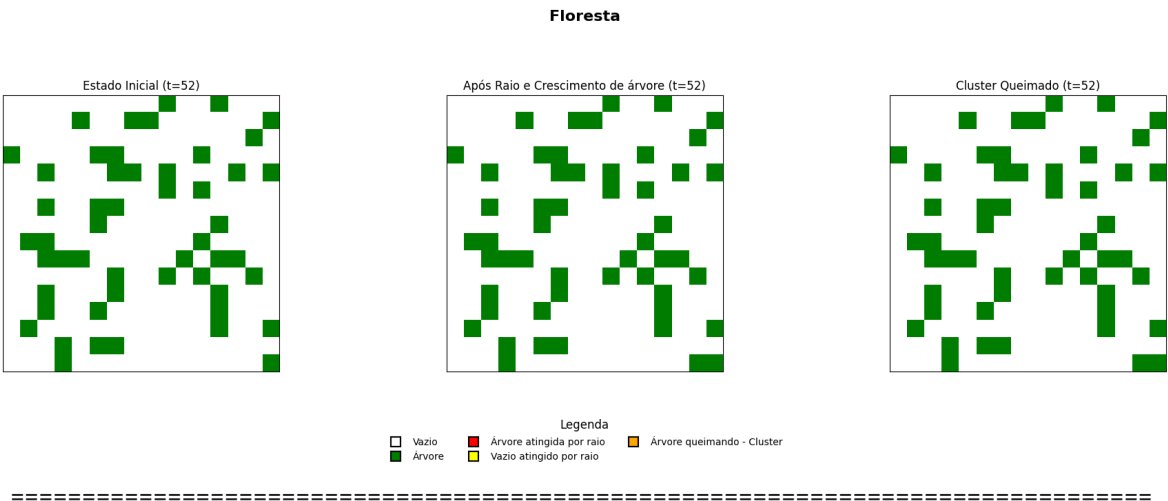
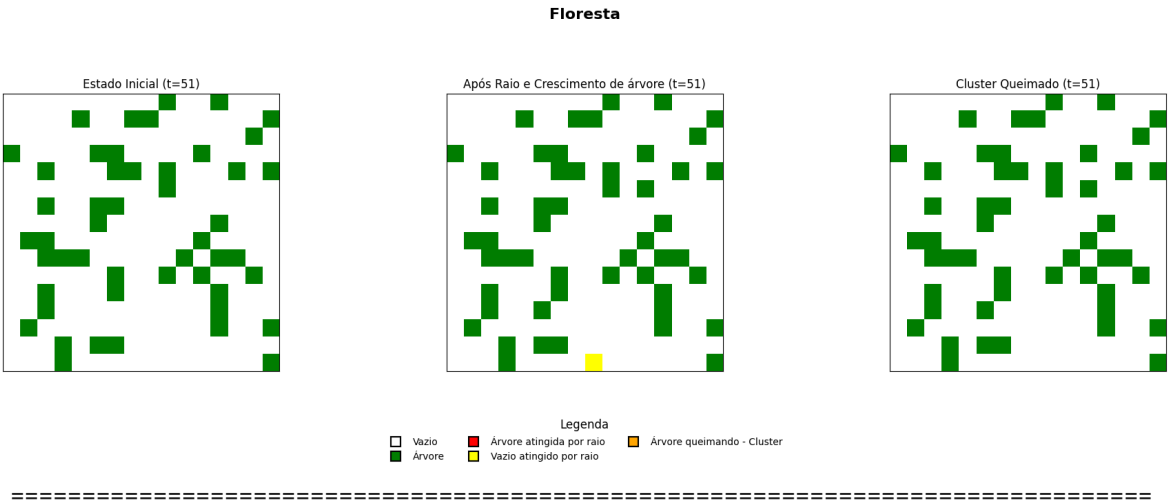
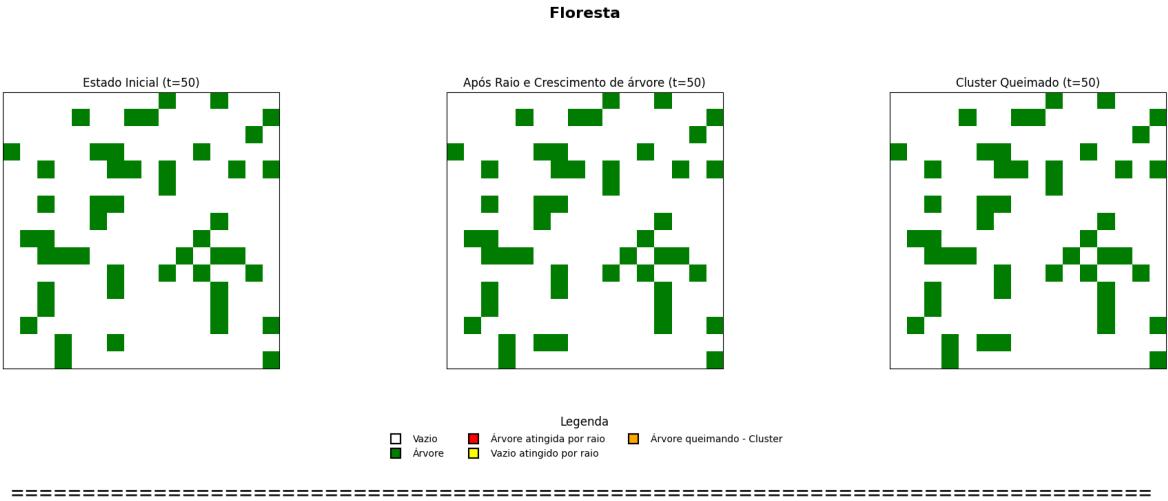


Floresta

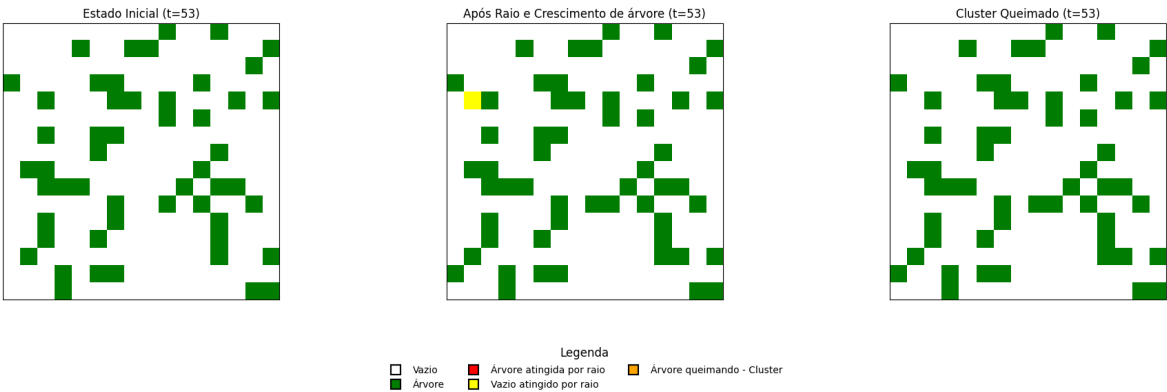


Floresta

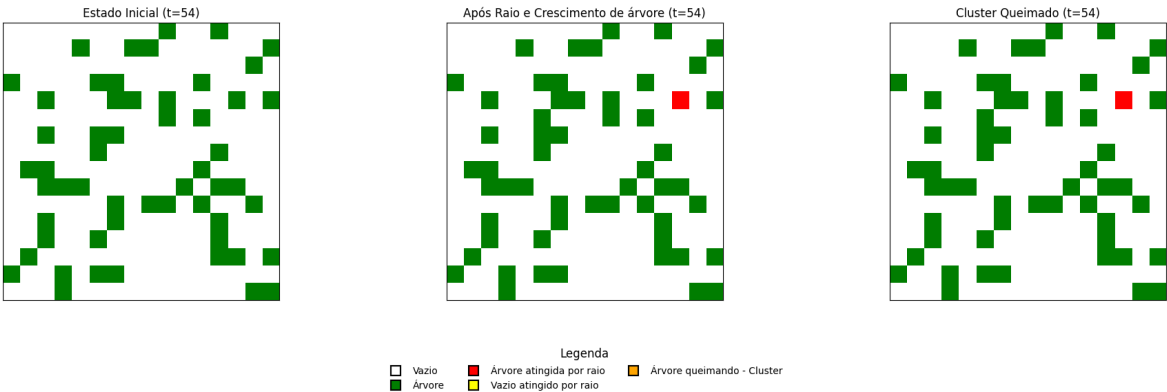




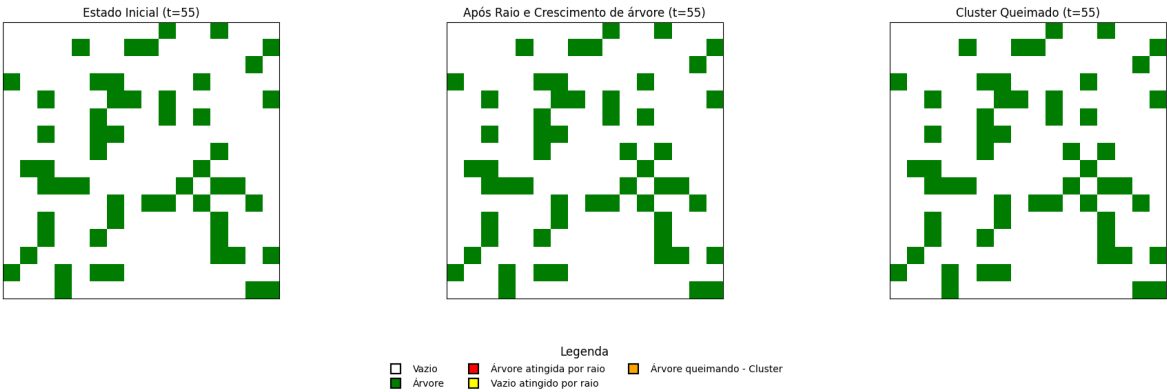
Floresta

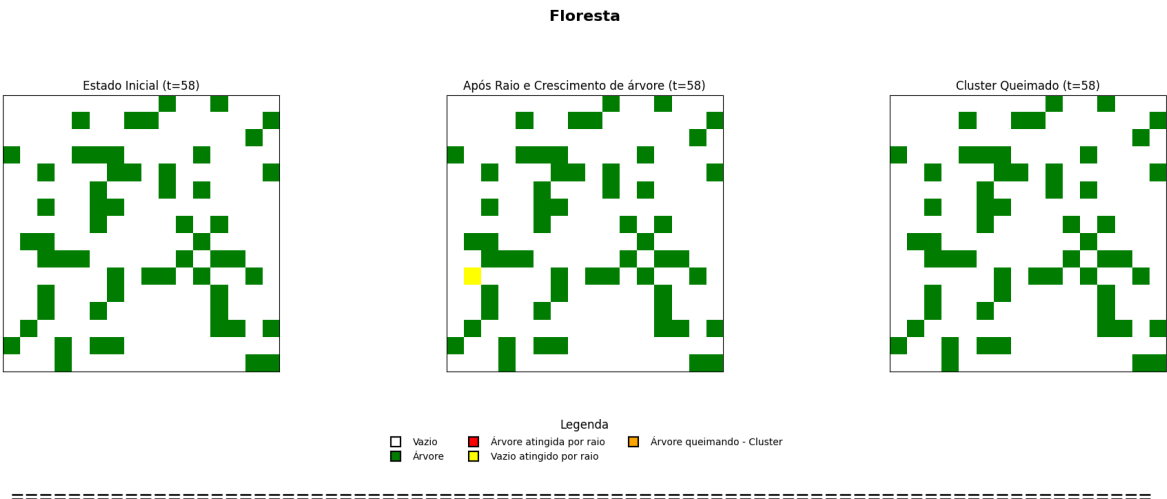
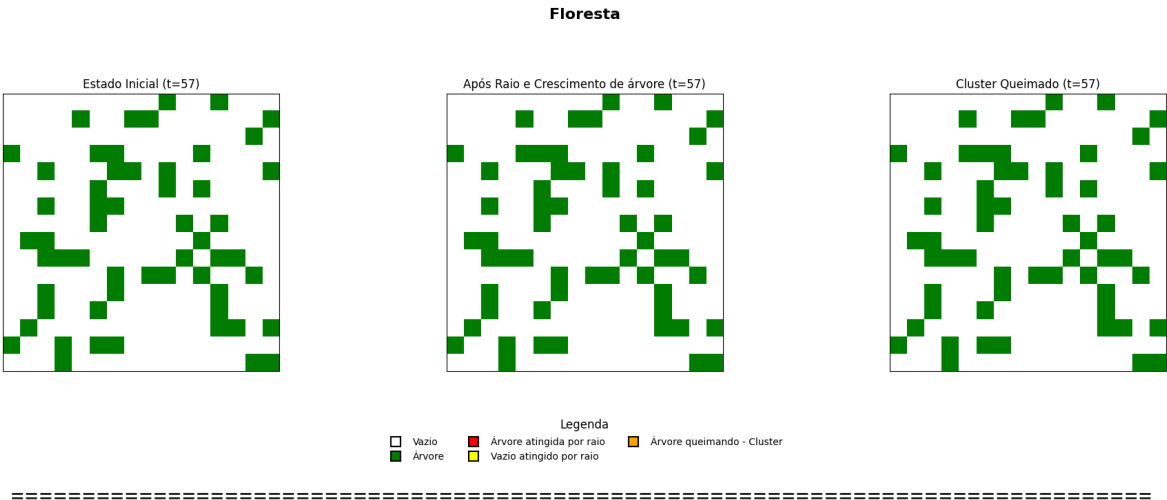
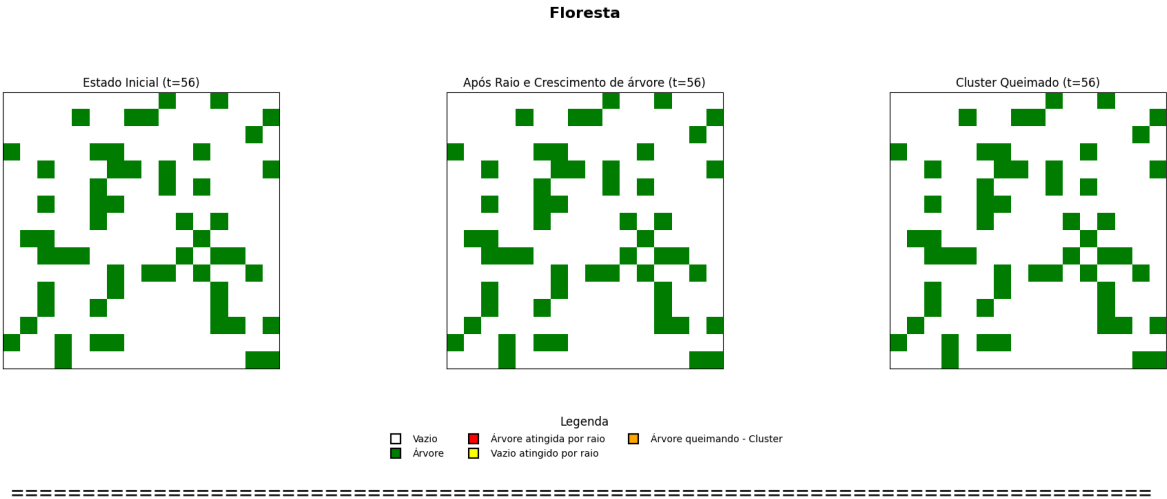


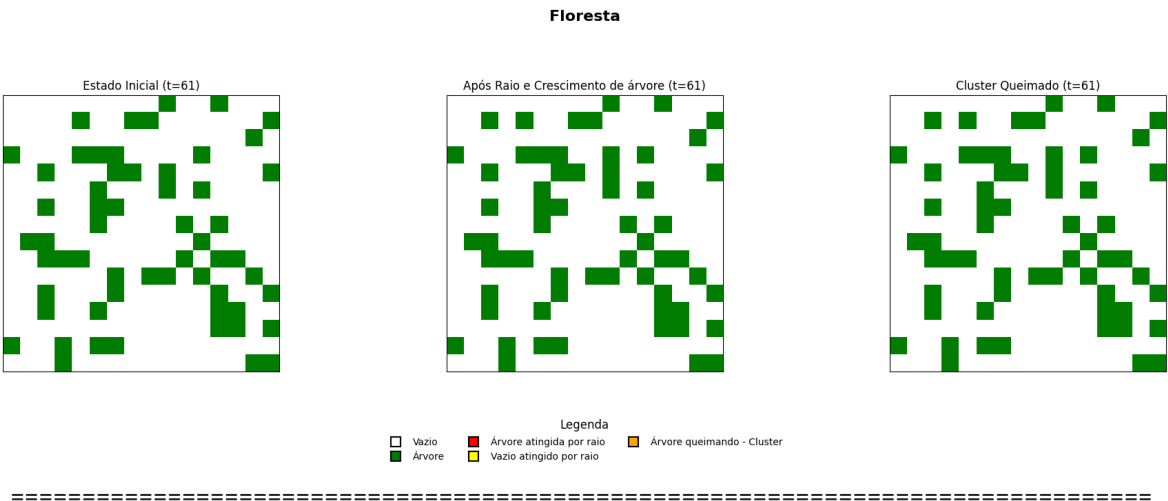
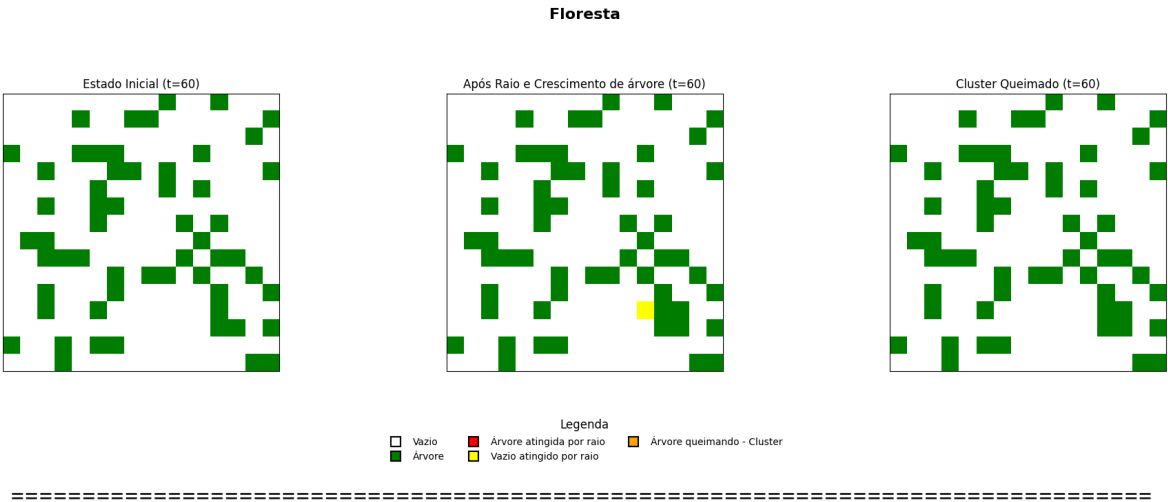
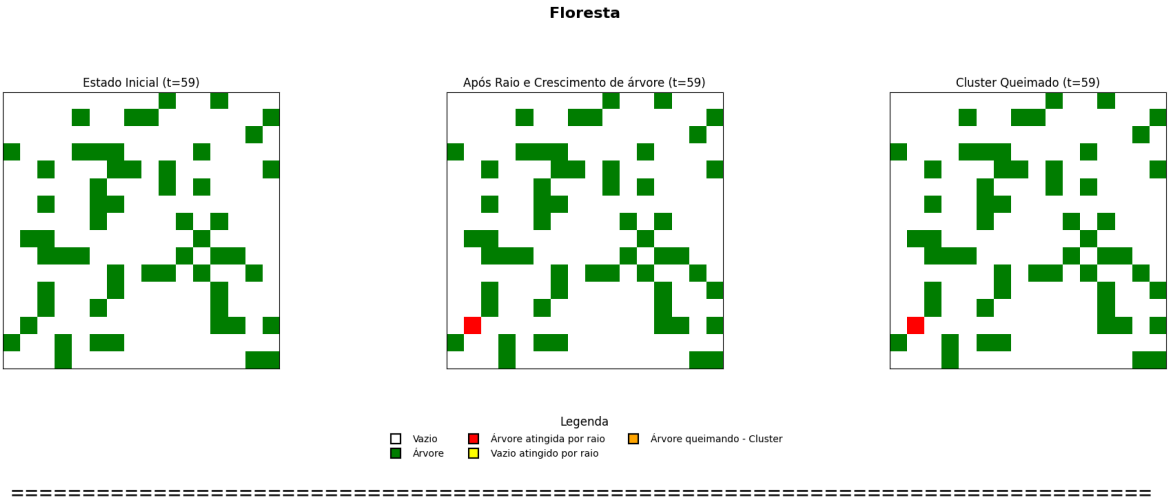
Floresta

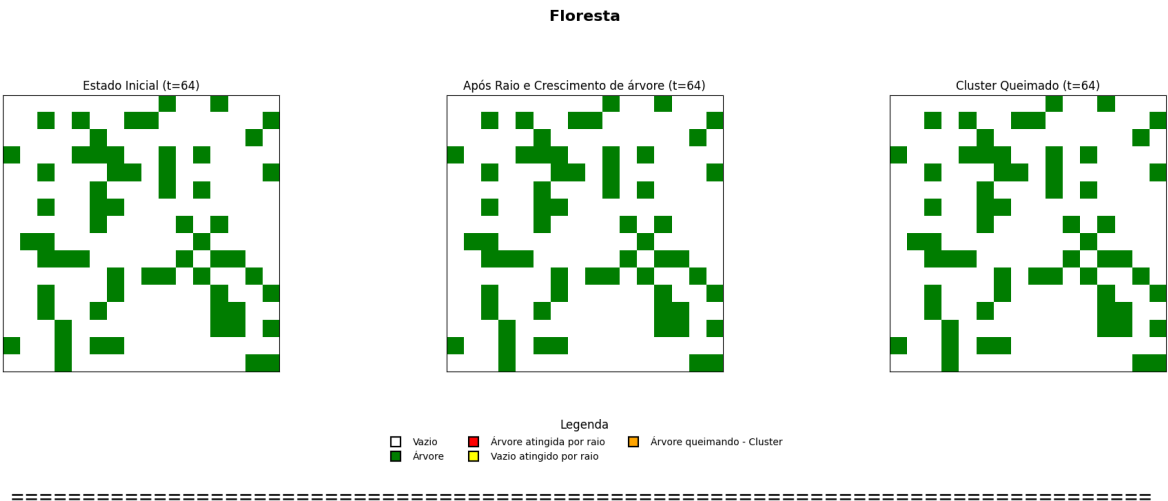
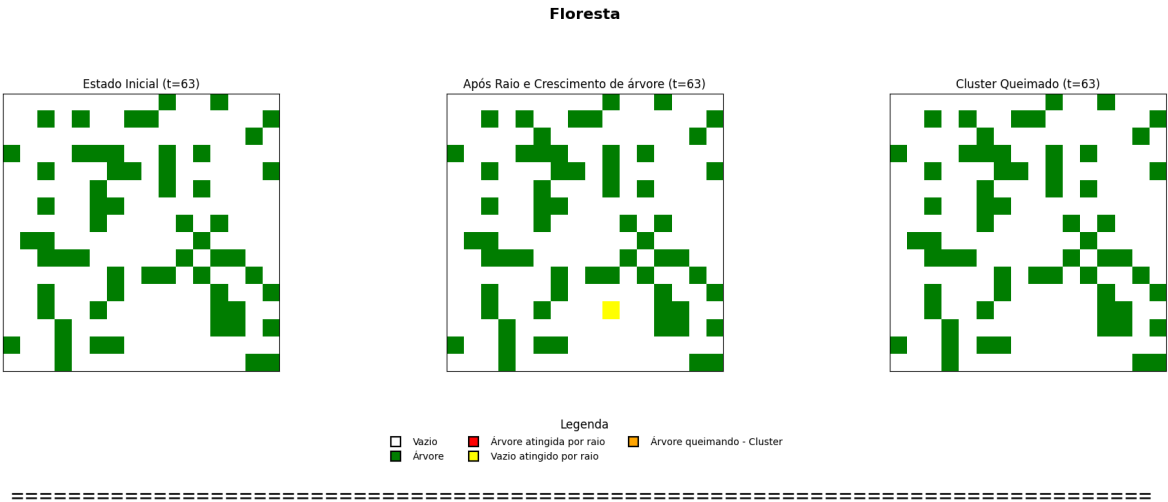
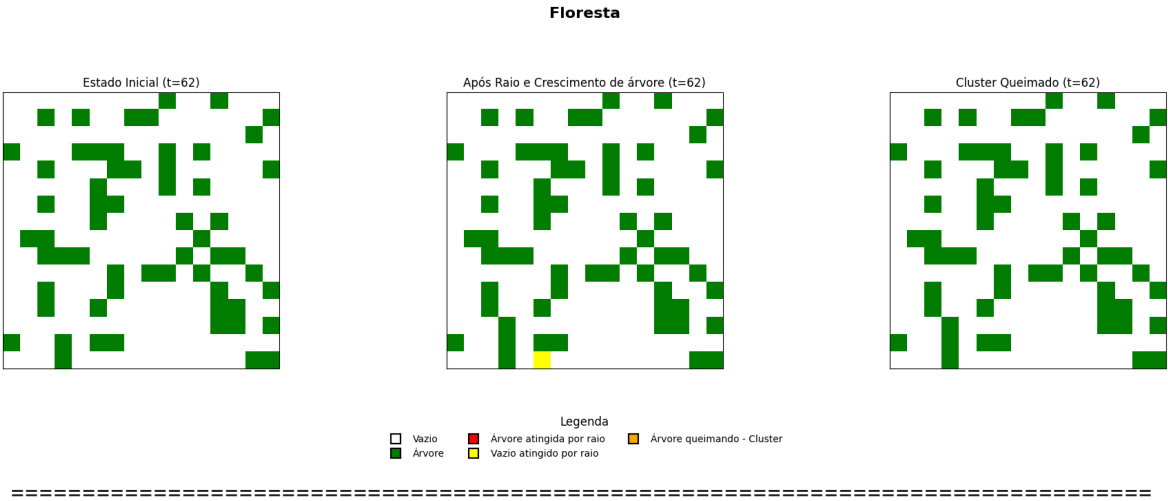


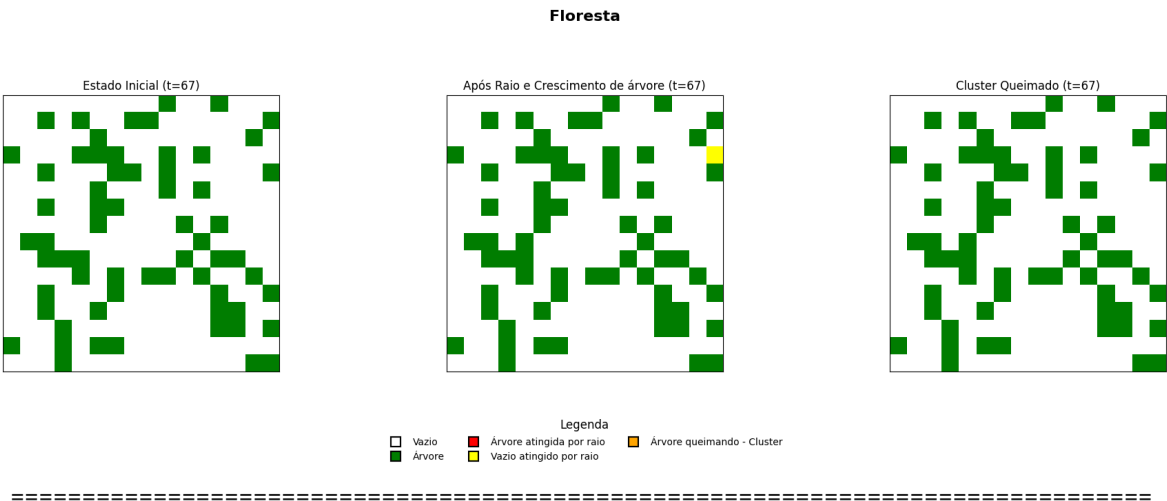
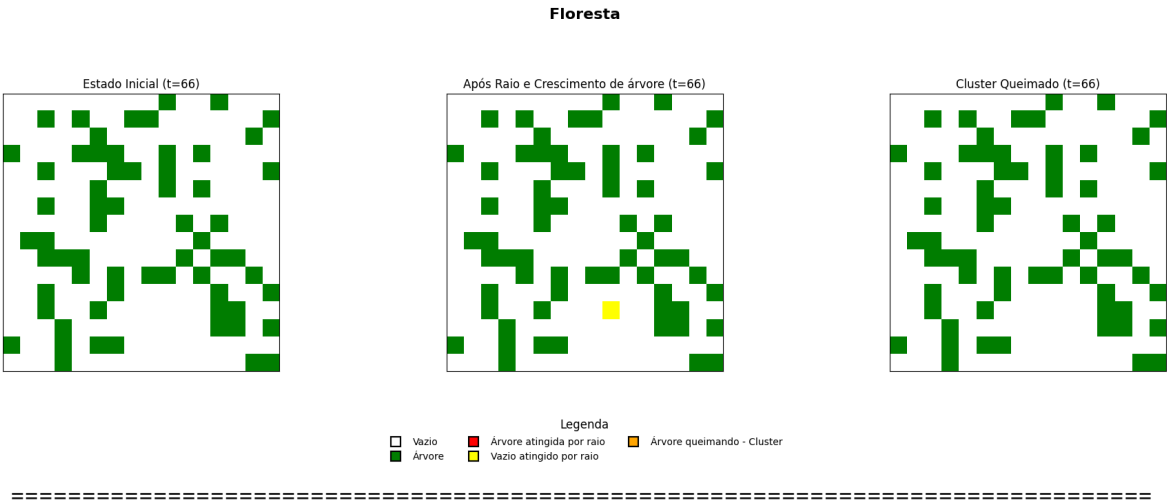
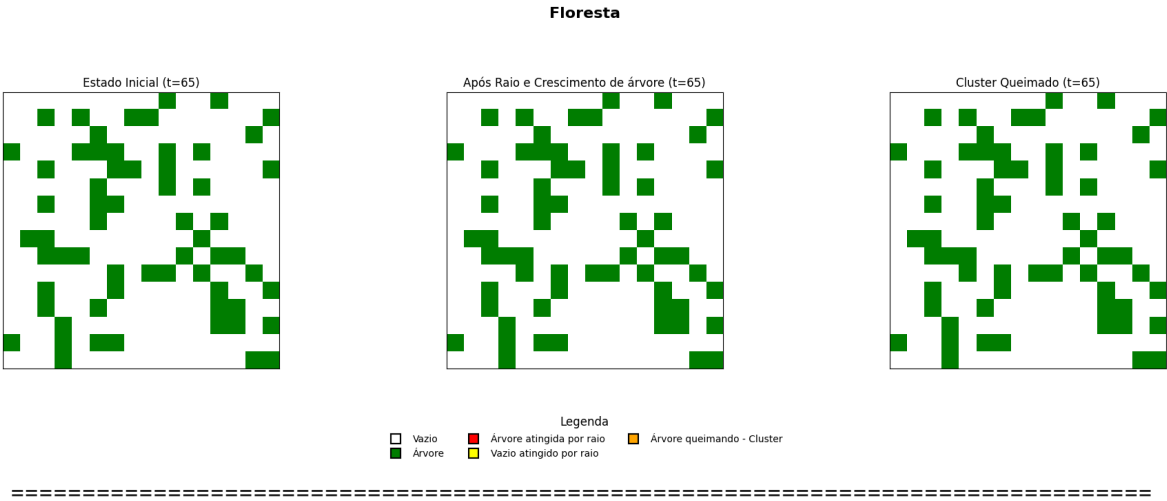
Floresta



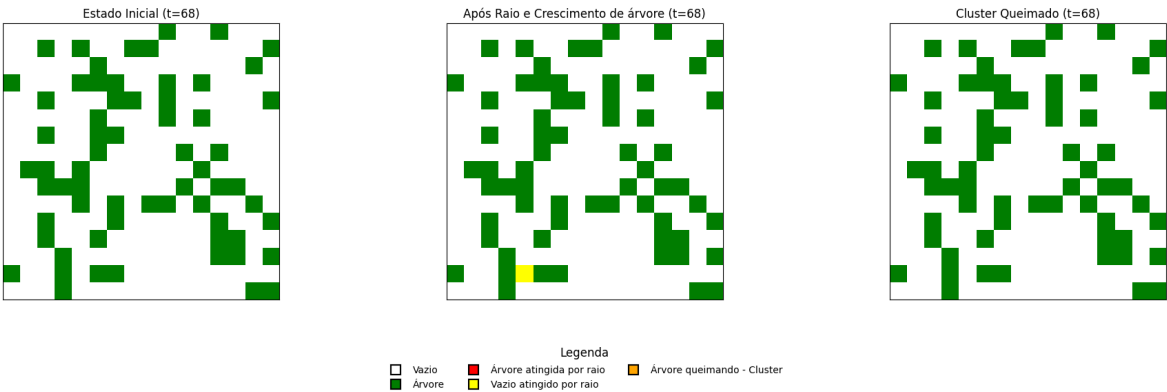




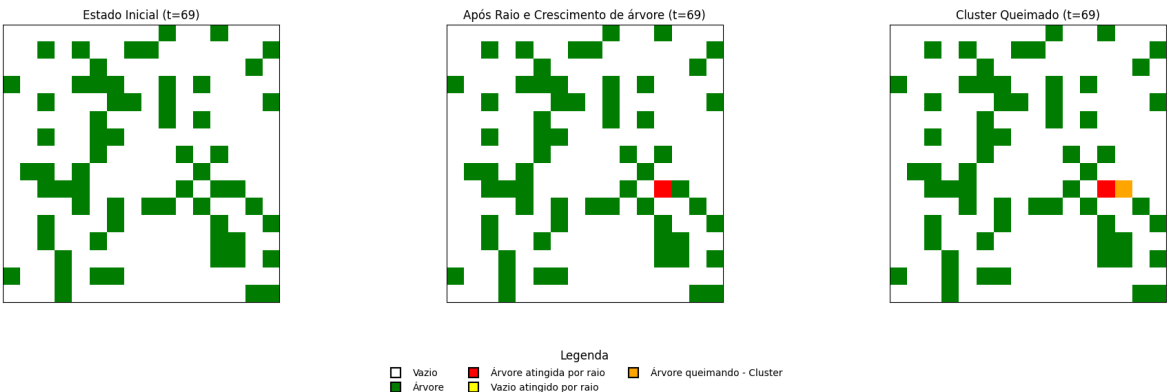




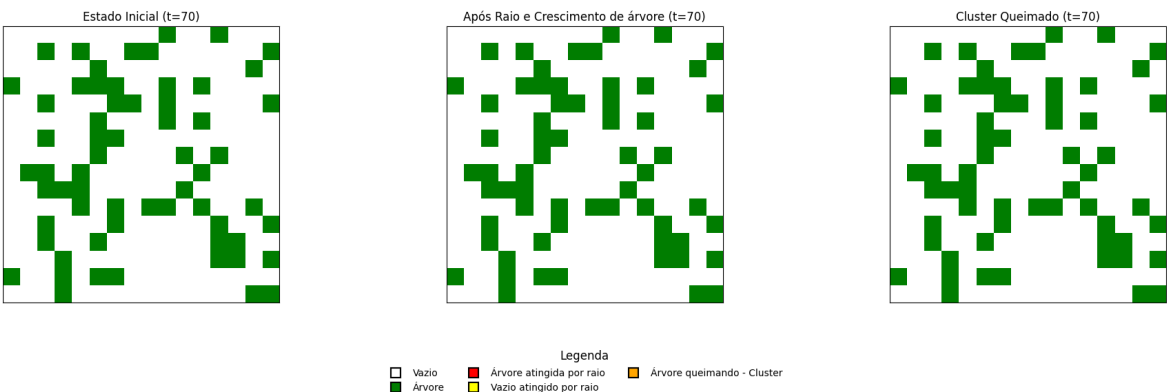
Floresta



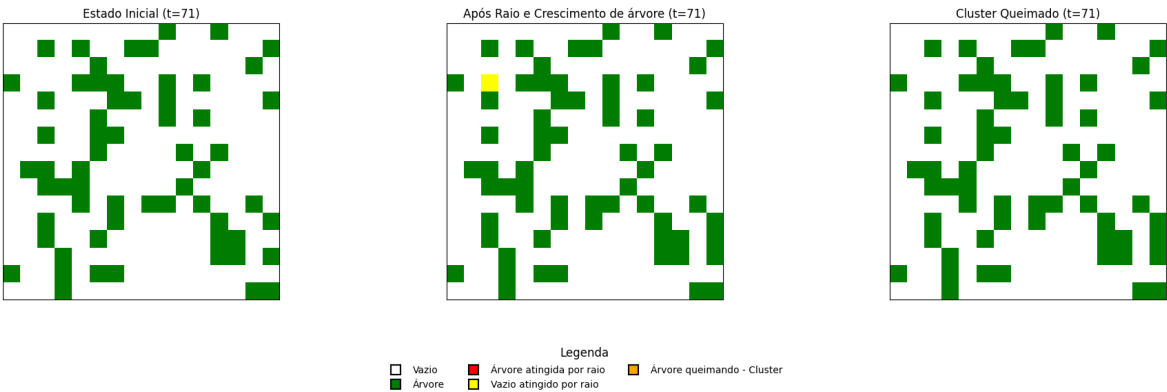
Floresta



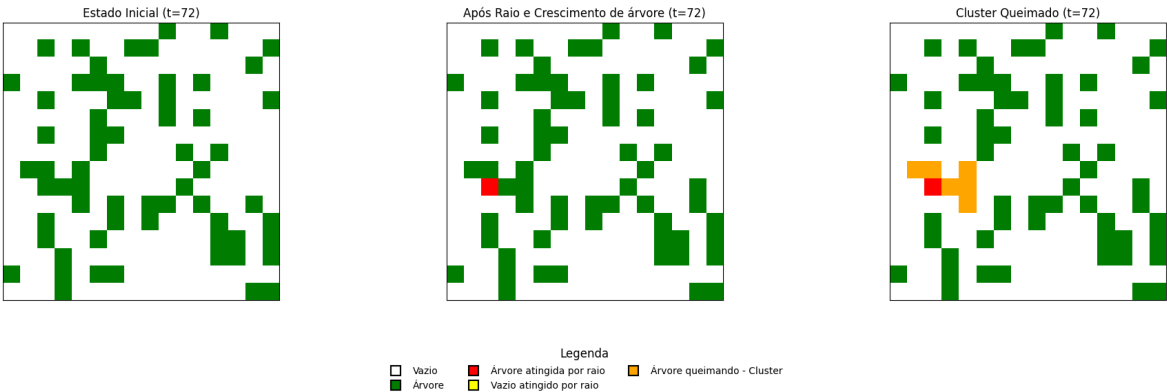
Floresta



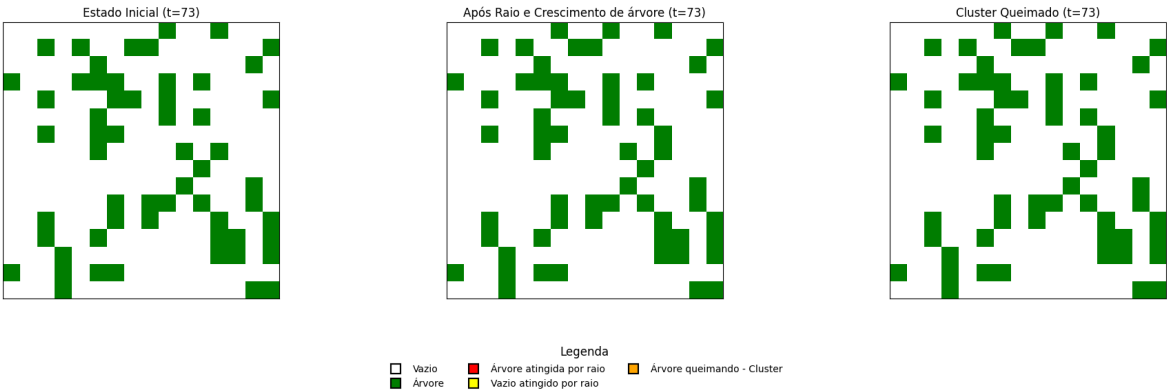
Floresta

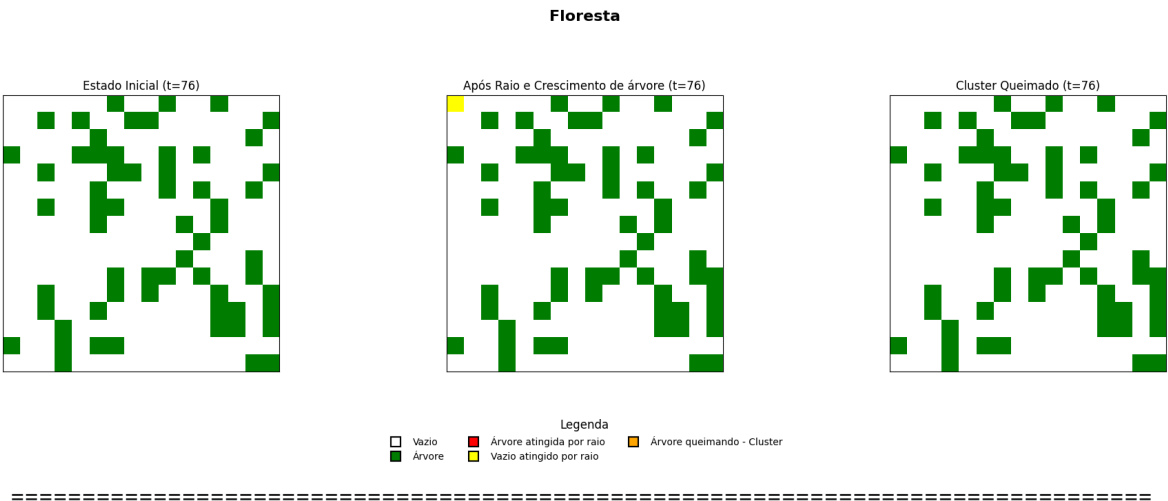
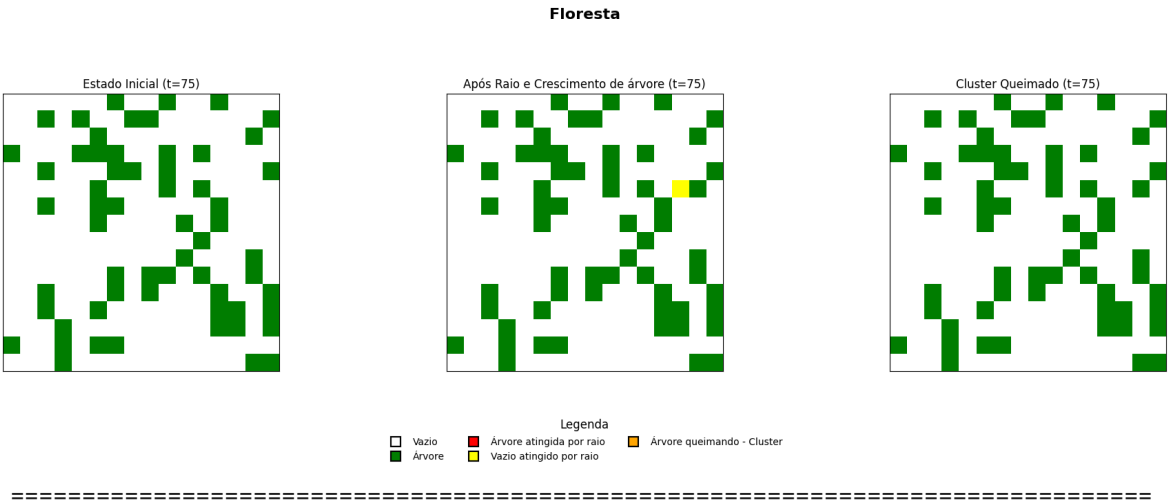
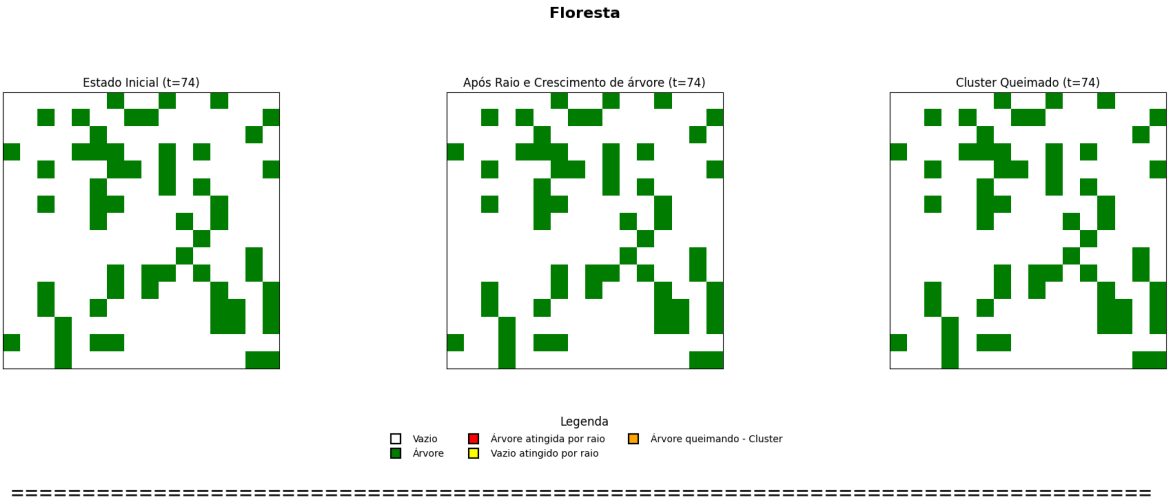


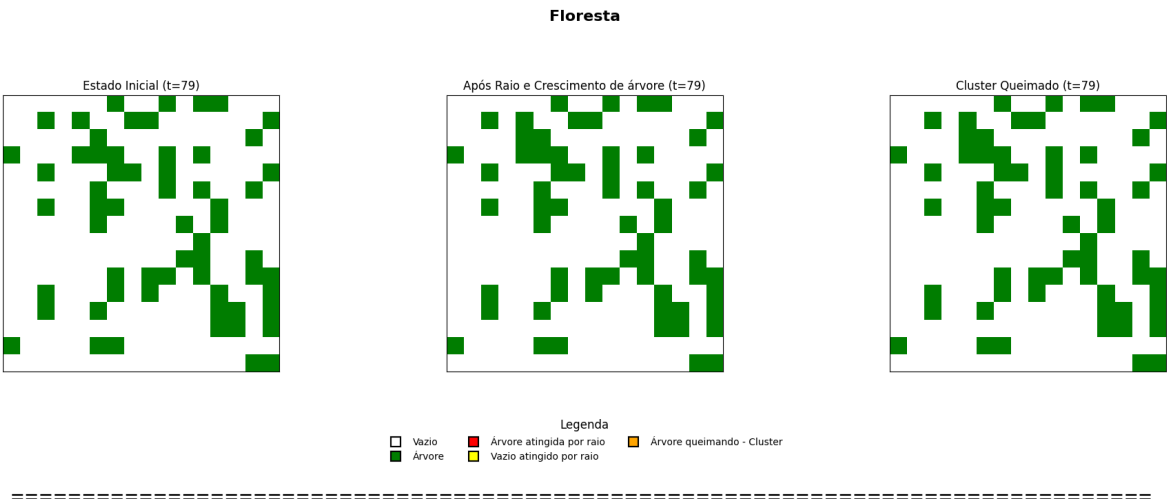
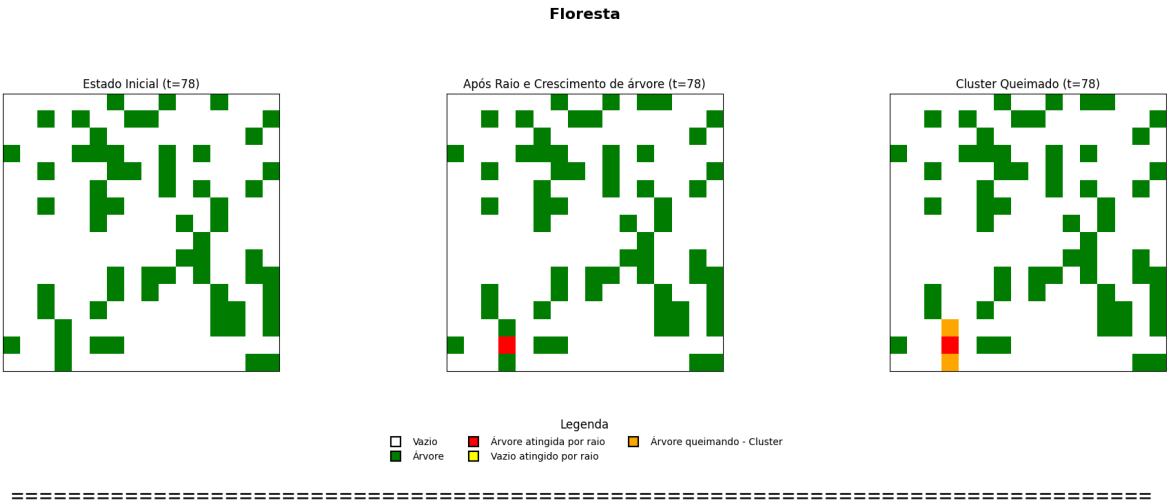
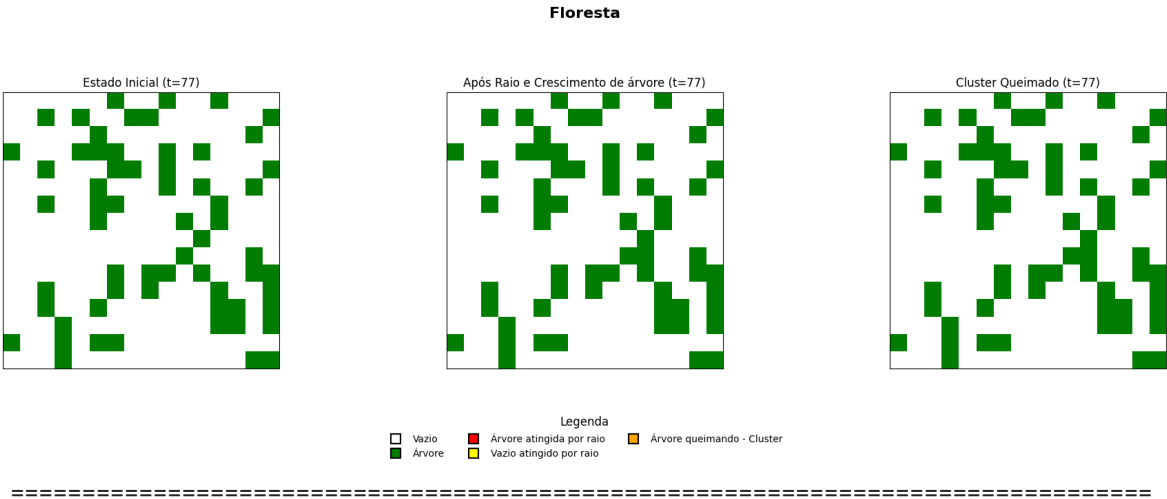
Floresta



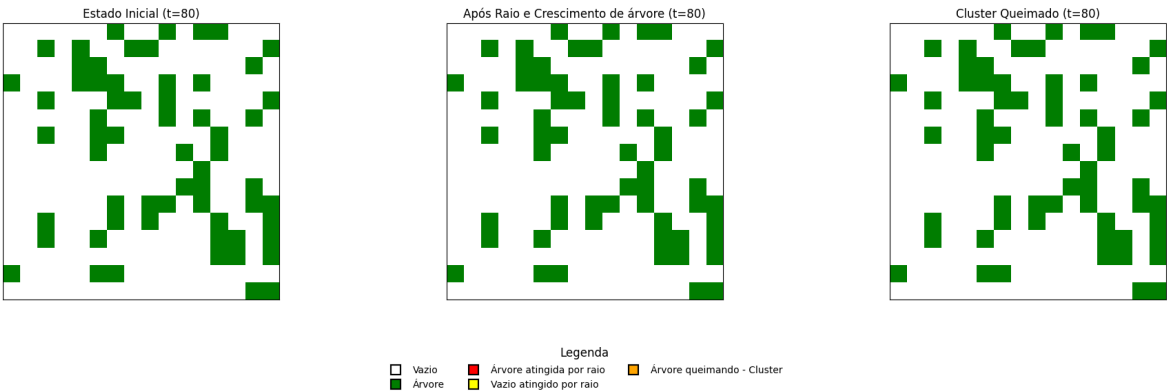
Floresta



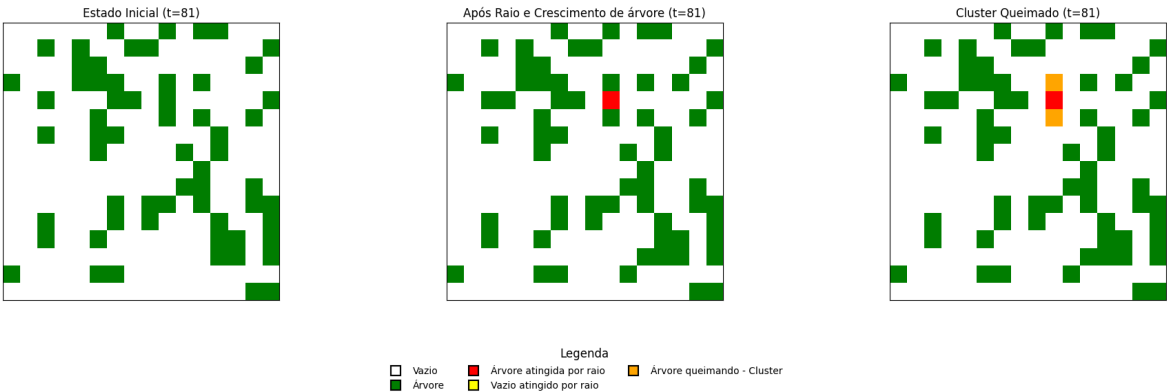




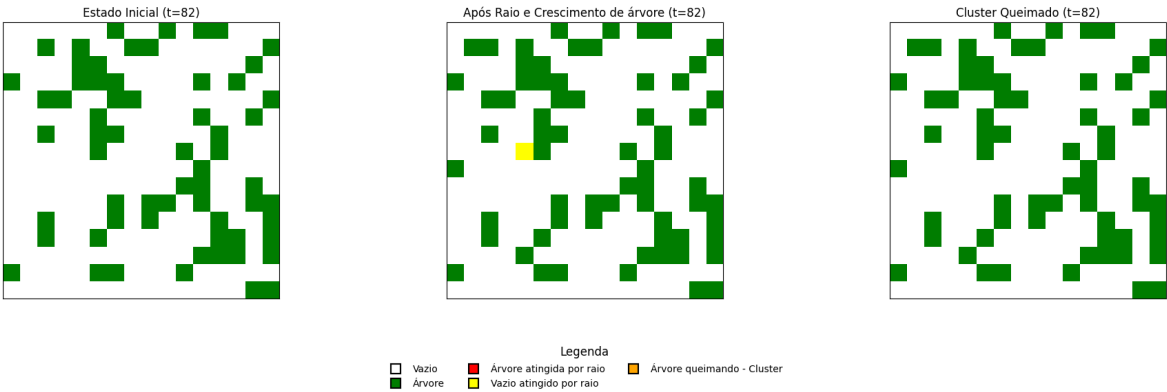
Floresta



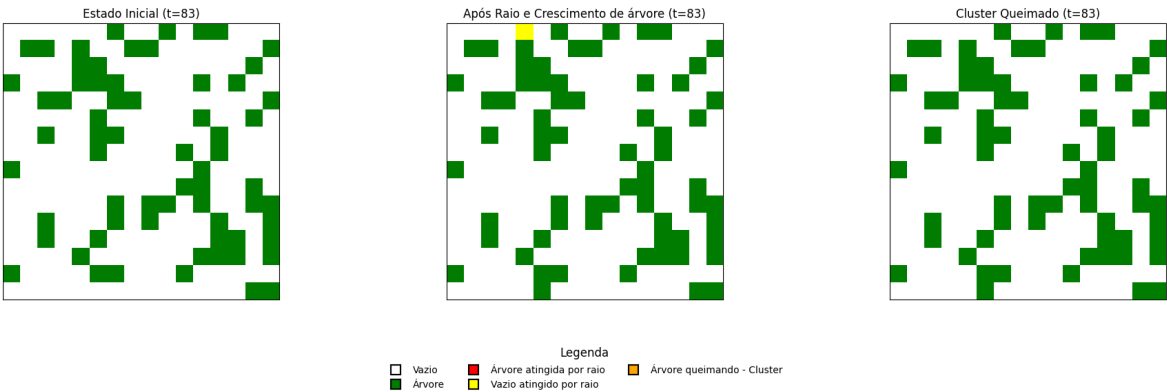
Floresta



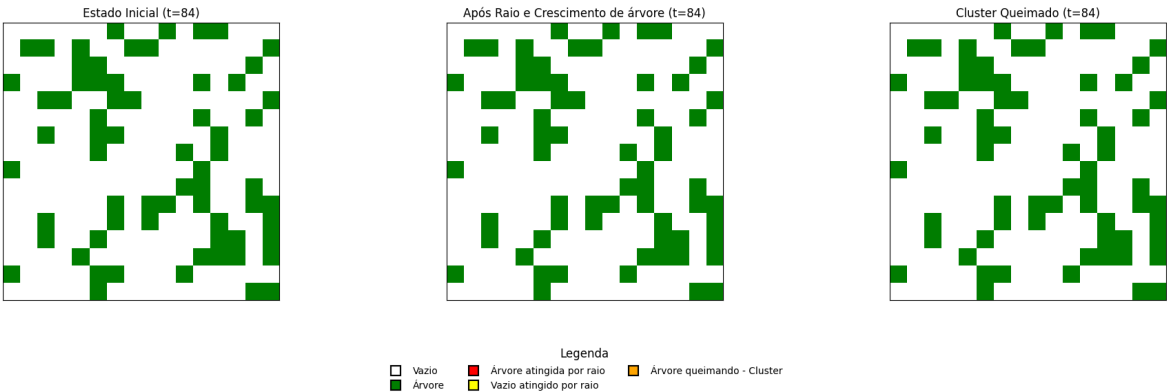
Floresta



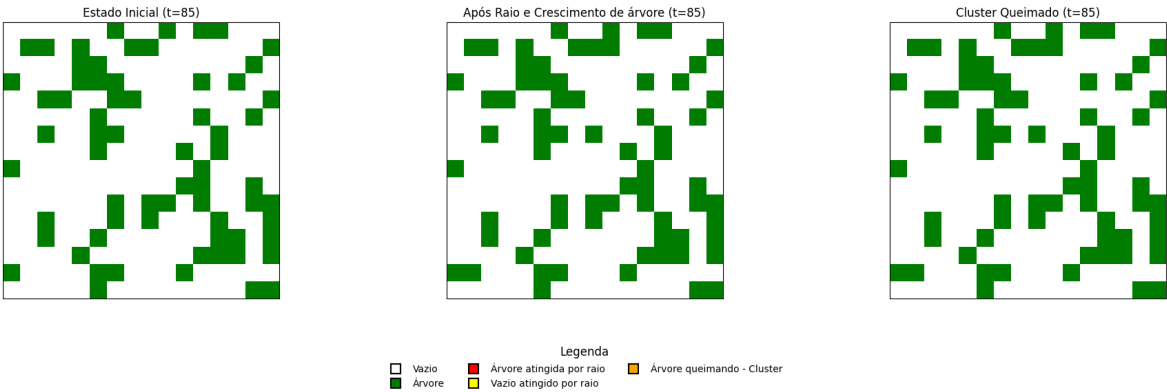
Floresta



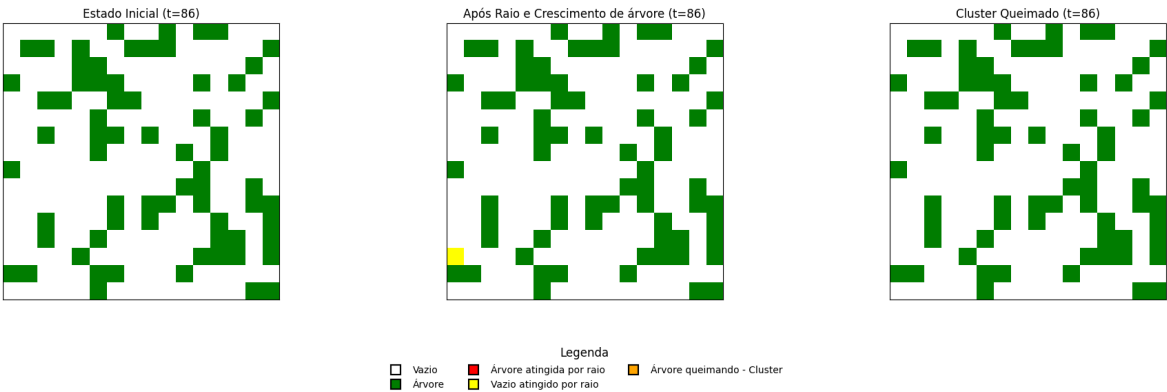
Floresta



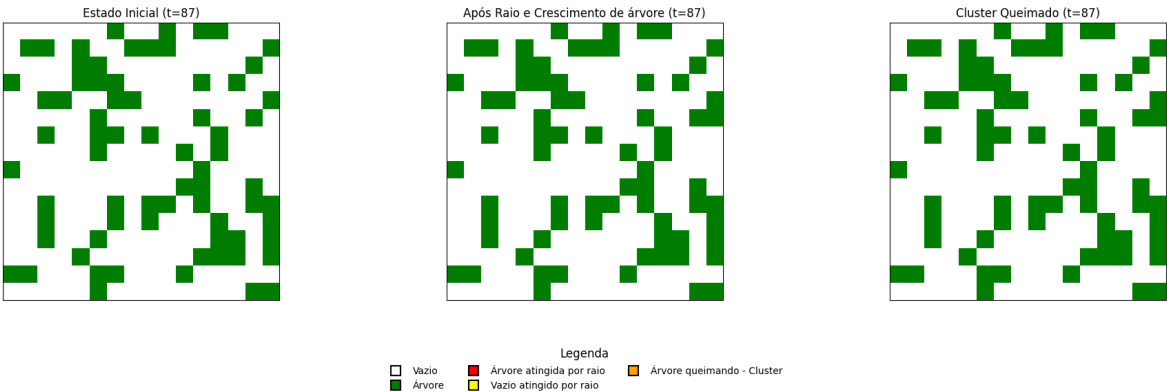
Floresta



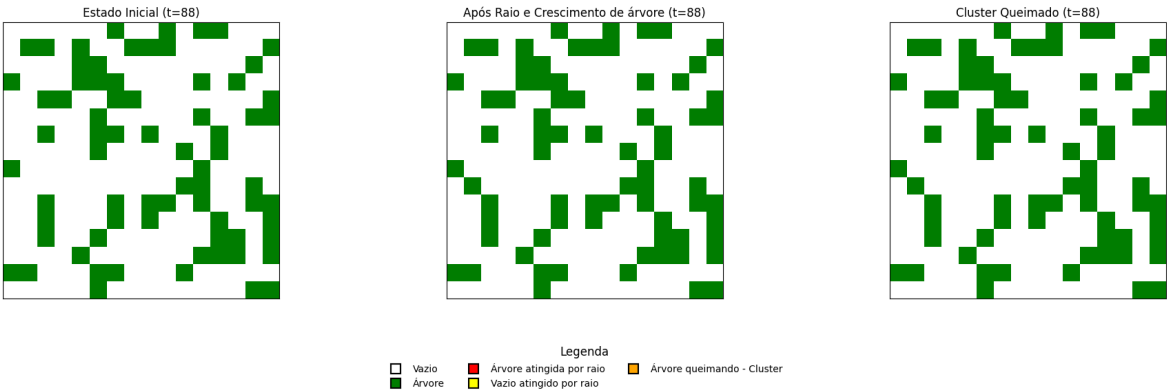
Floresta



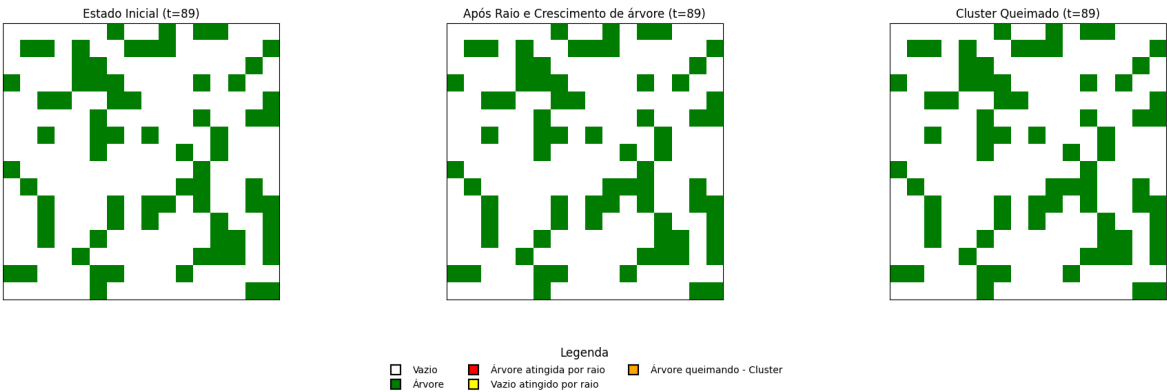
Floresta



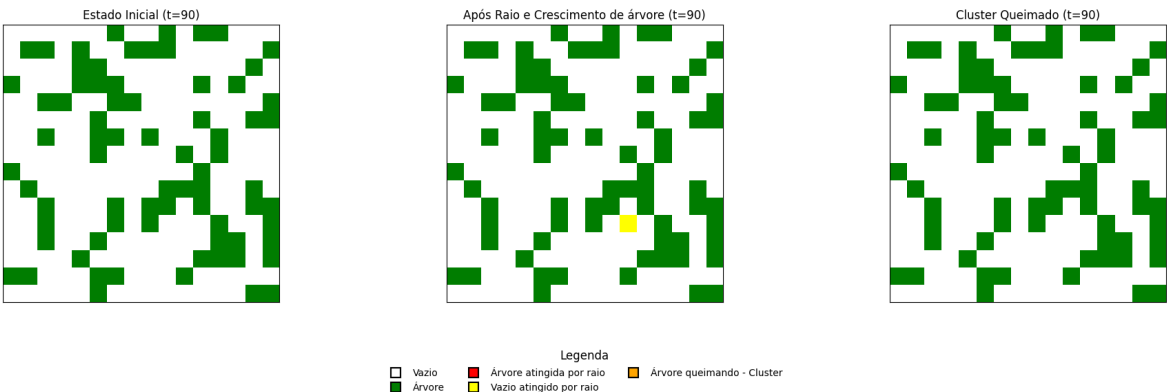
Floresta



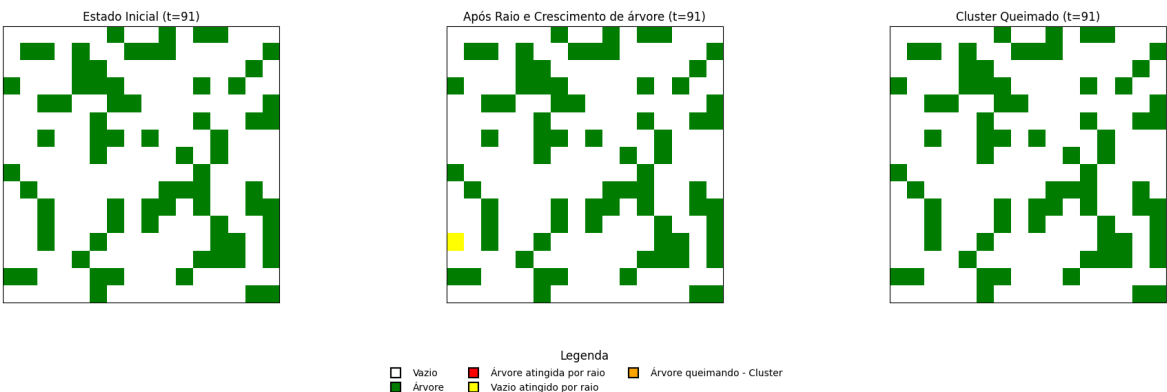
Floresta



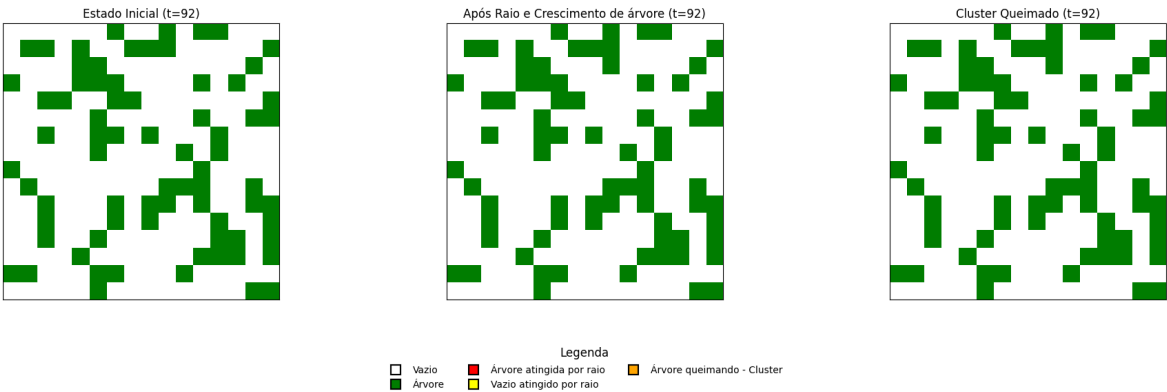
Floresta



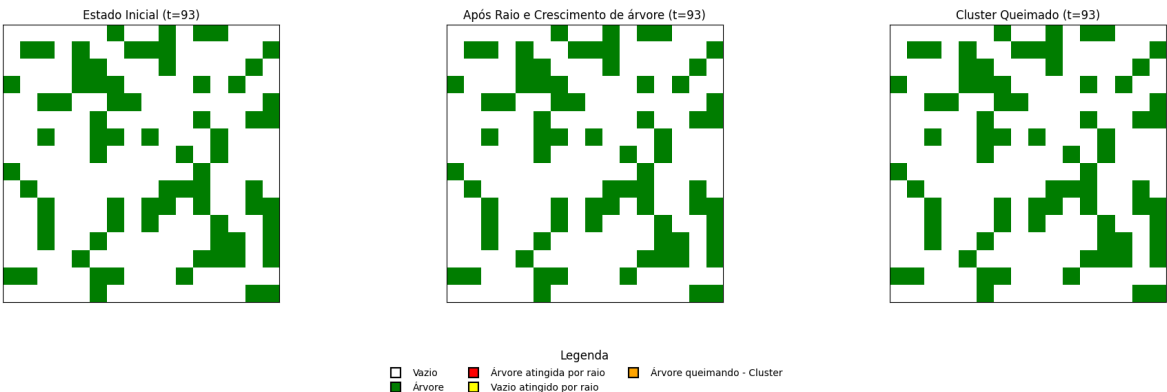
Floresta



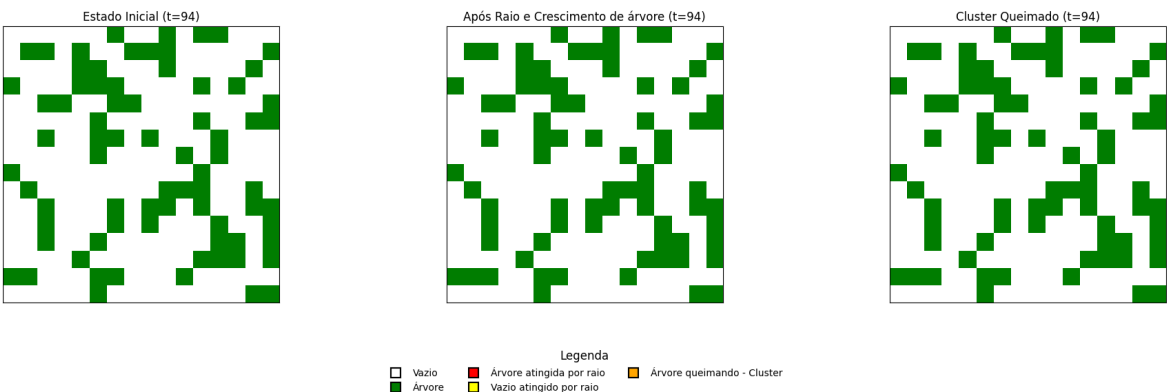
Floresta



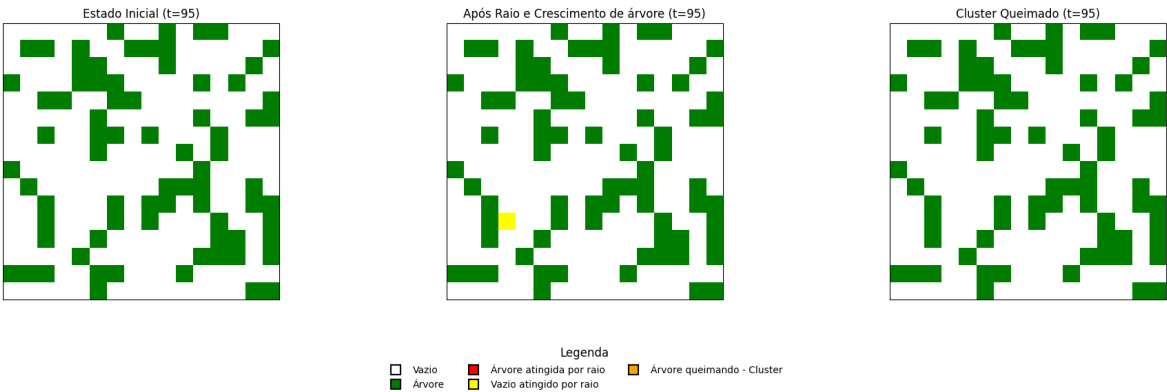
Floresta



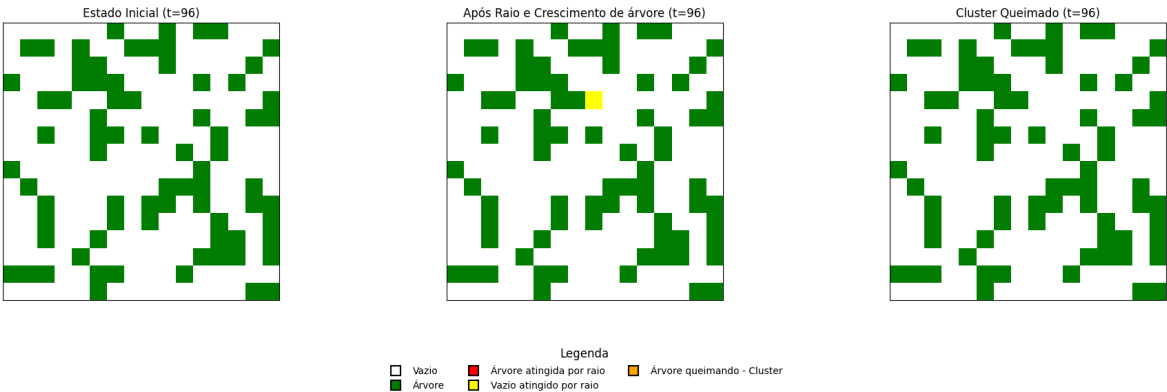
Floresta



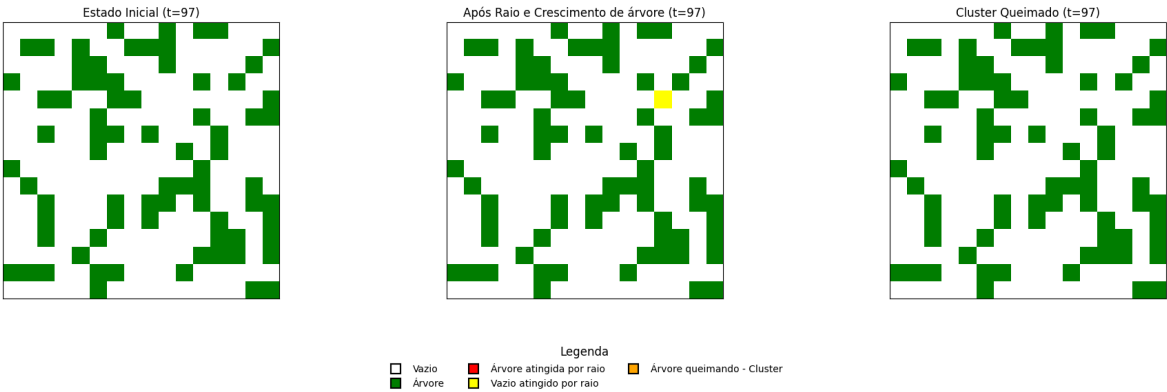
Floresta

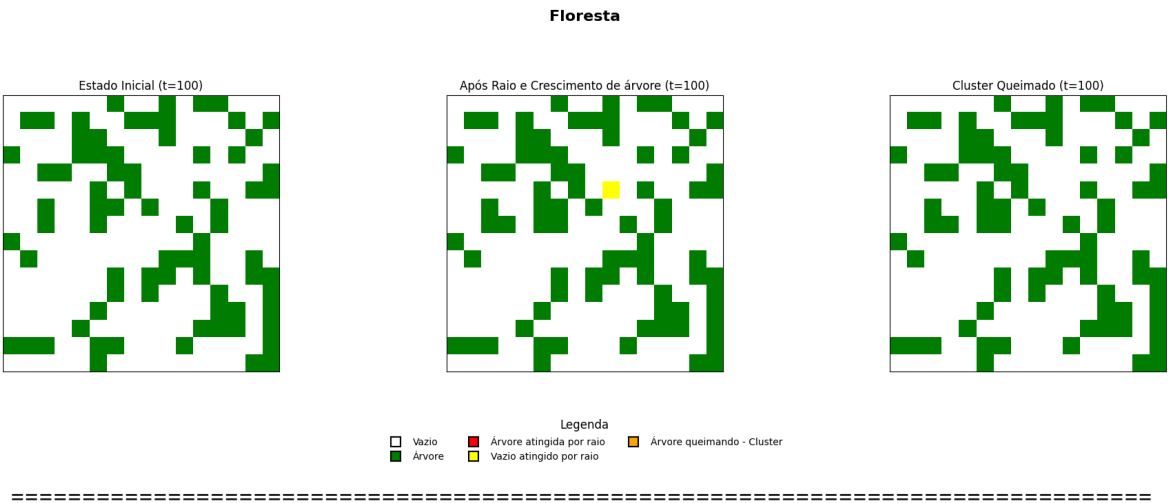
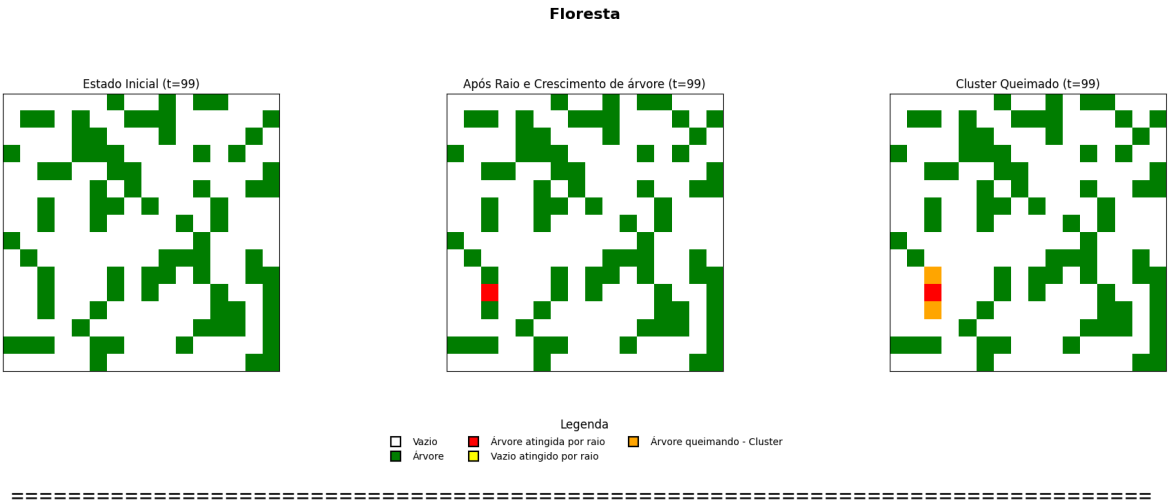


Floresta



Floresta





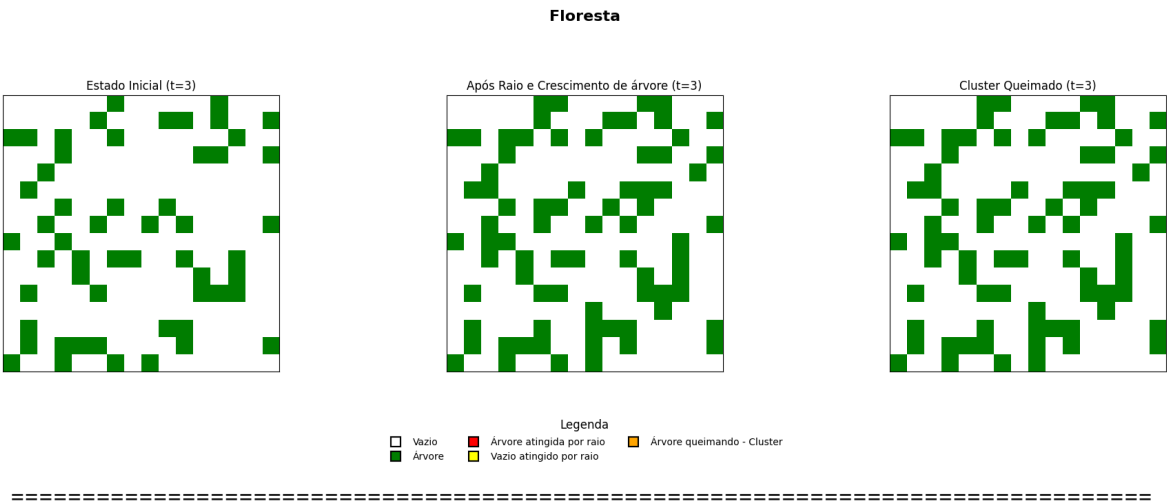
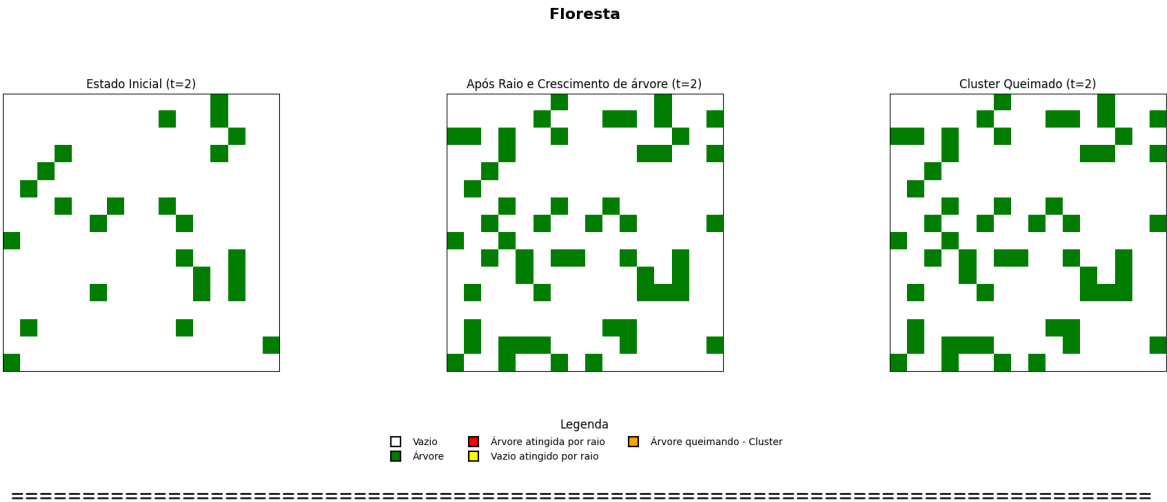
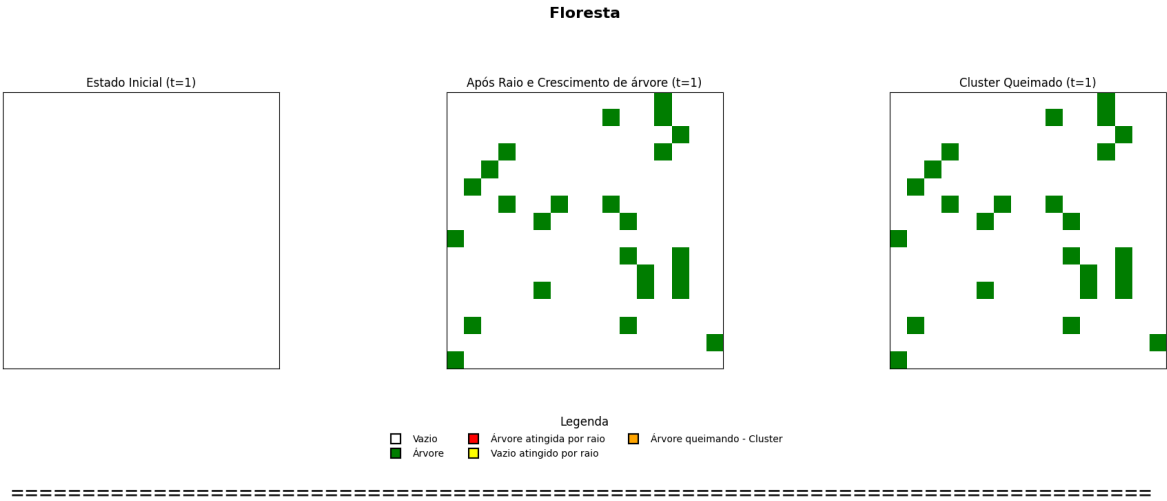
Incêndios grandes - Alto p e baixo f

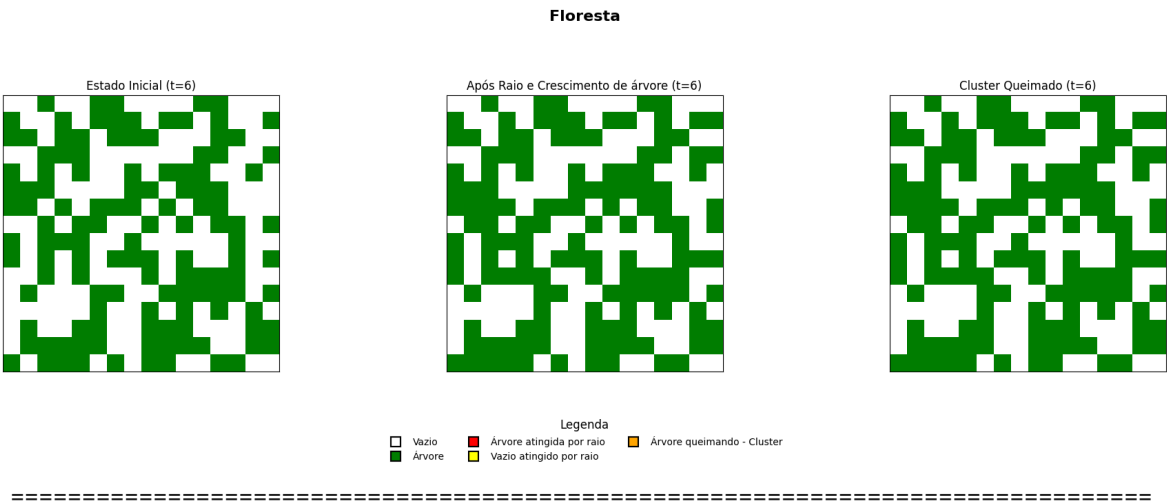
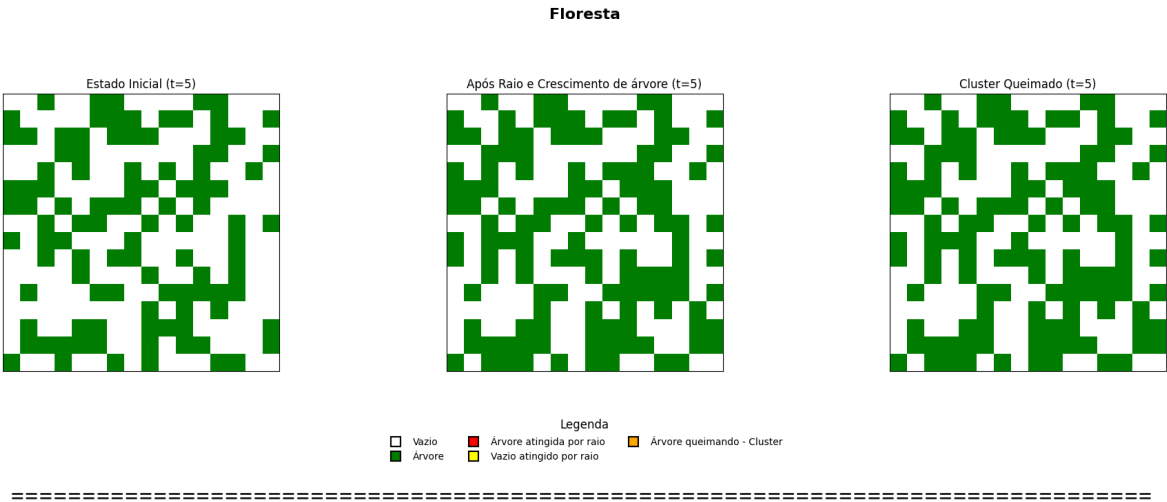
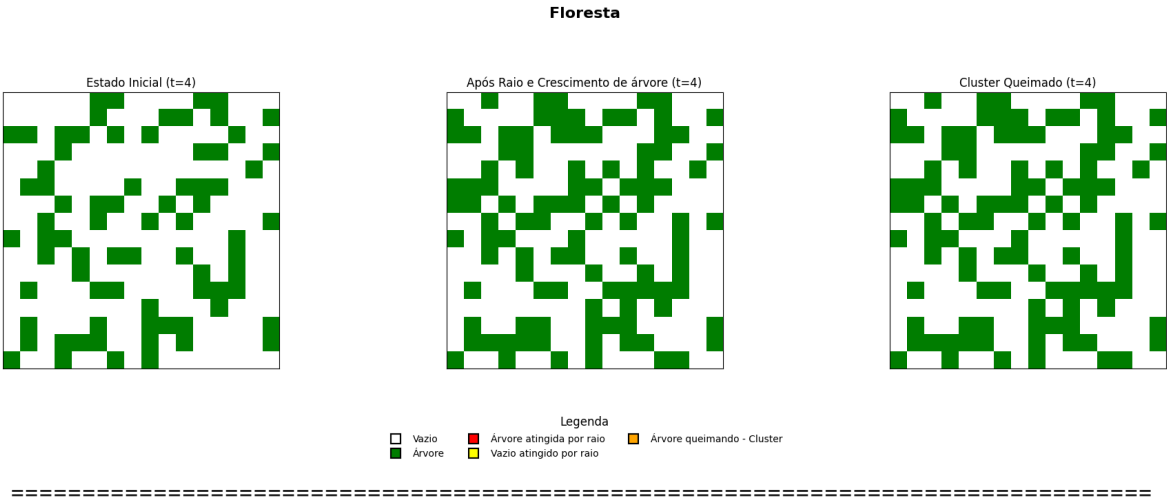
$p = 0.1$

$f = 0.1$

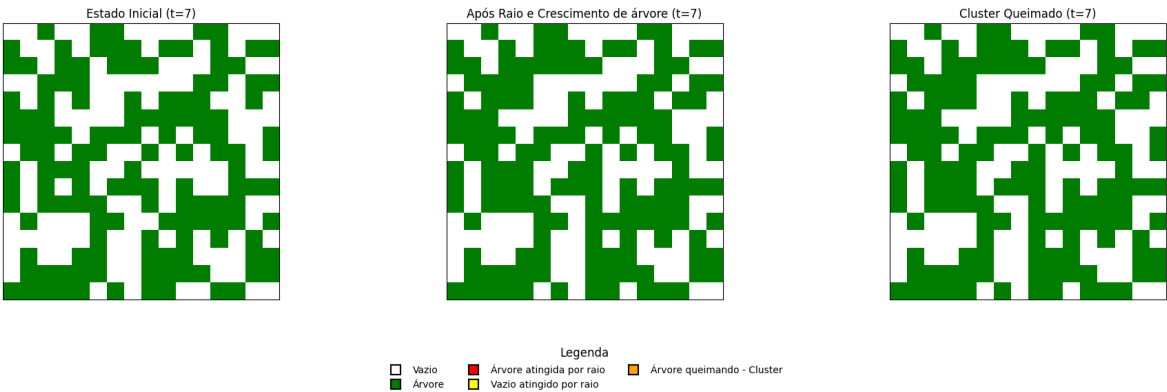
```
In [133... p = 0.1
           f = 0.1

In [134... _ = simula_floresta(L, p, f, iteracoes, vizualizar)
```





Floresta



Legenda

Vazio

Árvore

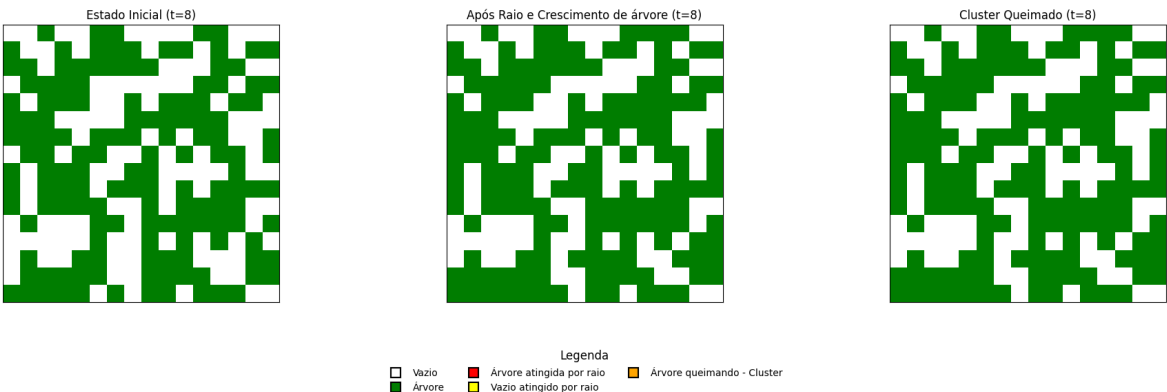
Árvore atingida por raio

Vazio atingido por raio

Árvore queimando - Cluster

=====

Floresta



Legenda

Vazio

Árvore

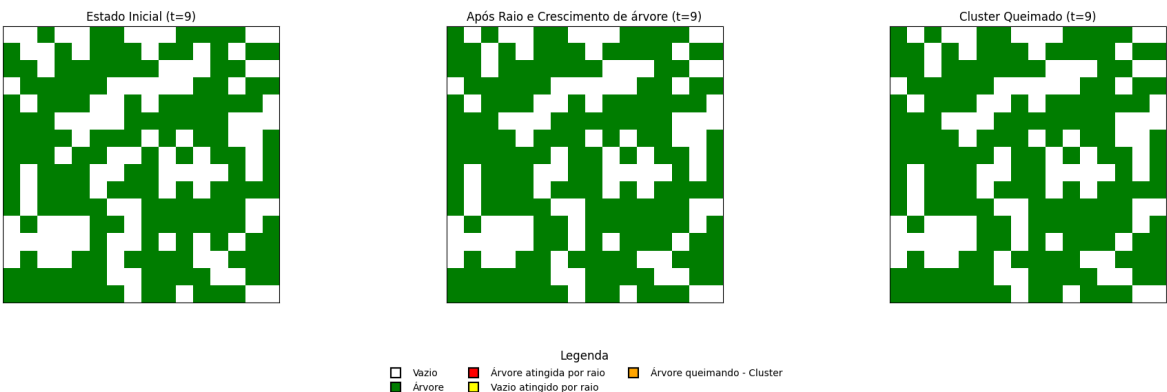
Árvore atingida por raio

Vazio atingido por raio

Árvore queimando - Cluster

=====

Floresta



Legenda

Vazio

Árvore

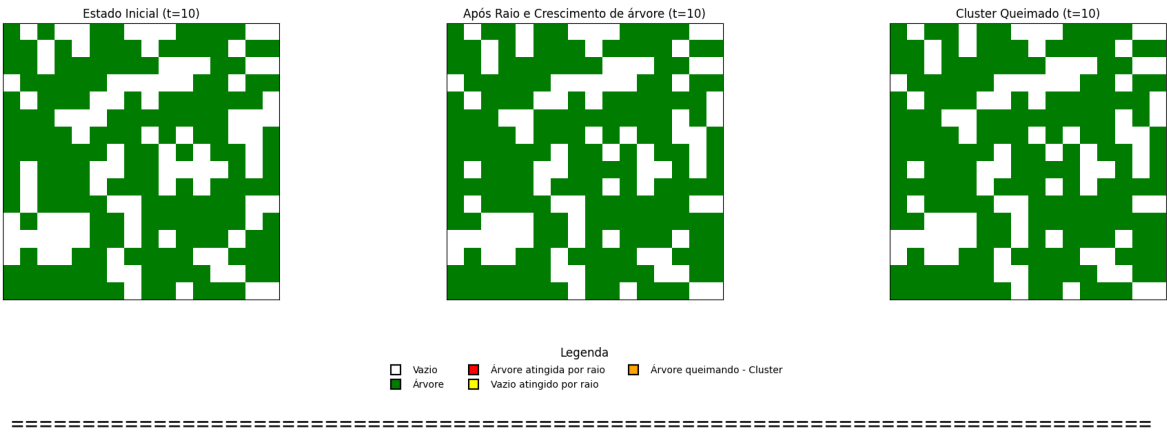
Árvore atingida por raio

Vazio atingido por raio

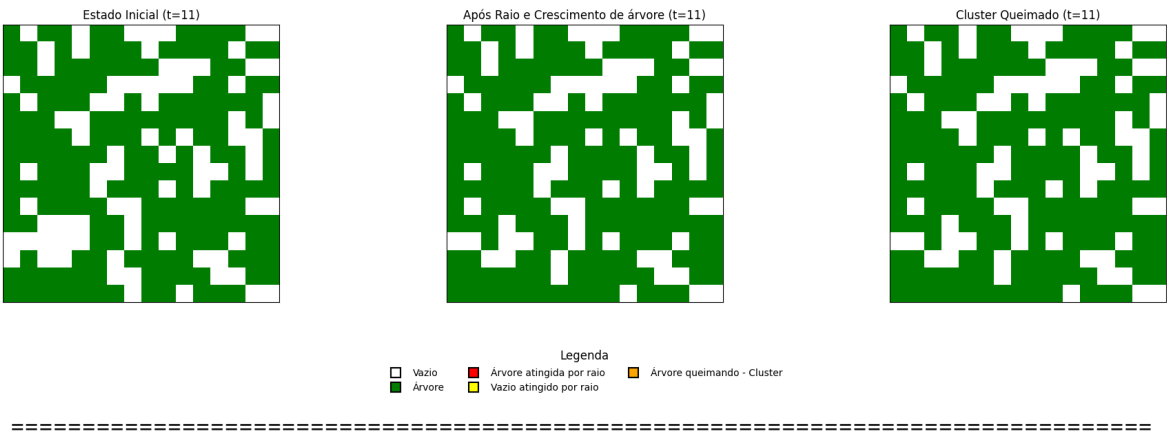
Árvore queimando - Cluster

=====

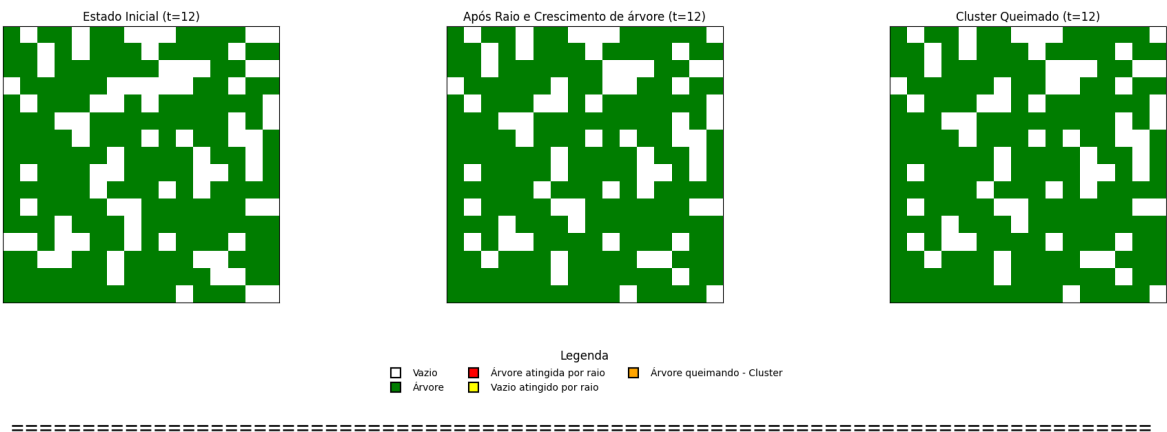
Floresta

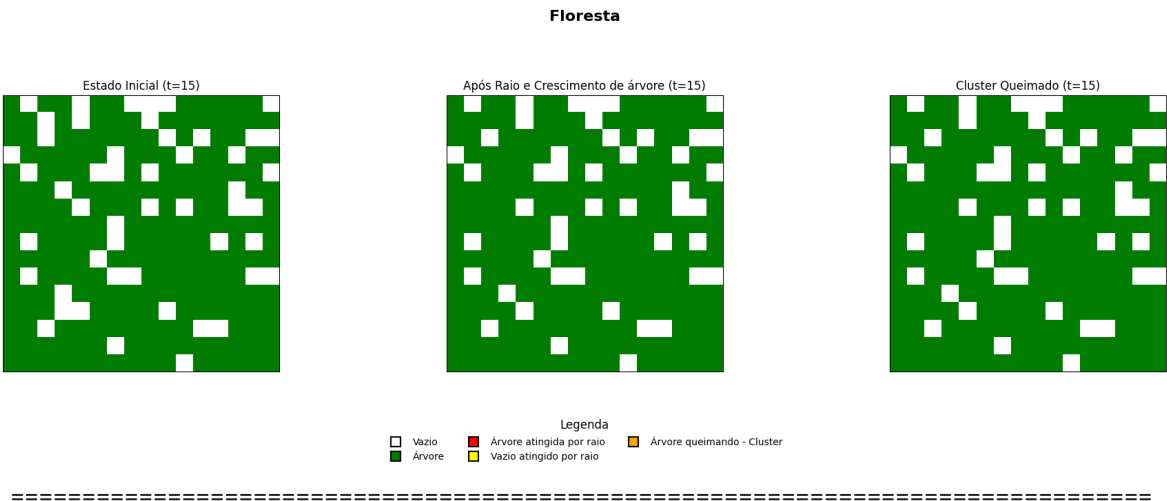
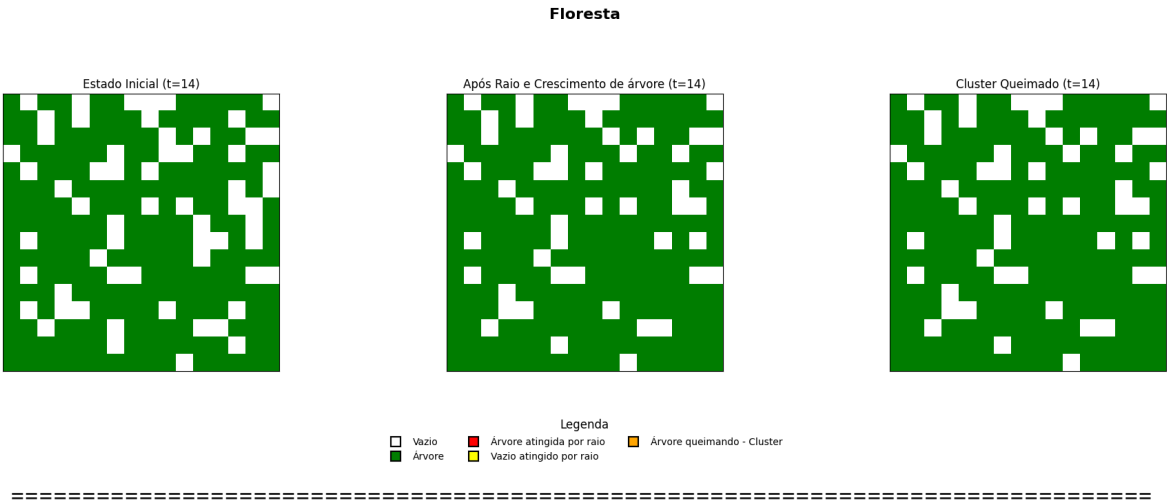
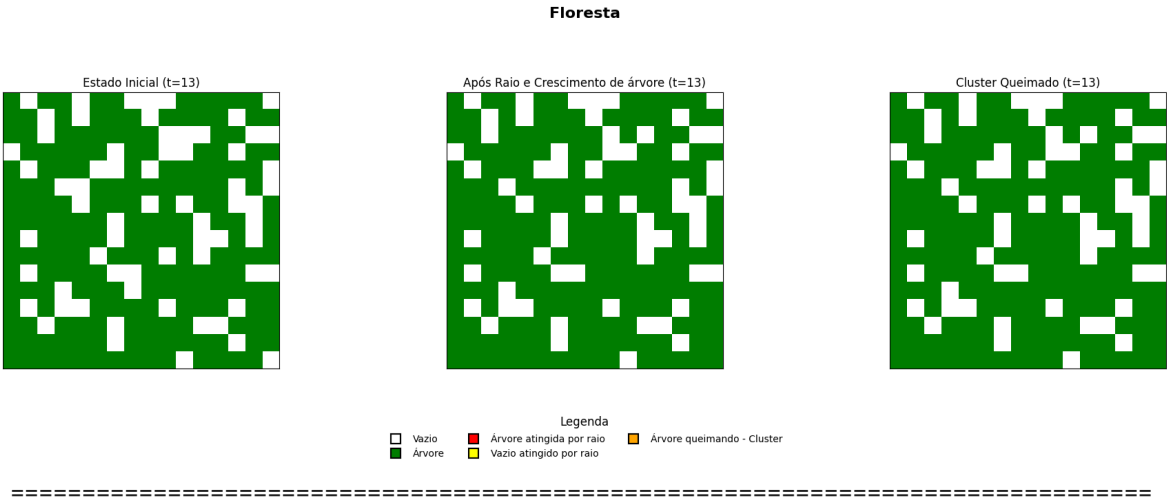


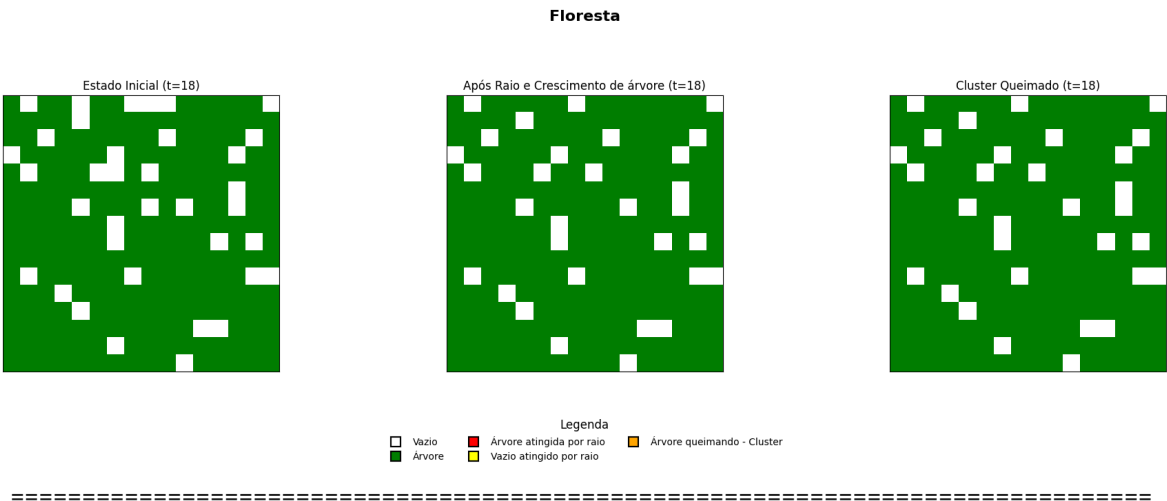
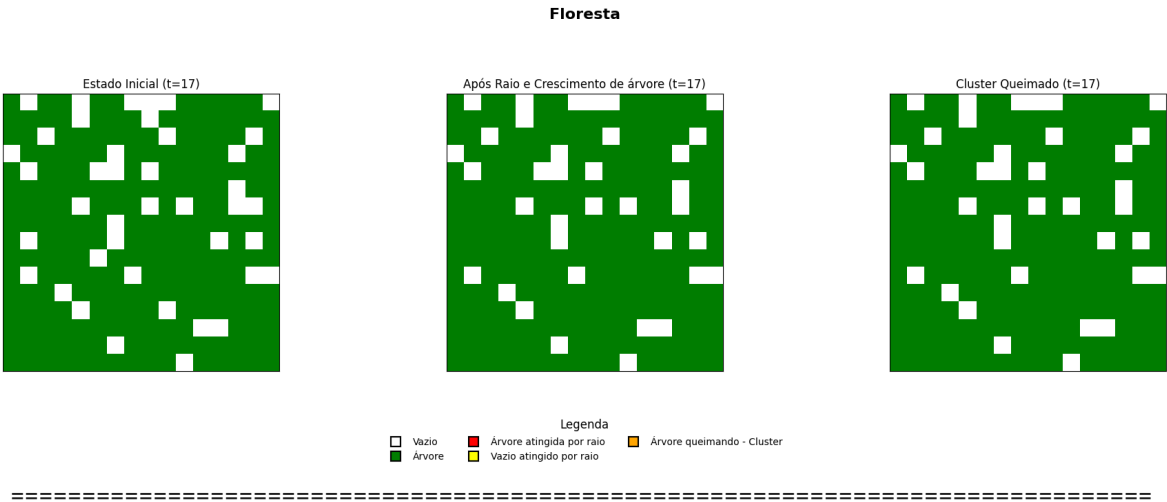
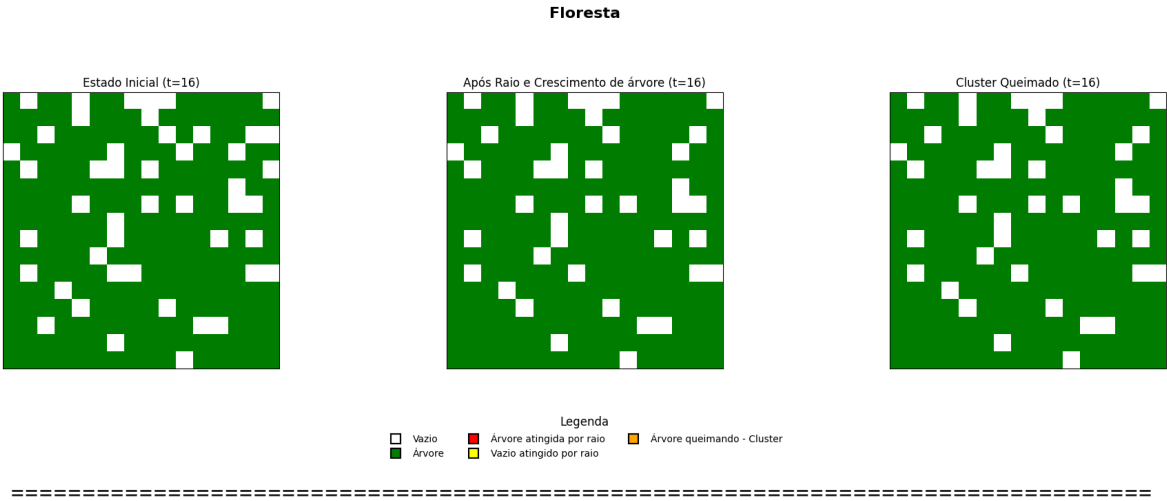
Floresta



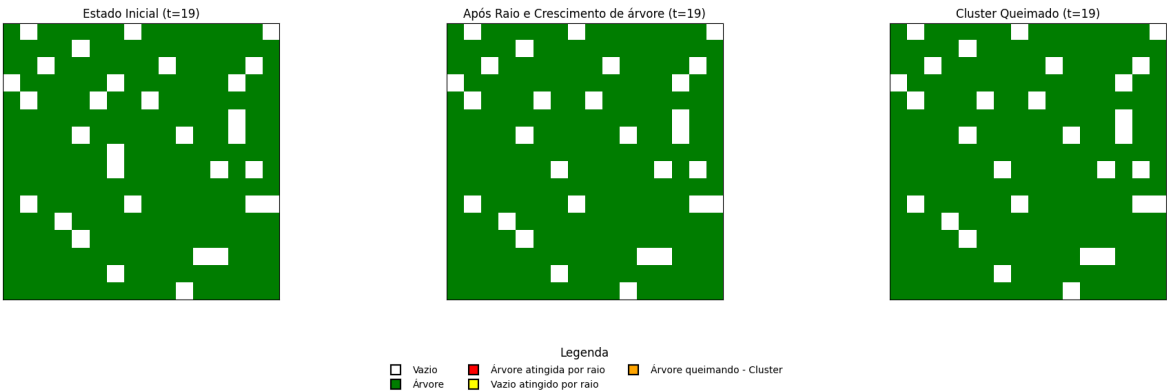
Floresta



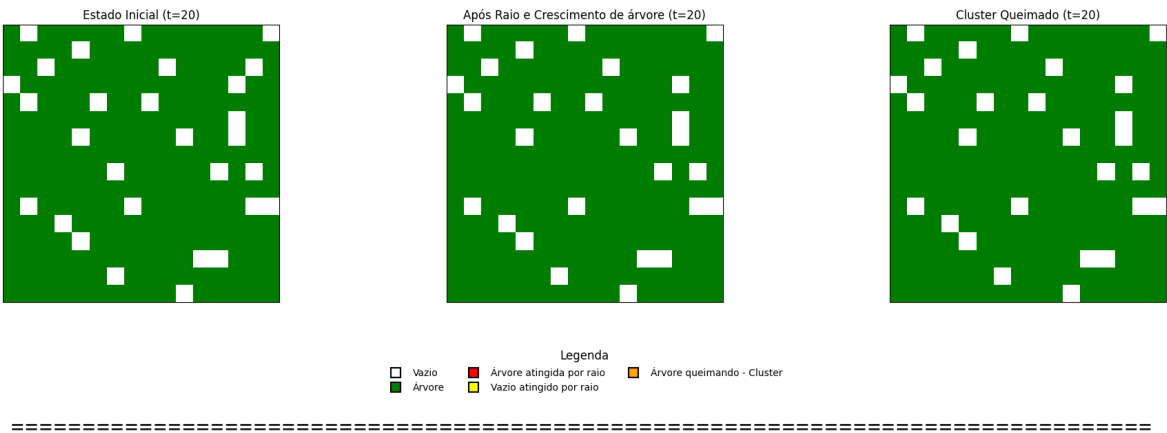




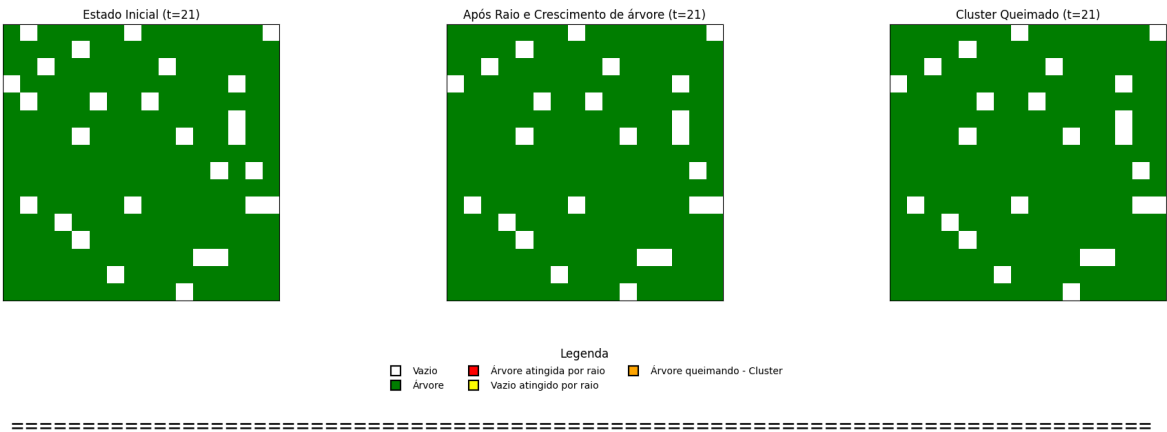
Floresta



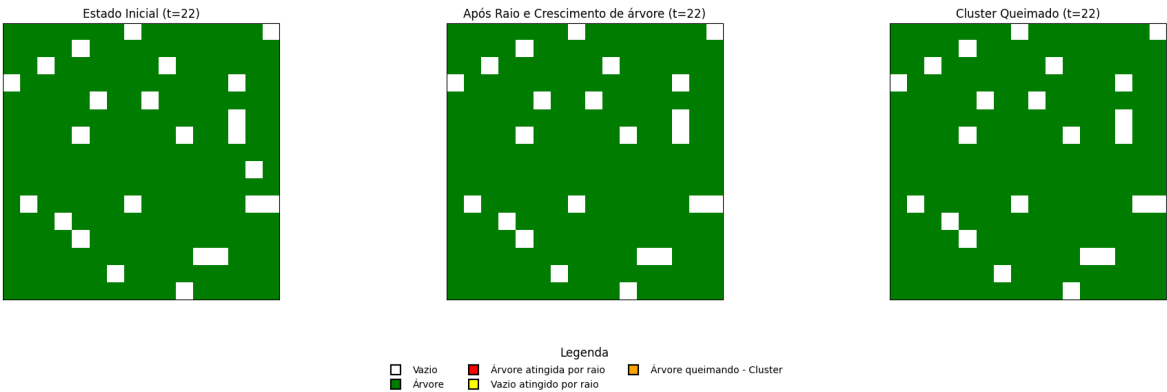
Floresta



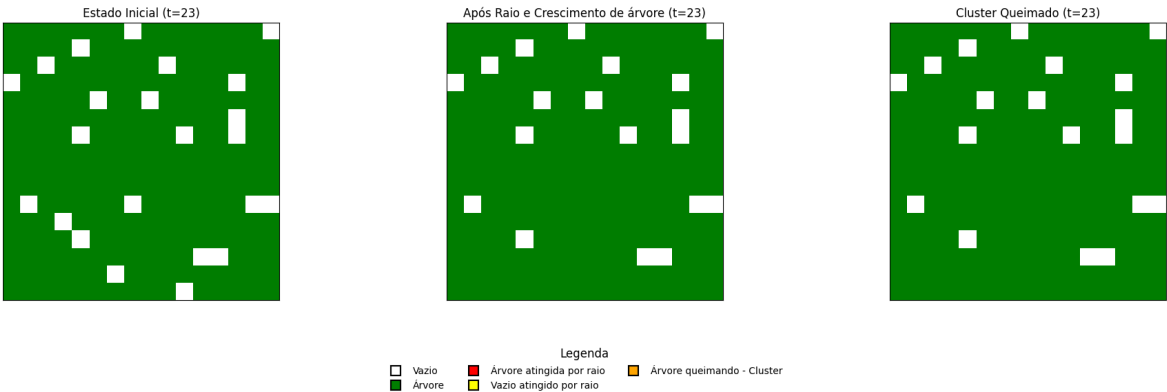
Floresta



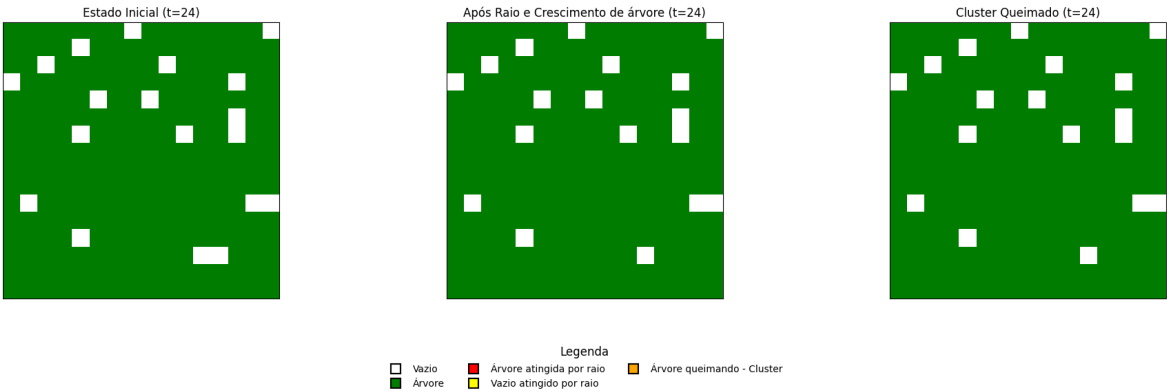
Floresta



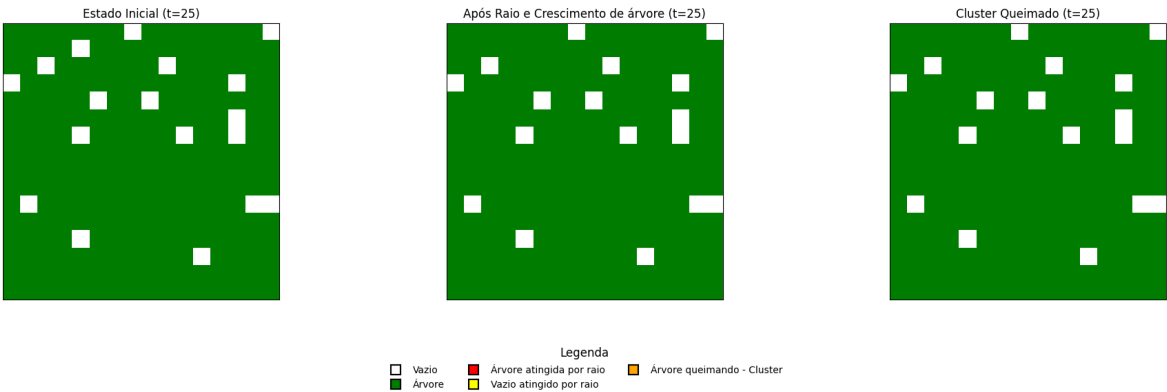
Floresta



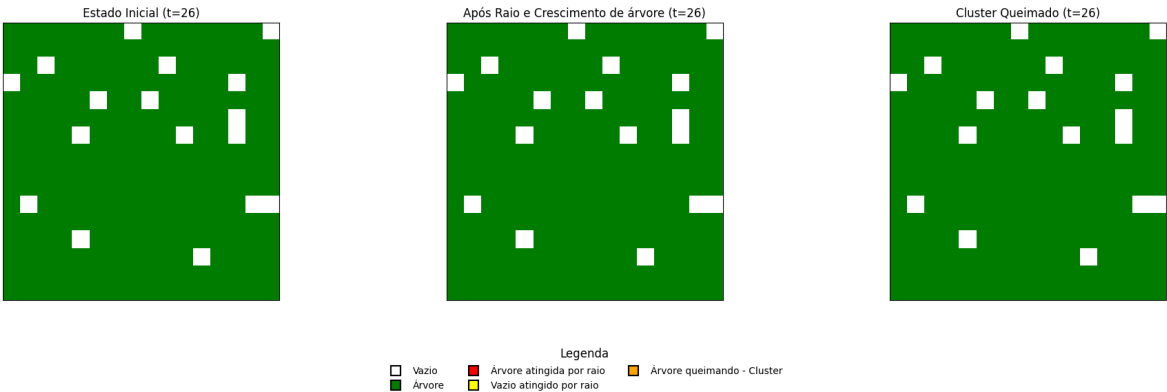
Floresta



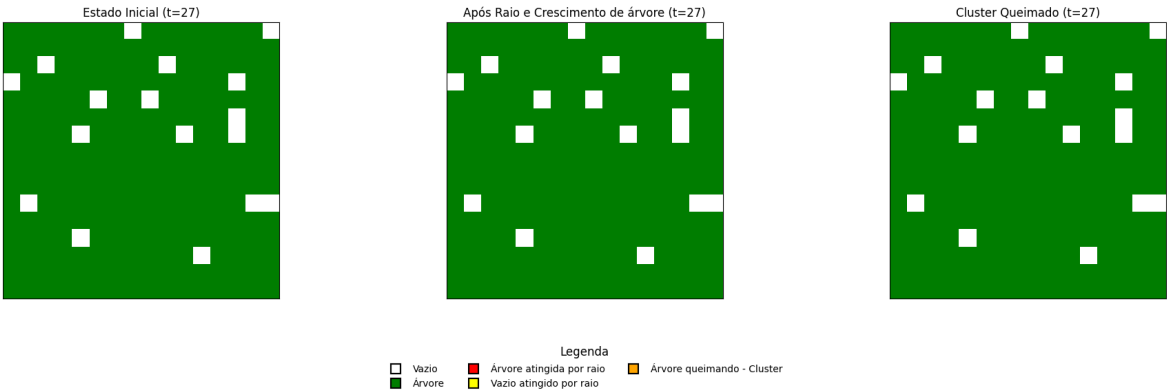
Floresta



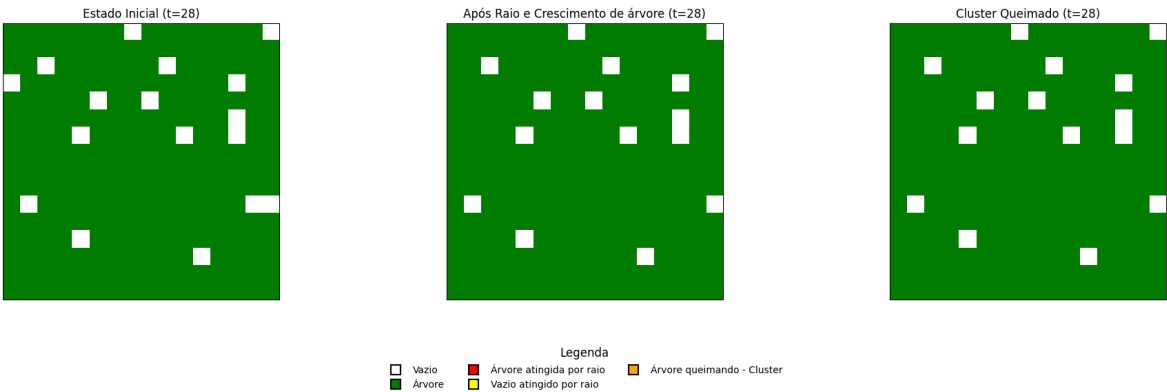
Floresta



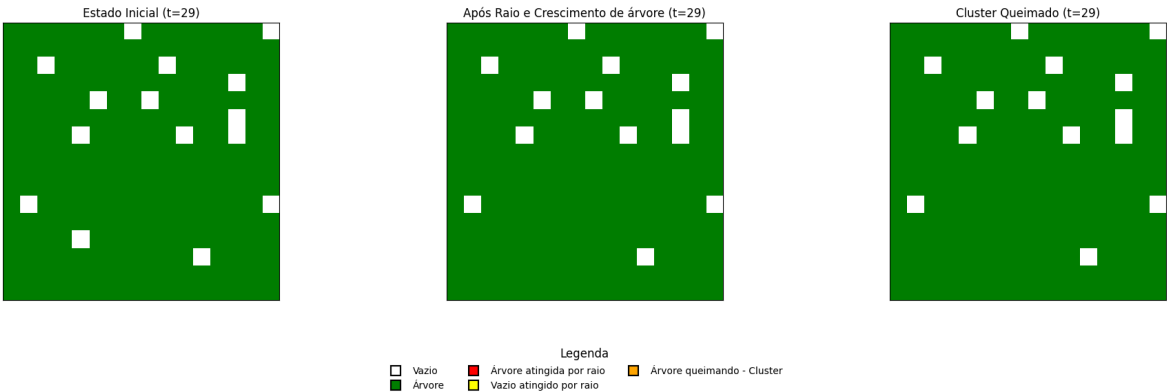
Floresta



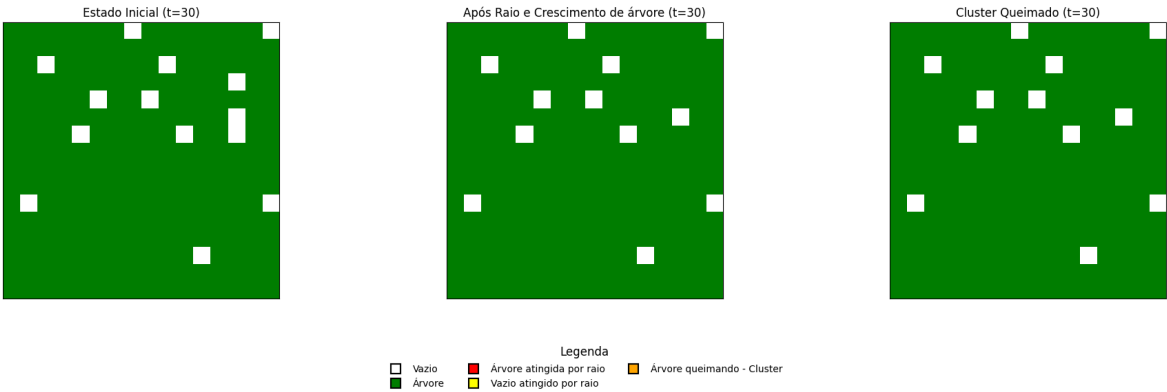
Floresta



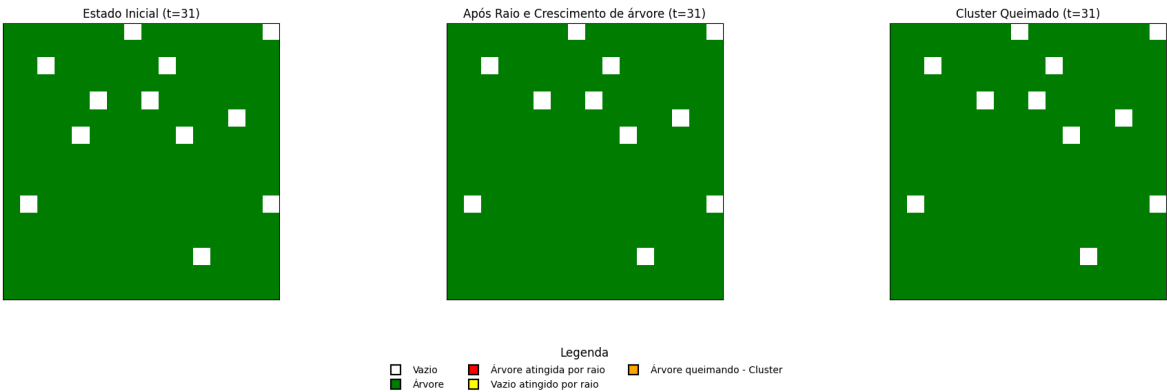
Floresta



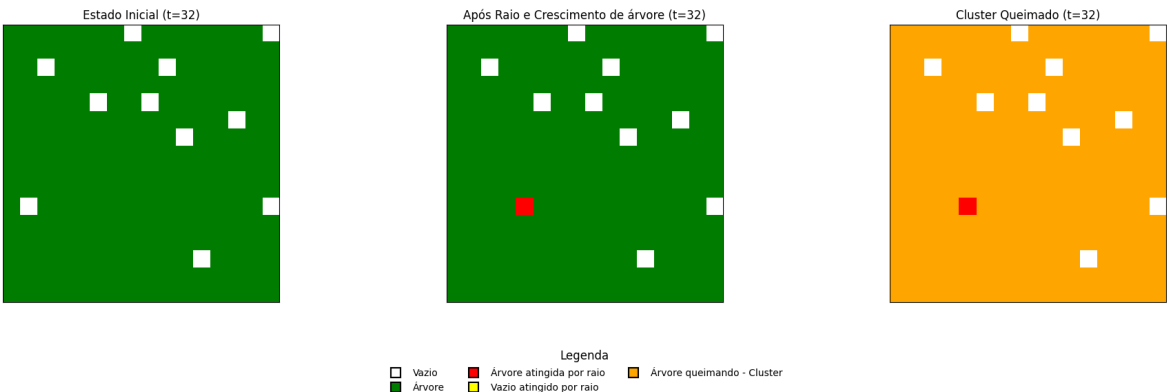
Floresta



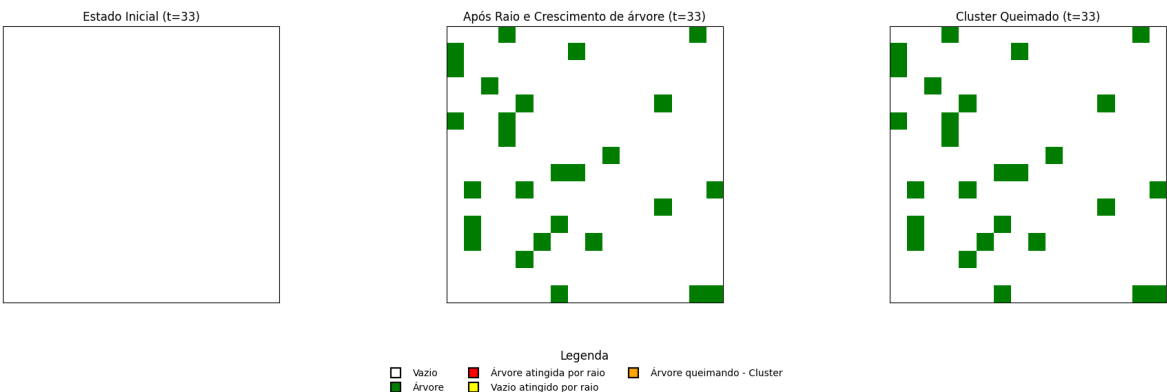
Floresta

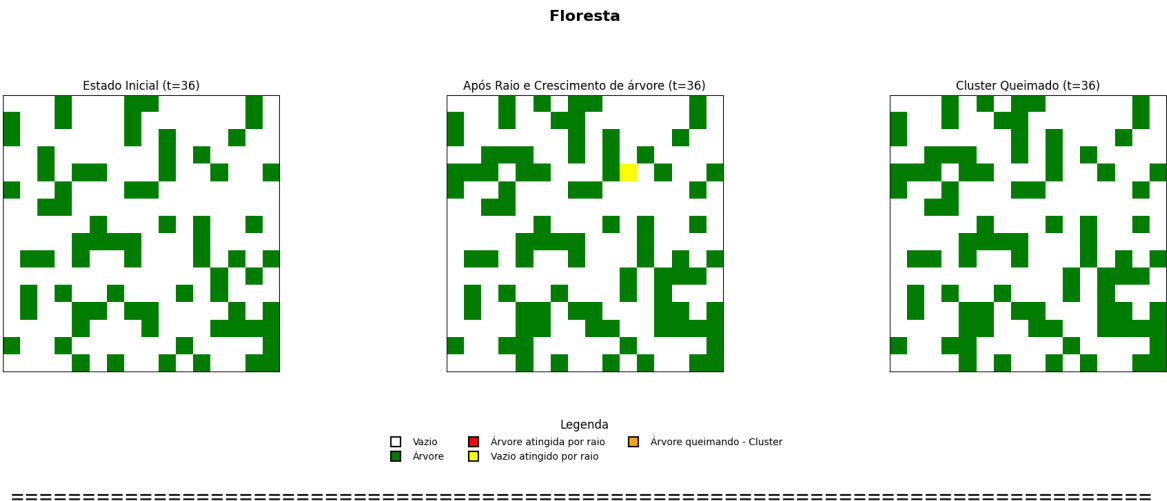
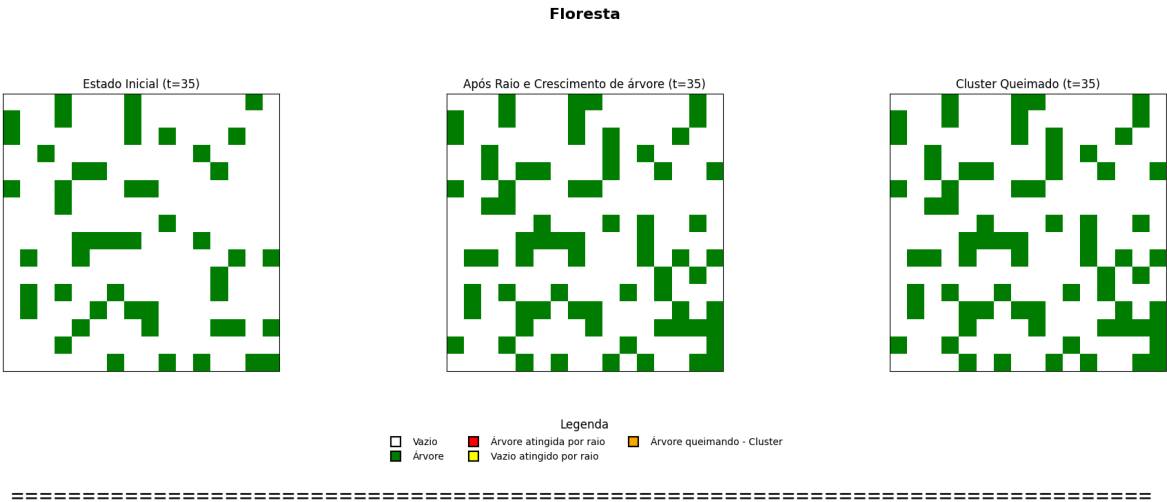
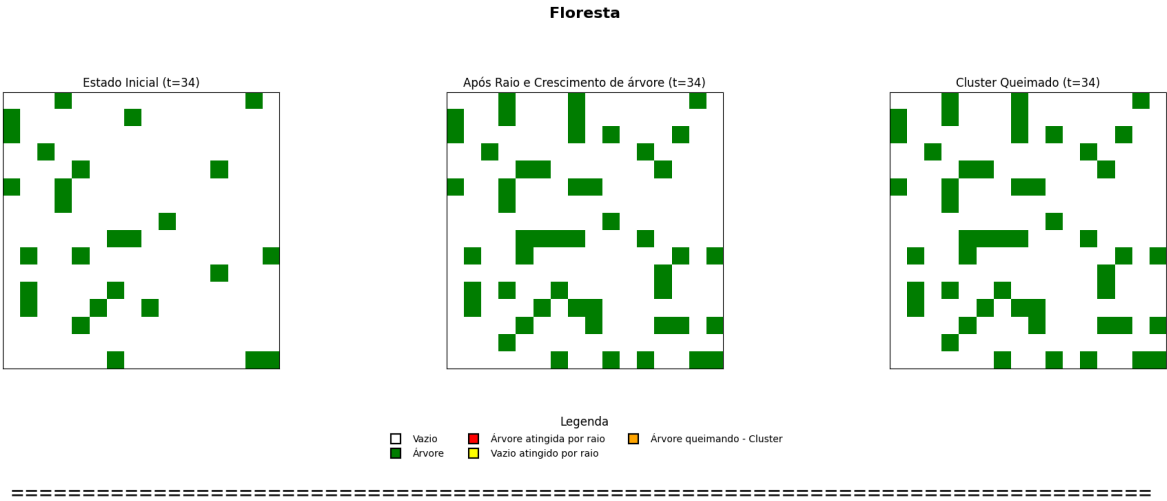


Floresta

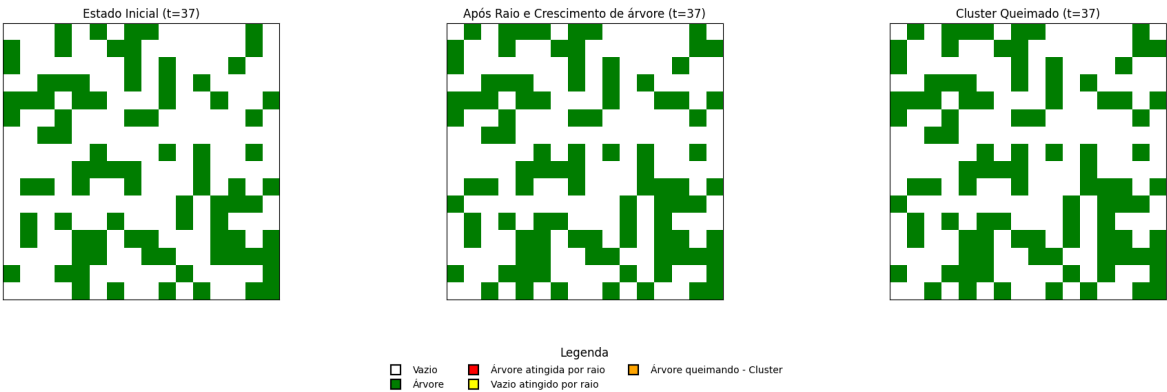


Floresta

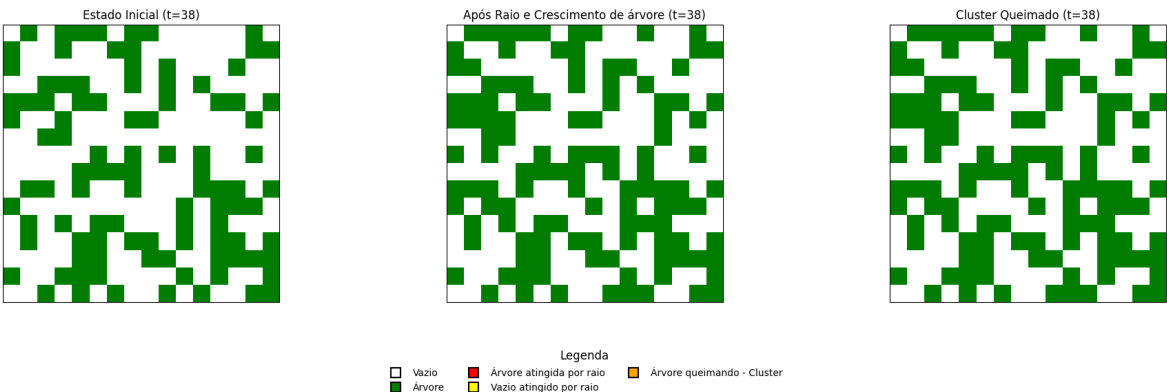




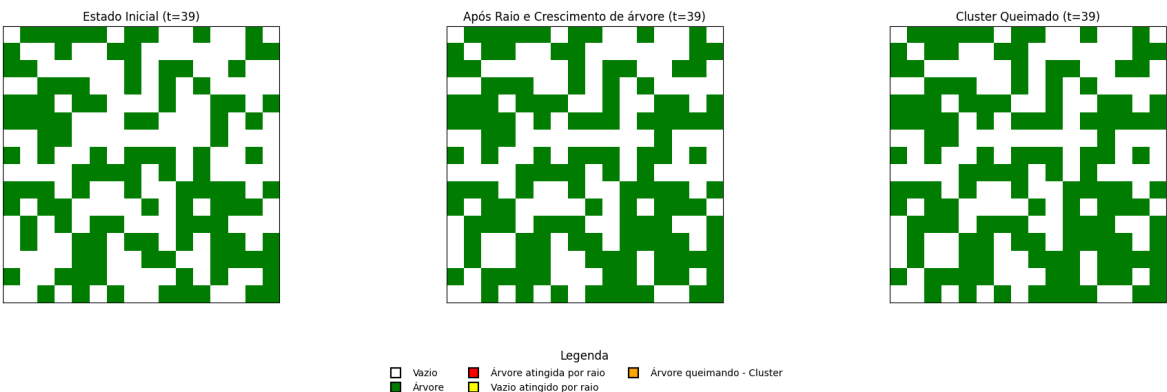
Floresta



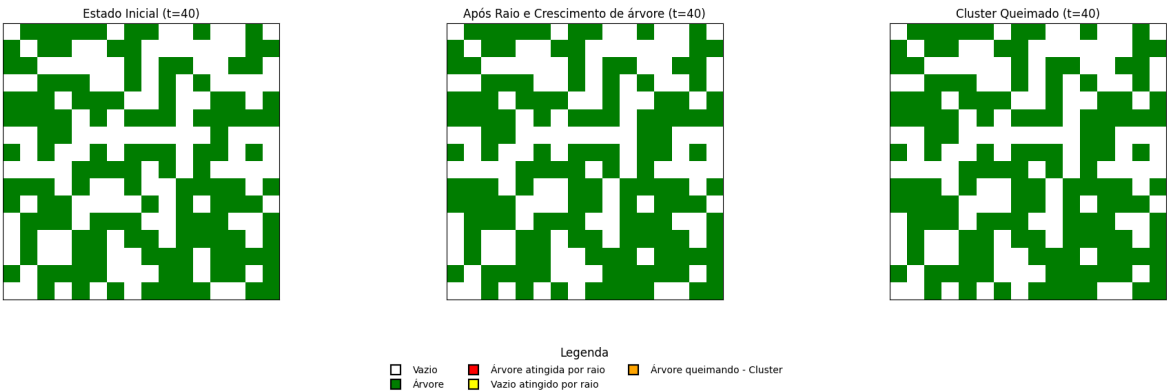
Floresta



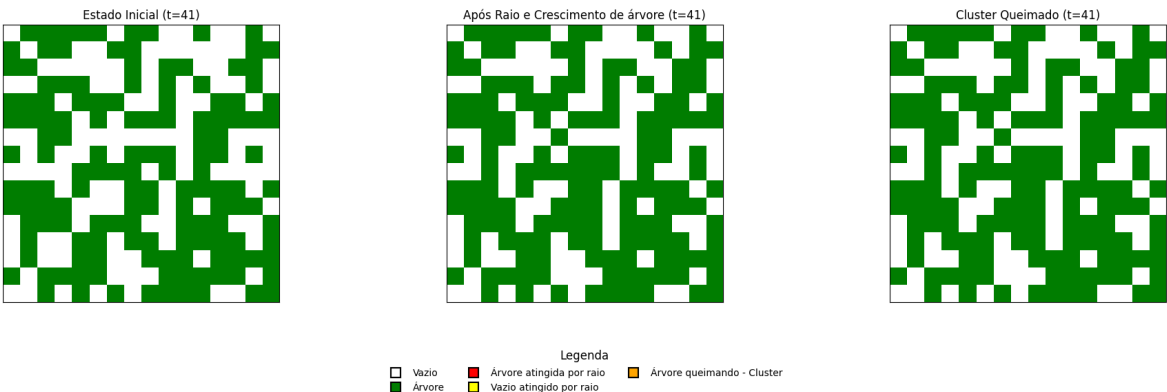
Floresta



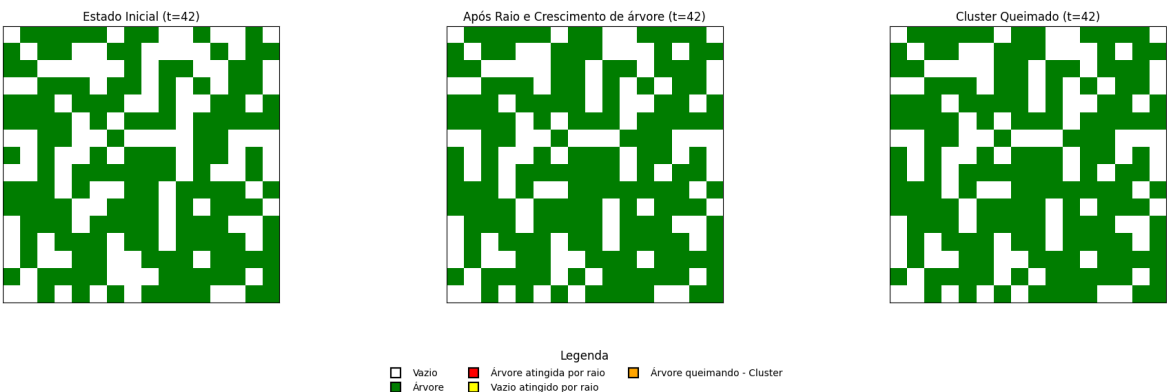
Floresta



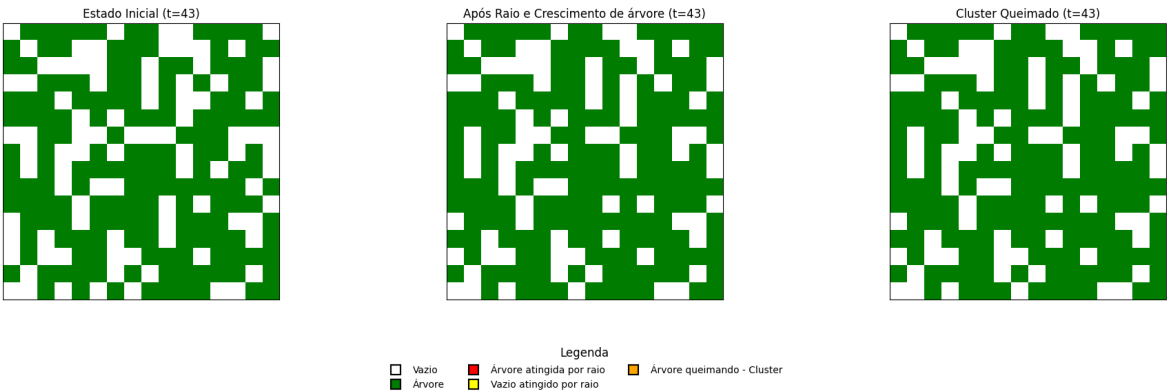
Floresta



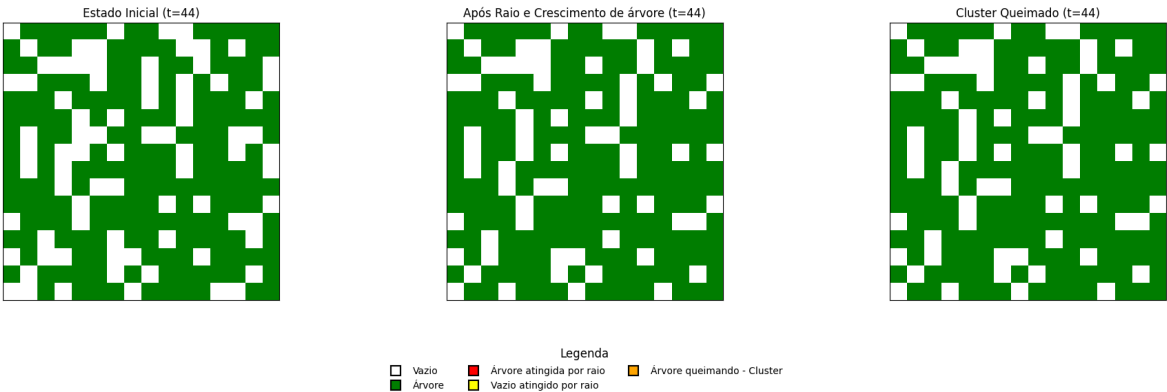
Floresta



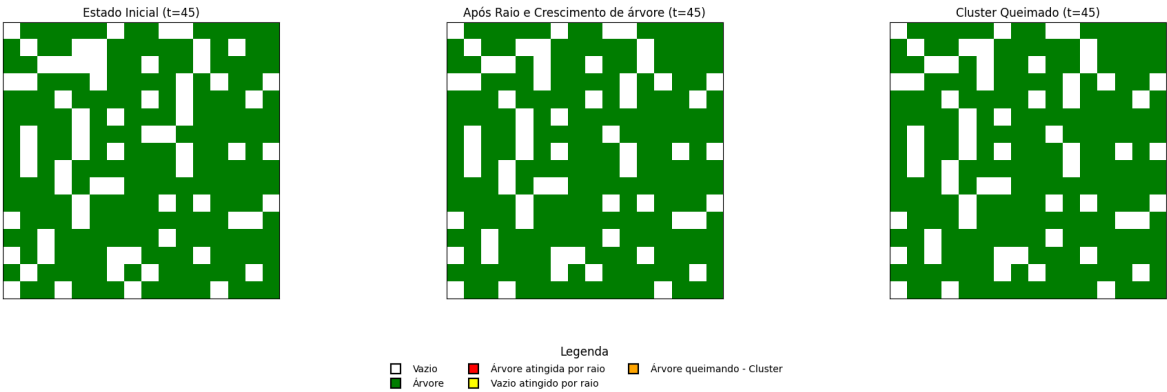
Floresta



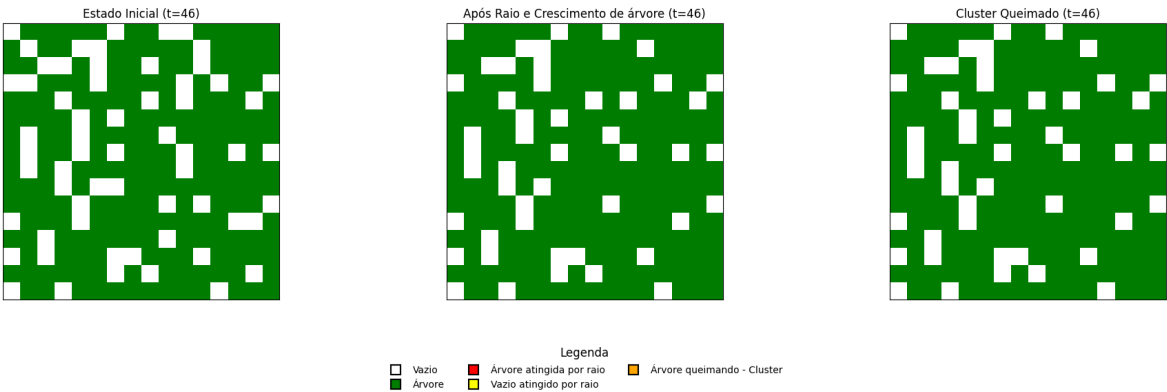
Floresta



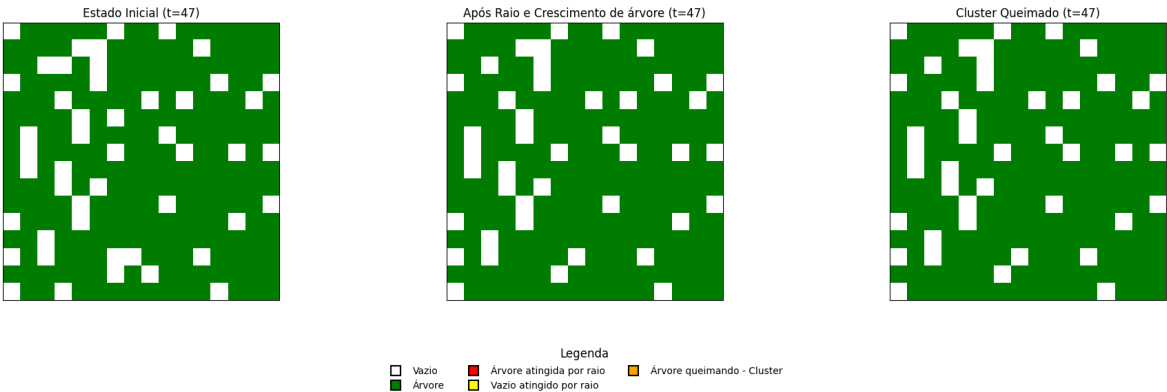
Floresta



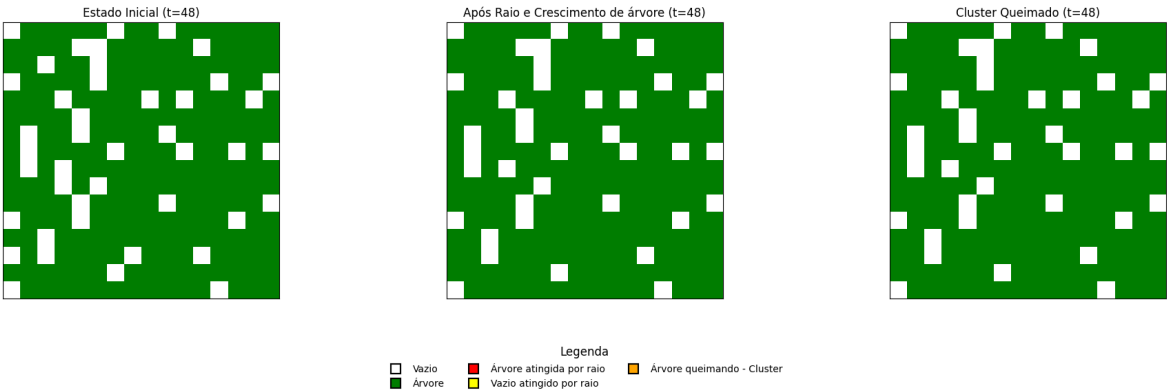
Floresta



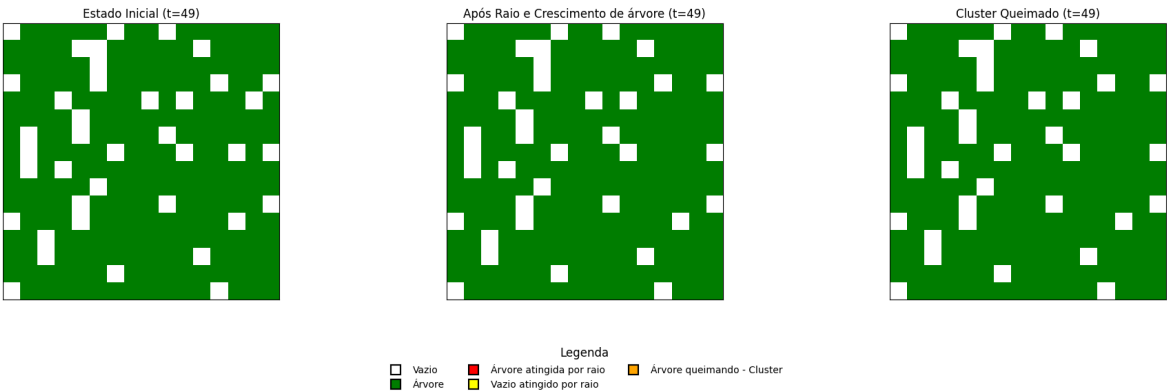
Floresta



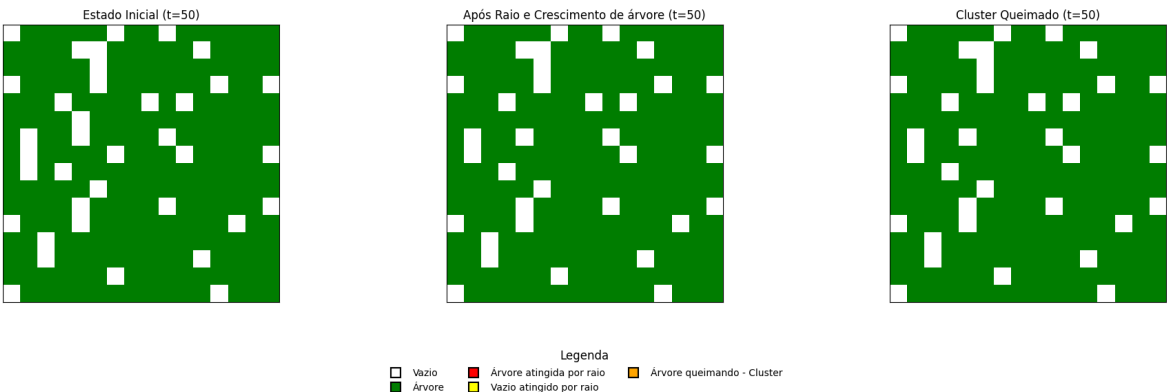
Floresta



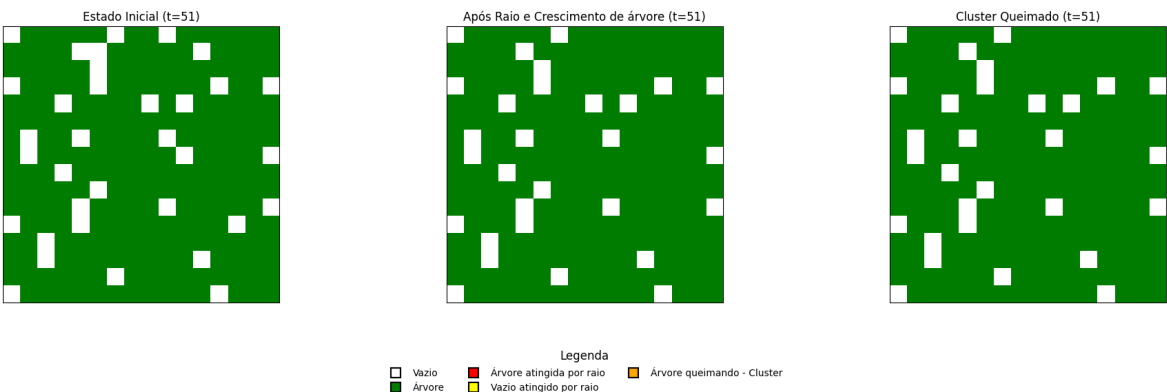
Floresta



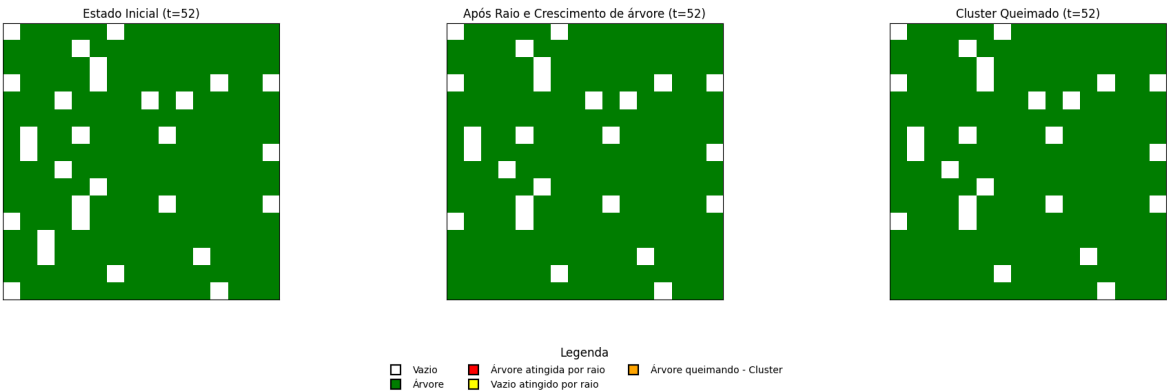
Floresta



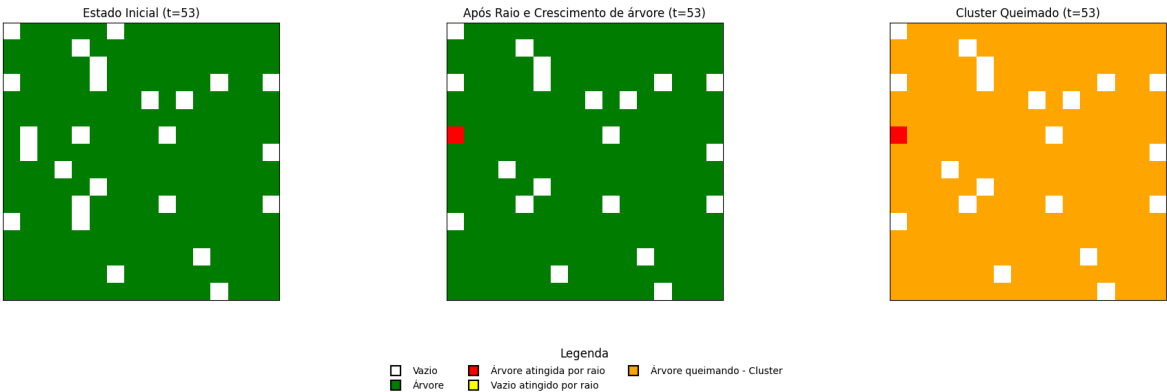
Floresta



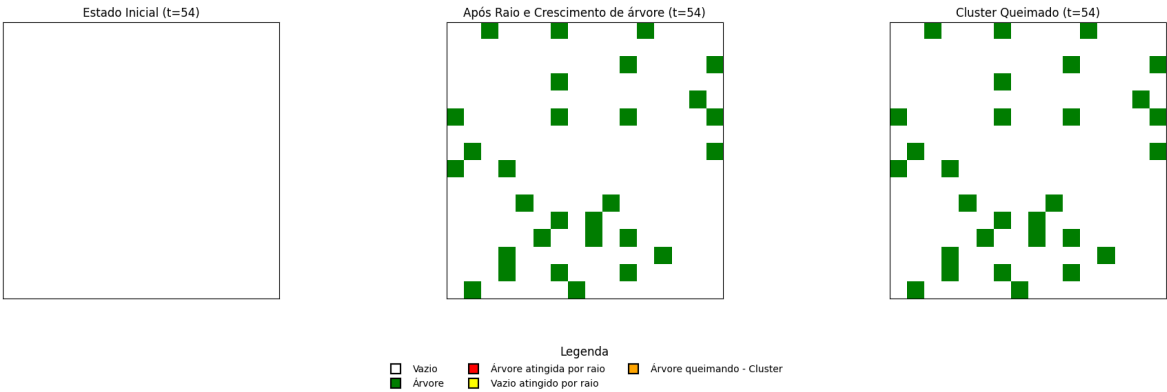
Floresta

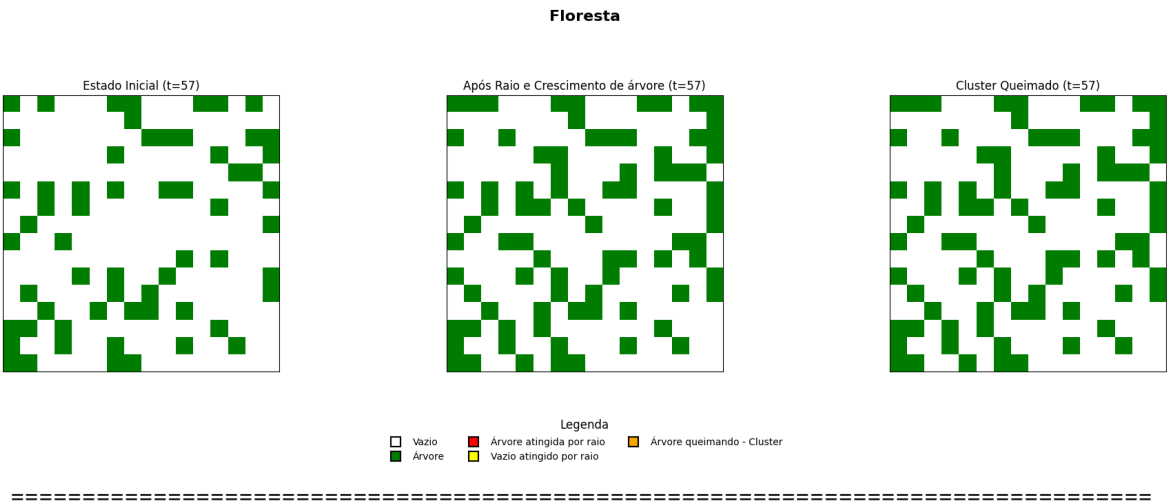
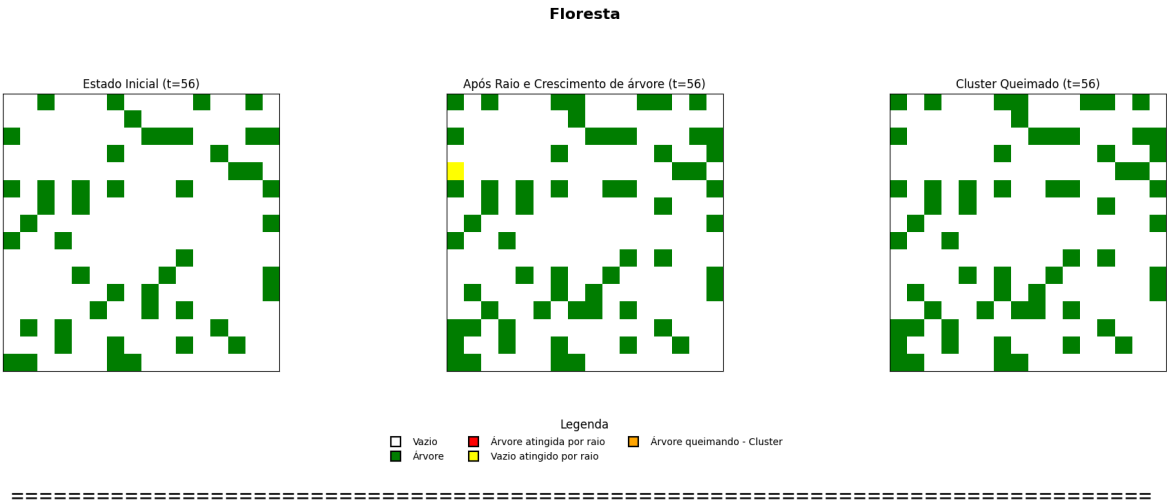
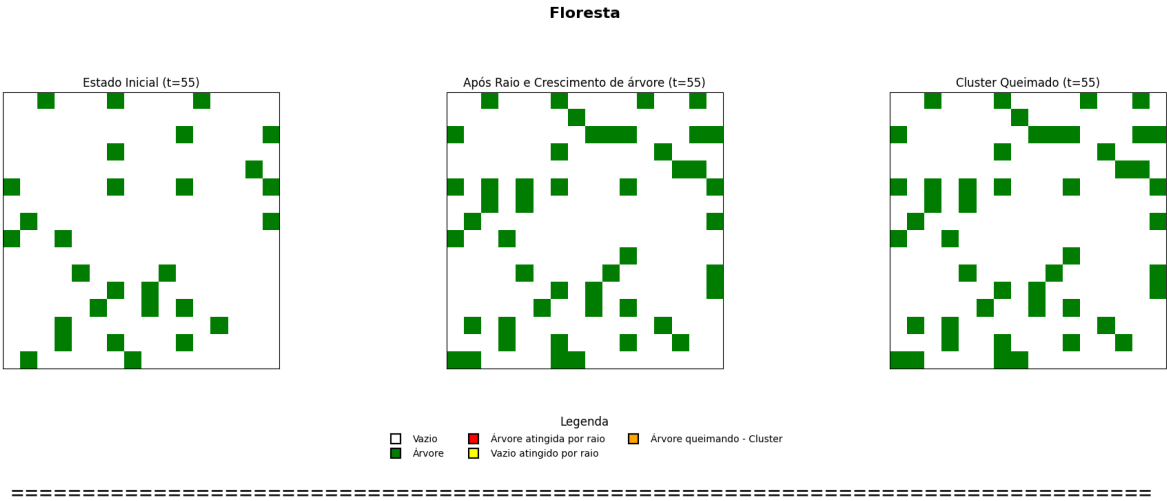


Floresta

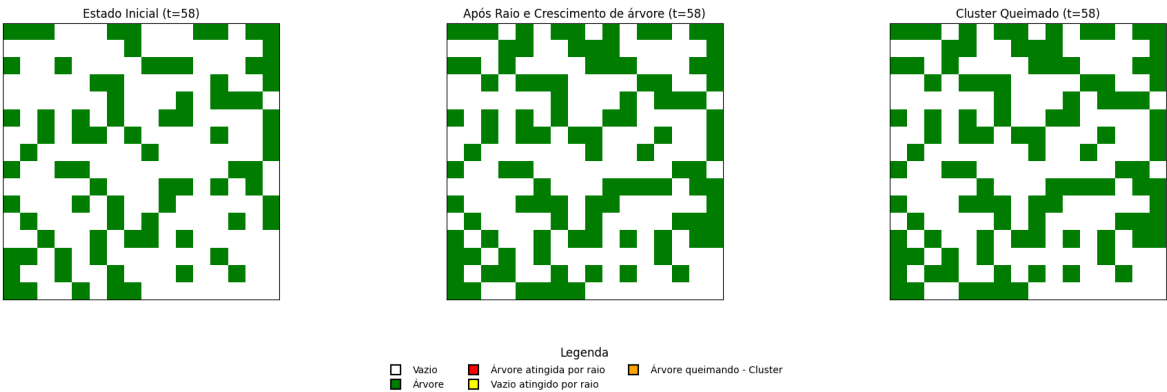


Floresta

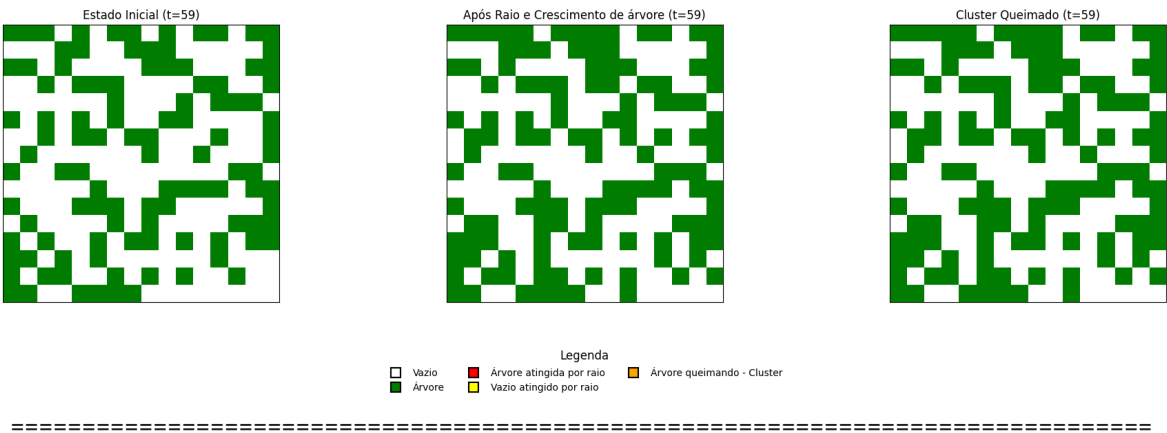




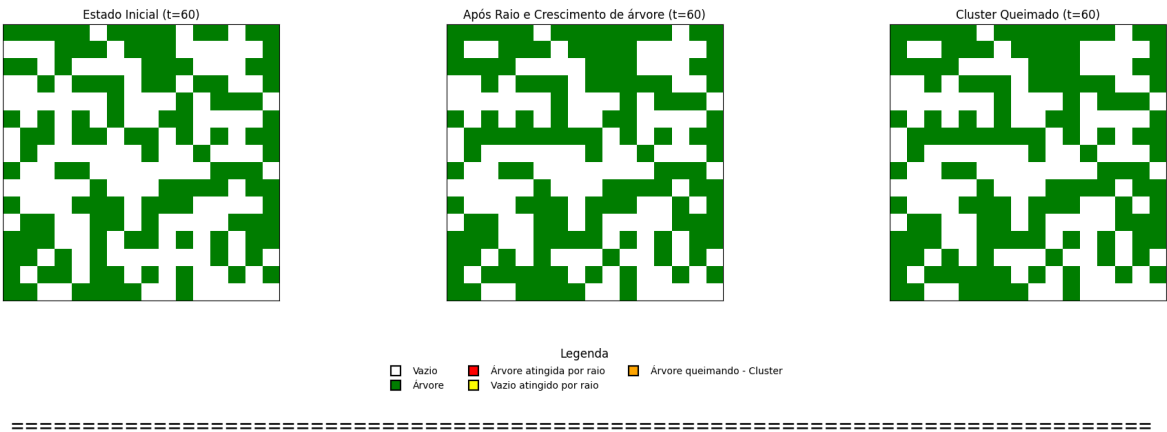
Floresta



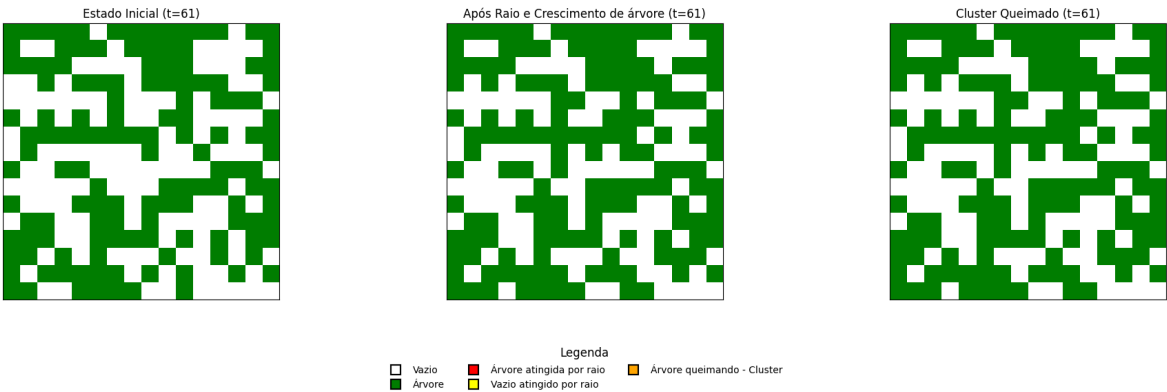
Floresta



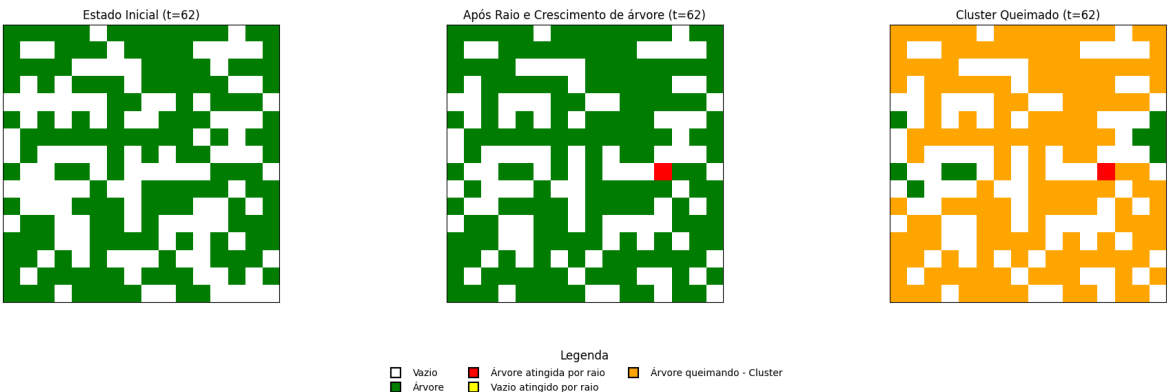
Floresta



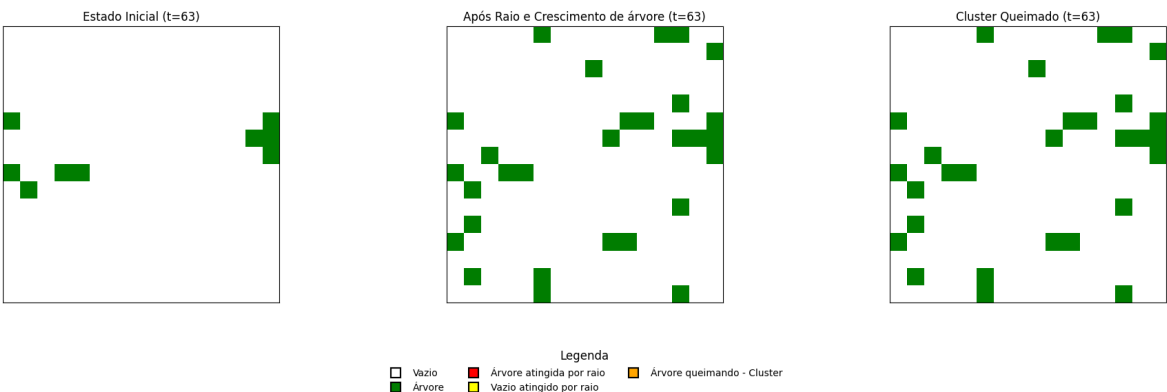
Floresta



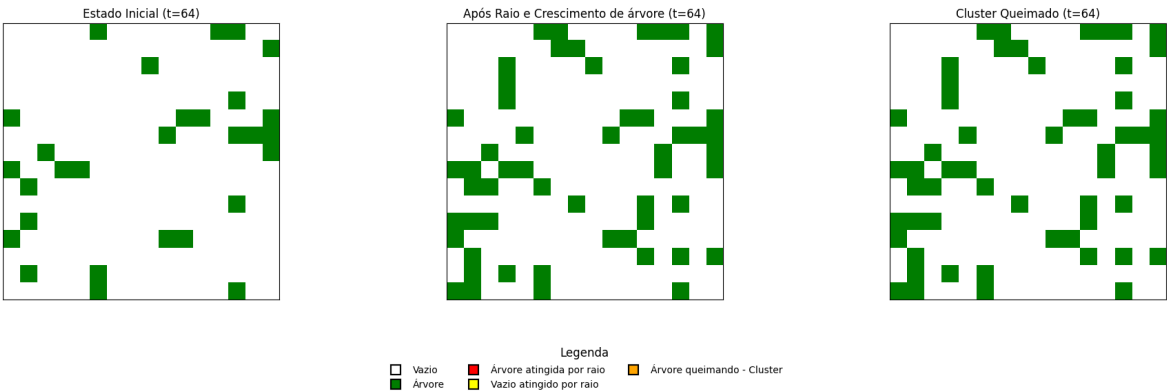
Floresta



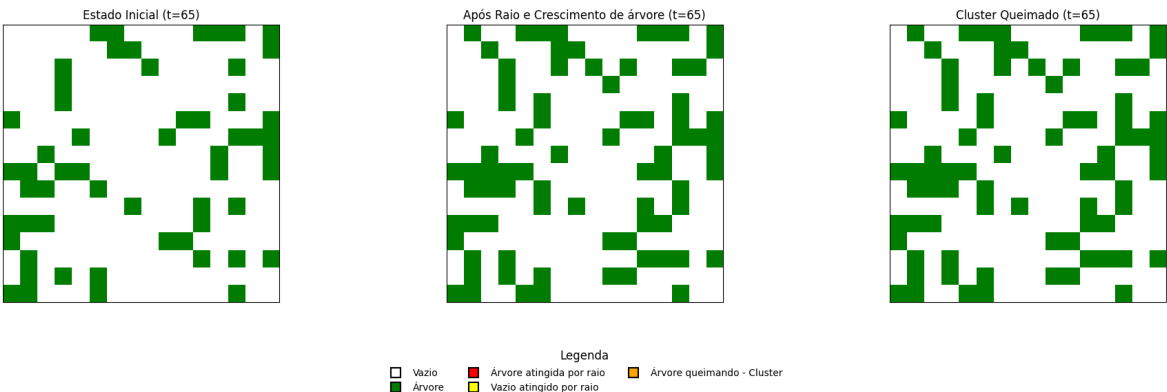
Floresta



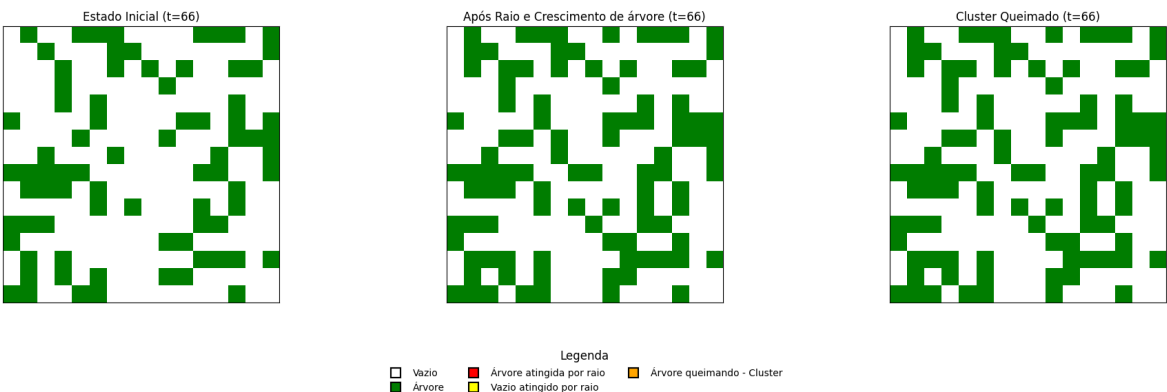
Floresta



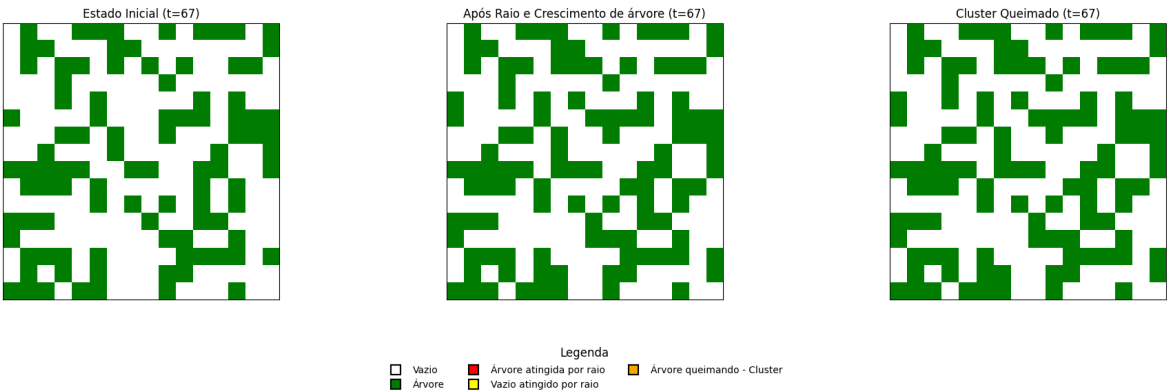
Floresta



Floresta

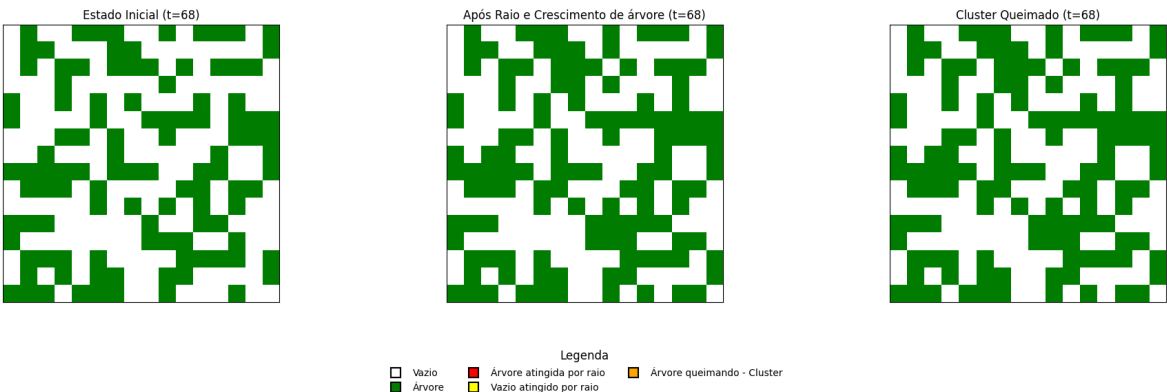


Floresta



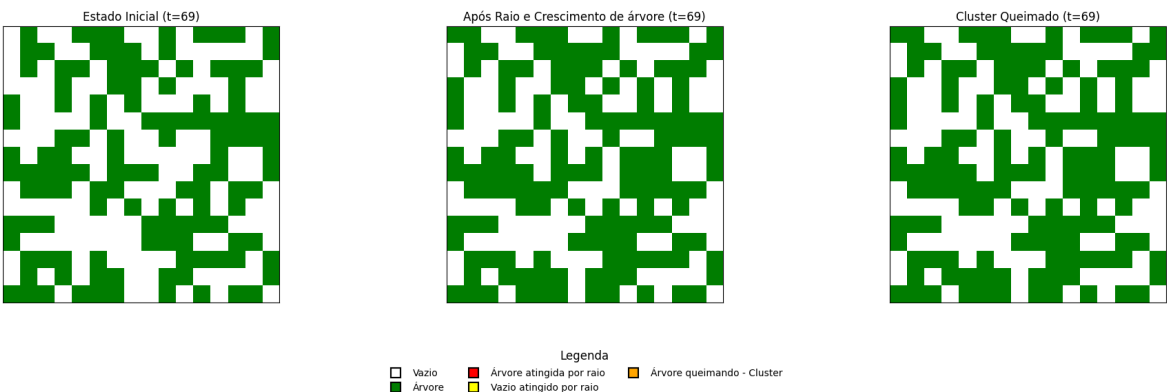
=====

Floresta



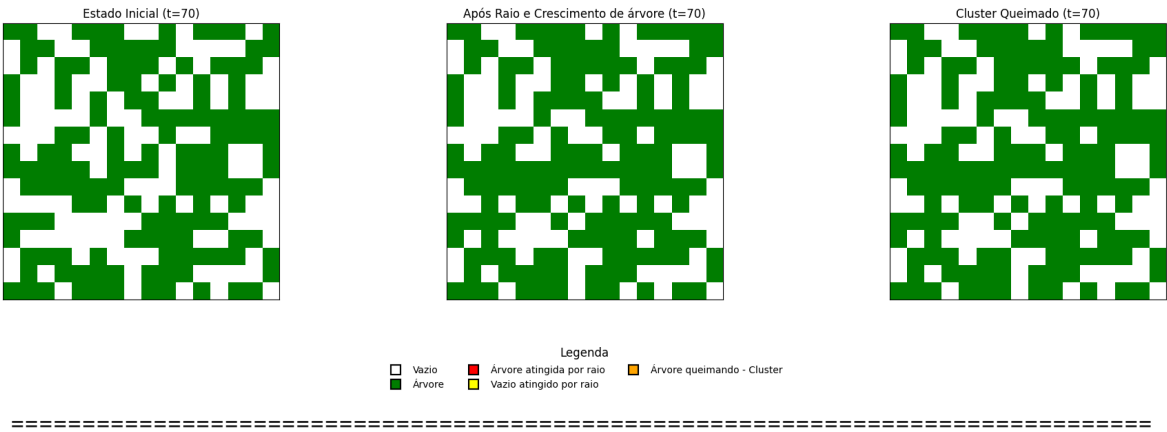
=====

Floresta

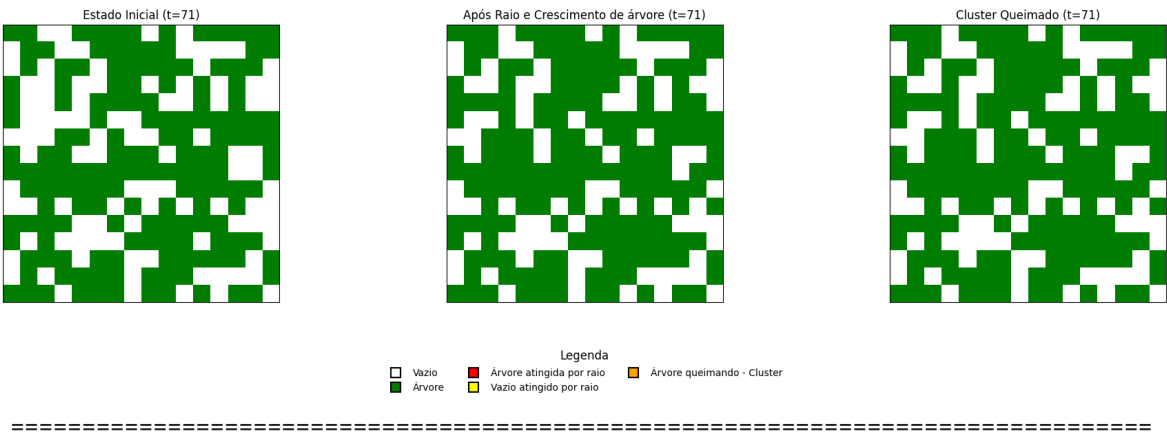


=====

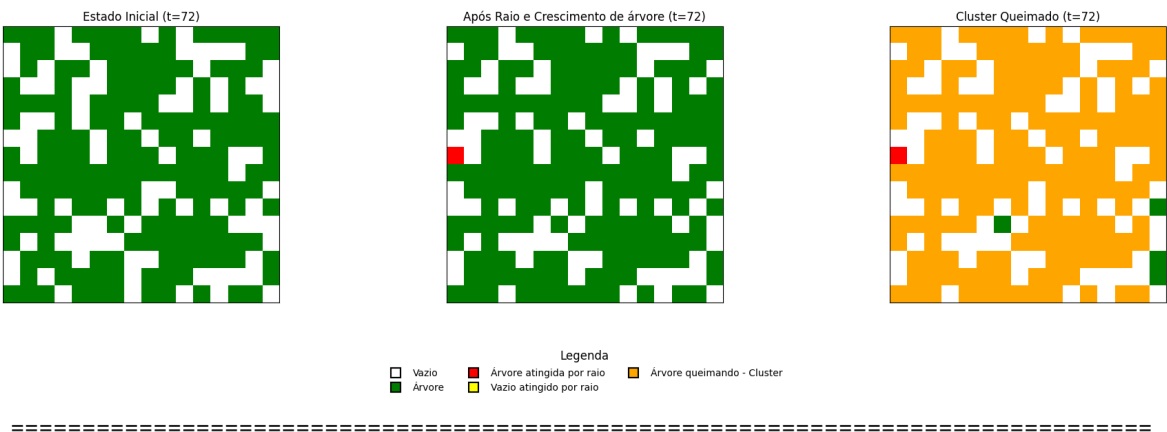
Floresta

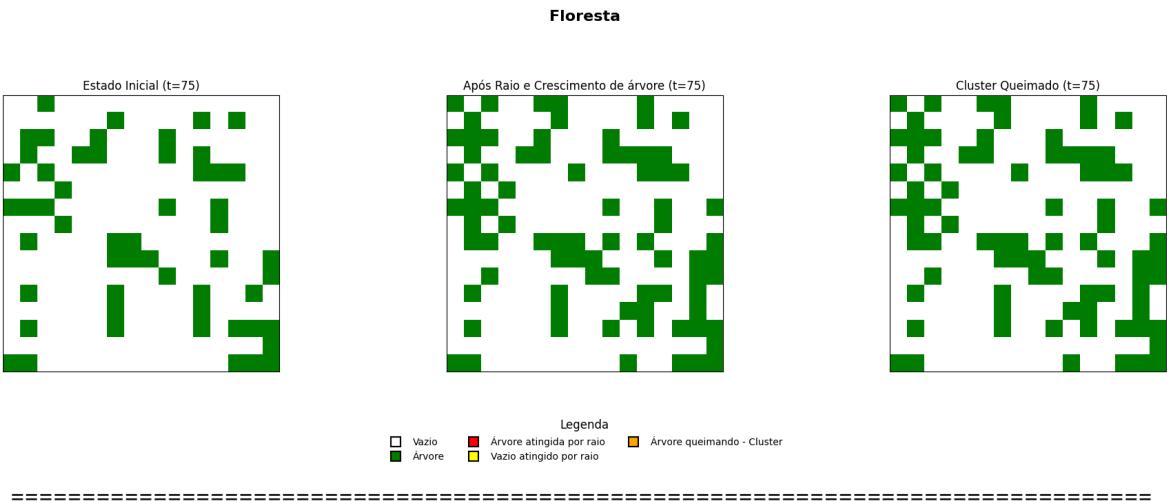
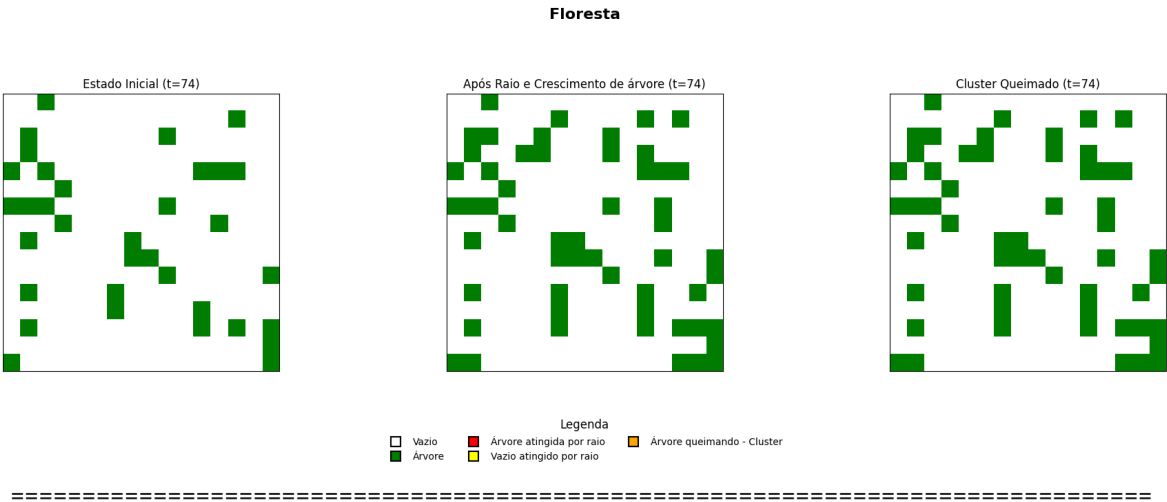
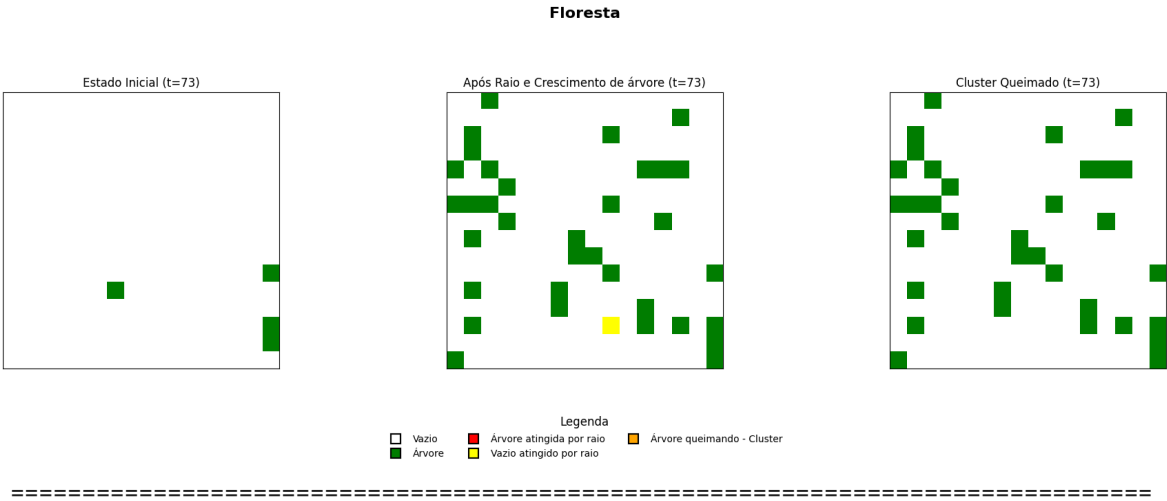


Floresta

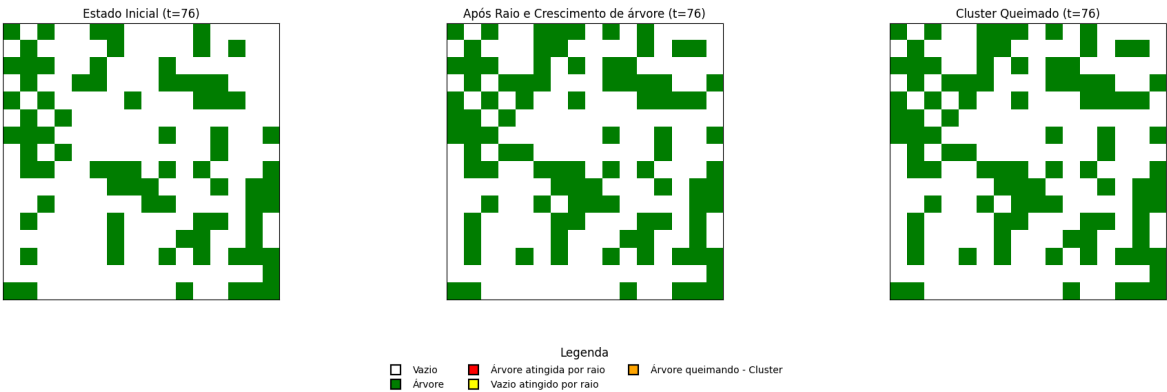


Floresta

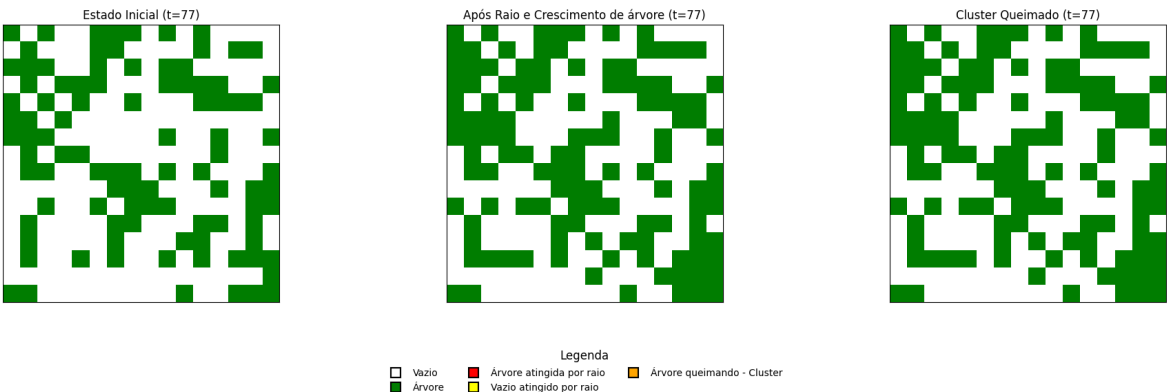




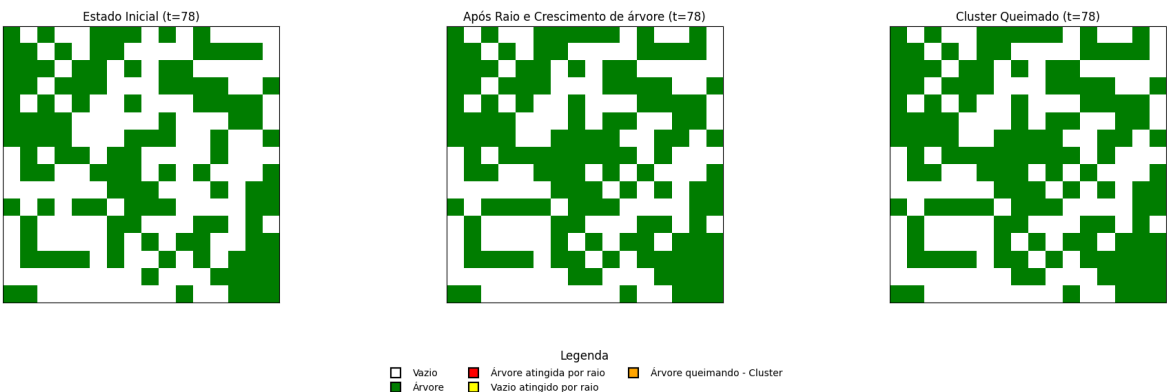
Floresta



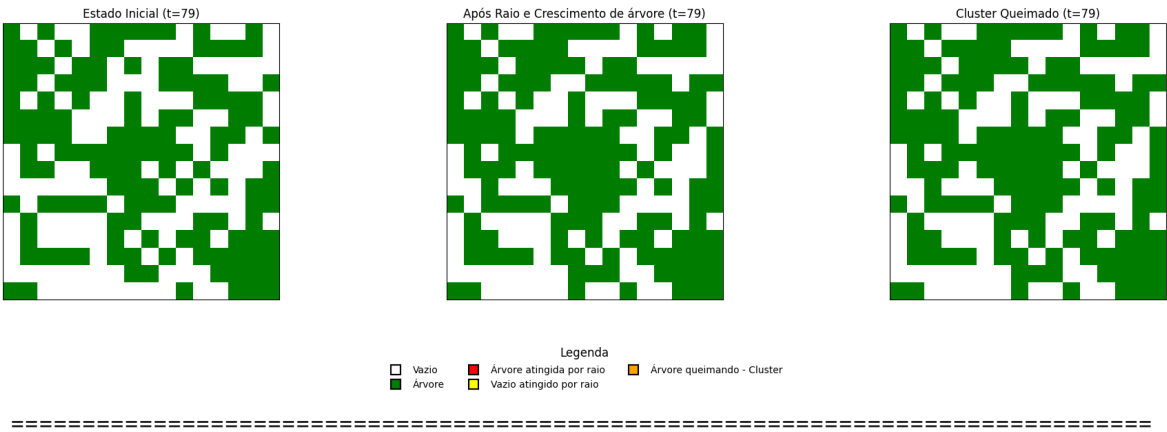
Floresta



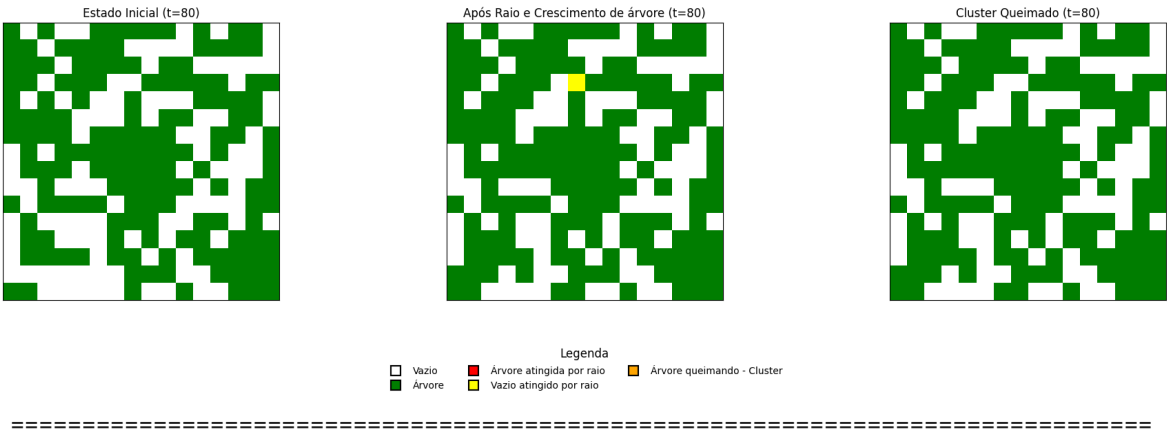
Floresta



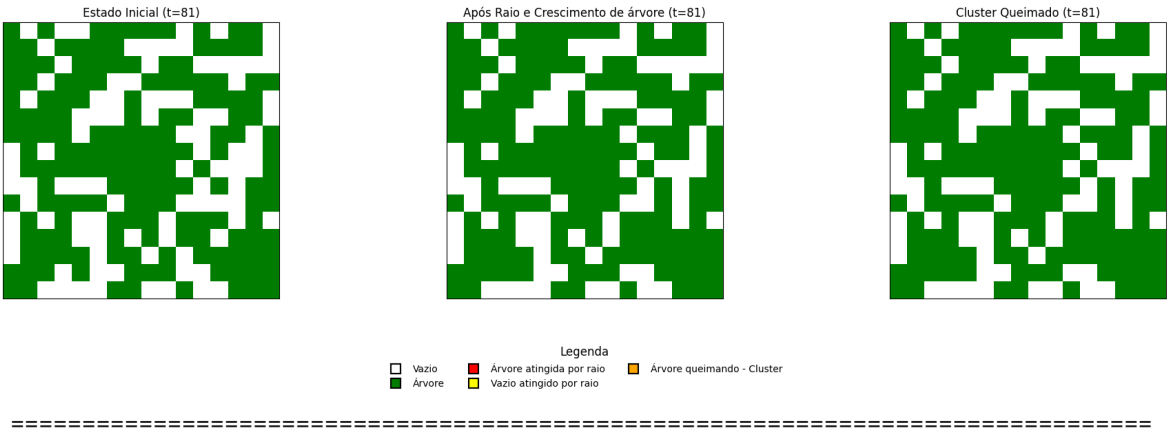
Floresta



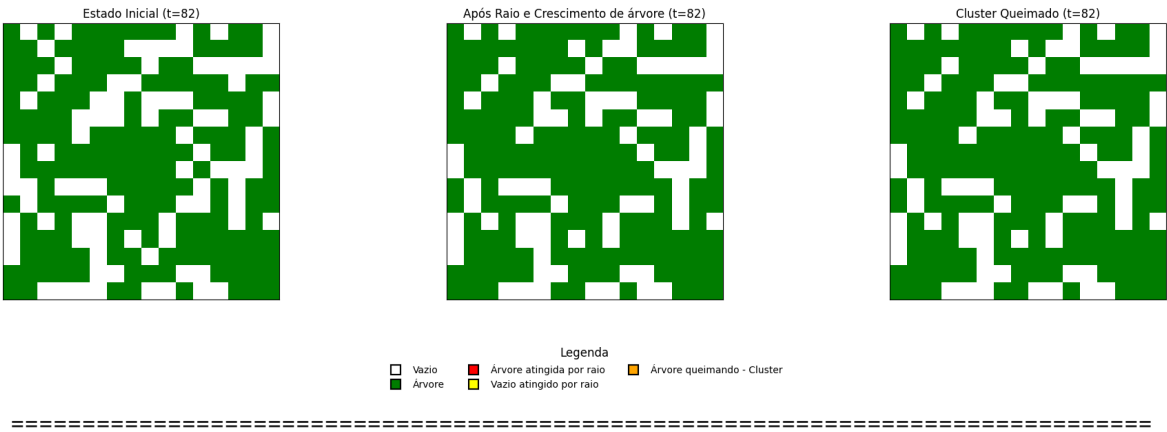
Floresta



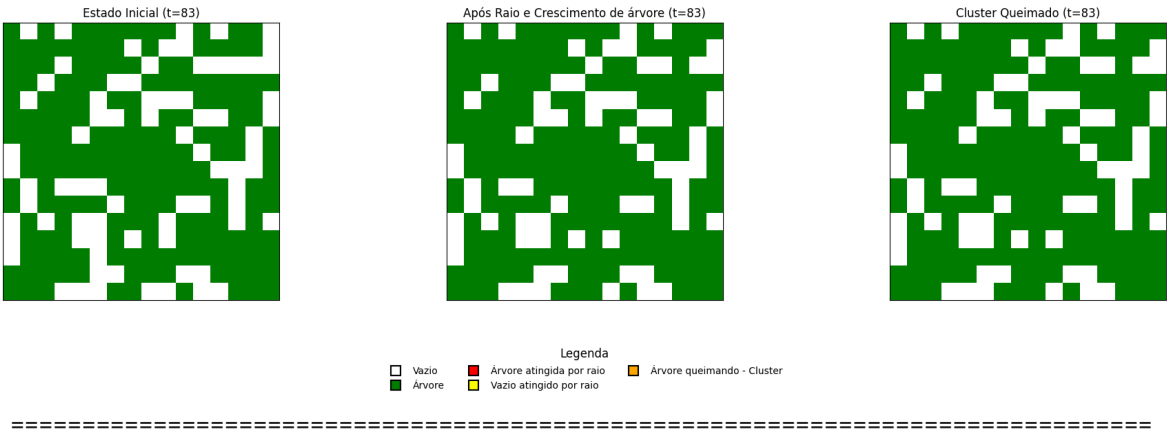
Floresta



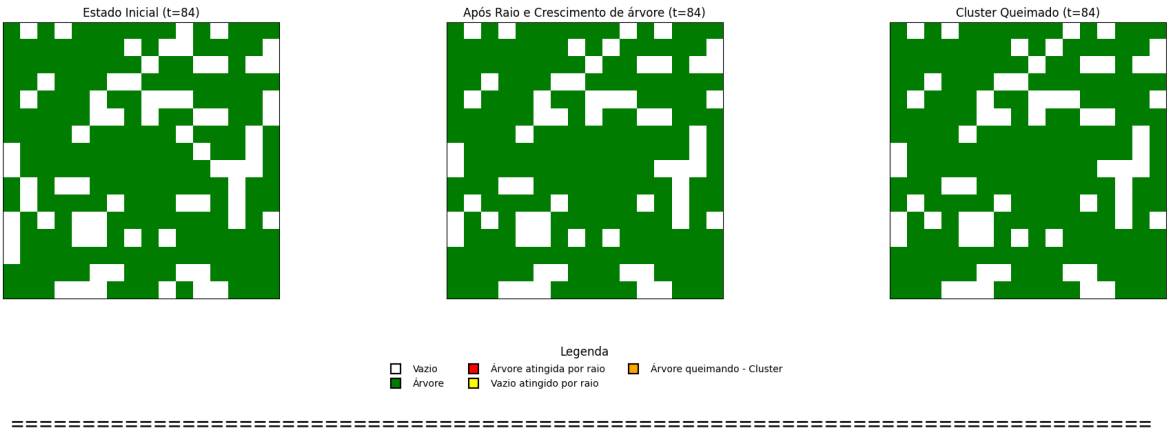
Floresta



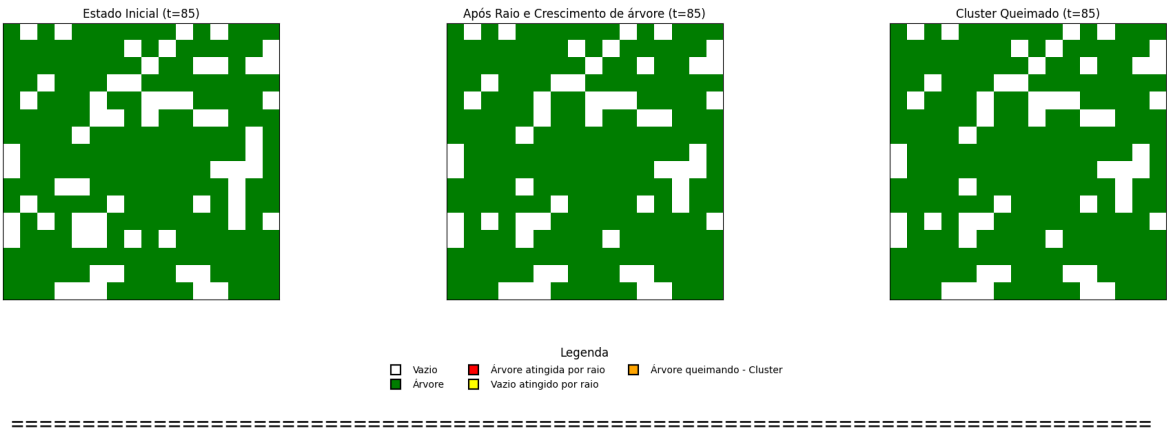
Floresta



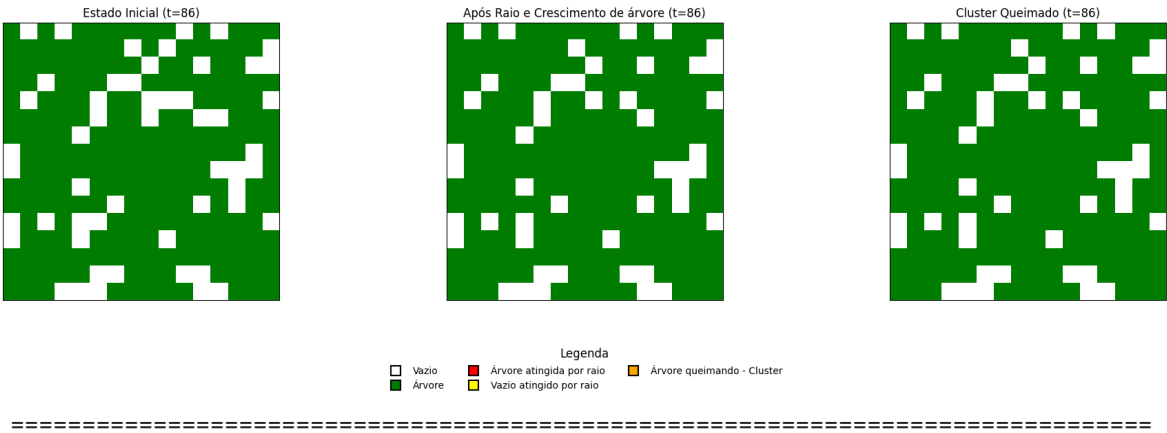
Floresta



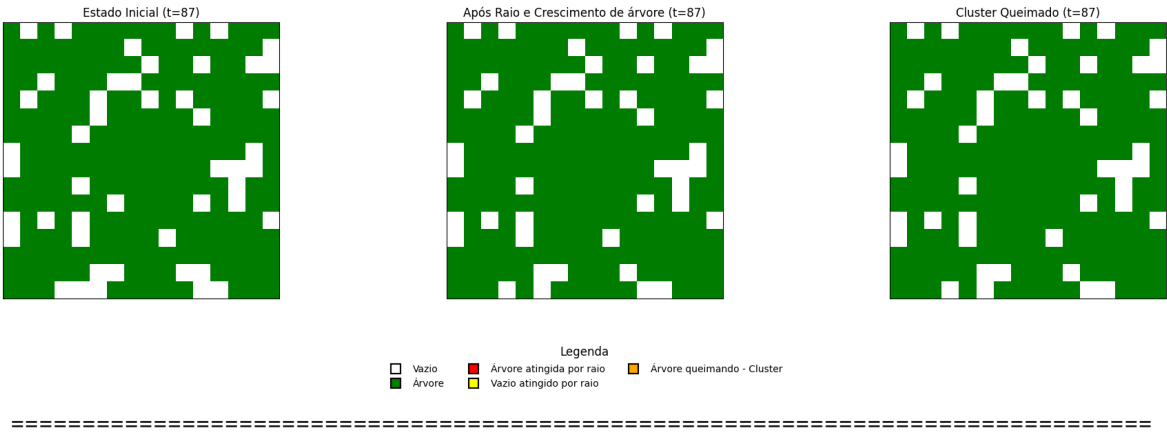
Floresta



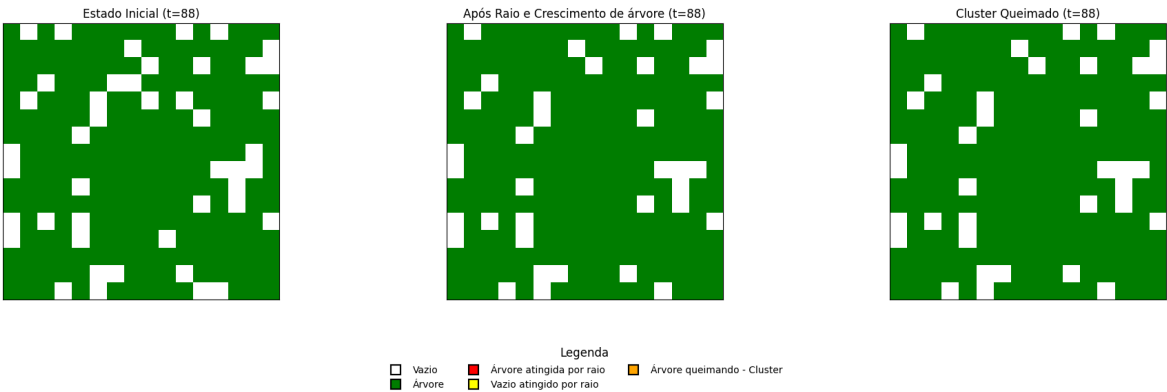
Floresta



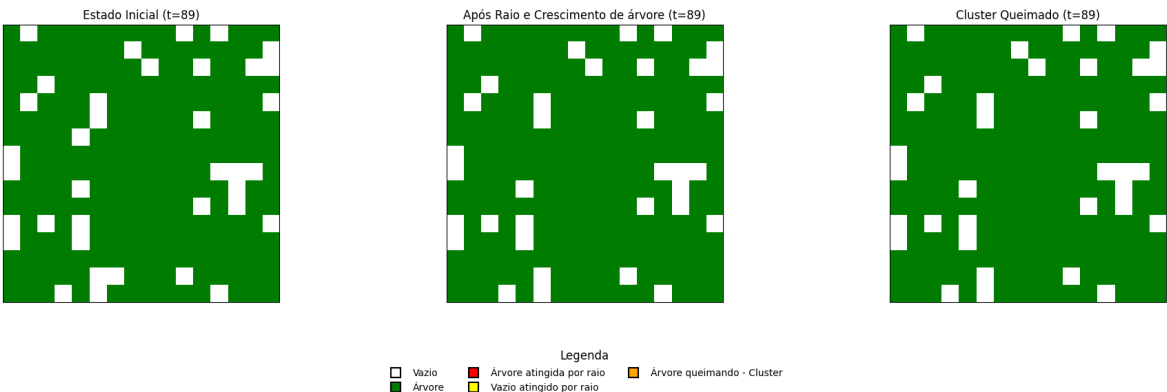
Floresta



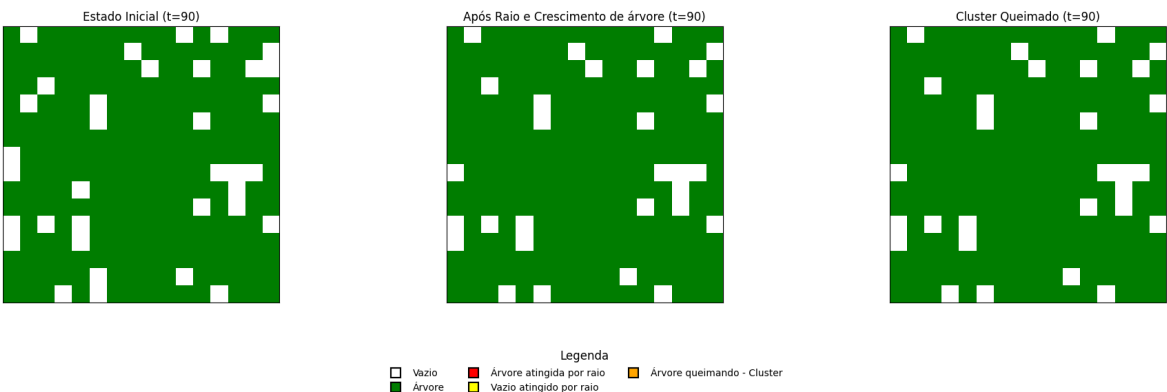
Floresta

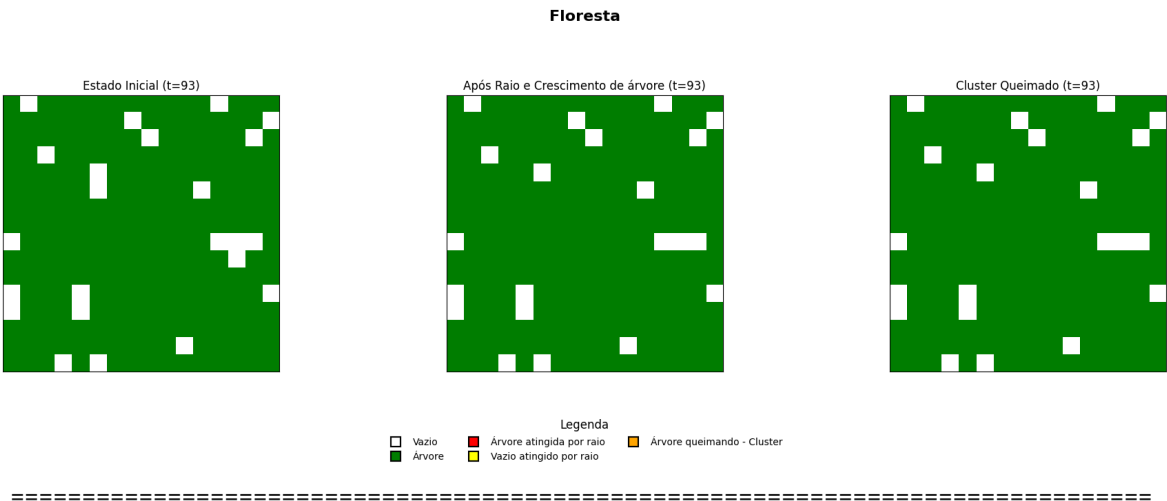
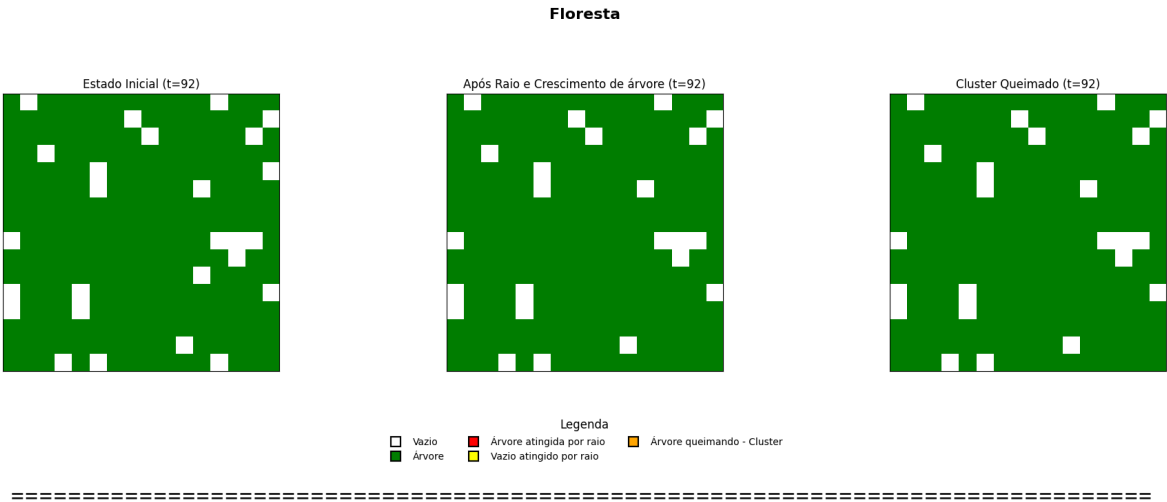
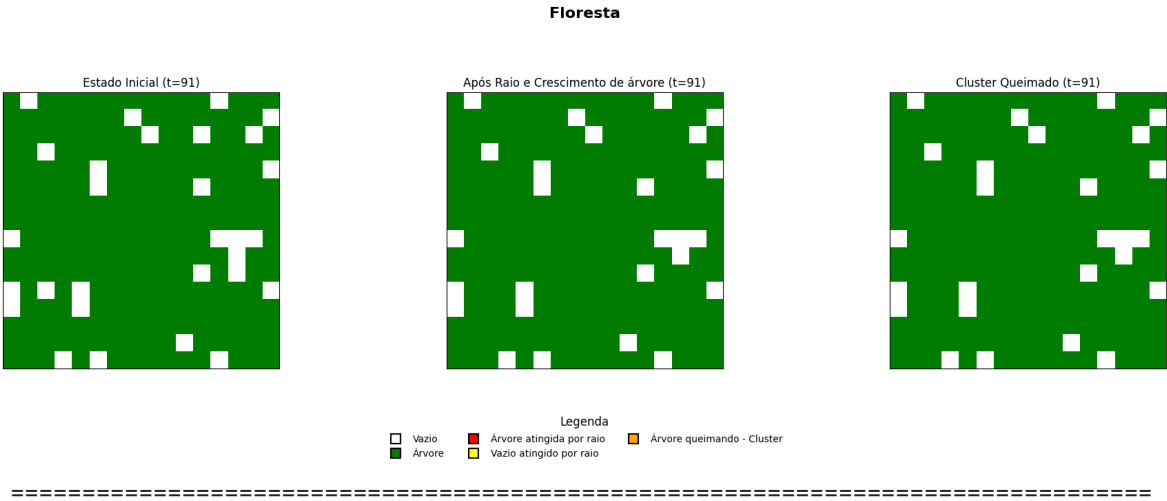


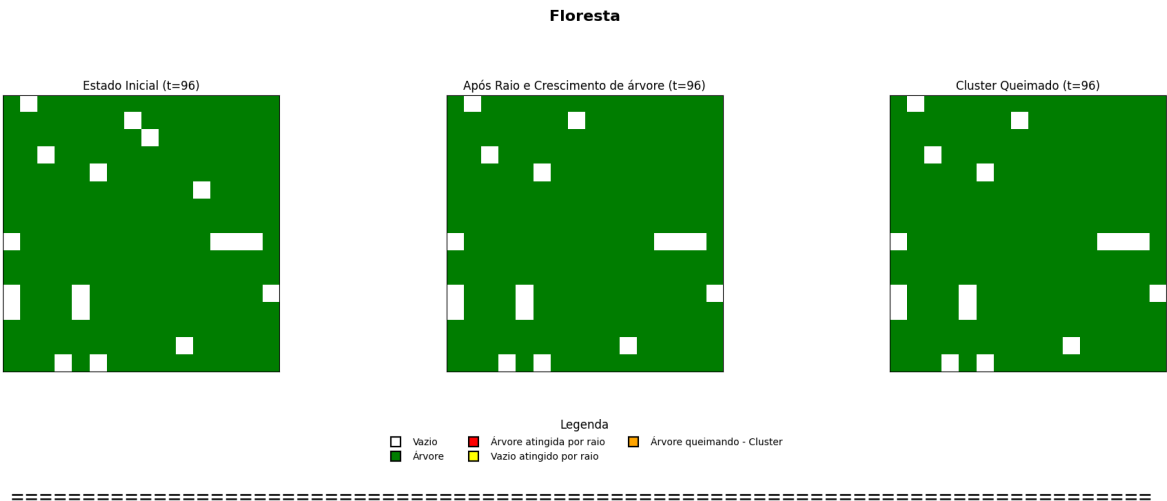
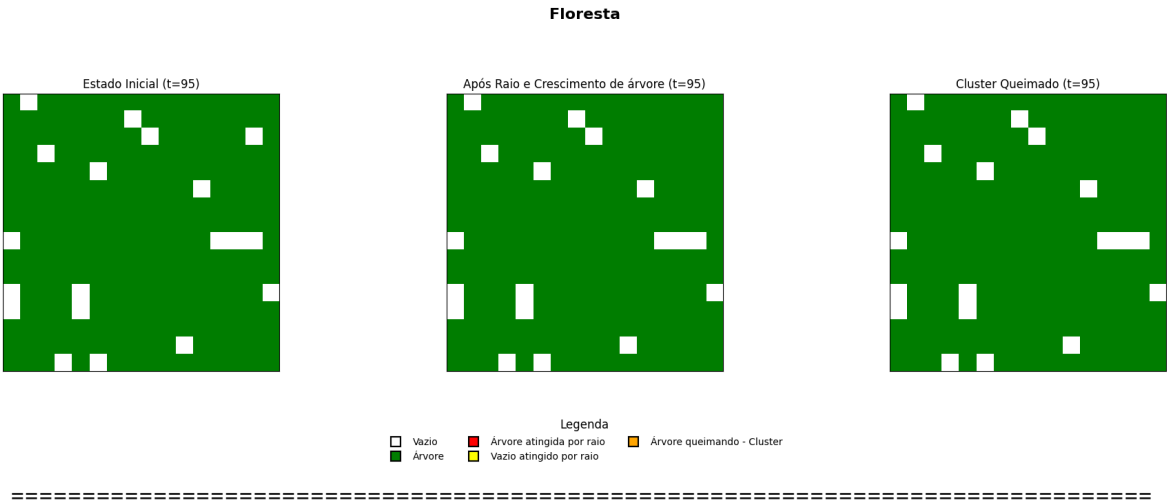
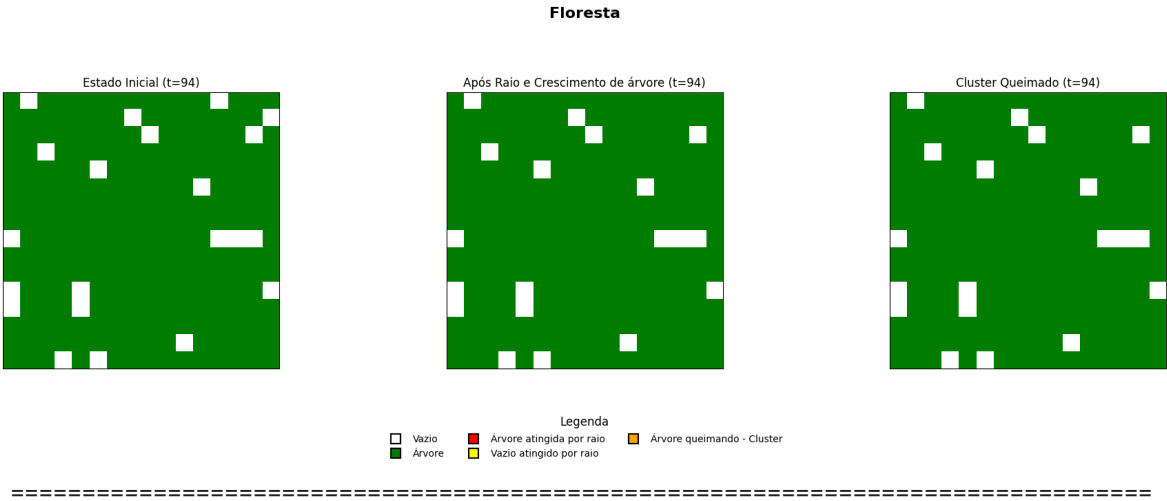
Floresta

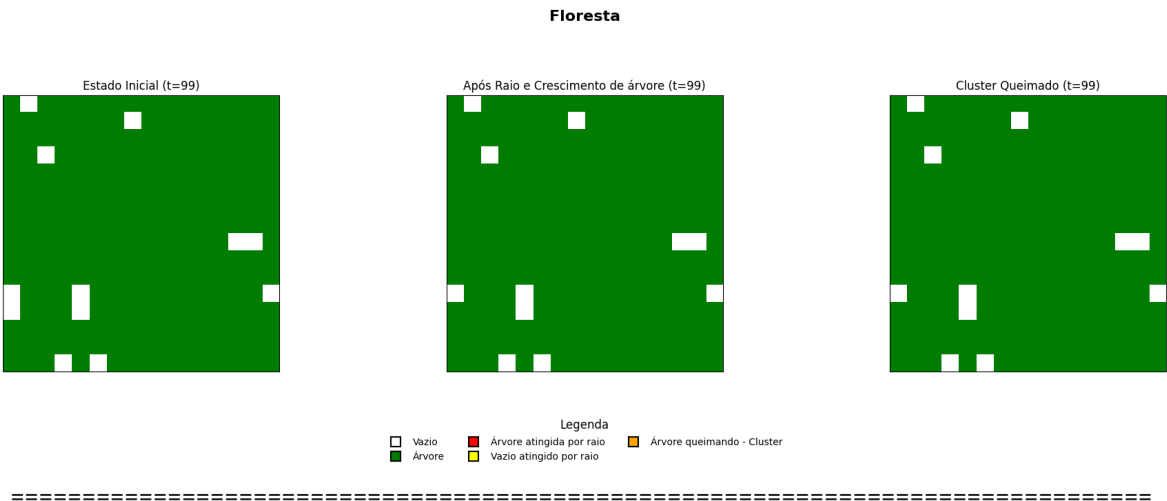
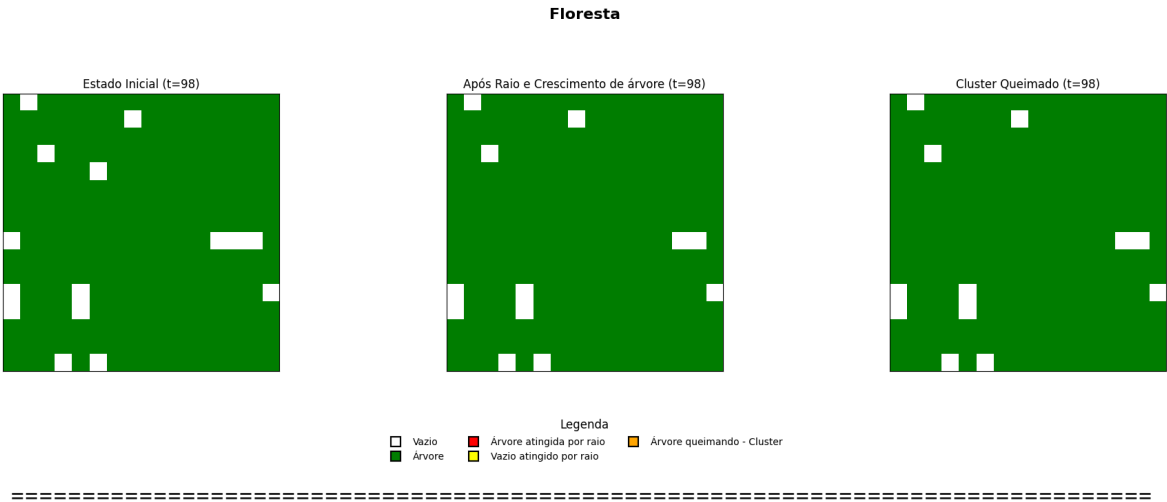
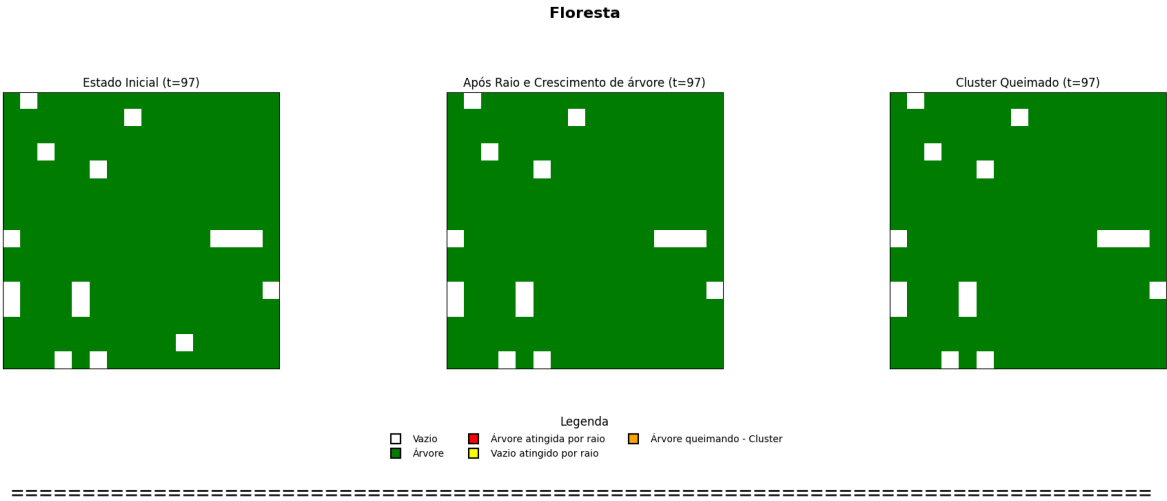


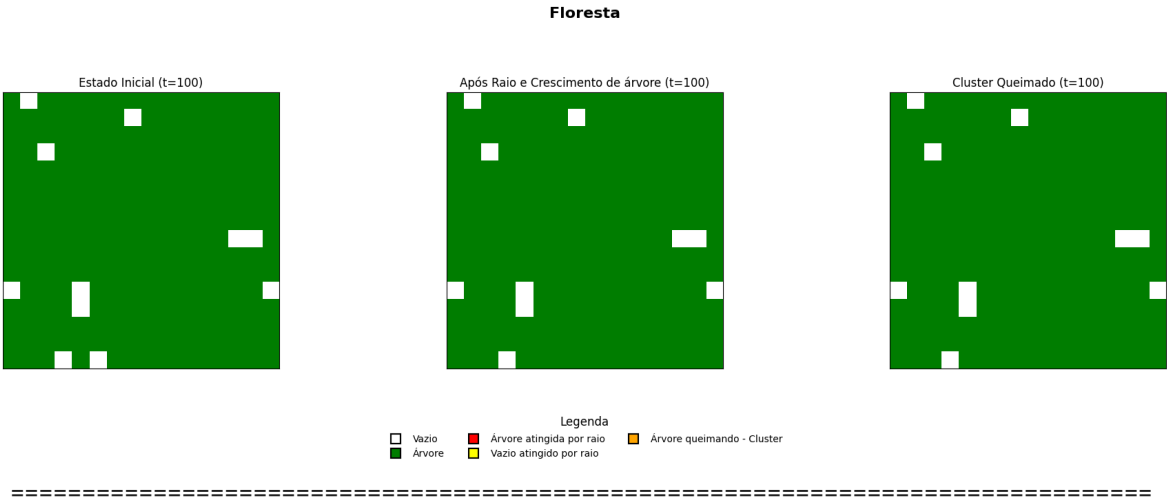
Floresta











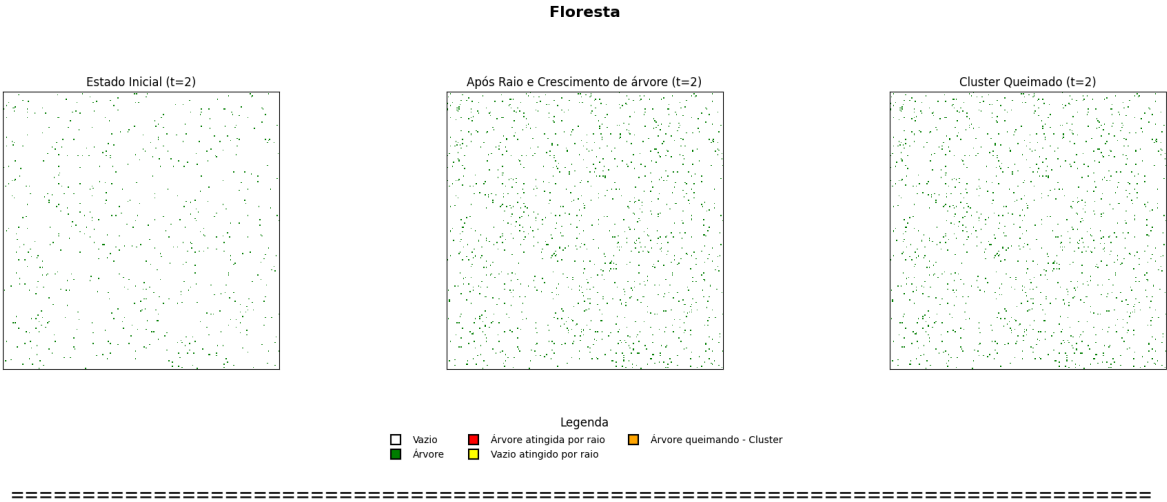
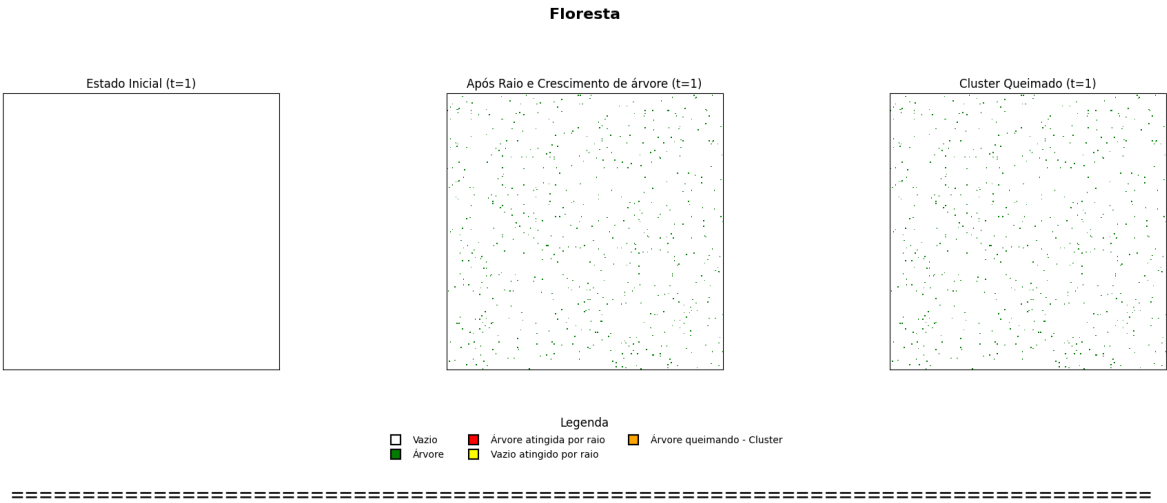
C.

In [135...

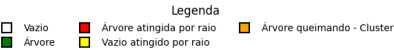
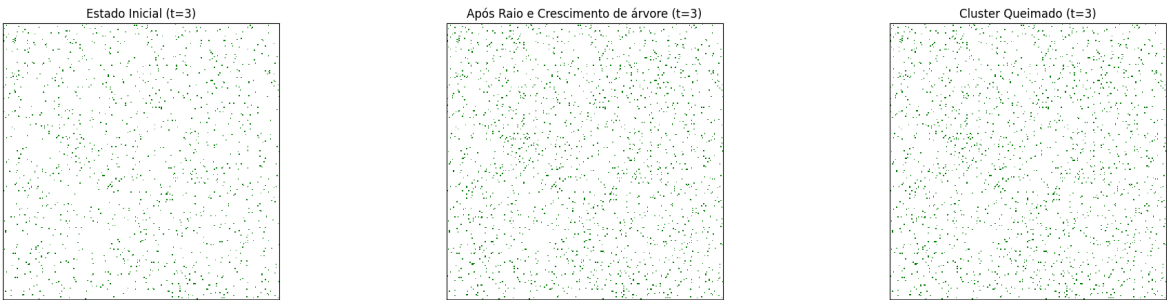
```
L = 256
p = 0.01
f = 0.2
iteracoes = 104
```

In [136...

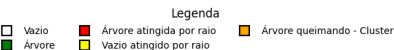
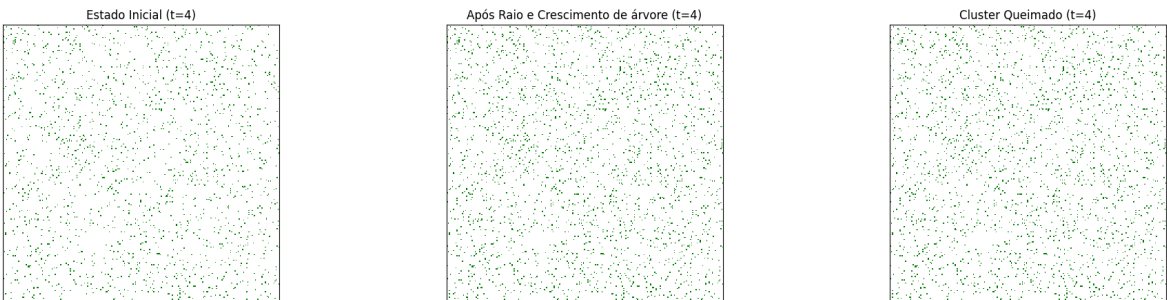
```
tam_incendios = simula_floresta(L, p, f, iteracoes, vizualizar)
```



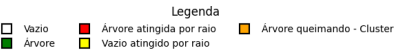
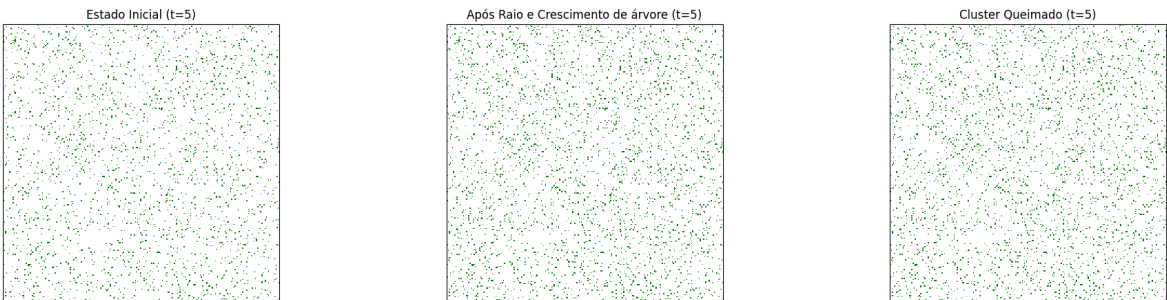
Floresta



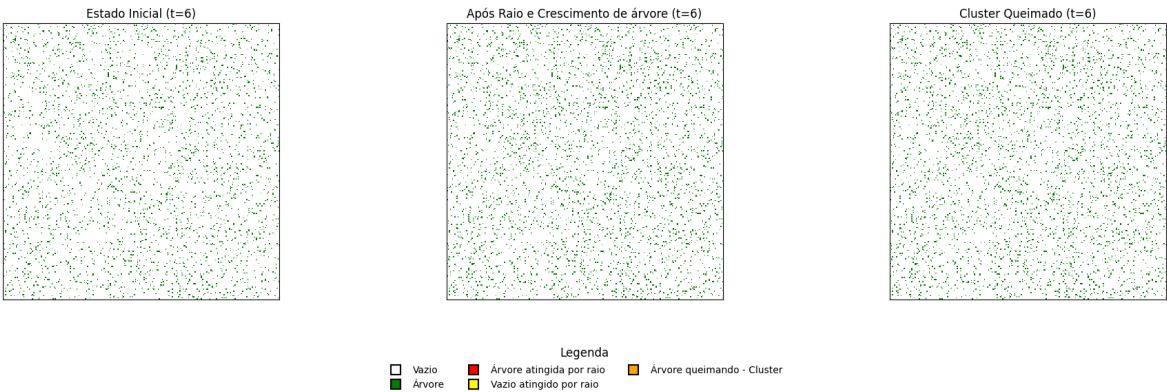
Floresta



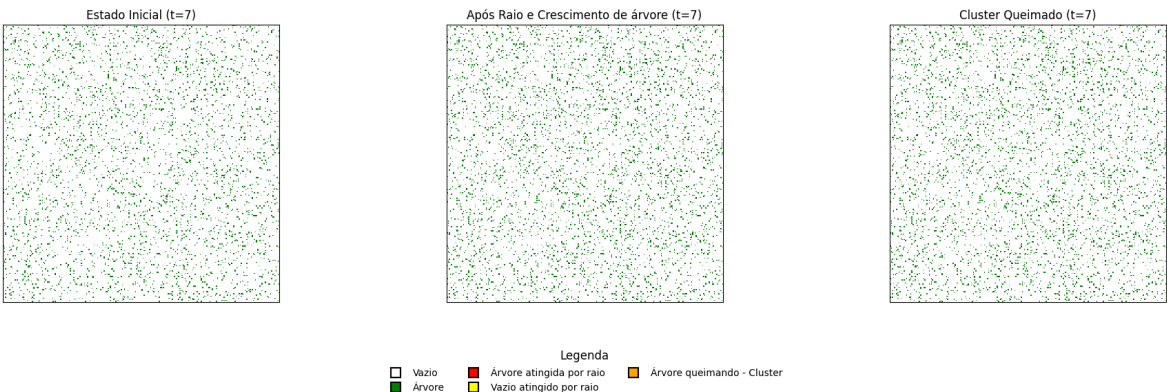
Floresta



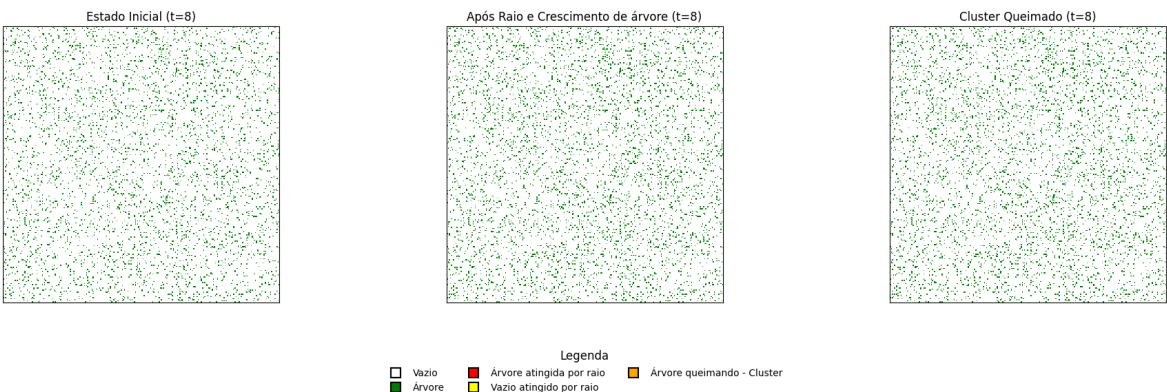
Floresta



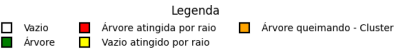
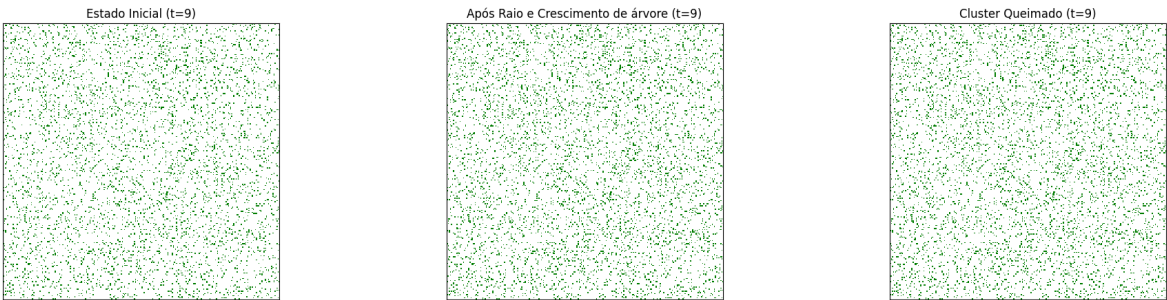
Floresta



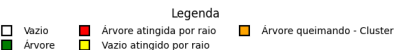
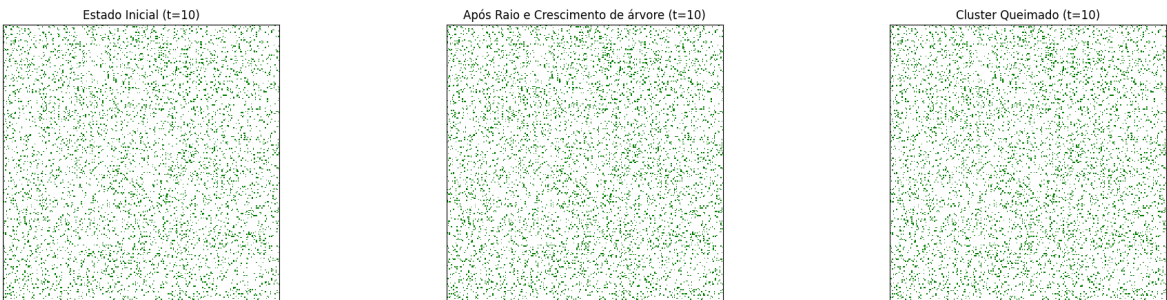
Floresta



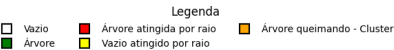
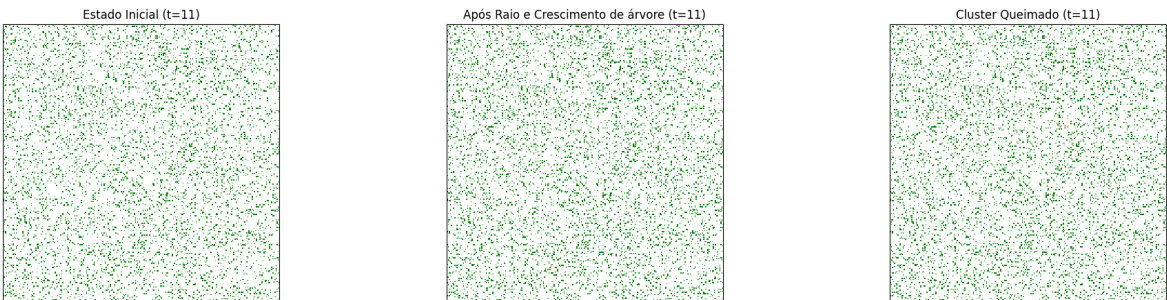
Floresta



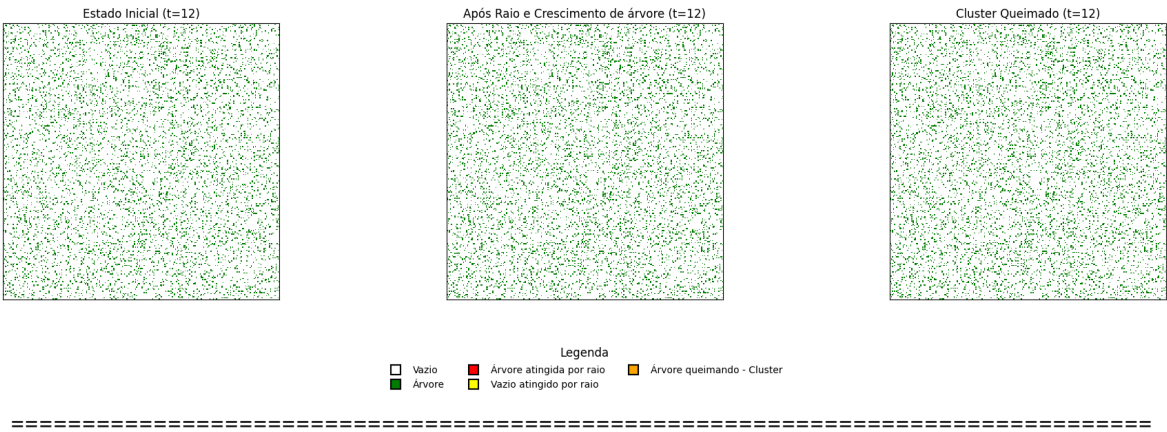
Floresta



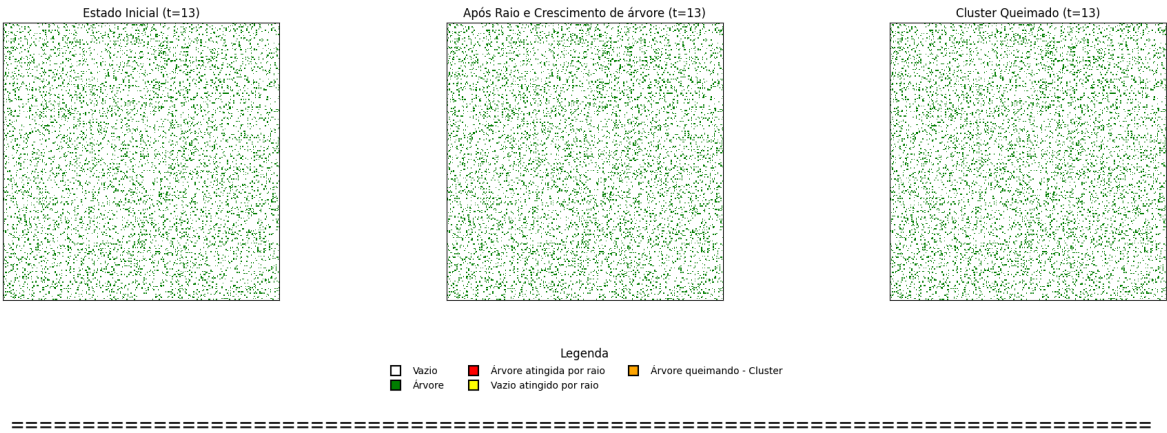
Floresta



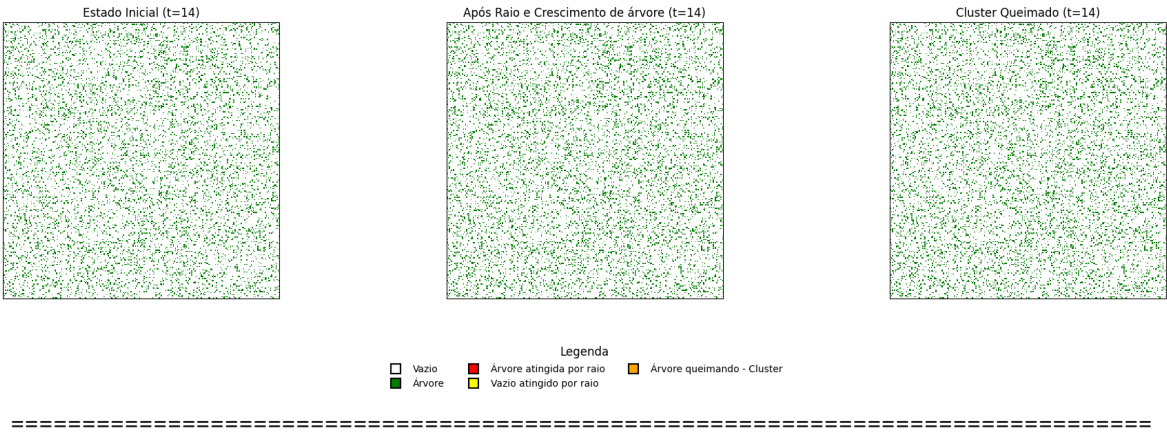
Floresta



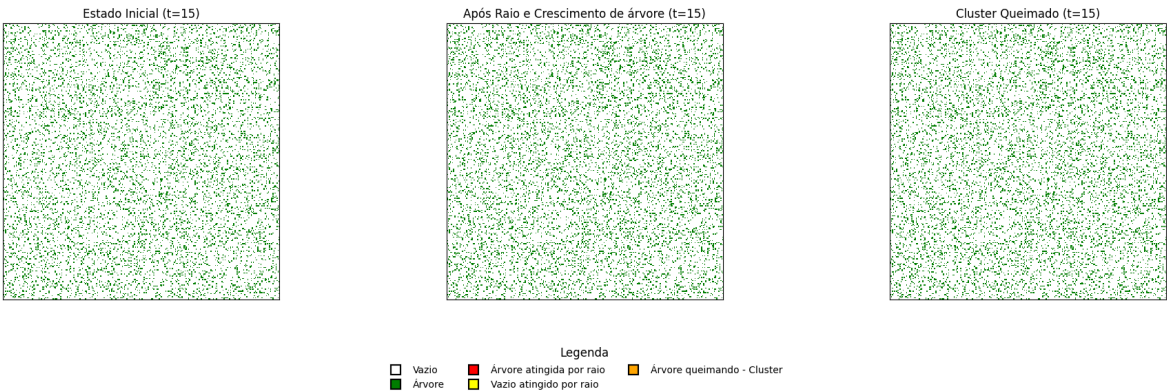
Floresta



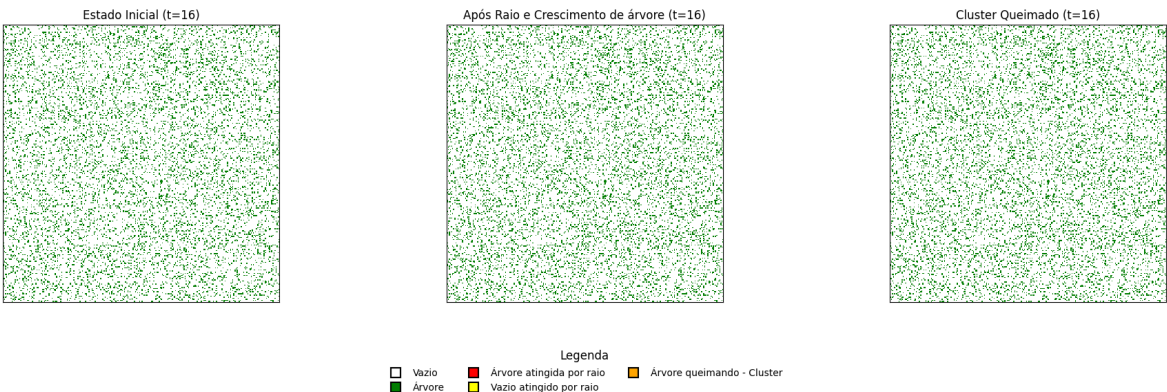
Floresta



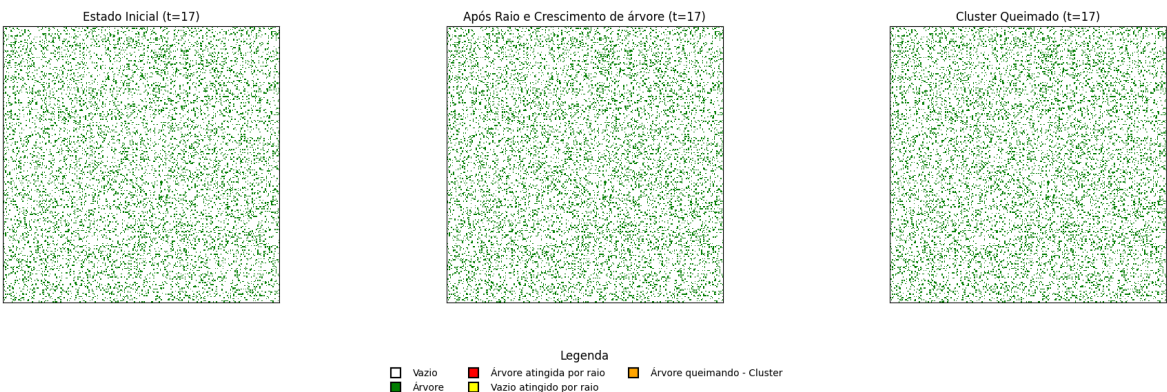
Floresta



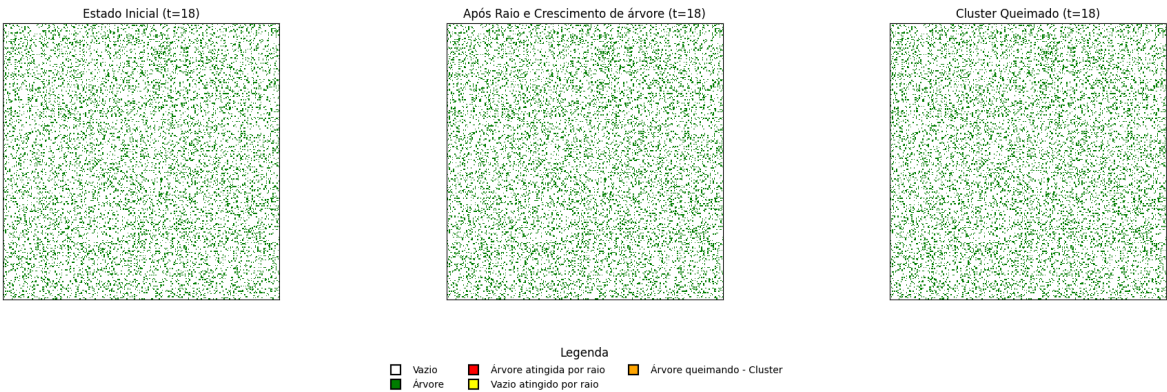
Floresta



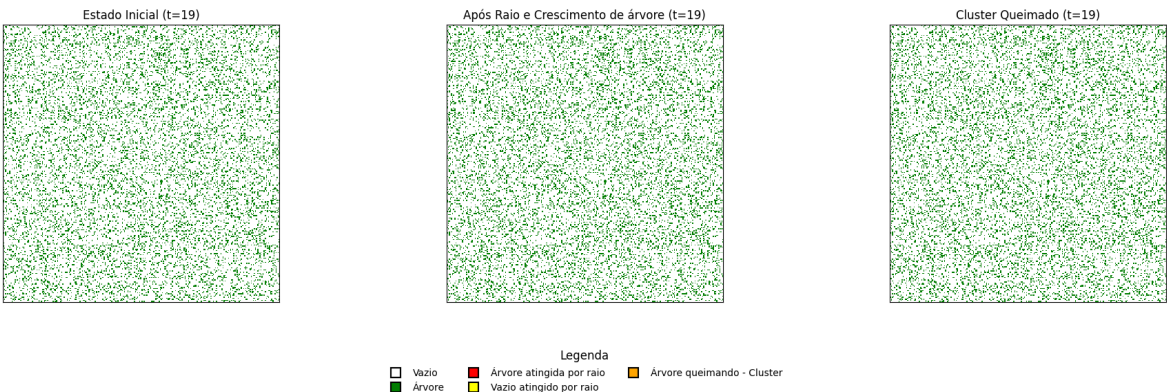
Floresta



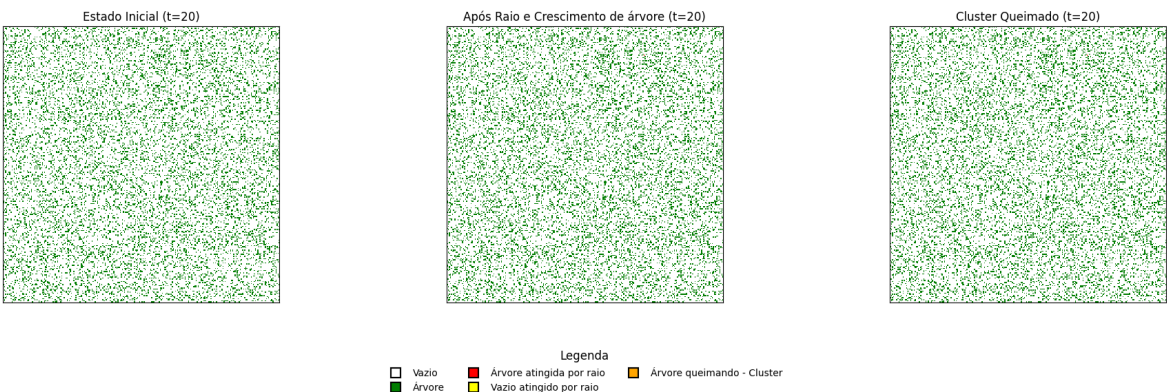
Floresta



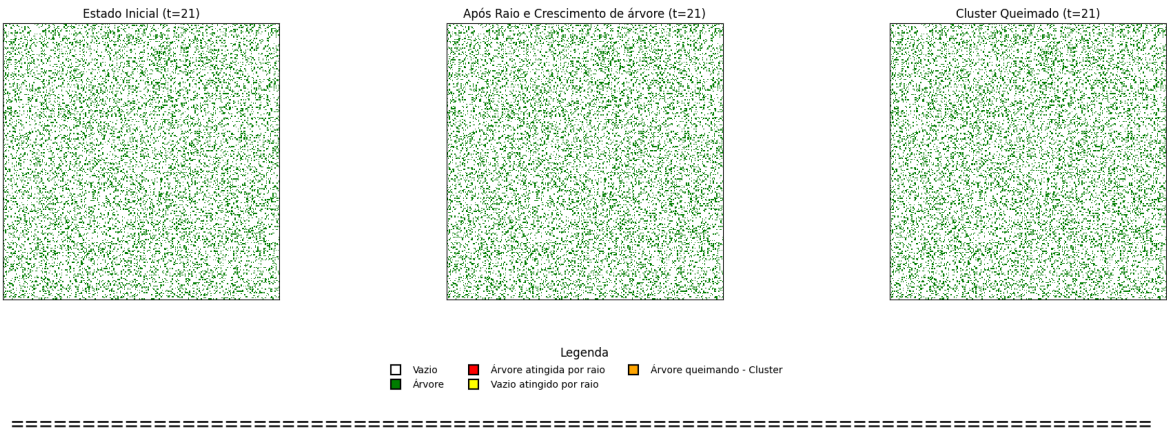
Floresta



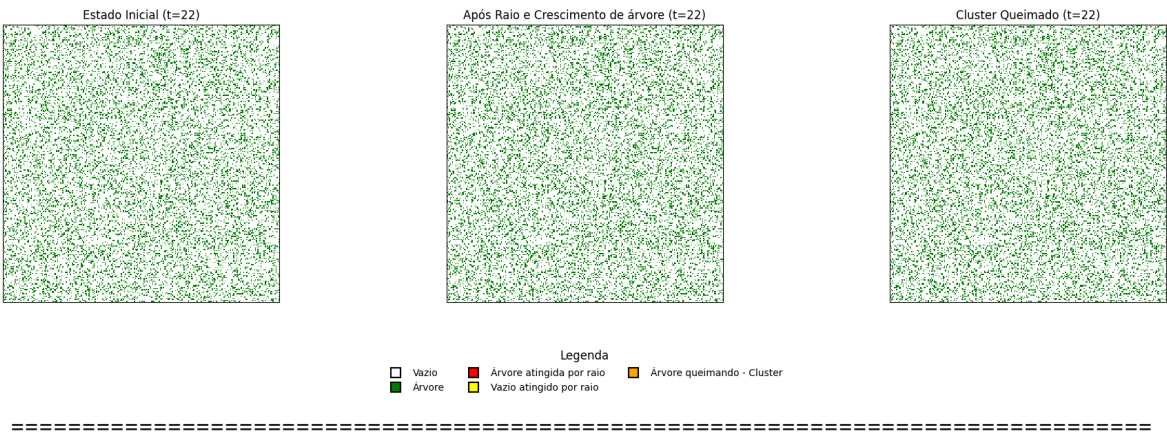
Floresta



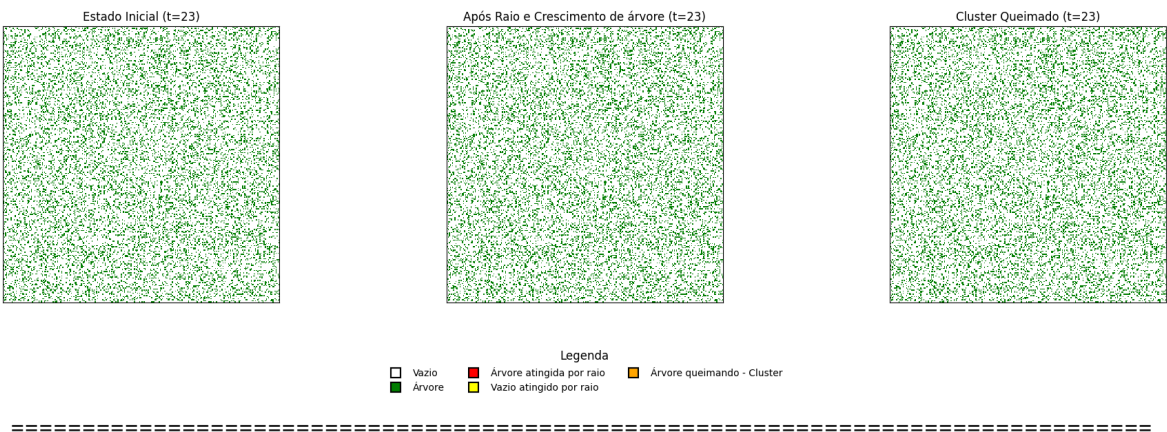
Floresta



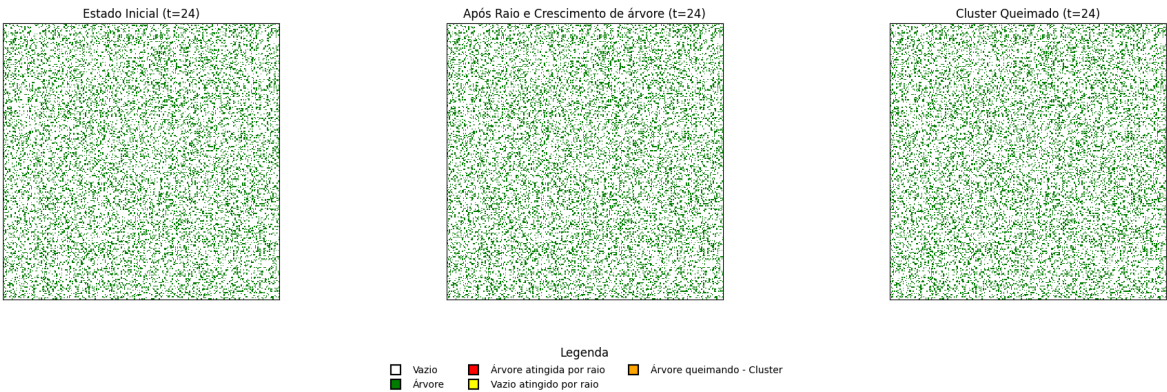
Floresta



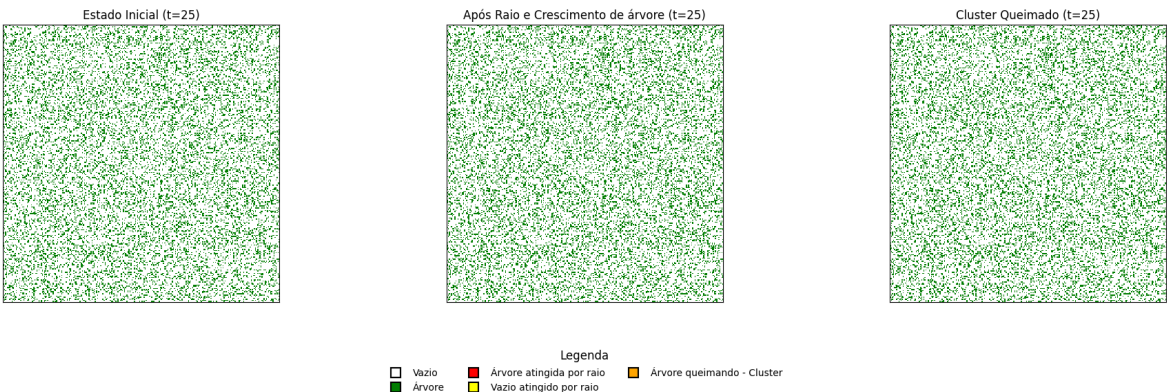
Floresta



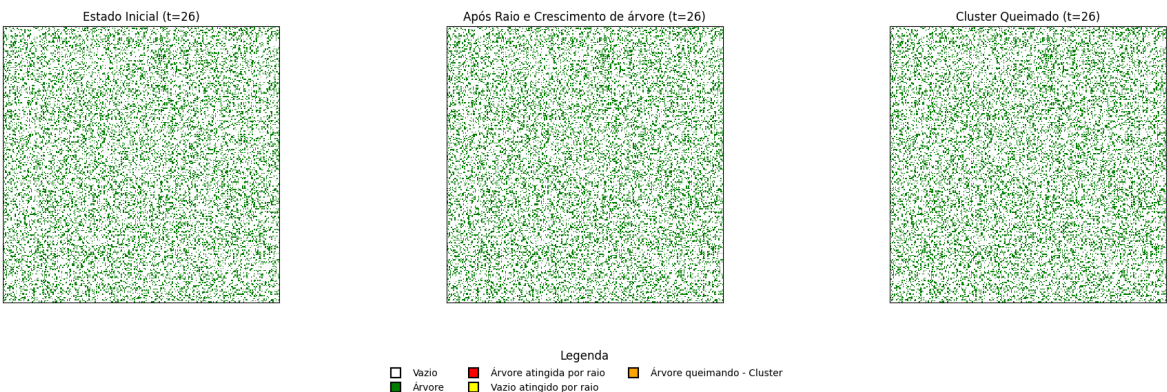
Floresta



Floresta



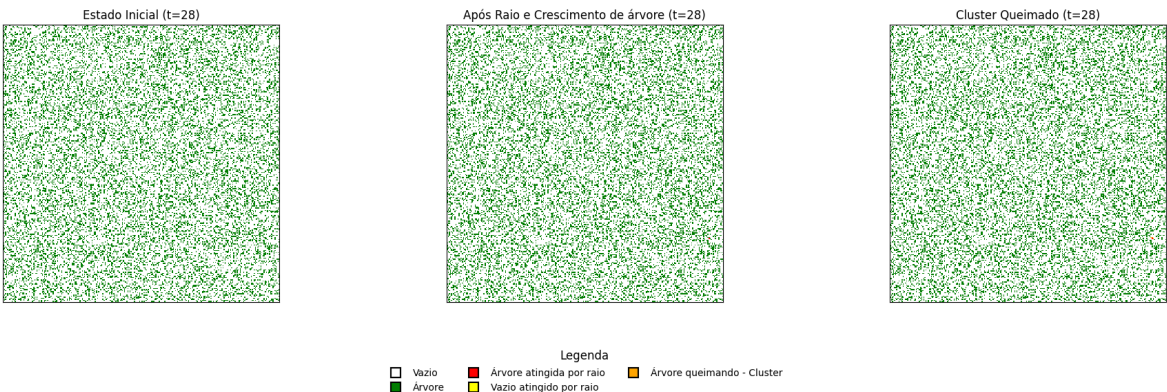
Floresta



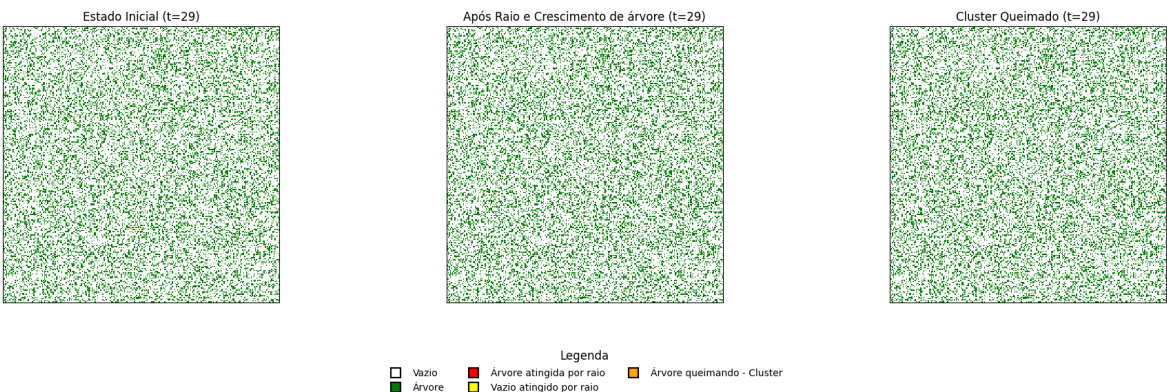
Floresta



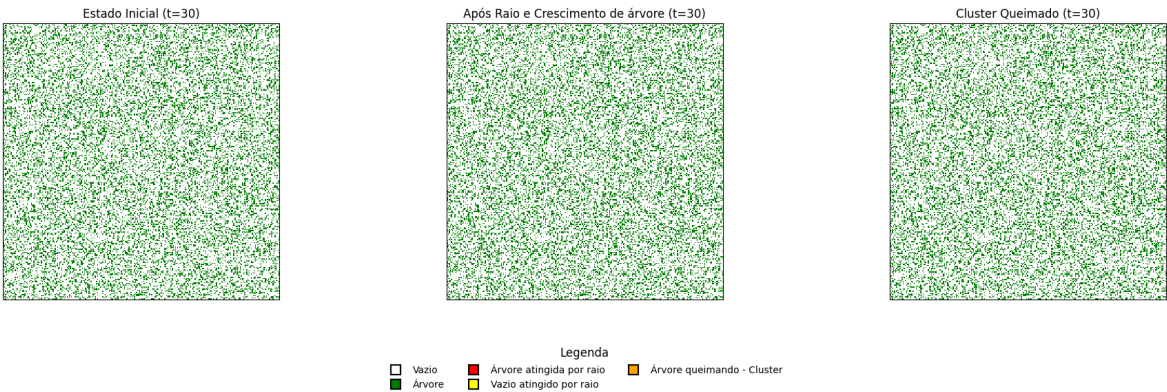
Floresta



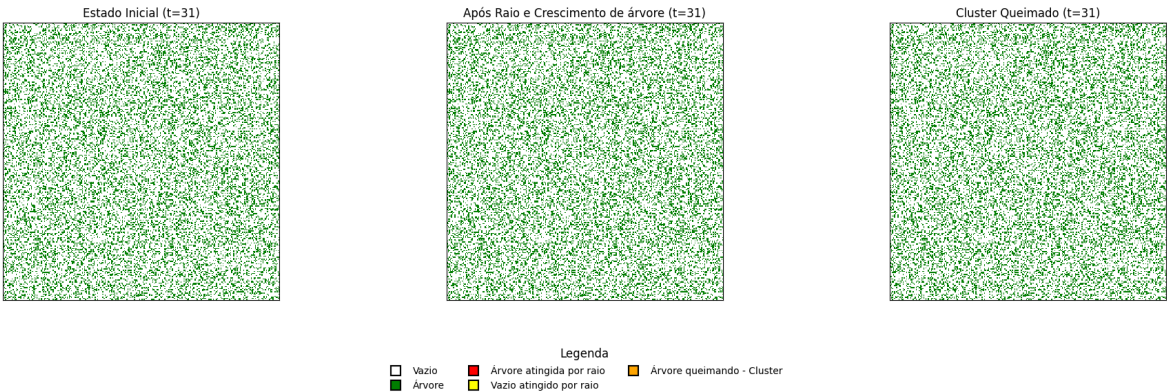
Floresta



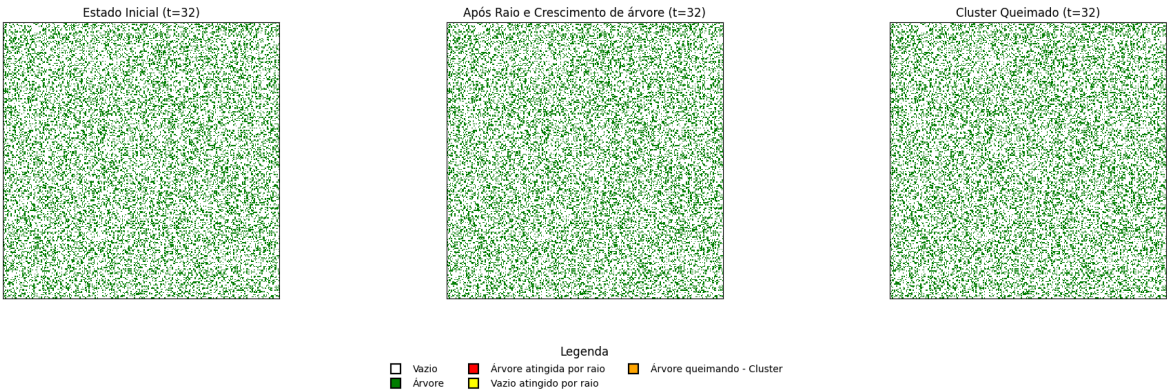
Floresta



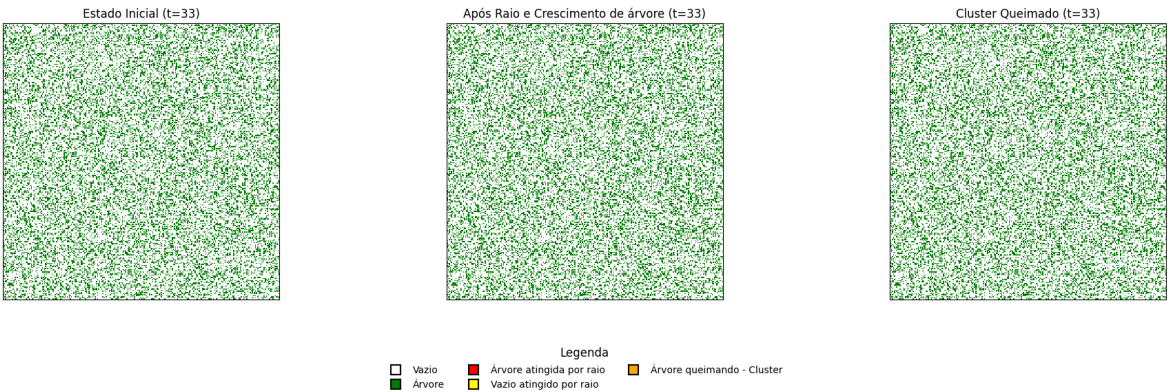
Floresta



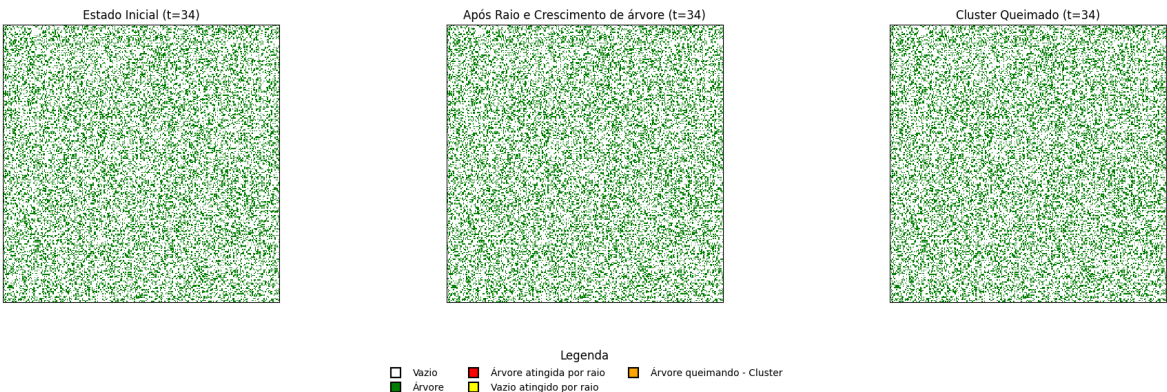
Floresta



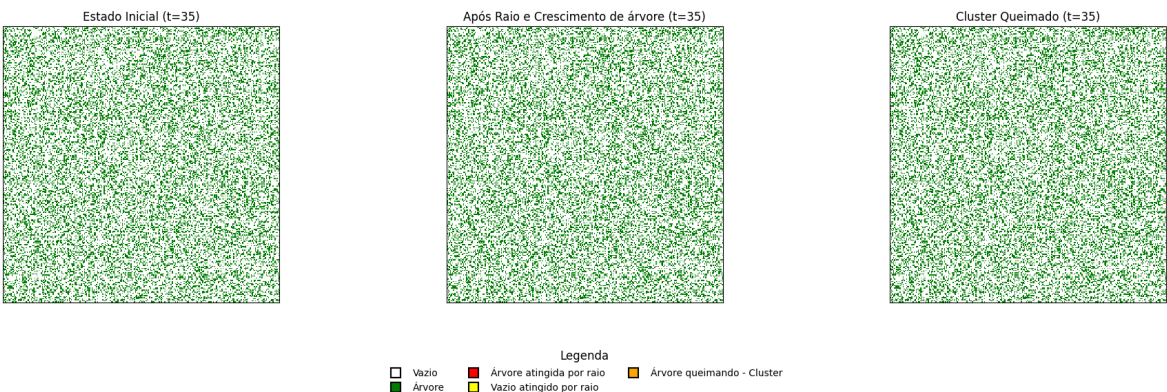
Floresta



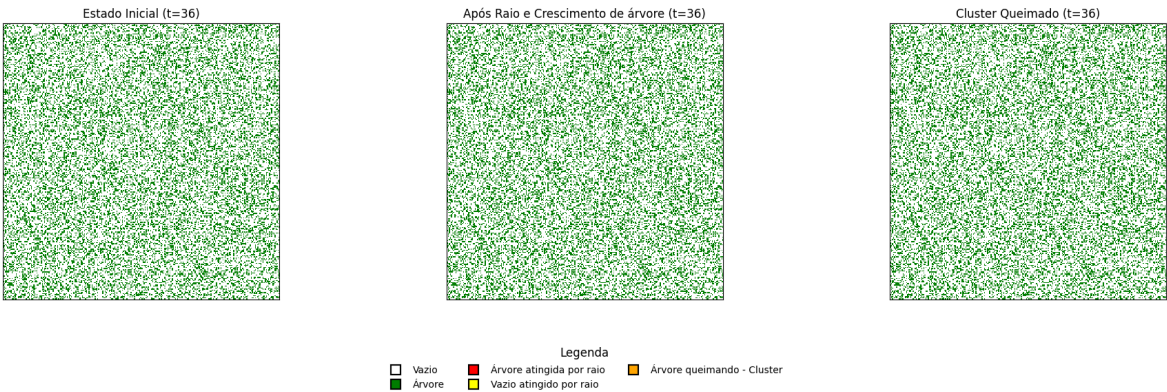
Floresta



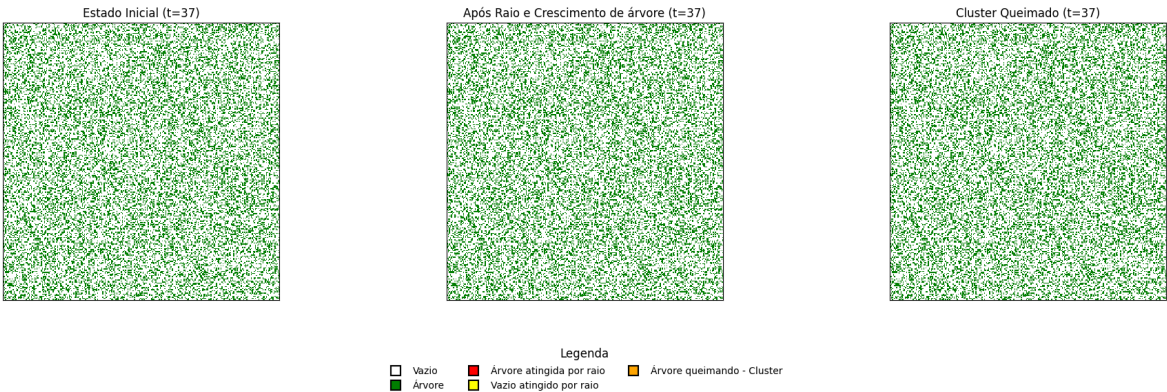
Floresta



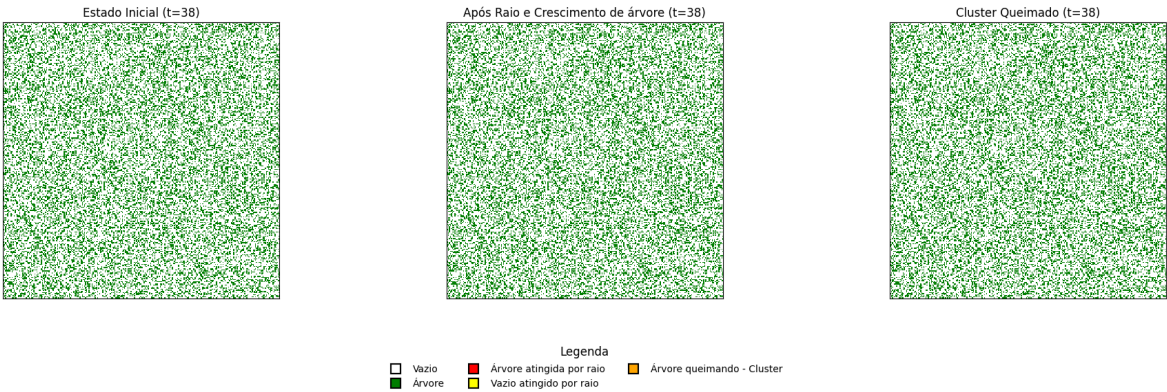
Floresta



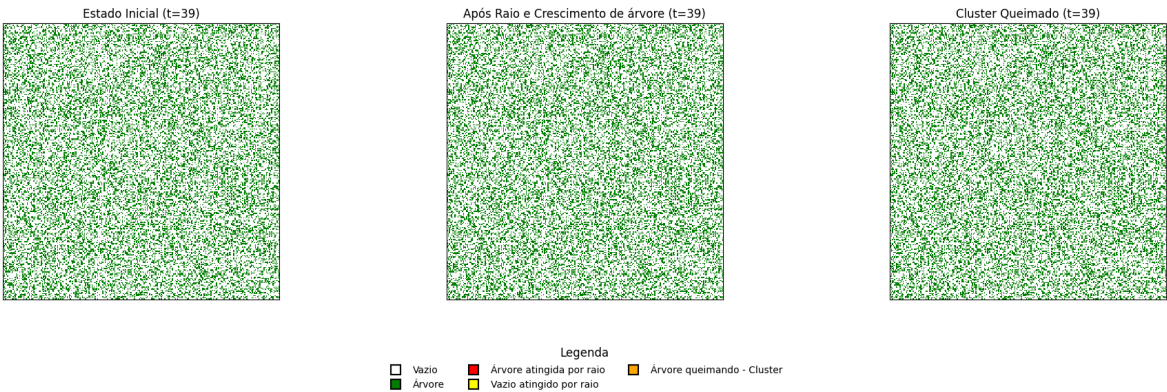
Floresta



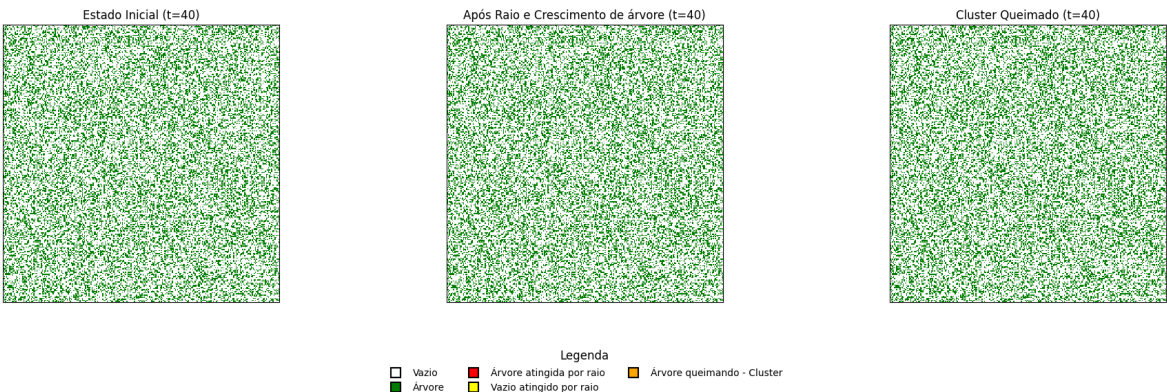
Floresta



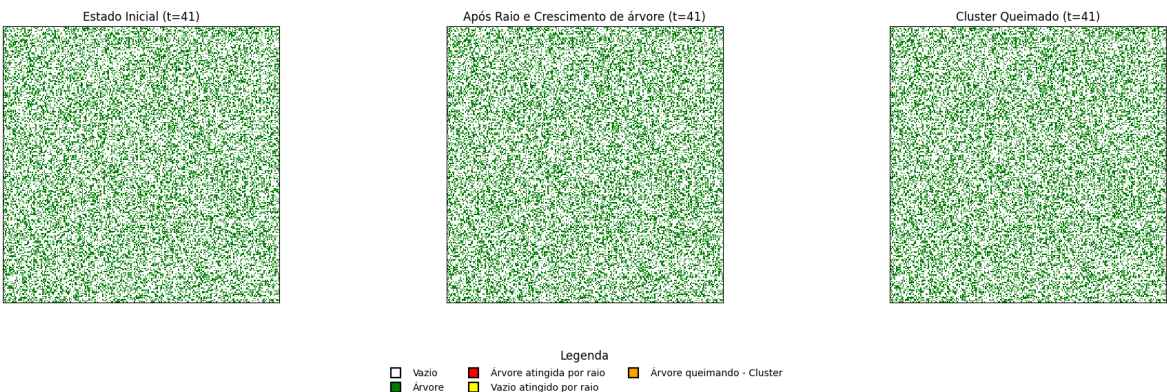
Floresta



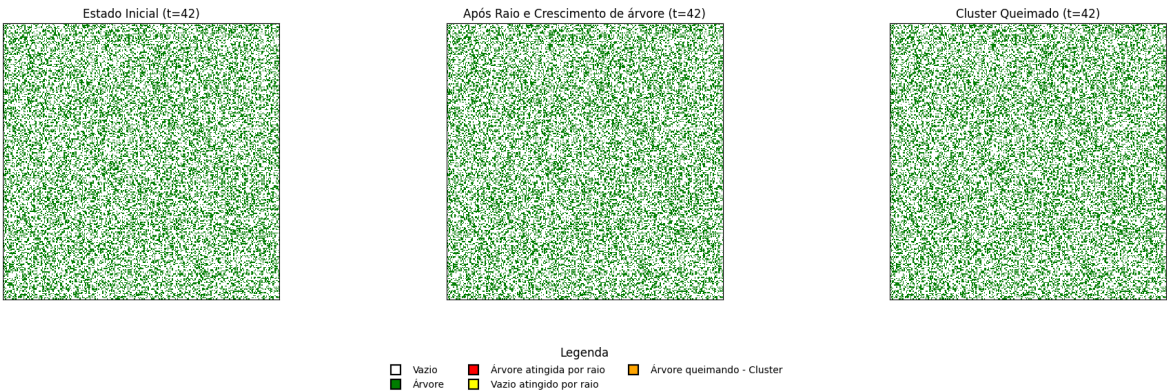
Floresta



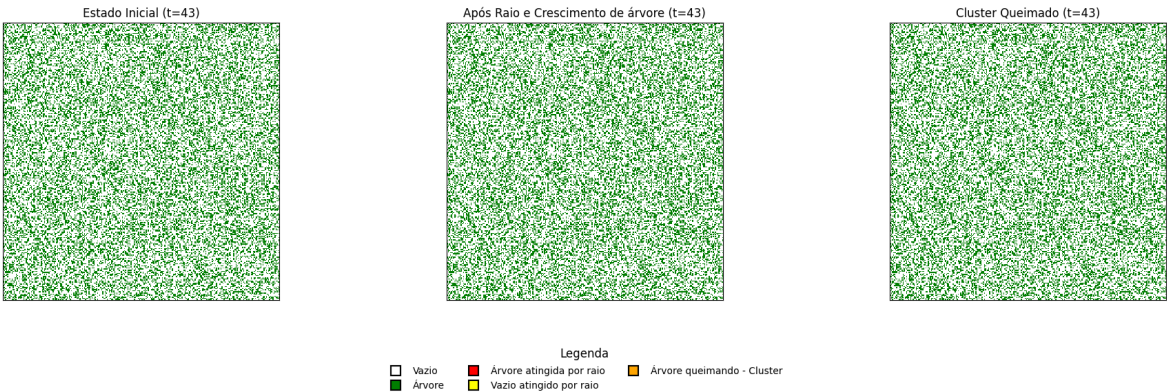
Floresta



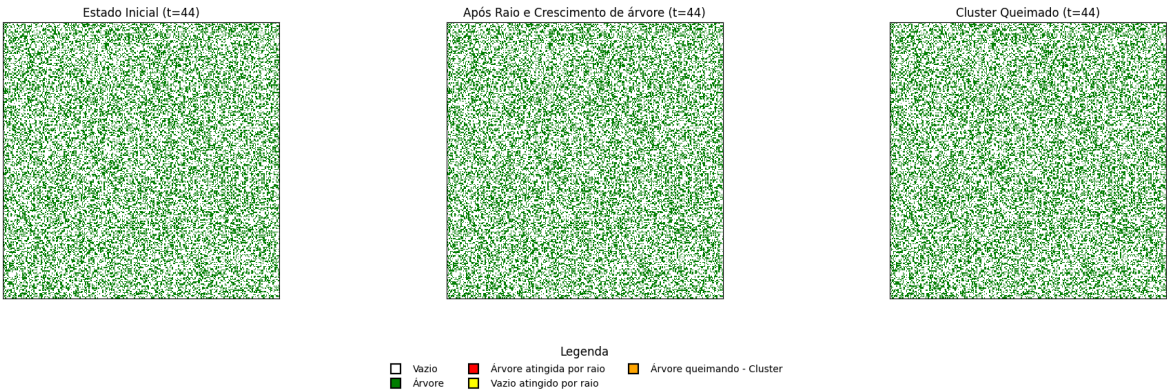
Floresta



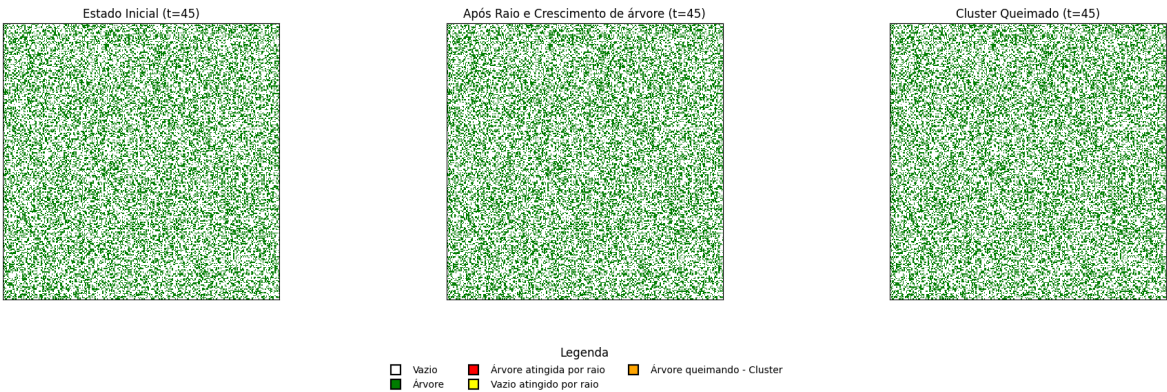
Floresta



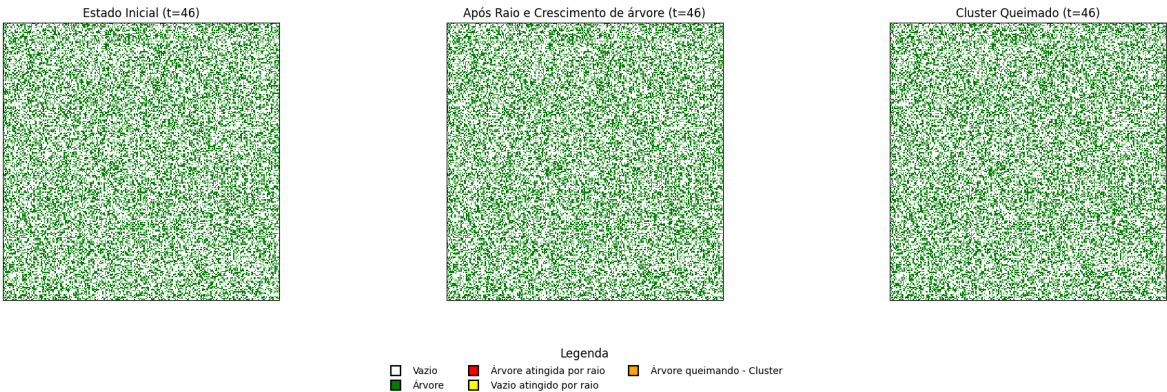
Floresta



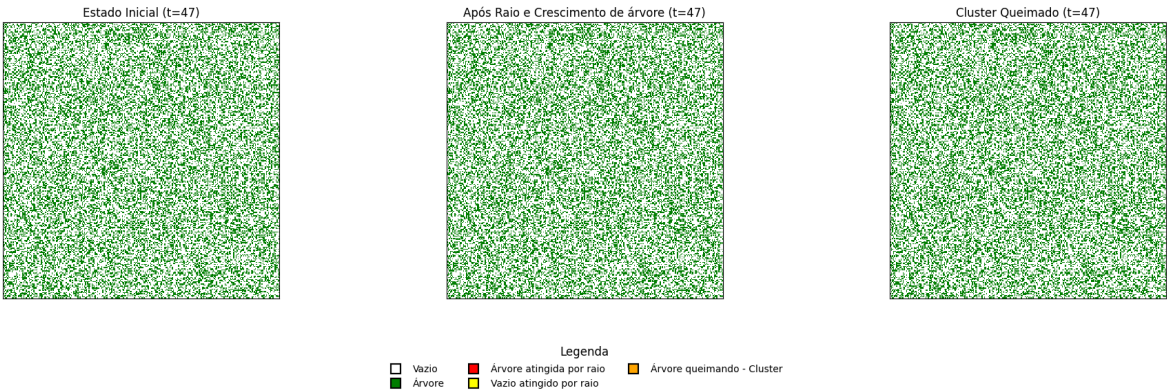
Floresta



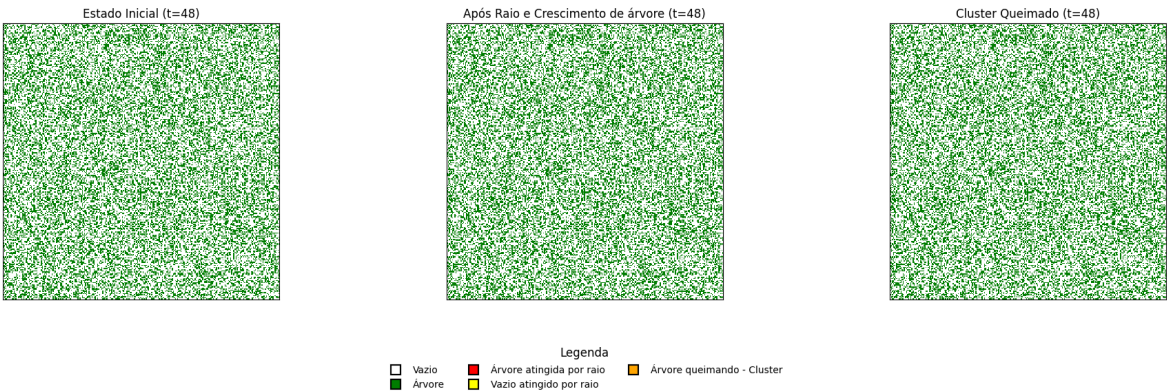
Floresta



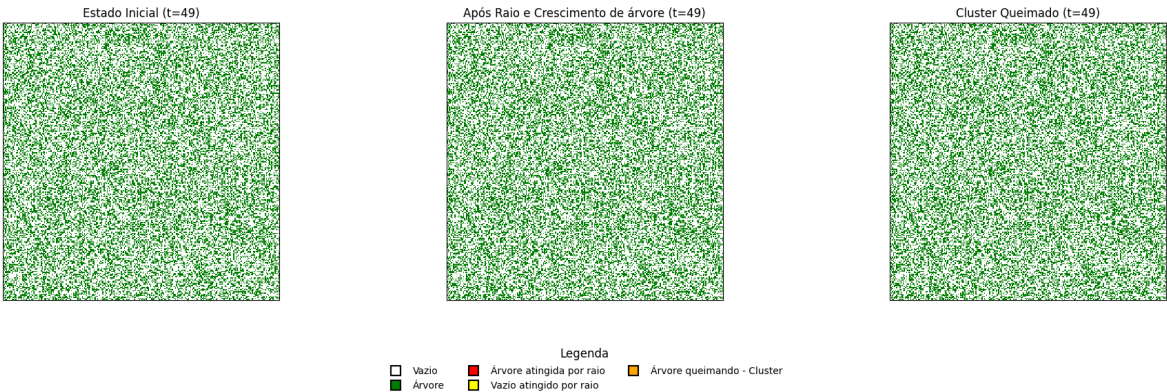
Floresta



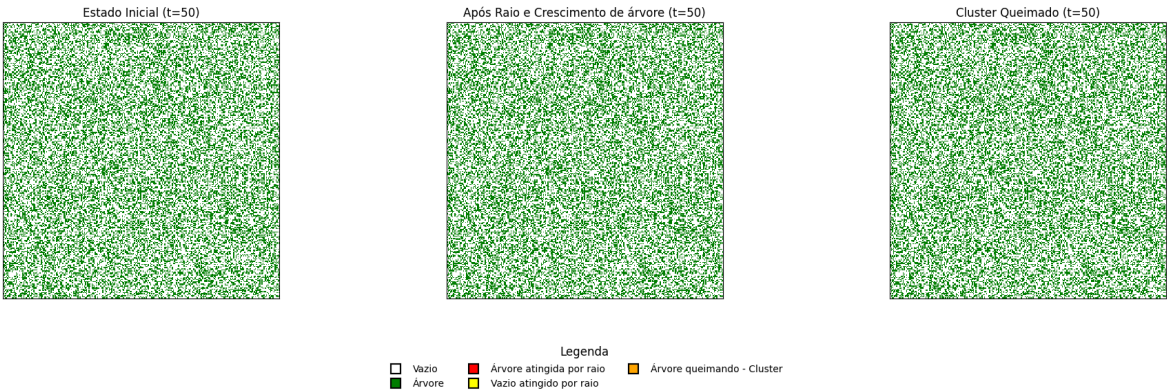
Floresta



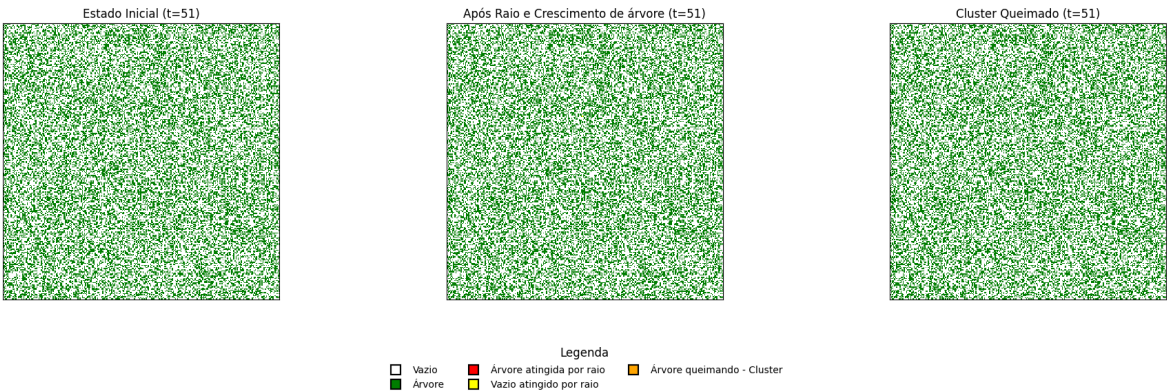
Floresta



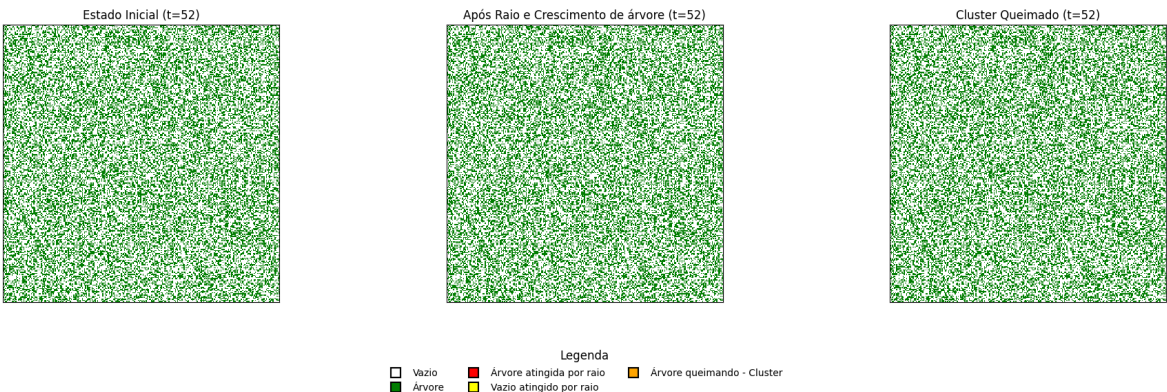
Floresta



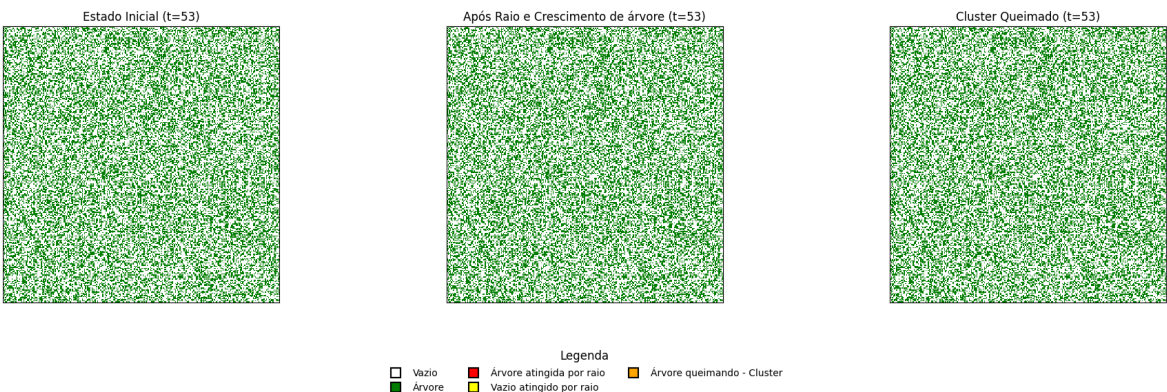
Floresta



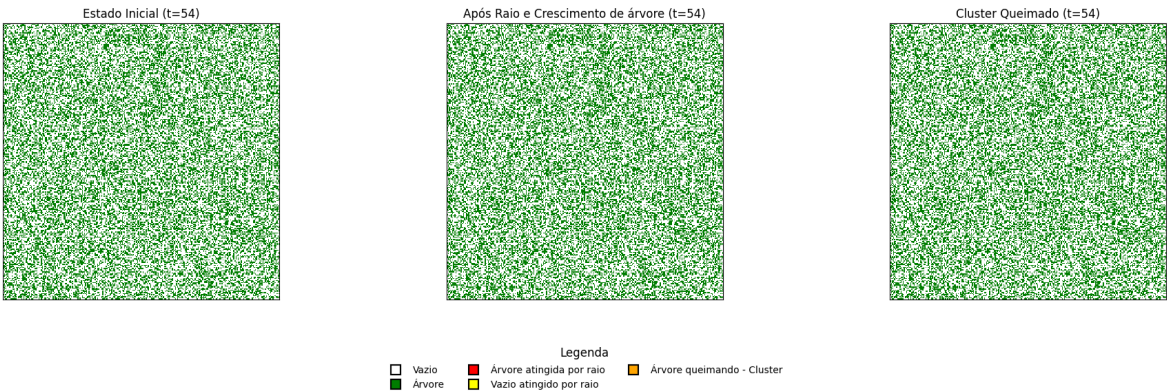
Floresta



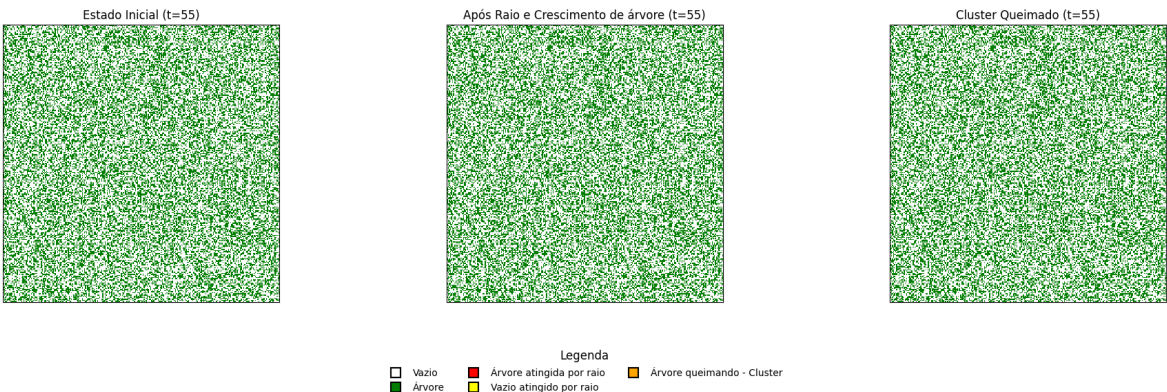
Floresta



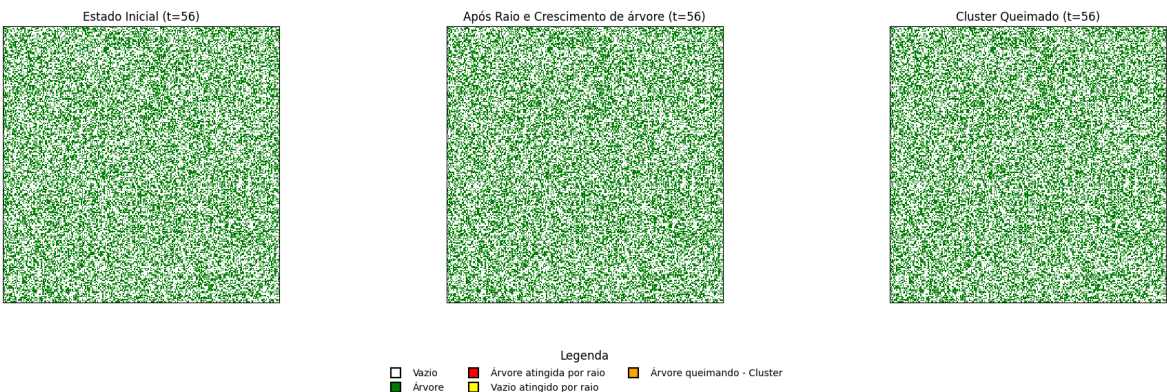
Floresta



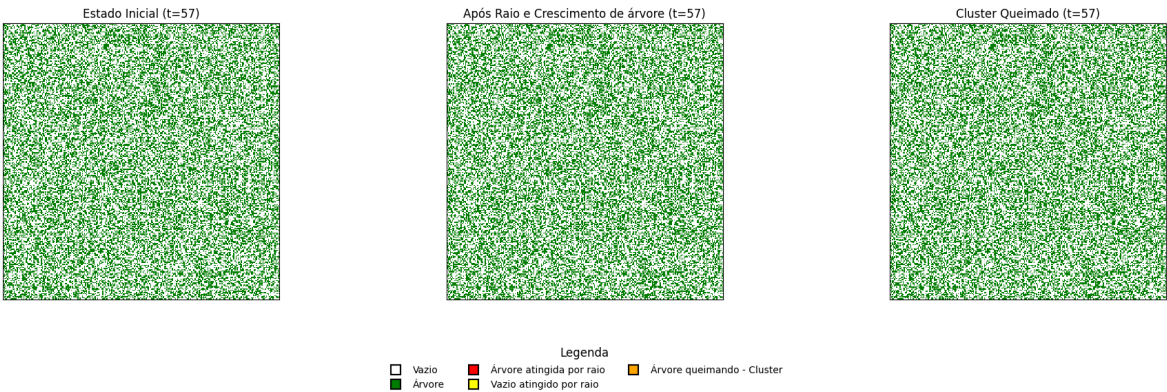
Floresta



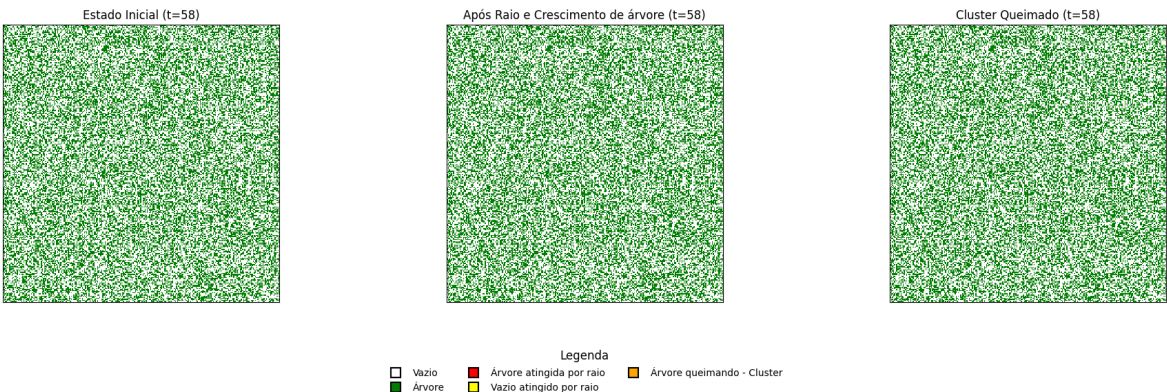
Floresta



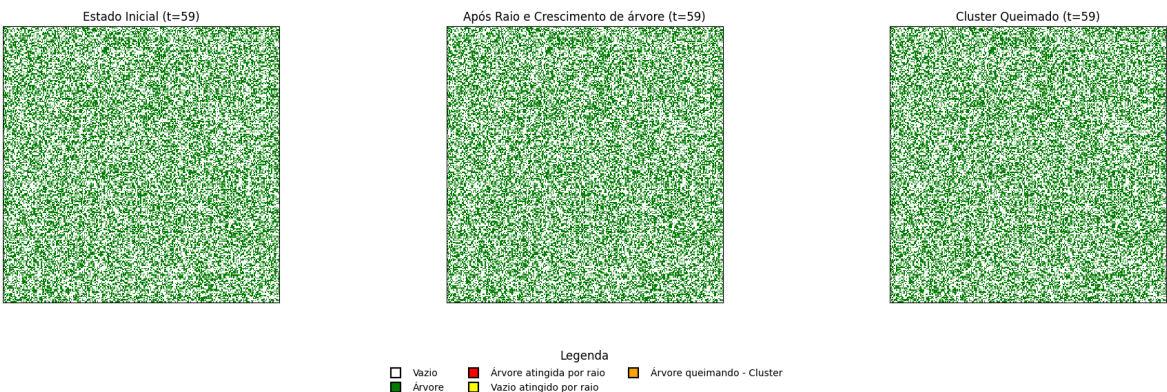
Floresta



Floresta



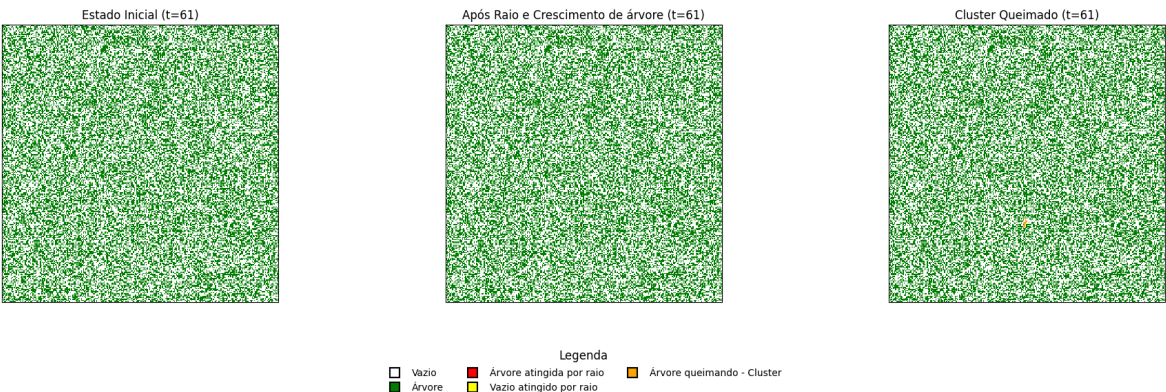
Floresta



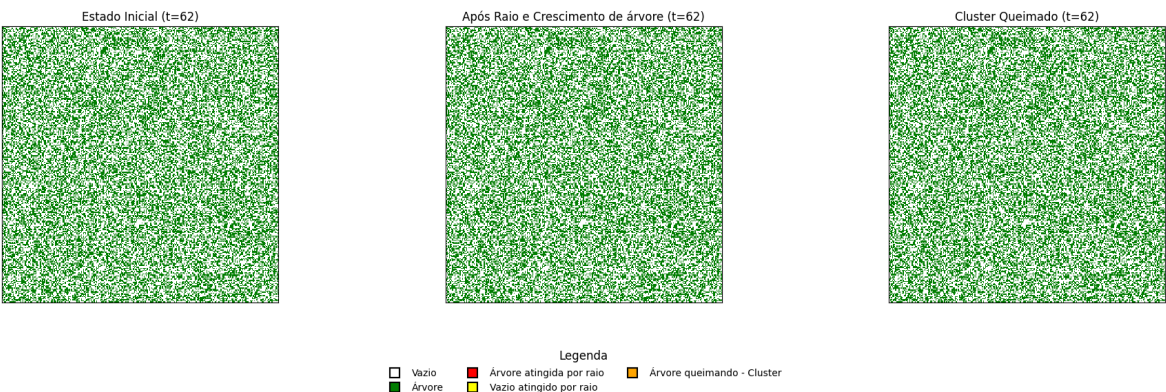
Floresta



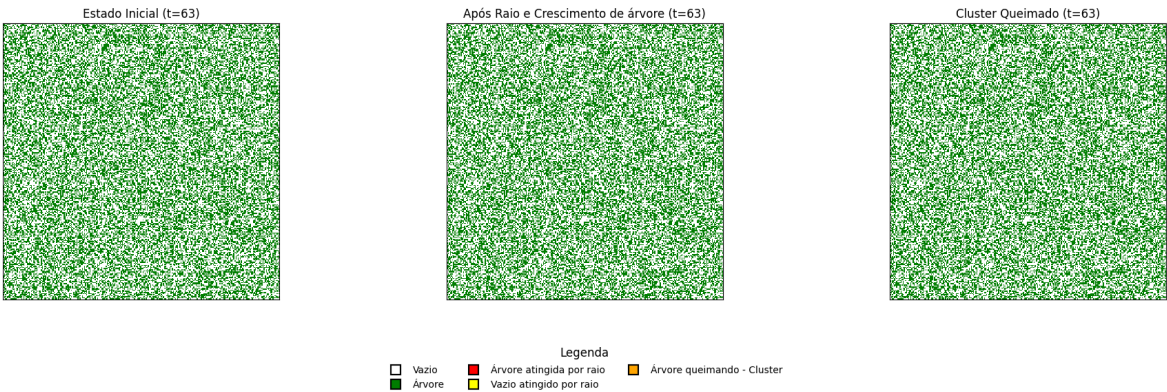
Floresta



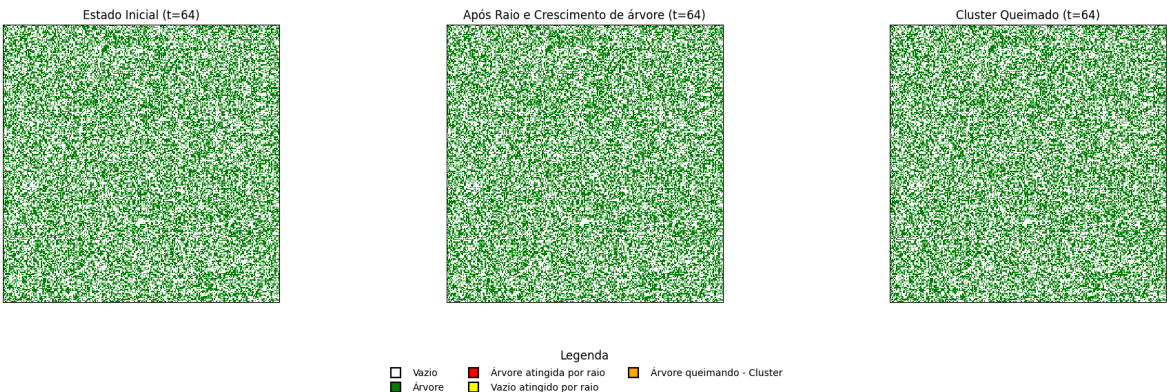
Floresta



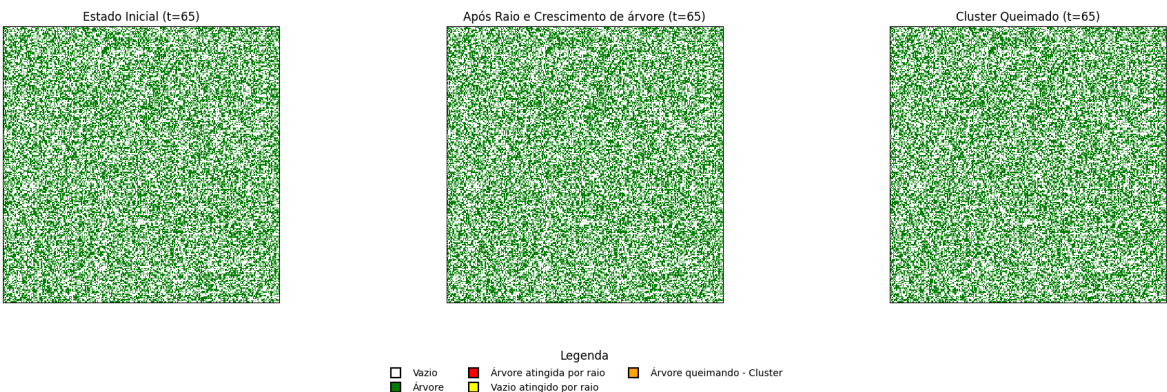
Floresta



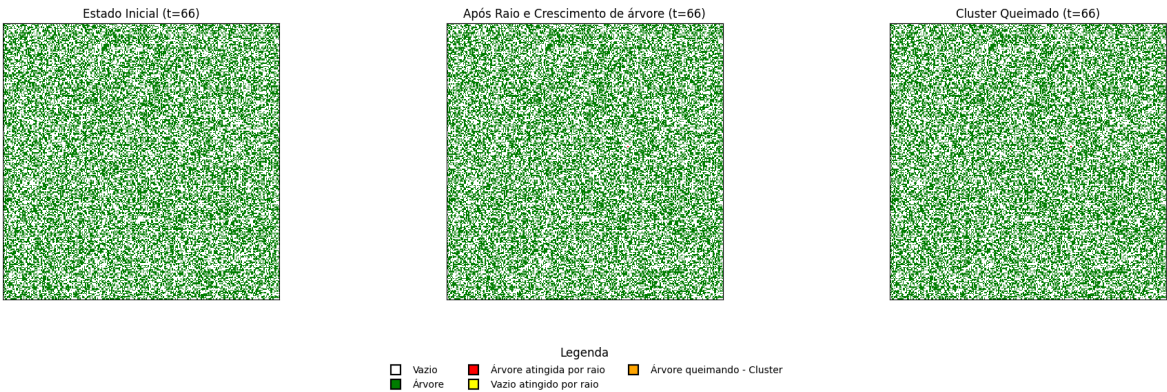
Floresta



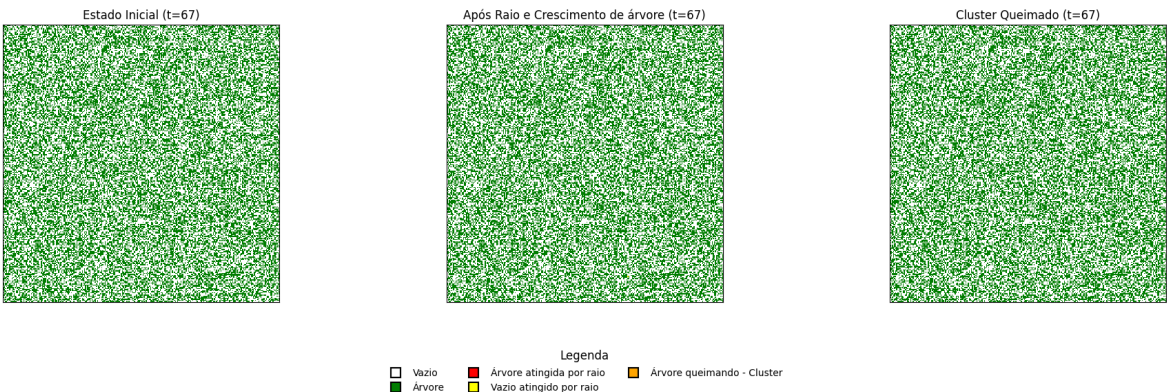
Floresta



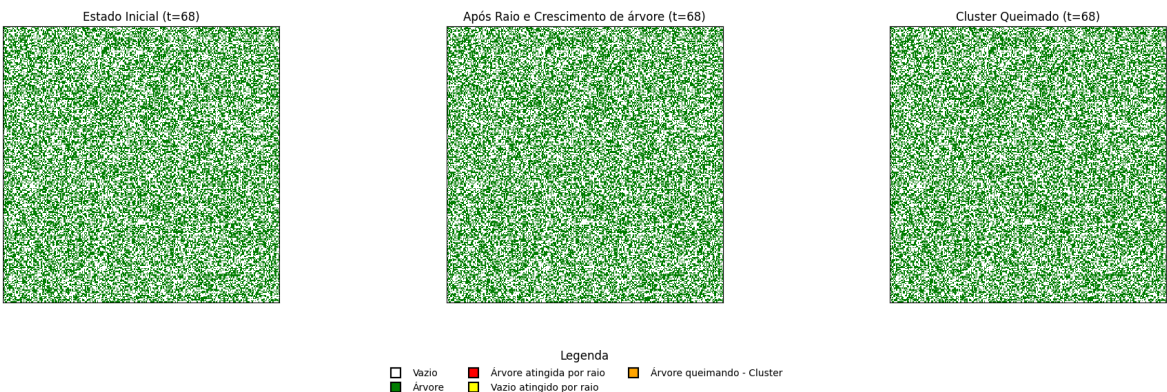
Floresta



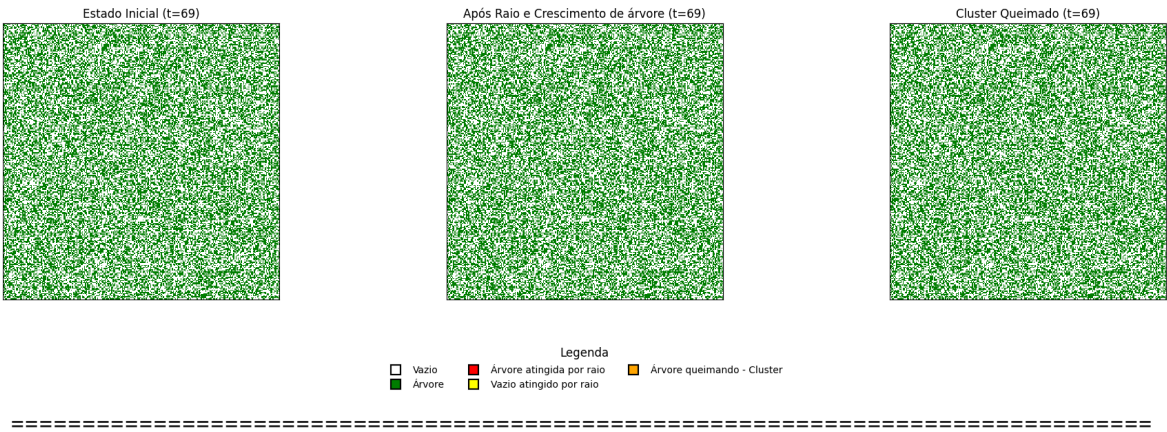
Floresta



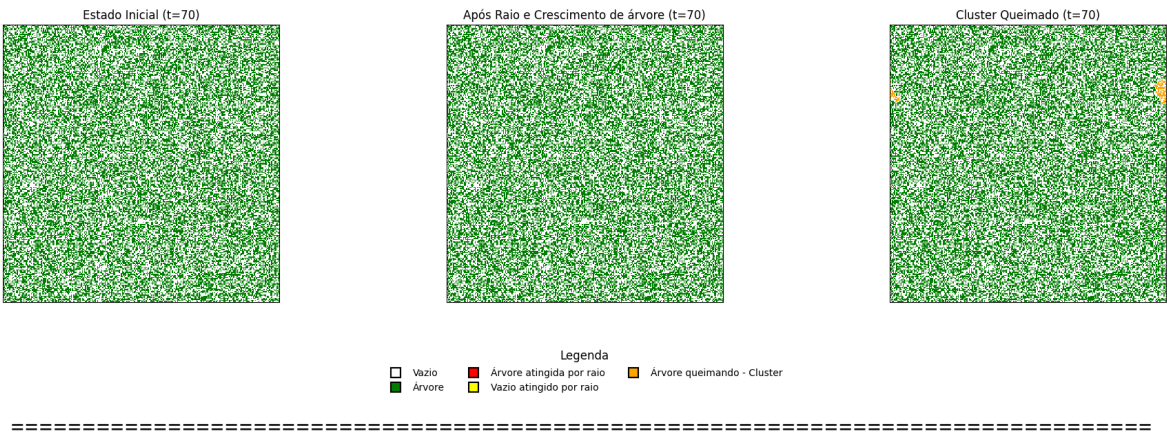
Floresta



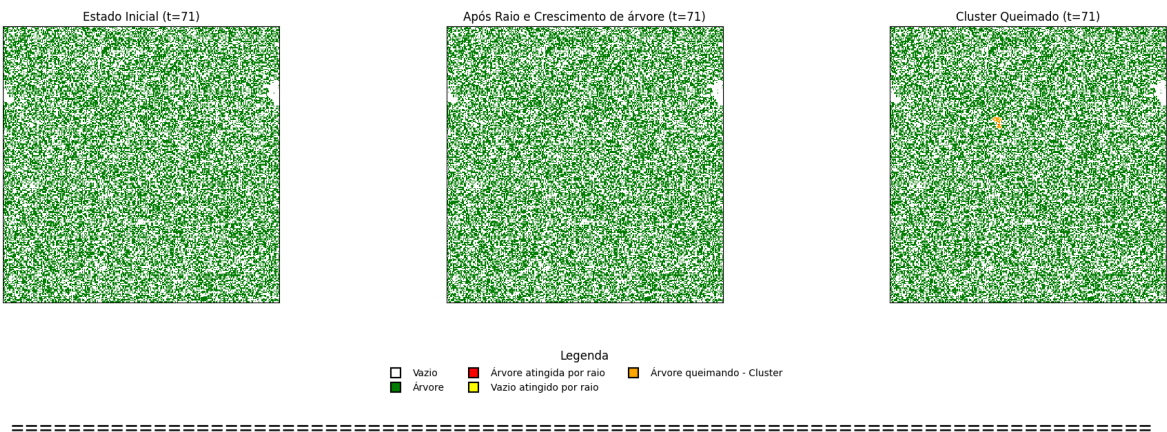
Floresta



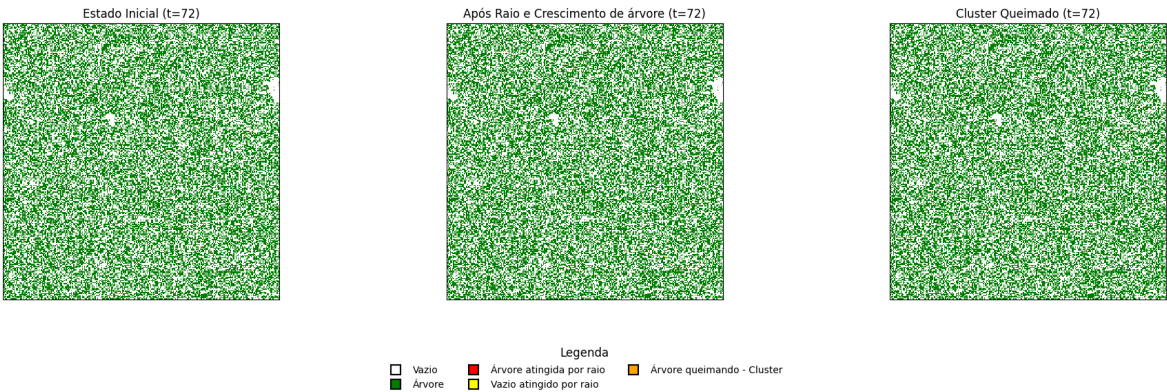
Floresta



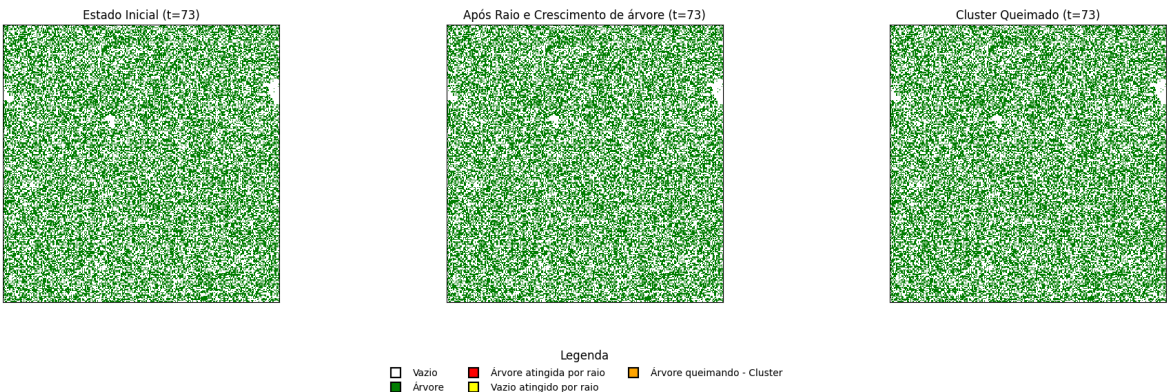
Floresta



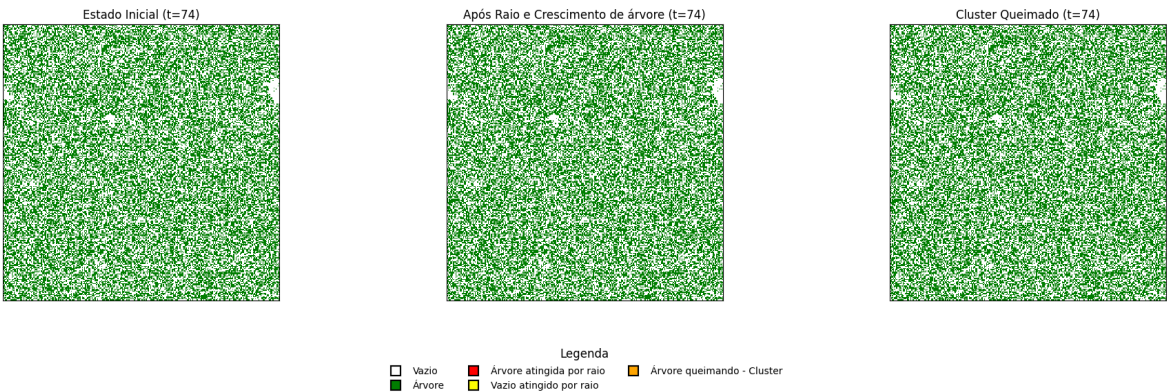
Floresta



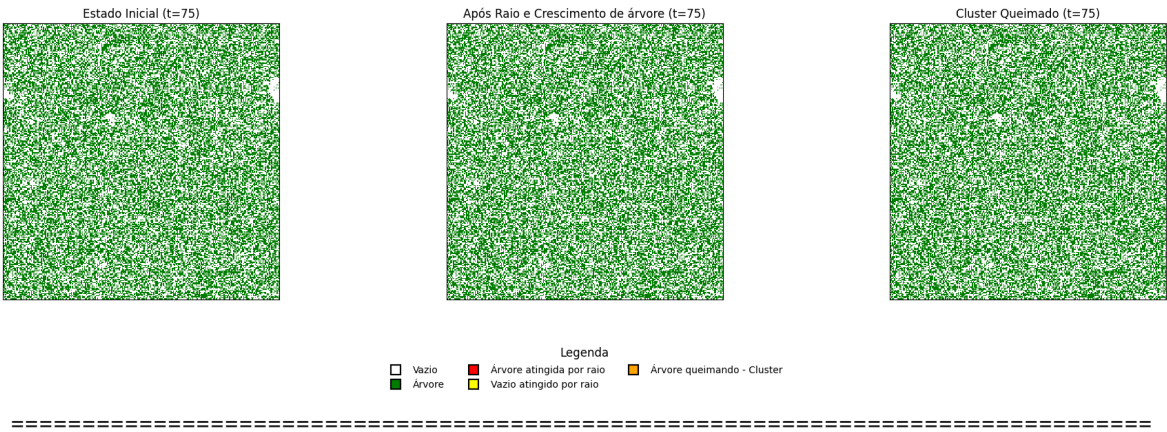
Floresta



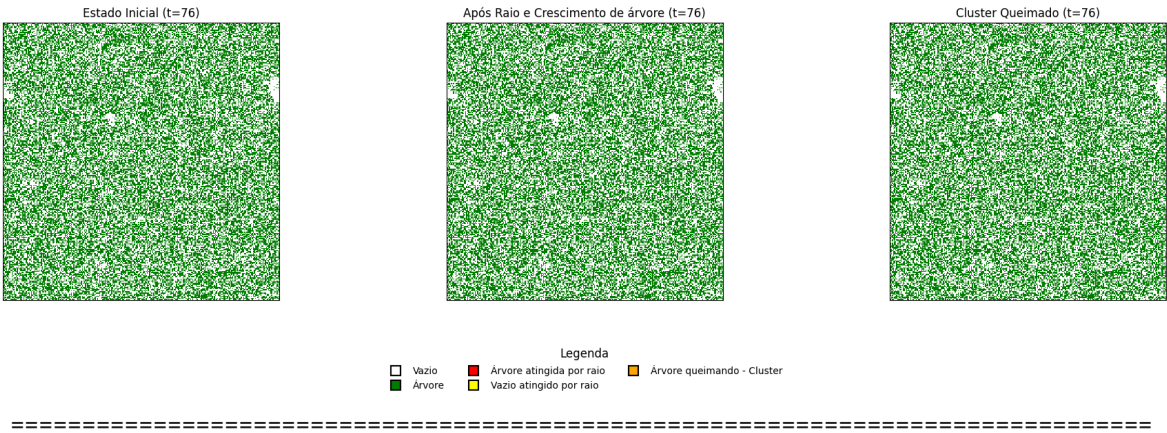
Floresta



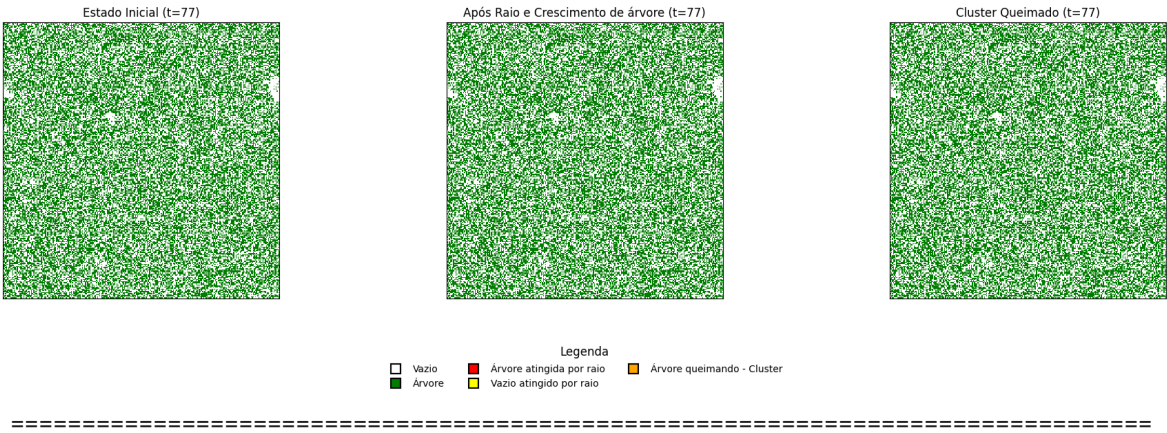
Floresta



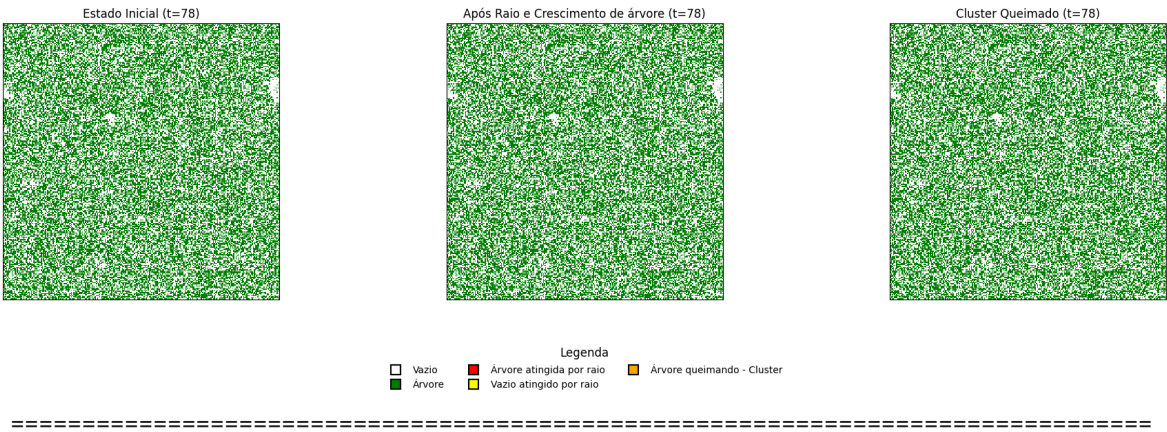
Floresta



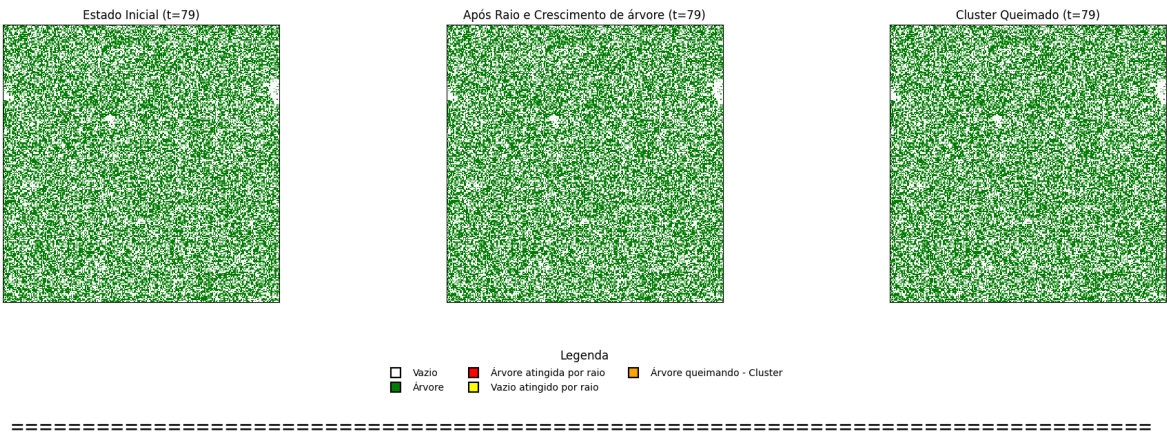
Floresta



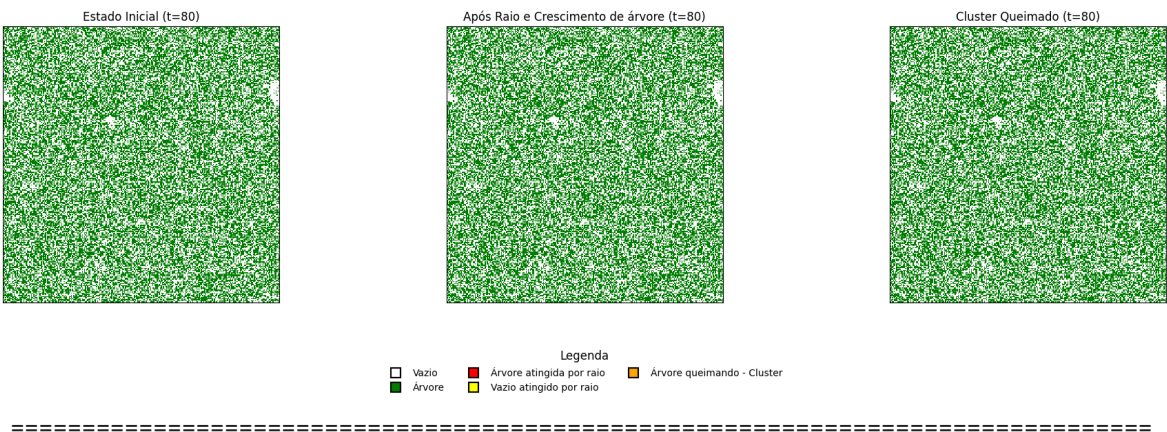
Floresta



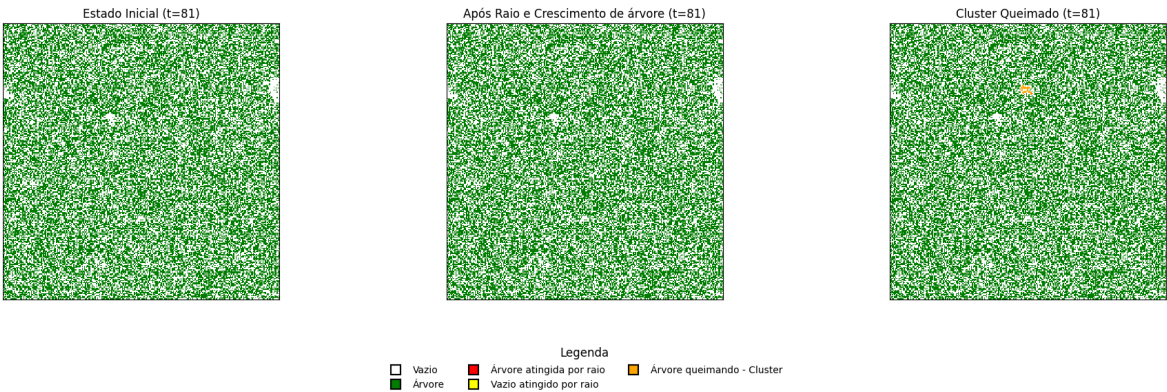
Floresta



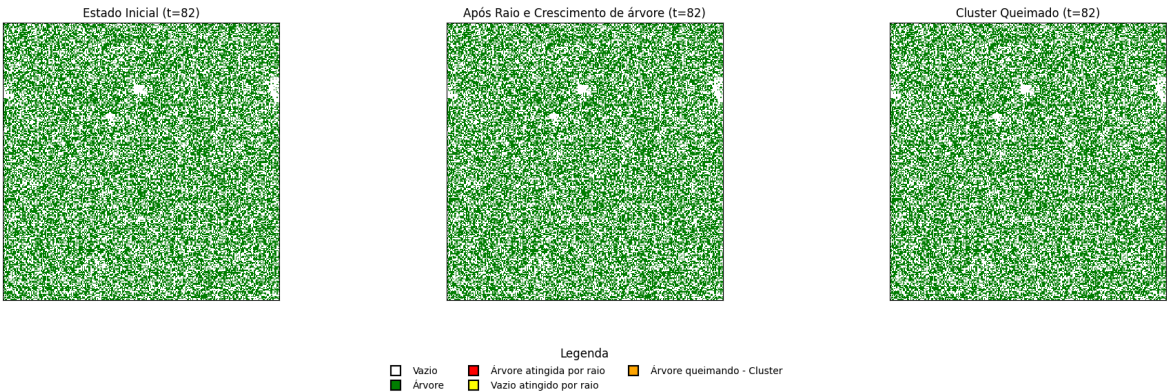
Floresta



Floresta



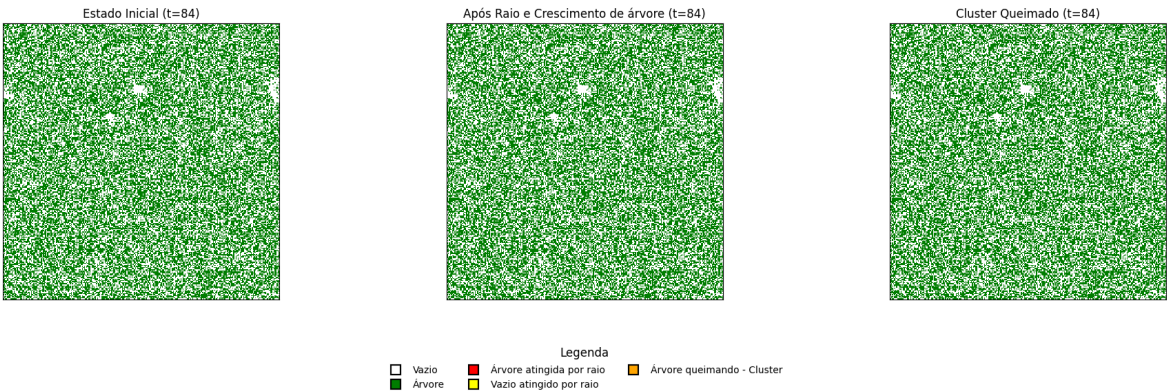
Floresta



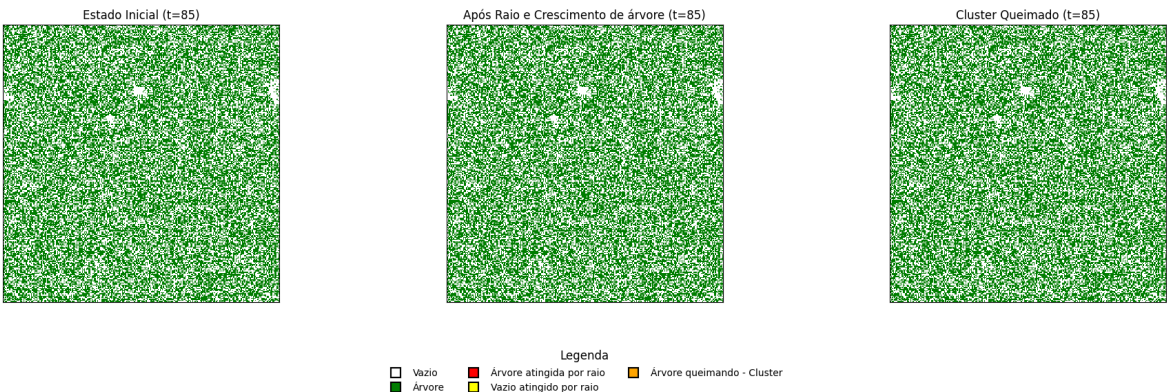
Floresta



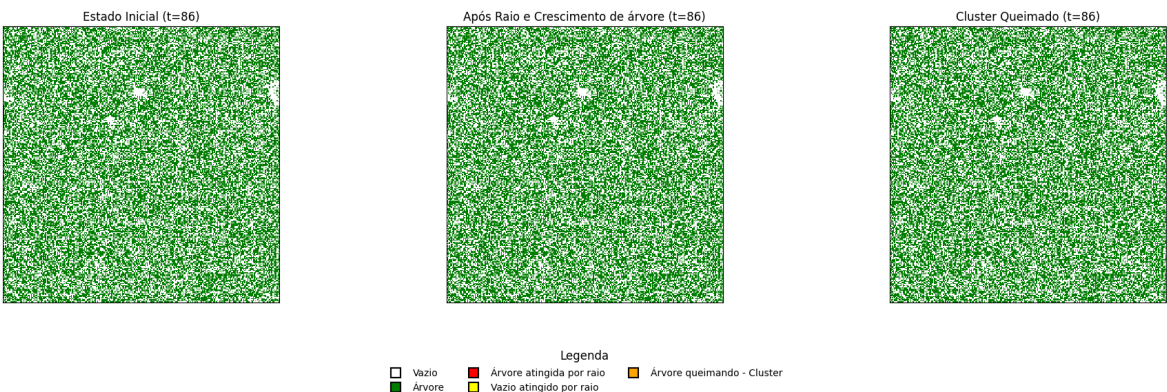
Floresta



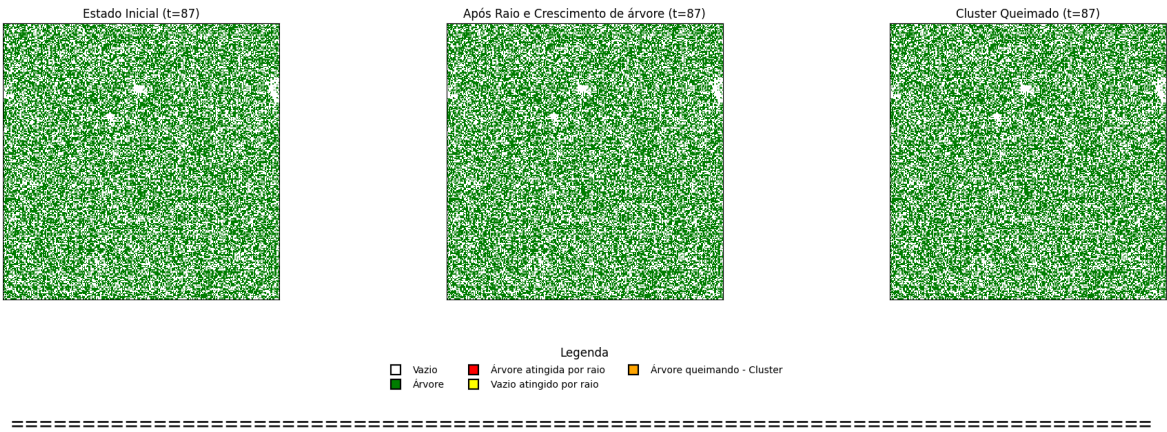
Floresta



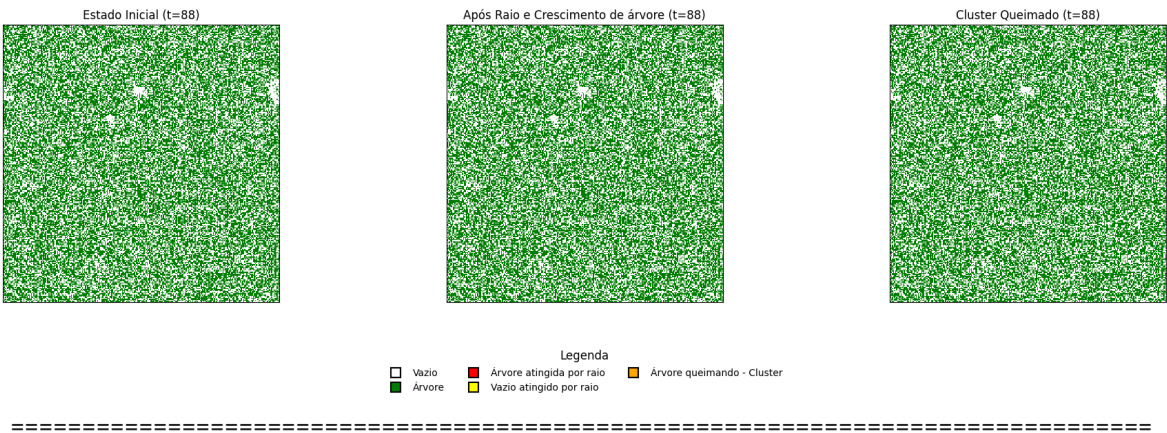
Floresta



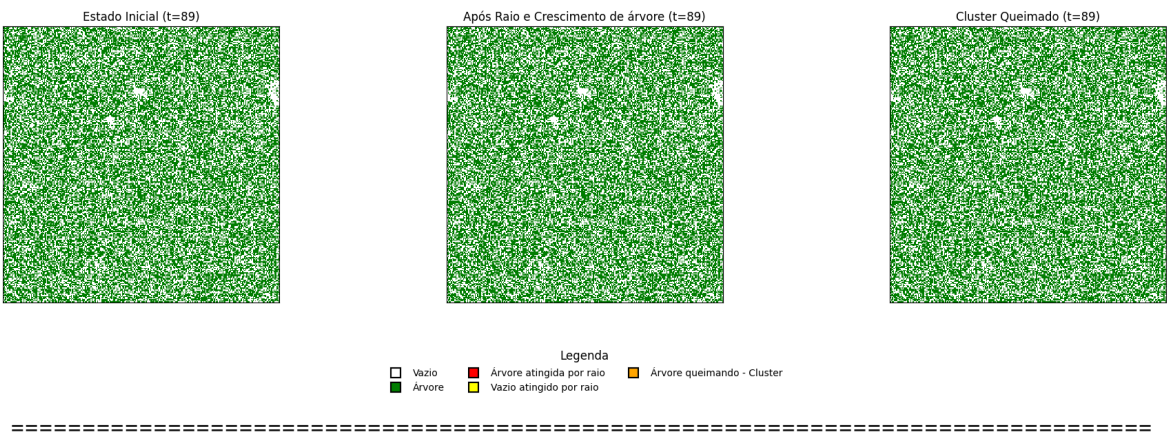
Floresta



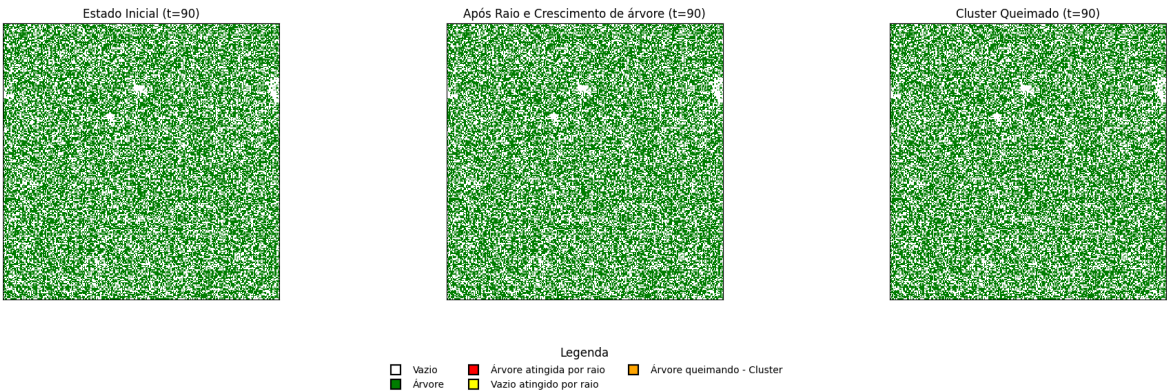
Floresta



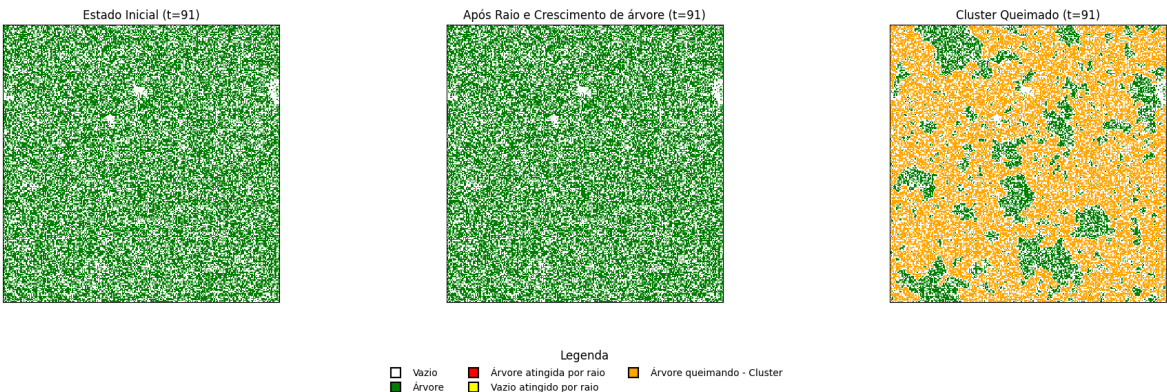
Floresta



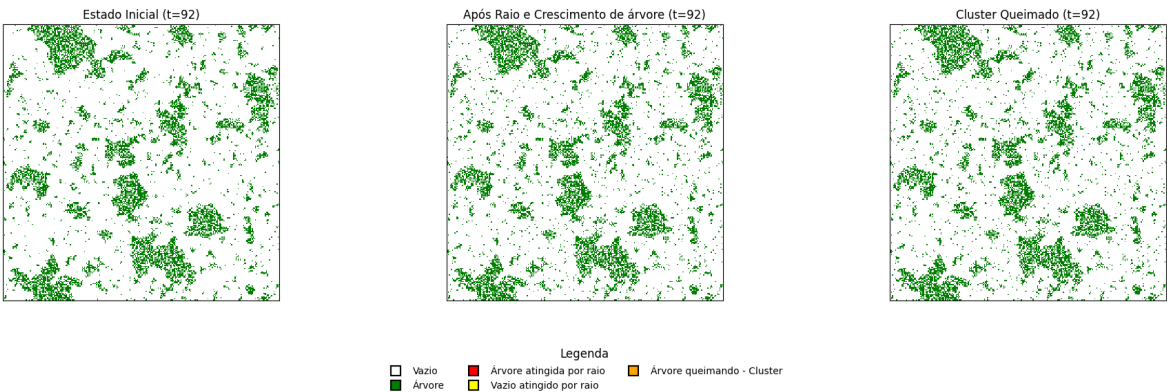
Floresta



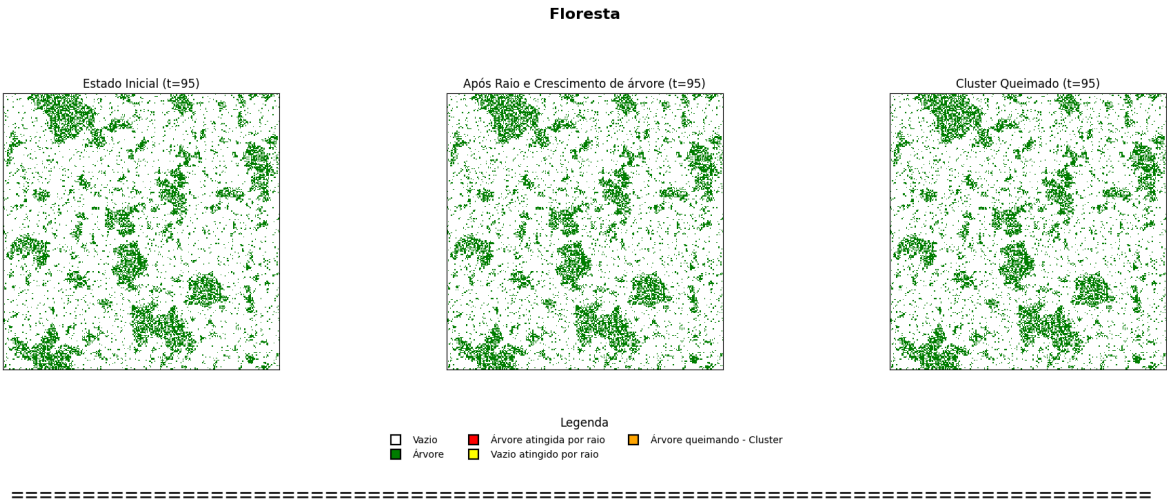
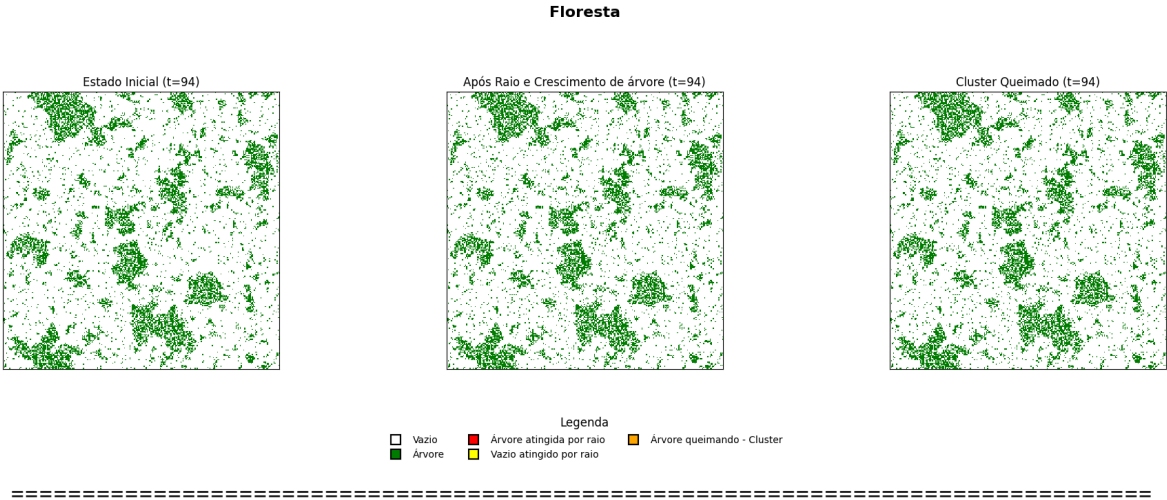
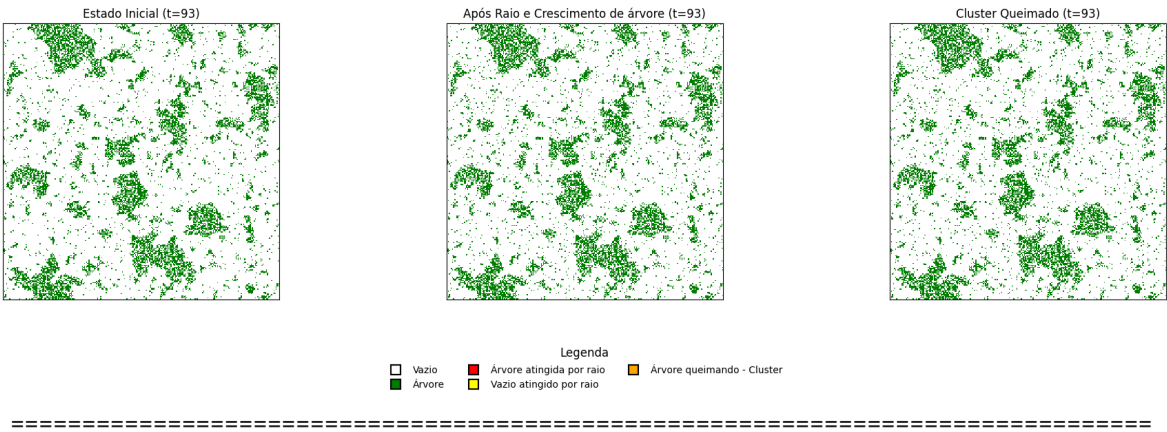
Floresta



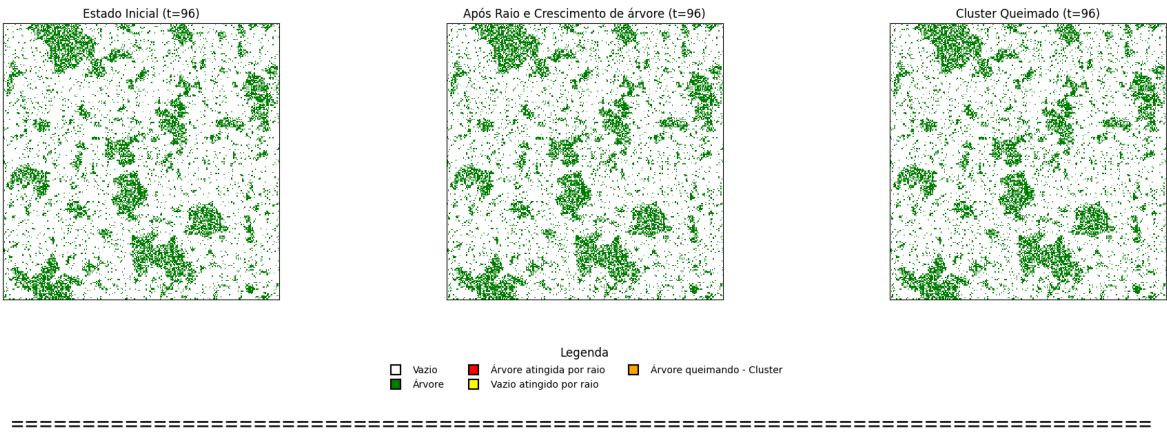
Floresta



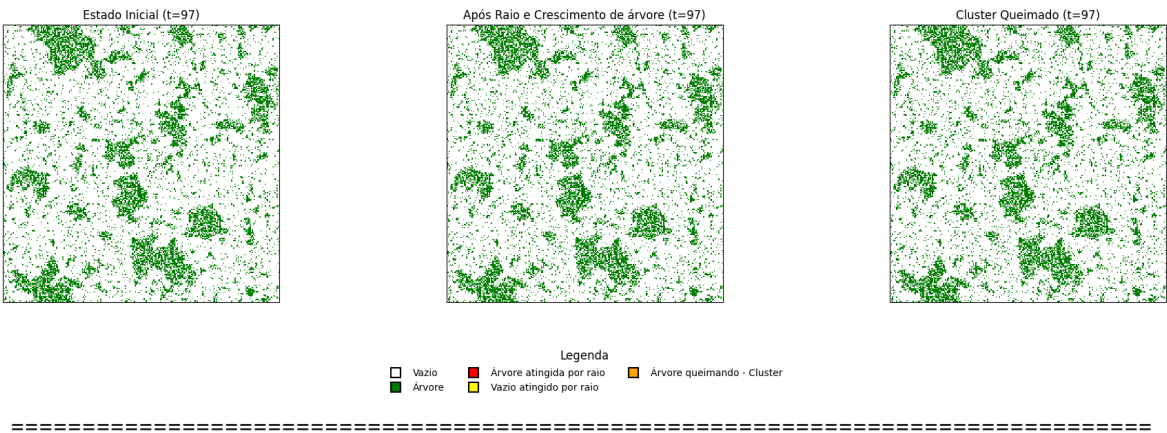
Floresta



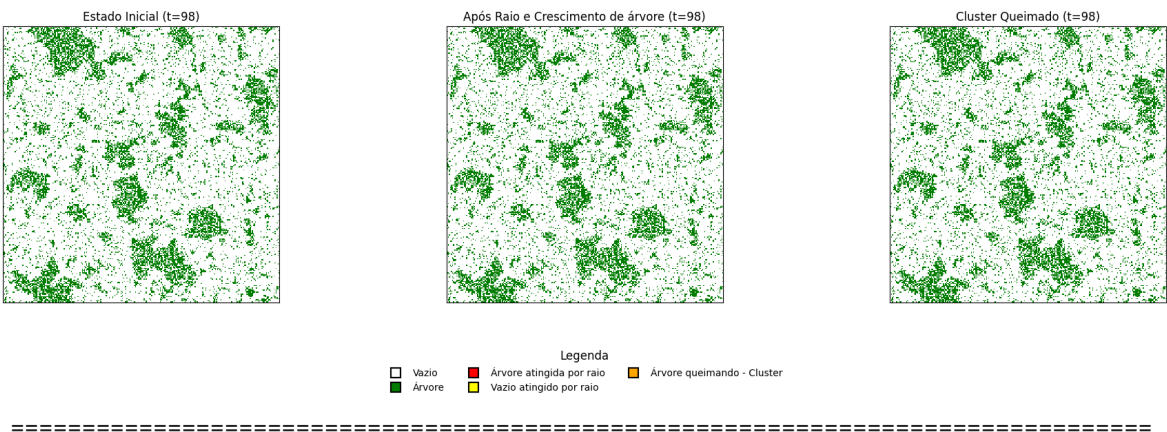
Floresta



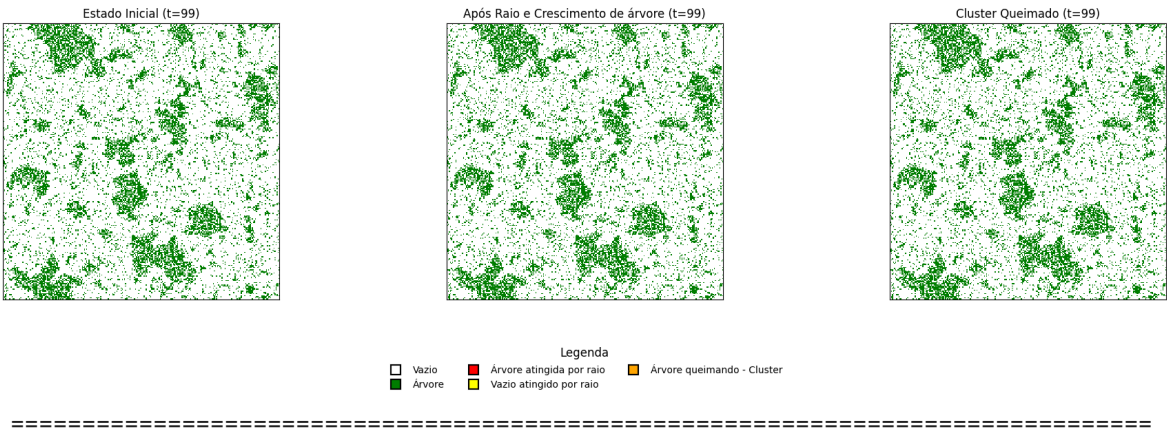
Floresta



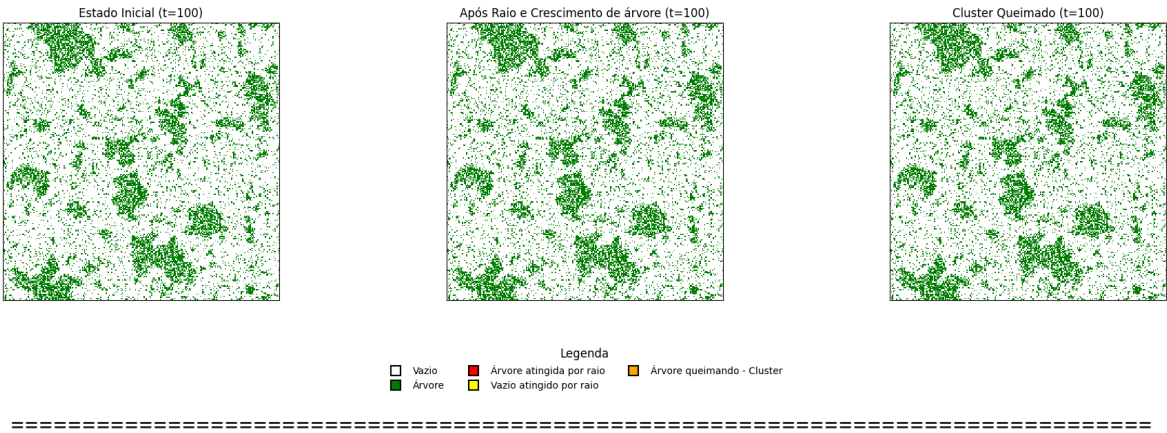
Floresta



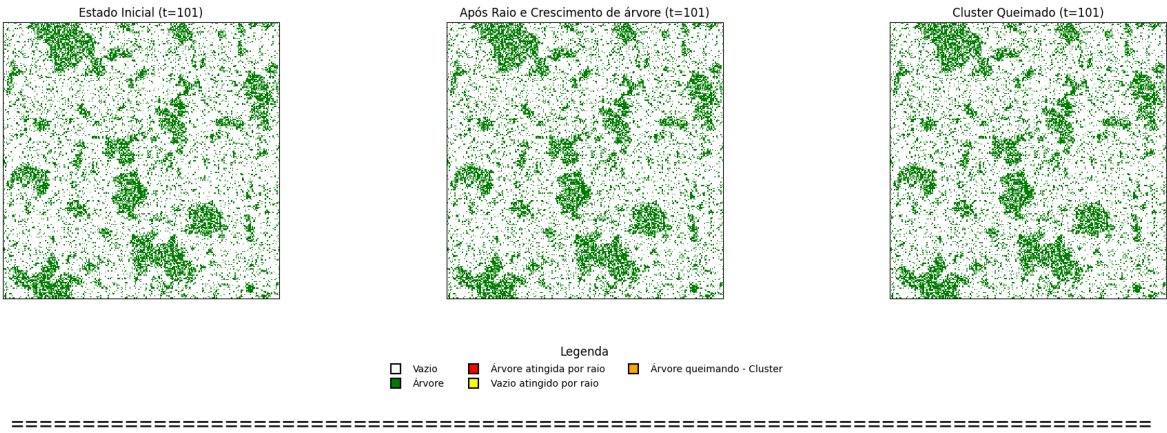
Floresta

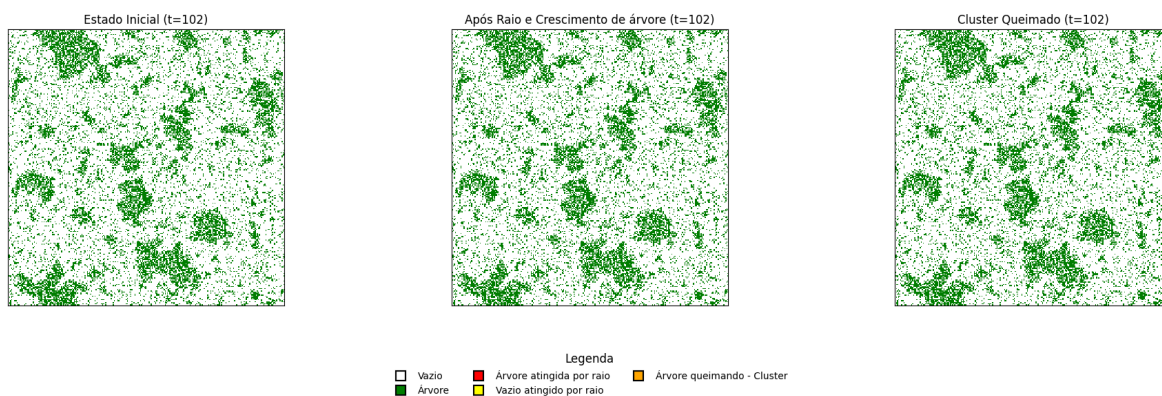
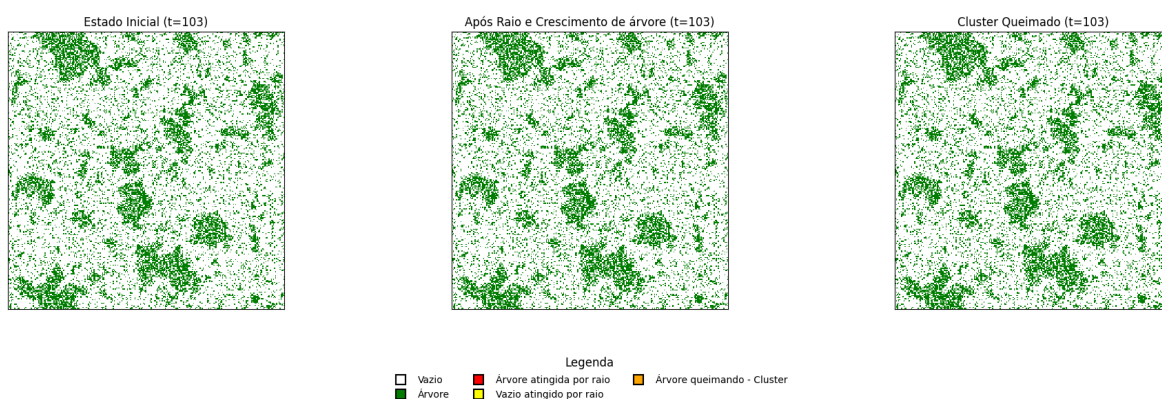
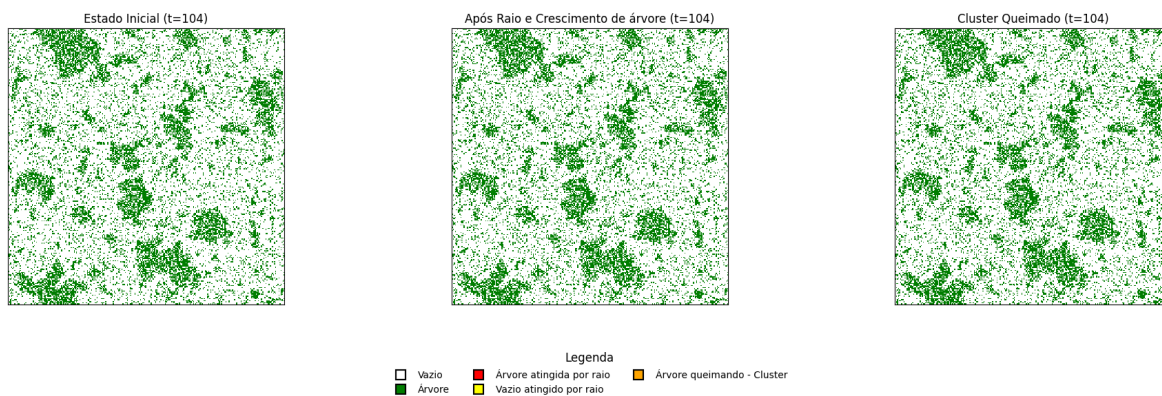


Floresta



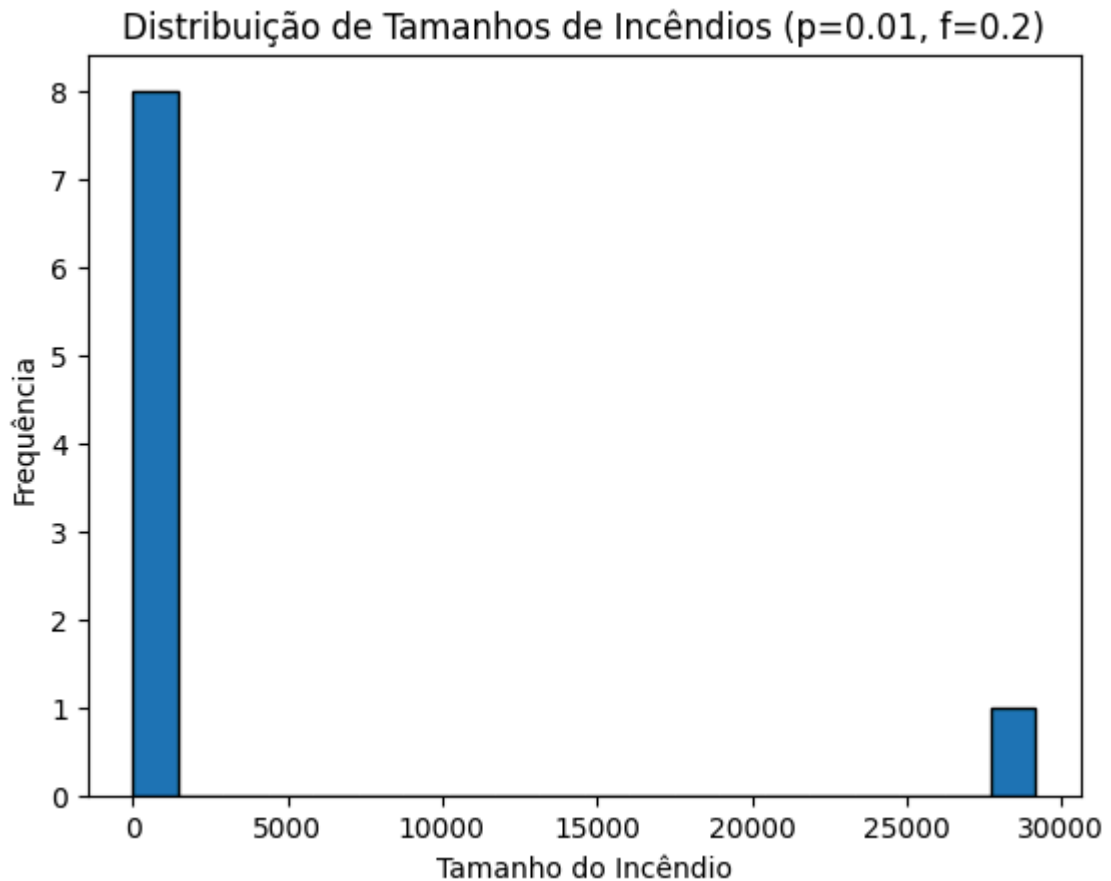
Floresta



Floresta**Floresta****Floresta**

In [137...

```
plt.hist(tam_incendios, bins=20, edgecolor='black')
plt.title(f"Distribuição de Tamanhos de Incêndios (p={p}, f={f})")
plt.xlabel("Tamanho do Incêndio")
plt.ylabel("Frequência")
plt.show()
```



Apesar da representação ruim do histograma gerado, é possível observar que ocorreram diversos incêndios pequenos, mas teve um incêndio muito grande quando a floresta cresceu muito (perto das últimas iterações).

Podemos observar a evolução da floresta nos gráficos anteriores, que condizem com o histograma.

Parte 2

Implementação

```
In [138... @jit(nopython=True)
def simula_floresta_2(L, p, f, num_incendios):
    """
    Simula uma floresta com crescimento de árvores e incêndios periódicos.
    Compatível com Numba para maior desempenho, mantendo a funcionalidade origin
    """
    N = L * L # Número total de nós na rede.
    viz = vizinhos(N) # Calcula os vizinhos de cada nó da floresta.
    tam_incendios = np.zeros(num_incendios, dtype=np.int32) # Armazena tamanhos
    contador_incendios = 0 # Contador para o número de incêndios registrados.

    # Inicializa a floresta.
    floresta = np.zeros(shape=N, dtype=np.int8)

    while contador_incendios < num_incendios:
        # Crescimento de árvores.
        crescimento = np.random.rand(N) < p
```



```

floresta[(floresta == 0) & crescimento] = 1

# Queda de raio
raio_caiu = False
local_raio = -1
if np.random.rand() < f:
    local_raio = np.random.randint(0, N)
    raio_caiu = True
    if floresta[local_raio] == 1:
        floresta[local_raio] = 2 # Vermelho para árvore atingida.
    else:
        floresta[local_raio] = 3 # Amarelo para local vazio.

# Propagação do fogo
if raio_caiu and floresta[local_raio] == 2:
    queimando = np.zeros(N, dtype=np.int8) # Array para células em chama
    cluster = np.zeros(N, dtype=np.int8) # Array para células do cluster
    queimando[local_raio] = 1
    tamanho_cluster = 0

    while np.any(queimando == 1): # Enquanto houver células queimando.
        for idx in range(N):
            if queimando[idx] == 1:
                queimando[idx] = 0
                cluster[idx] = 1 # Marca como parte do cluster.
                tamanho_cluster += 1
                for v in viz[idx]: # Propaga para os vizinhos.
                    if floresta[v] == 1:
                        floresta[v] = 4 # Laranja para árvores queimando
                        queimando[v] = 1

    tam_incendios[contador_incendios] = tamanho_cluster
    contador_incendios += 1

elif raio_caiu and floresta[local_raio] == 3:
    floresta[local_raio] = 0 # Limpa o local vazio atingido pelo raio.

# Limpa árvores queimadas e reseta floresta.
for i in range(N):
    if floresta[i] == 4: # Laranja (árvore queimando no cluster).
        floresta[i] = 0
    elif floresta[i] == 2: # Vermelho (árvore atingida diretamente).
        floresta[i] = 0

return tam_incendios

```

In [139...

```

def calcular_ccdf(tam_incendios, L):
    """
    Calcula a cCDF dos tamanhos de incêndios.
    """
    N = L * L
    tam_incendios = np.array(sorted(tam_incendios, reverse=True))

    # cCDF: Fração de incêndios maiores ou iguais a cada tamanho
    ccdf = np.arange(1, len(tam_incendios) + 1) / len(tam_incendios)

    # Tamanhos relativos
    tam_relativos = tam_incendios / N

```



```
return ccdf, tam_relativos
```

```
In [140... def plotar_ccdf(tam_relativo, ccdf, L):
    """
    Plota a cCDF em função do tamanho relativo do incêndio.
    """
    plt.figure(figsize=(8, 6))
    plt.loglog(tam_relativo, ccdf, marker='o', linestyle='-', color='blue', label=L)
    plt.title("cCDF de Tamanhos Relativos de Incêndios")
    plt.xlabel("Tamanho Relativo do Incêndio (n / L²)")
    plt.ylabel("C(n)")
    plt.grid(True, which="both", linestyle="--", linewidth=0.5)
    plt.legend()
    plt.show()
```

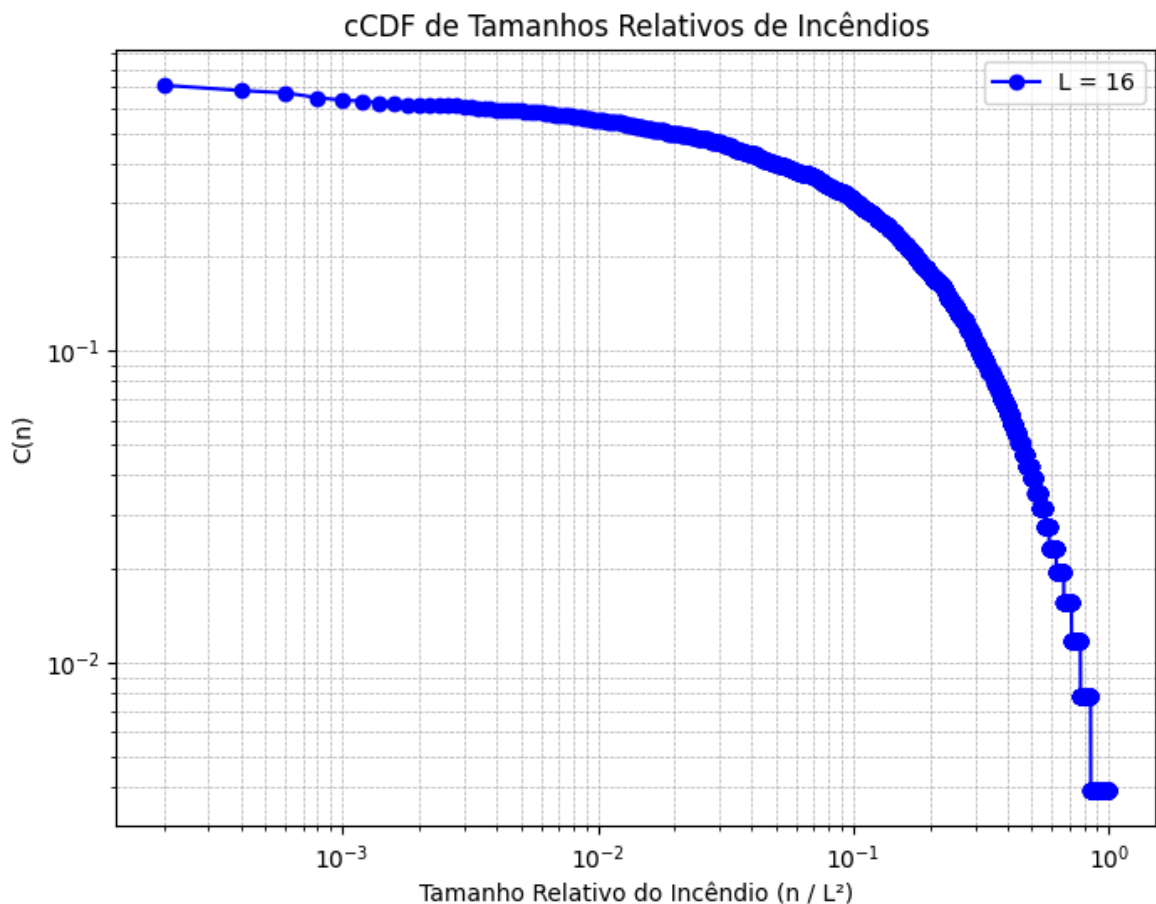
a.

```
In [141... L = 16
num_incendios = 5000
```

```
In [142... tam_incendios = simula_floresta_2(L, p, f, num_incendios)

ccdf, tam_relativos = calcular_ccdf(tam_incendios, L)

plotar_ccdf(ccdf, tam_relativos, L)
```



b.

In [143...

L = 256

In [144...

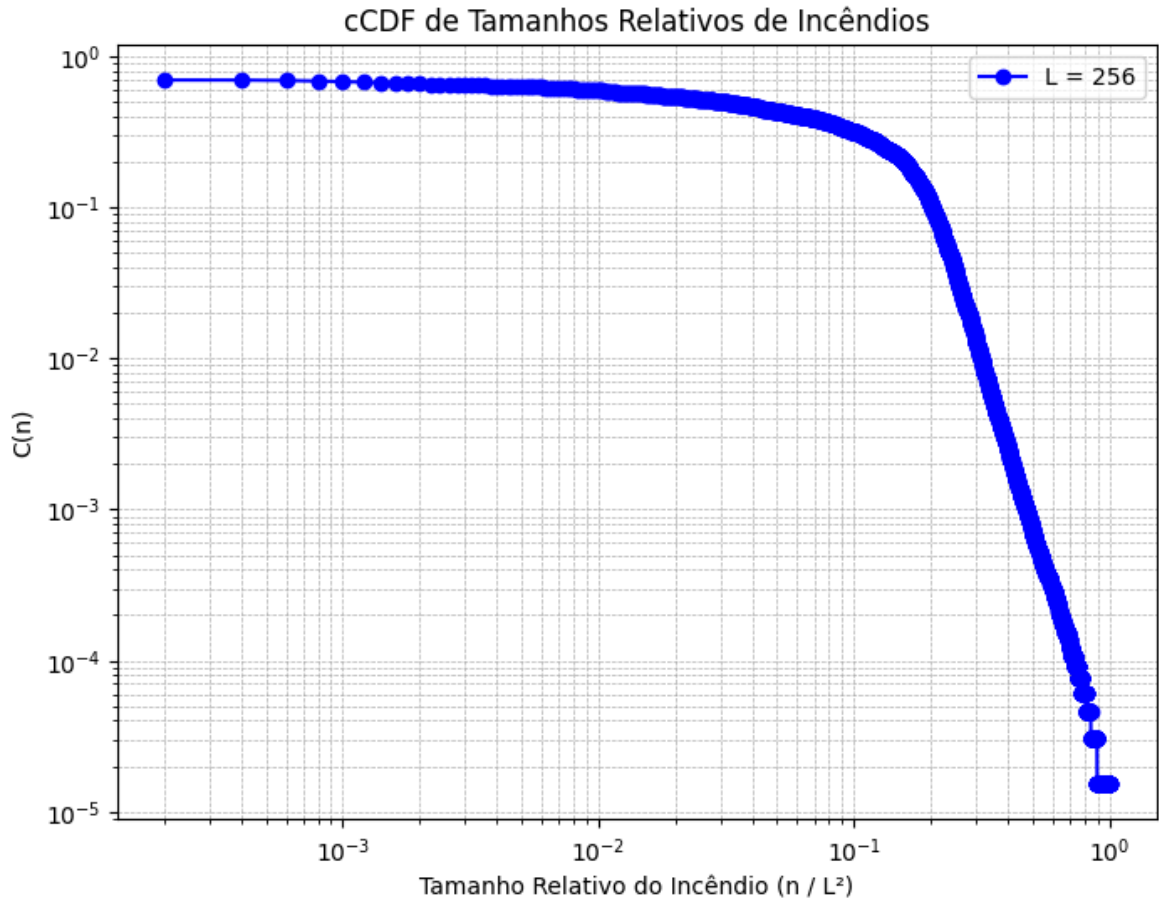
```

tam_incendios = simula_floresta_2(L, p, f, num_incendios)

ccdf, tam_relativos = calcular_ccdf(tam_incendios, L)

plotar_ccdf(ccdf, tam_relativos, L)

```



Parte 3

Implementação

In [145...

```

def simula_floresta_3(L, p, f, num_incendios, vizualizar):
    """
    Simula uma floresta com crescimento de árvores e incêndios periódicos.
    """
    N = L * L # Número total de nós na rede.
    viz = vizinhos(N) # Calcula os vizinhos de cada nó da floresta.
    tam_incendios_evoluidos = [] # Tamanhos dos incêndios na floresta evoluída.
    tam_incendios_aleatorios = [] # Tamanhos dos incêndios na floresta aleatória.
    contador_incendios = 0 # Contador para o número de incêndios registrados.

    # Inicializa a floresta
    floresta = np.zeros(shape=N, dtype=np.int8)

    while contador_incendios < num_incendios:
        # Estado inicial da iteração
        estado_inicial = floresta.copy()

```

```

# Crescimento de árvores
crescimento = np.random.rand(N) < p
floresta[(floresta == 0) & crescimento] = 1

# Queda de raio
raio_caiu = False
local_raio = -1
if np.random.rand() < f:
    local_raio = np.random.randint(0, N)
    raio_caiu = True
    if floresta[local_raio] == 1:
        floresta[local_raio] = 2 # Vermelho para árvore atingida
    else:
        floresta[local_raio] = 3 # Amarelo para local vazio

estado_raio = floresta.copy()

# Propagação do fogo
if raio_caiu and floresta[local_raio] == 2:

    # Número de árvores antes do incêndio
    num_arvores = np.sum(floresta == 1) + 1

    queimando = [local_raio]
    cluster = []
    while queimando:
        idx = queimando.pop()
        if idx not in cluster:
            cluster.append(idx)
            for v in viz[idx]:
                if floresta[v] == 1:
                    floresta[v] = 4
                    queimando.append(v)
    tam_incendios_evoluidos.append(len(cluster))
    contador_incendios += 1

    # Gerar floresta aleatória com o mesmo número de árvores
    floresta_aleatoria = np.zeros(shape=N, dtype=np.int8)
    indices_arvores = np.random.choice(np.arange(N), size=num_arvores, r
    floresta_aleatoria[indices_arvores] = 1

    estado_inicial_aleatoria = floresta_aleatoria.copy()

    # Incêndio na floresta aleatória
    floresta_aleatoria[indices_arvores[0]] = 2
    estado_raio_aleatoria = floresta_aleatoria.copy()
    queimando = [indices_arvores[0]] # Escolhe uma árvore aleatória par
    cluster = []
    while queimando:
        idx = queimando.pop()
        if idx not in cluster:
            cluster.append(idx)
            for v in viz[idx]:
                if floresta_aleatoria[v] == 1:
                    floresta_aleatoria[v] = 4
                    queimando.append(v)
    tam_incendios_aleatorios.append(len(cluster)) # Tamanho do incêndio

    estado_cluster_aleatoria = floresta_aleatoria.copy()

```

```

elif raio_caiu and floresta[local_raio] == 3:
    floresta[local_raio] = 0

# Estado do cluster queimado
estado_cluster = floresta.copy()

# Visualizações
if visualizar:
    visualizar_floresta(estado_inicial, estado_raio, estado_cluster, L,
    if raio_caiu and floresta[local_raio] == 2:
        visualizar_floresta(estado_inicial_aleatoria, estado_raio_aleatori
    print("\n" + "=" * 80 + "\n")

# Limpa arvores queimadas
for i in range(N):
    if floresta[i] == 4:
        floresta[i] = 0
    elif floresta[i] == 2:
        floresta[i] = 0

return tam_incendios_evoluidos, tam_incendios_aleatorios

```

```

In [146... def plotar_ccdf_multiplas(ccdfs, tam_relativos, labels, L):
    """
    Plota múltiplas cCDFs em um único gráfico para comparação.
    """
    plt.figure(figsize=(10, 7))

    # Itera sobre as cCDFs e seus rótulos
    for ccdf, tam_relativo, label in zip(ccdfs, tam_relativos, labels):
        plt.loglog(tam_relativo, ccdf, marker='o', linestyle='-', label=label)

    # Configurações do gráfico
    plt.title(f"Comparação das cCDFs para L = {L}")
    plt.xlabel("Tamanho Relativo do Incêndio (n / L²)")
    plt.ylabel("C(n)")
    plt.grid(True, which="both", linestyle="--", linewidth=0.5)
    plt.legend(fontsize=10)
    plt.show()

```

a.

```

In [147... L = 128
num_incendios = 1000
visualizar = False

```

```

In [148... tam_incendios_evoluidos, tam_incendios_aleatorios = simula_floresta_3(L, p, f, n

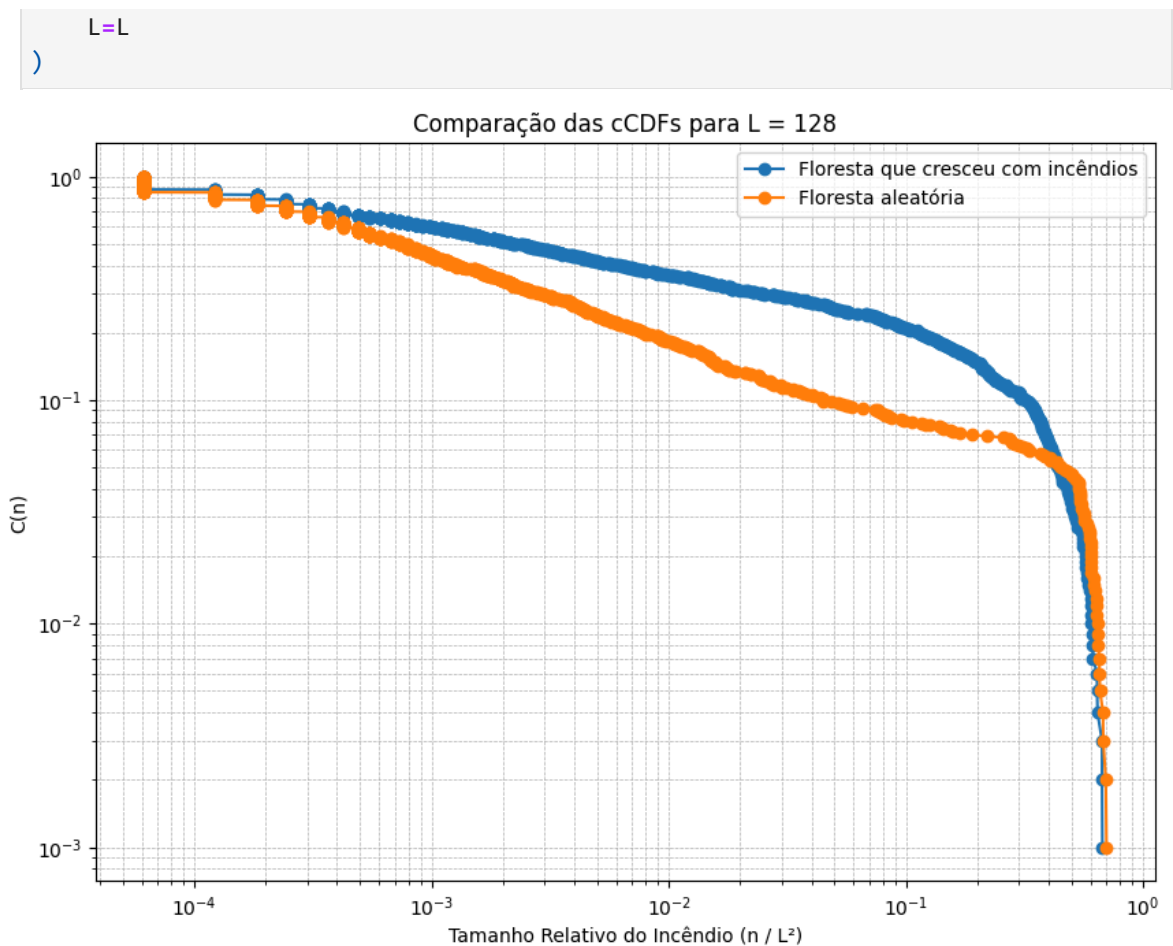
```

```

In [149... ccdf_evoluidos, tam_relativos_evoluidos = calcular_ccdf(tam_incendios_evoluidos,
ccdf_aleatorios, tam_relativos_aleatorios = calcular_ccdf(tam_incendios_aleatori

# Plota as cCDFs
plotar_ccdf_multiplas(
    ccdfs=[ccdf_evoluidos, ccdf_aleatorios],
    tam_relativos=[tam_relativos_evoluidos, tam_relativos_aleatorios],
    labels=["Floresta que cresceu com incêndios", "Floresta aleatória"],

```



b.

Incêndios muito pequenos - Baixo p e alto f $p = 0.005$ $f = 0.6$

In [150...]

```
p = 0.005
f = 0.6
```

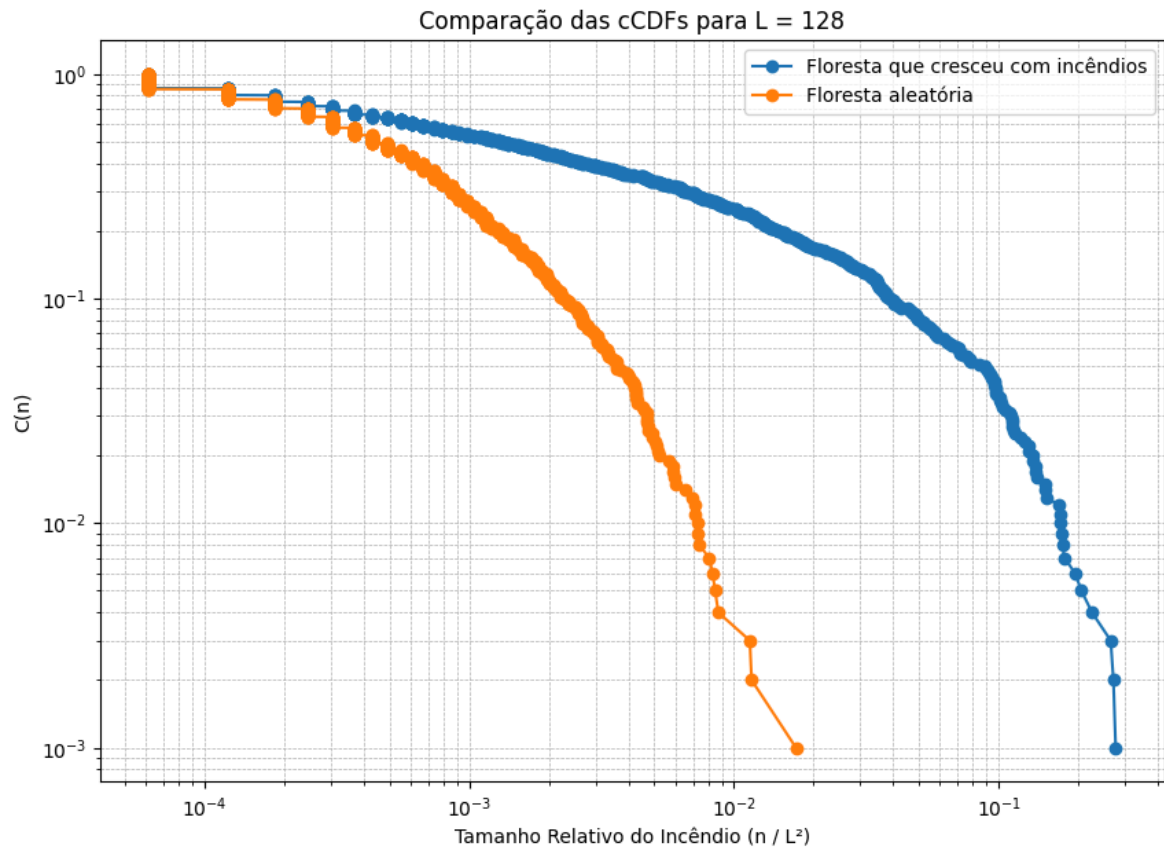
In [151...]

```
tam_incendios_evoluidos, tam_incendios_aleatorios = simula_floresta_3(L, p, f, n
```

In [152...]

```
ccdf_evoluidos, tam_relativos_evoluidos = calcular_ccdf(tam_incendios_evoluidos,
ccdf_aleatorios, tam_relativos_aleatorios = calcular_ccdf(tam_incendios_aleatori

# Plota as cCDFs
plotar_ccdf_multiplas(
    ccdfs=[ccdf_evoluidos, ccdf_aleatorios],
    tam_relativos=[tam_relativos_evoluidos, tam_relativos_aleatorios],
    labels=["Floresta que cresceu com incêndios", "Floresta aleatória"],
    L=L
)
```



Incêndios grandes - Alto p e baixo f

$p = 0.1$

$f = 0.1$

In [153...

```
p = 0.1
f = 0.1
```

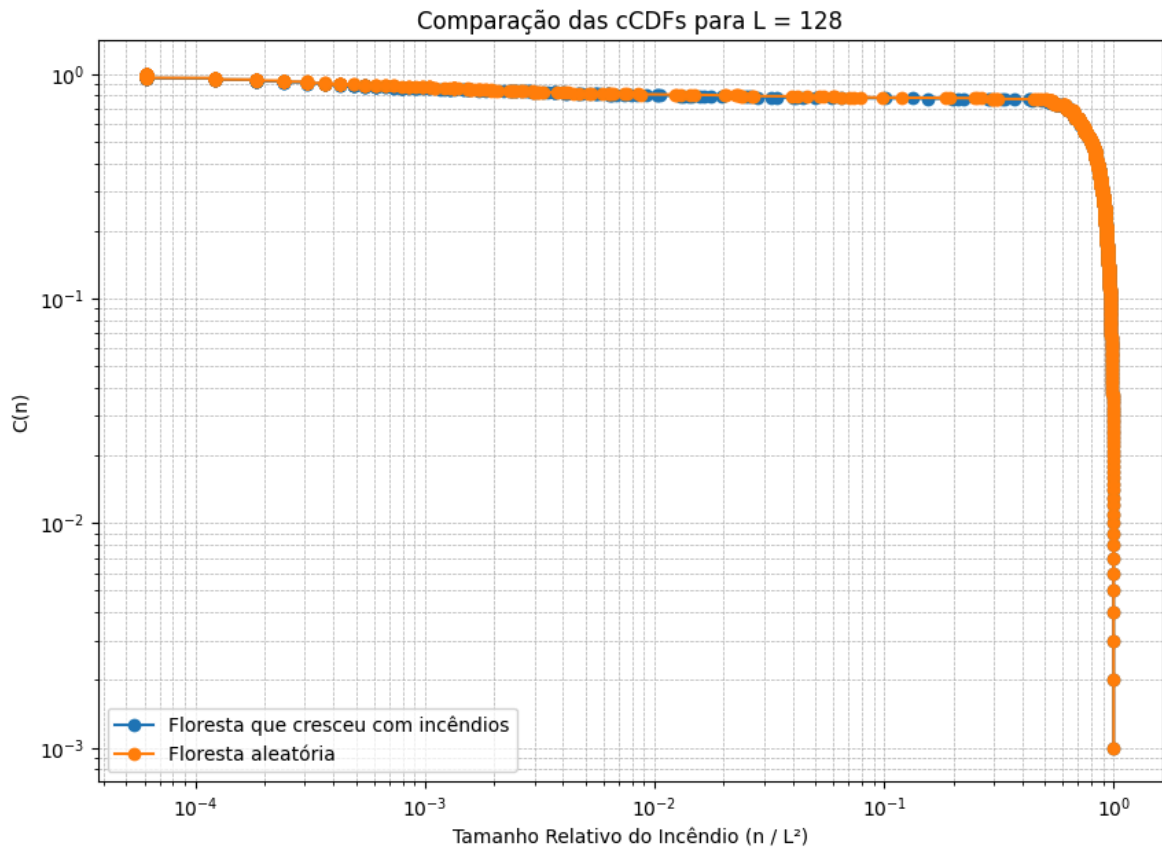
In [154...

```
tam_incendios_evoluidos, tam_incendios_aleatorios = simula_floresta_3(L, p, f, n
```

In [155...

```
ccdf_evoluidos, tam_relativos_evoluidos = calcular_ccdf(tam_incendios_evoluidos,
ccdf_aleatorios, tam_relativos_aleatorios = calcular_ccdf(tam_incendios_aleatori

# Plota as cCDFs
plotar_ccdf_multiplas(
    ccdfs=[ccdf_evoluidos, ccdf_aleatorios],
    tam_relativos=[tam_relativos_evoluidos, tam_relativos_aleatorios],
    labels=["Floresta que cresceu com incêndios", "Floresta aleatória"],
    L=L
)
```



Podemos observar que para incêndios pequenos, os gráficos diferem muito entre a floresta que cresceu com incêndio e a floresta aleatória. Entretanto, para incêndios grandes, os gráficos são muito semelhantes.

Parte 4

Implementação

```
In [156... def ajustar_expoente(tam_relativos, ccdf):
    """
    Ajusta a inclinação da região linear no gráfico log-log da cCDF.
    """

    # Converte para log-log
    log_n = np.log(tam_relativos)
    log_c = np.log(ccdf)

    # Seleção da região linear (ajustar limites conforme necessário)
    mask = (log_n > -6) & (log_n < -1) # Ajustar os limites
    log_n_linear = log_n[mask]
    log_c_linear = log_c[mask]

    # Realiza o ajuste linear:  $\log(C(n)) = \beta * \log(n) + \text{intercept}$ 
    beta, intercept = np.polyfit(log_n_linear, log_c_linear, 1)
    alpha = 1 - beta # Calcula o expoente da Lei de potência
    return alpha, beta, log_n_linear, log_c_linear, intercept

In [157... def plotar_ccdf_e_ajuste(tam_relativo, ccdf, alpha, beta, log_n_linear, intercept):
    """
    Plota a cCDF em escala log-log e inclui o ajuste linear calculado para os me
```

```
plt.figure(figsize=(8, 6)) # Tamanho consistente com a função plotar_ccdf

# cCDF original
plt.loglog(tam_relativo, ccdf, marker='o', linestyle='-', color='blue', label='ccdf')

# Ajuste linear calculado para os mesmos pontos do cCDF
ajuste_y = np.exp(beta * np.log(tam_relativo) + intercept) # Aplicando o ajuste
plt.loglog(tam_relativo, ajuste_y, linestyle='--', color='red', label=f"Ajuste Linear")

# Configurações do gráfico
plt.title("cCDF de Tamanhos Relativos de Incêndios com Ajuste Linear")
plt.xlabel("Tamanho Relativo do Incêndio (n / L²)")
plt.ylabel("C(n)")
plt.grid(True, which="both", linestyle="--", linewidth=0.5)
plt.legend()
plt.show()
```

a.

In [158...

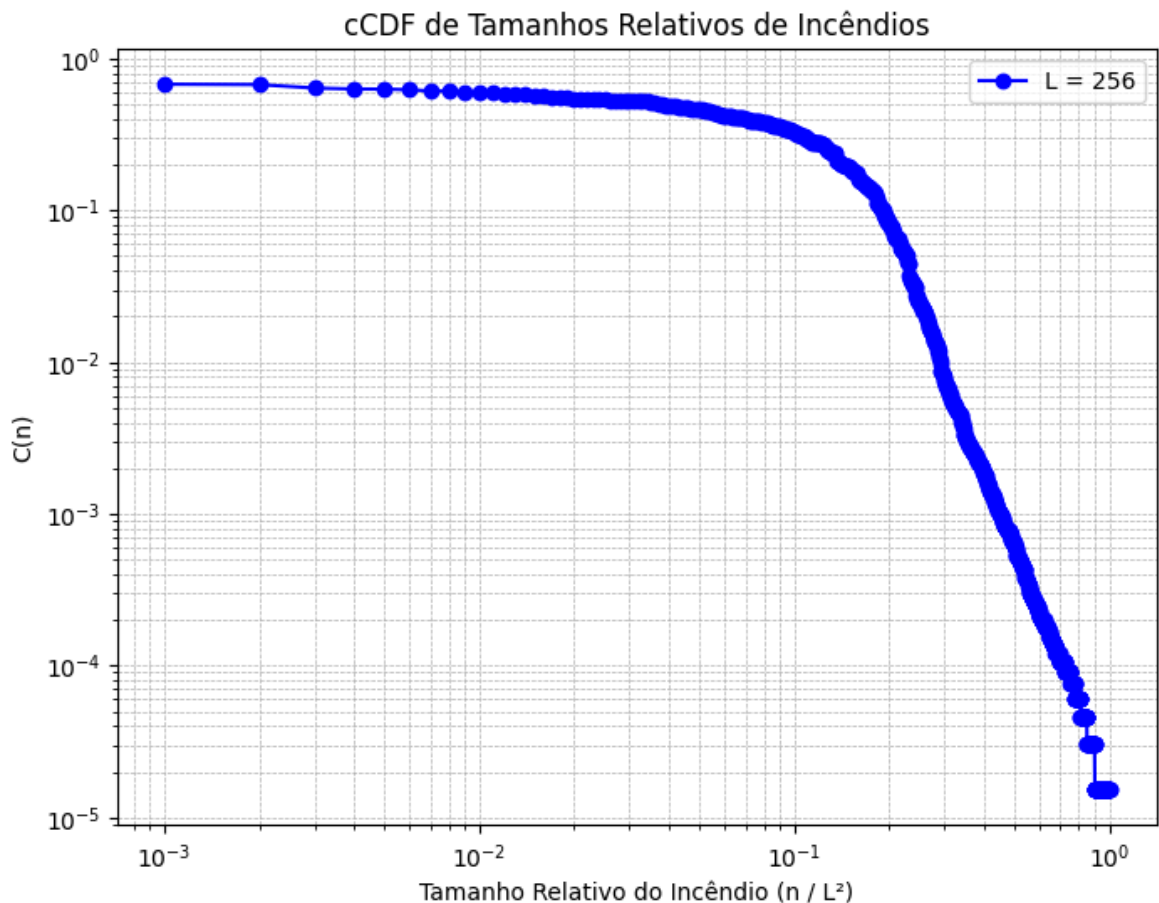
```
L = 256
p = 0.01
f = 0.2
```

In [159...

```
# Simulação
tam_incendios = simula_floresta_2(L, p, f, num_incendios)

ccdf, tam_relativos = calcular_ccdf(tam_incendios, L)

plotar_ccdf(ccdf, tam_relativos, L)
```

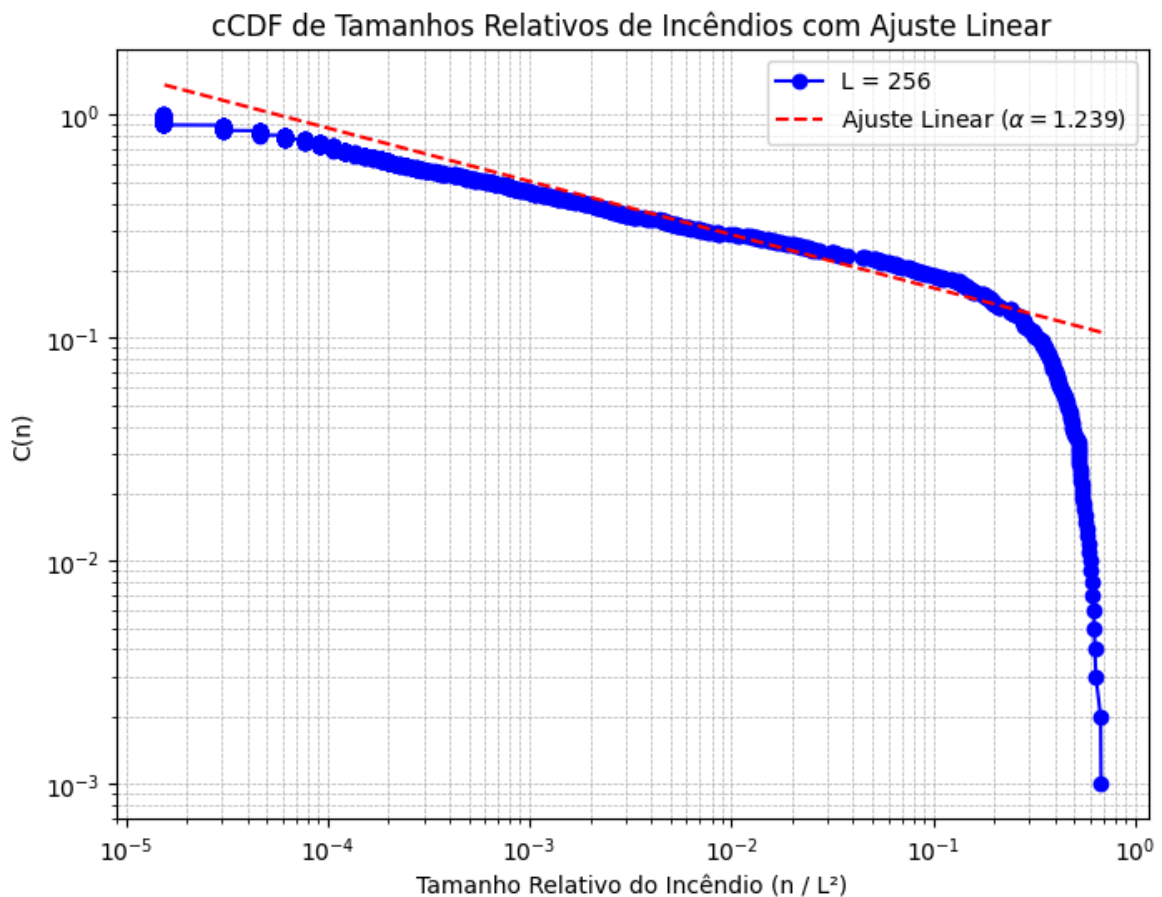


b.

```
In [160... alpha, beta, log_n_linear, log_c_linear, intercept = ajustar_expoente(tam_relati
print(f"Expoente da Lei de Potência ( $\alpha$ ): {alpha:.3f}")
print(f"Inclinação Ajustada ( $\beta$ ): {beta:.3f}")
```

Expoente da Lei de Potência (α): 1.239Inclinação Ajustada (β): -0.239

```
In [161... plotar_ccdf_e_ajuste(tam_relativos, ccdf, alpha, beta, log_n_linear, intercept,
```



c.

```
In [162... # Valor de referência
alpha_referencia = 1.15
```

```
In [163... # Diferença e erro relativo
diferenca = abs(alpha - alpha_referencia)
erro_relativo = (diferenca / alpha_referencia) * 100

# Mostrar os resultados
print(f"Valor calculado de  $\alpha$ : {alpha:.3f}")
print(f"Valor de referência de  $\alpha$ : {alpha_referencia}")
print(f"Diferença: {diferenca:.3f}")
print(f"Erro relativo: {erro_relativo:.2f}%")
```

Valor calculado de α : 1.239
 Valor de referência de α : 1.15
 Diferença: 0.089
 Erro relativo: 7.74%

d.

Incêndios muito pequenos - Baixo p e alto f

$p = 0.005$

$f = 0.6$

In [164...

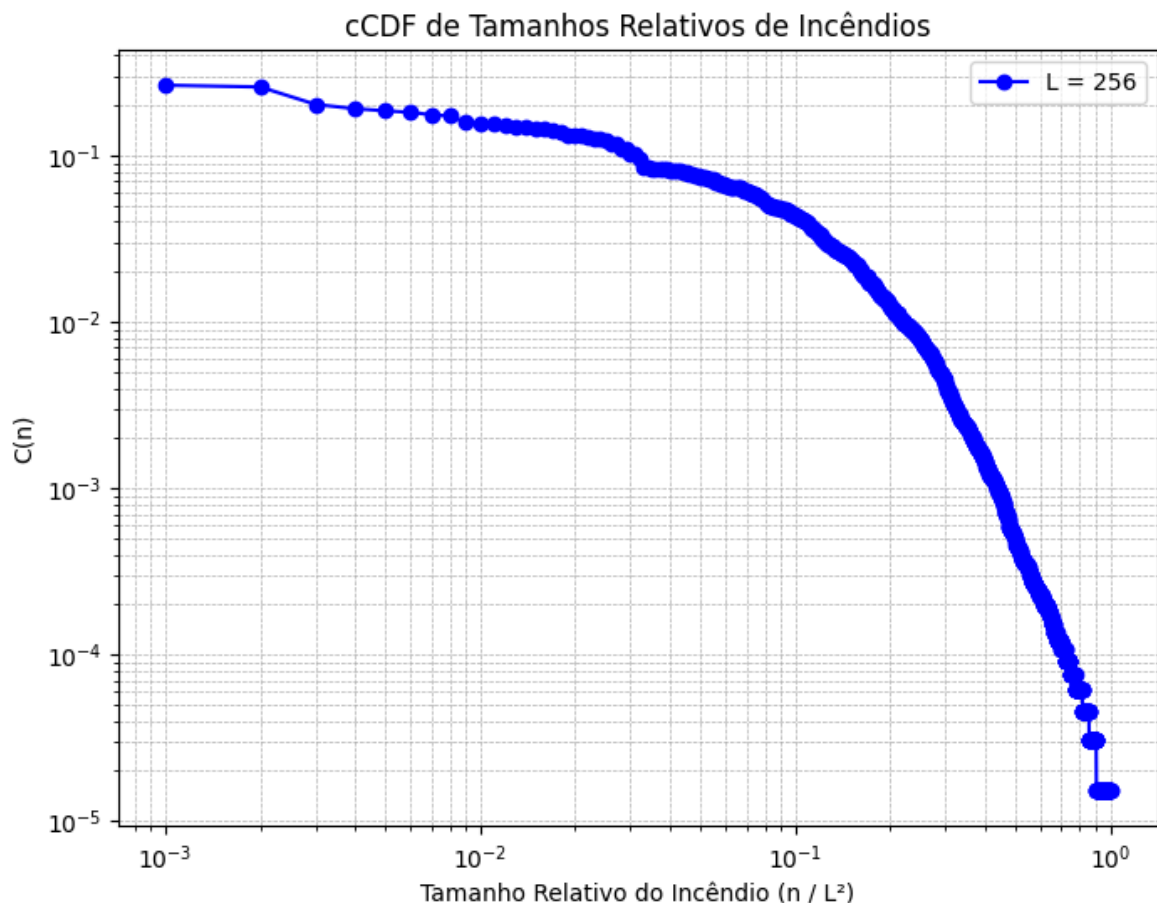
```
p = 0.005
f = 0.6
```

In [165...

```
# Simulação
tam_incendios = simula_floresta_2(L, p, f, num_incendios)

ccdf, tam_relativos = calcular_ccdf(tam_incendios, L)

plotar_ccdf(ccdf, tam_relativos, L)
```



In [166...

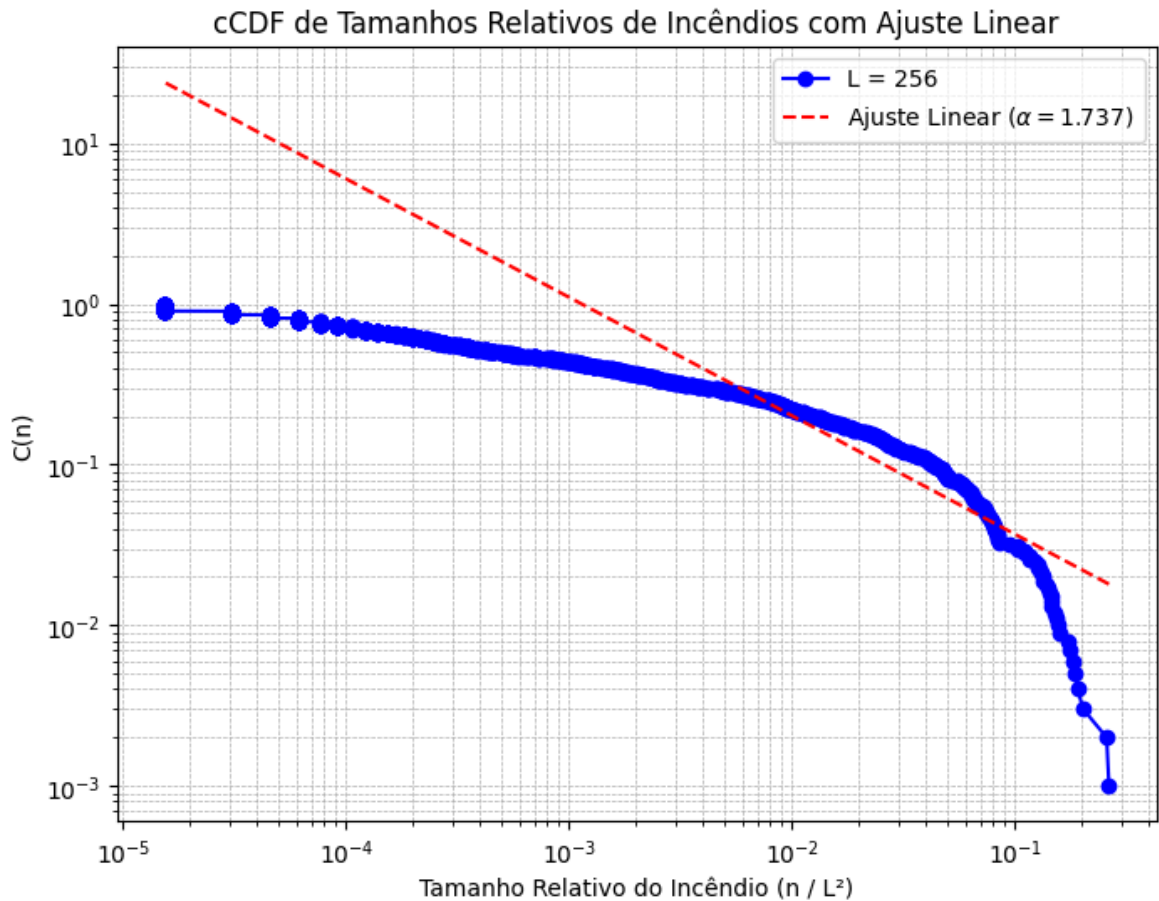
```
alpha, beta, log_n_linear, log_c_linear, intercept = ajustar_expoente(tam_relati

print(f"Expoente da Lei de Potência ( $\alpha$ ): {alpha:.3f}")
print(f"Inclinação Ajustada ( $\beta$ ): {beta:.3f}")
```

Expoente da Lei de Potência (α): 1.737
 Inclinação Ajustada (β): -0.737

In [167...

```
plotar_ccdf_e_ajuste(tam_relativos, ccdf, alpha, beta, log_n_linear, intercept,
```



In [168...

```
# Diferença e erro relativo
diferenca = abs(alpha - alpha_referencia)
erro_relativo = (diferenca / alpha_referencia) * 100

# Mostrar os resultados
print(f"Valor calculado de  $\alpha$ : {alpha:.3f}")
print(f"Valor de referência de  $\alpha$ : {alpha_referencia}")
print(f"Diferença: {diferenca:.3f}")
print(f"Erro relativo: {erro_relativo:.2f}%")
```

Valor calculado de α : 1.737Valor de referência de α : 1.15

Diferença: 0.587

Erro relativo: 51.09%

Incêndios grandes - Alto p e baixo f

 $p = 0.1$ $f = 0.1$

In [179...

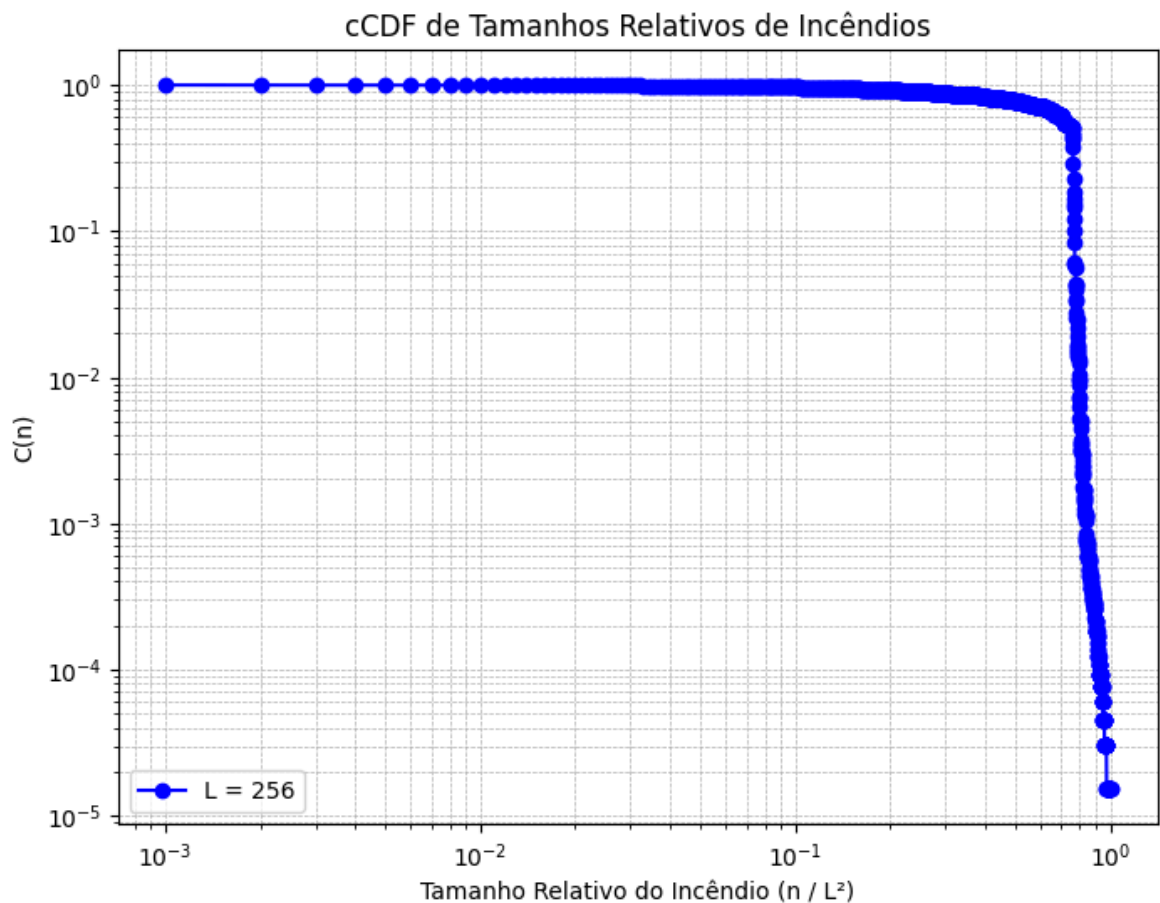
```
p = 0.1
f = 0.1
```

In [180...

```
# Simulação
tam_incendios = simula_floresta_2(L, p, f, num_incendios)

ccdf, tam_relativos = calcular_ccdf(tam_incendios, L)
```

```
plotar_ccdf(ccdf, tam_relativos, L)
```

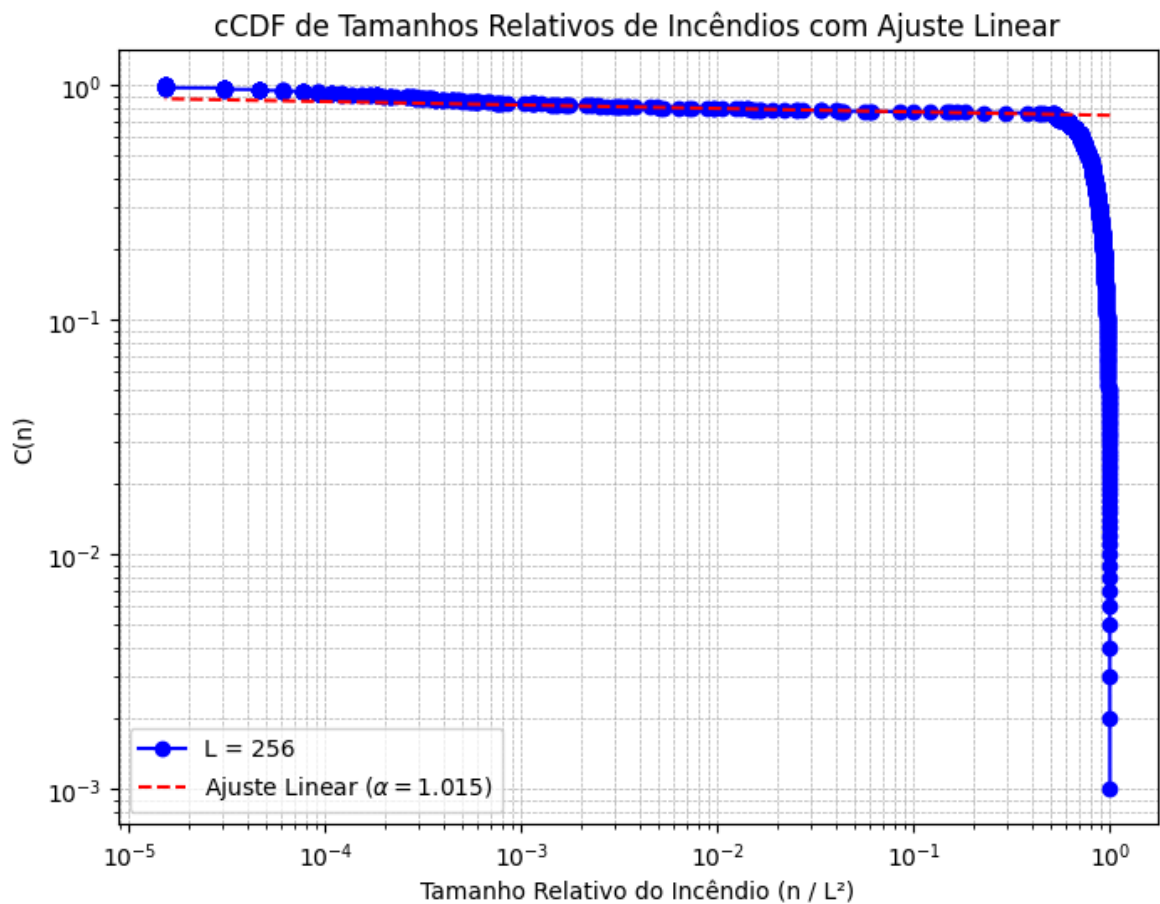


```
In [181... alpha, beta, log_n_linear, log_c_linear, intercept = ajustar_expoente(tam_relati
print(f"Expoente da Lei de Potência ( $\alpha$ ): {alpha:.3f}")
print(f"Inclinação Ajustada ( $\beta$ ): {beta:.3f}")
```

Expoente da Lei de Potência (α): 1.015

Inclinação Ajustada (β): -0.015

```
In [182... plotar_ccdf_e_ajuste(tam_relativos, ccdf, alpha, beta, log_n_linear, intercept,
```



In [183...

```
# Diferença e erro relativo
diferenca = abs(alpha - alpha_referencia)
erro_relativo = (diferenca / alpha_referencia) * 100

# Mostrar os resultados
print(f"Valor calculado de  $\alpha$ : {alpha:.3f}")
print(f"Valor de referência de  $\alpha$ : {alpha_referencia}")
print(f"Diferença: {diferenca:.3f}")
print(f"Erro relativo: {erro_relativo:.2f}%")
```

Valor calculado de α : 1.015Valor de referência de α : 1.15

Diferença: 0.135

Erro relativo: 11.77%

Podemos observar que para incêndios grades, o trecho com linearidade é maior que para incêncios pequenos.